```
apps/api/tests/test_identity_constraints.py-68-            )
apps/api/tests/test_identity_constraints.py-69-
apps/api/tests/test_identity_constraints.py-70-    try:
apps/api/tests/test_identity_constraints.py-71-        with TestClient(app) as client:
apps/api/tests/test_identity_constraints.py:72:            anonymous =
client.get("/api/v1/admin/authorization/capabilities")
apps/api/tests/test_identity_constraints.py-73-            assert anonymous.status_code ==
401
apps/api/tests/test_identity_constraints.py-74-            assert anonymous.json() ==
{"detail": "Authentication required"}
apps/api/tests/test_identity_constraints.py-75-
apps/api/tests/test_identity_constraints.py-76-            invalid_session = client.get(
apps/api/tests/test_identity_constraints.py:77:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-78-                cookies={SESSION_COOKIE:
"invalid-session-token"},
apps/api/tests/test_identity_constraints.py-79-            )
apps/api/tests/test_identity_constraints.py-80-            assert
invalid_session.status_code == 401
apps/api/tests/test_identity_constraints.py-81-
apps/api/tests/test_identity_constraints.py-82-            allowed = client.get(
apps/api/tests/test_identity_constraints.py:83:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-84-                cookies={SESSION_COOKIE:
tokens[0]},
apps/api/tests/test_identity_constraints.py-85-            )
apps/api/tests/test_identity_constraints.py-86-            #₹ The request_id header is
optional, but if supplied it must be validated and preserved as the correlation_id.
apps/api/tests/test_identity_constraints.py-87-            correlated = client.get(
apps/api/tests/test_identity_constraints.py:88:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-89-                cookies={SESSION_COOKIE:
tokens[0]},
apps/api/tests/test_identity_constraints.py-90-                headers={"request_id":
"client-correlation-123"},
apps/api/tests/test_identity_constraints.py-91-            )
apps/api/tests/test_identity_constraints.py-92-            assert correlated.status_code ==
200
apps/api/tests/test_identity_constraints.py-93-            correlated_body =
correlated.json()
```

```
apps/api/tests/test_identity_constraints.py-94-            assert
correlated_body["request_id"].startswith("req_")
apps/api/tests/test_identity_constraints.py-95-            assert
correlated_body["request_id"] != "client-correlation-123"
apps/api/tests/test_identity_constraints.py-96-            assert
correlated_body["correlation_id"] == "client-correlation-123"
apps/api/tests/test_identity_constraints.py-97-
apps/api/tests/test_identity_constraints.py-98-            malformed = client.get(
apps/api/tests/test_identity_constraints.py:99:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-100-                cookies={SESSION_COOKIE:
tokens[0]},
apps/api/tests/test_identity_constraints.py-101-                headers={"request_id": "x" *
129},
apps/api/tests/test_identity_constraints.py-102-            )
apps/api/tests/test_identity_constraints.py-103-            assert malformed.status_code
== 400
apps/api/tests/test_identity_constraints.py-104-
apps/api/tests/test_identity_constraints.py-105-            # The platform-admin user
should be allowed to inspect the authorization policy.
apps/api/tests/test_identity_constraints.py-106-            assert allowed.status_code ==
200
apps/api/tests/test_identity_constraints.py-107-            body = allowed.json()
apps/api/tests/test_identity_constraints.py-108-            assert
body["data"]["subject"]["user_id"] == str(admin_id)
apps/api/tests/test_identity_constraints.py-109-            assert
body["data"]["subject"]["platform_role_code"] == "platform_admin"
apps/api/tests/test_identity_constraints.py-110-            assert
body["data"]["effective_permissions"] == ["platform.admin"]
apps/api/tests/test_identity_constraints.py-111-            assert body["data"]["capabilities"]
== ["inspect_authorization_policy"]
apps/api/tests/test_identity_constraints.py-112-            assert
body["data"]["policy_version"] == "sprint-1-platform-rbac-v1"
apps/api/tests/test_identity_constraints.py-113-            assert
body["request_id"].startswith("req_")
apps/api/tests/test_identity_constraints.py-114-            assert body["correlation_id"] ==
body["request_id"]
apps/api/tests/test_identity_constraints.py-115-
apps/api/tests/test_identity_constraints.py-116-            denied = client.get(
```

```
apps/api/tests/test_identity_constraints.py:117:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-118-            cookies={SESSION_COOKIE:
tokens[1]},
apps/api/tests/test_identity_constraints.py-119-            headers={"X-Role":
"platform_admin"},
apps/api/tests/test_identity_constraints.py-120-        )
apps/api/tests/test_identity_constraints.py-121-        assert denied.status_code ==
403
apps/api/tests/test_identity_constraints.py-122-        assert denied.json() == {"detail":
"Permission denied"}
apps/api/tests/test_identity_constraints.py-123-    finally:
apps/api/tests/test_identity_constraints.py-124-        with connect() as conn,
conn.cursor() as cur:
apps/api/tests/test_identity_constraints.py-125-            cur.execute("DELETE FROM
users WHERE user_id IN (%s, %s)", (admin_id, user_id))
apps/api/tests/test_identity_constraints.py-126-
apps/api/tests/test_identity_constraints.py-127-
apps/api/tests/test_identity_constraints.py-128-
apps/api/tests/test_identity_constraints.py-129-def
test_permission_grant_revocation_takes_effect_on_next_request():
apps/api/tests/test_identity_constraints.py-130-    suffix = uuid4().hex
apps/api/tests/test_identity_constraints.py-131-    role_key = f"test-admin-{suffix}"
apps/api/tests/test_identity_constraints.py-132-    email =
f"permission-revocation-{suffix}@example.test"
--
apps/api/tests/test_identity_constraints.py-174-            INSERT INTO auth_sessions
(user_id, refresh_token_hash, expires_at)
apps/api/tests/test_identity_constraints.py-175-            VALUES (%s, %s, %s)
apps/api/tests/test_identity_constraints.py-176-            """,
apps/api/tests/test_identity_constraints.py-177-            (user_id, digest,
session_expiry(5)),
apps/api/tests/test_identity_constraints.py-178-        )
apps/api/tests/test_identity_constraints.py-179-
apps/api/tests/test_identity_constraints.py-180-        with TestClient(app) as client:
apps/api/tests/test_identity_constraints.py-181-            first_response = client.get(
apps/api/tests/test_identity_constraints.py:182:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-183-            cookies={SESSION_COOKIE:
token},
```

```
apps/api/tests/test_identity_constraints.py-184-        )
apps/api/tests/test_identity_constraints.py-185-        assert
first_response.status_code == 200
apps/api/tests/test_identity_constraints.py-186-        assert
first_response.json()["data"]["effective_permissions"] == [
apps/api/tests/test_identity_constraints.py-187-            "platform.admin"
apps/api/tests/test_identity_constraints.py-188-        ]
apps/api/tests/test_identity_constraints.py-189-
apps/api/tests/test_identity_constraints.py-190-        with connect() as conn,
conn.cursor() as cur:
apps/api/tests/test_identity_constraints.py-191-            cur.execute(
apps/api/tests/test_identity_constraints.py-192-                "DELETE FROM
role_permissions WHERE role_id = %s AND permission_id = %s",
apps/api/tests/test_identity_constraints.py-193-                (role_id, permission_id),
apps/api/tests/test_identity_constraints.py-194-            )
apps/api/tests/test_identity_constraints.py-195-
apps/api/tests/test_identity_constraints.py-196-        next_response = client.get(
apps/api/tests/test_identity_constraints.py:197:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-198-            cookies={SESSION_COOKIE:
token},
apps/api/tests/test_identity_constraints.py-199-        )
apps/api/tests/test_identity_constraints.py-200-        assert
next_response.status_code == 403
apps/api/tests/test_identity_constraints.py-201-        assert next_response.json() ==
{"detail": "Permission denied"}
apps/api/tests/test_identity_constraints.py-202-
apps/api/tests/test_identity_constraints.py-203-    finally:
apps/api/tests/test_identity_constraints.py-204-        with connect() as conn,
conn.cursor() as cur:
apps/api/tests/test_identity_constraints.py-205-            if user_id is not None:
apps/api/tests/test_identity_constraints.py-206-                cur.execute("DELETE FROM
users WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py-207-            if role_id is not None:
apps/api/tests/test_identity_constraints.py-208-                cur.execute("DELETE FROM
roles WHERE role_id = %s", (role_id,))
apps/api/tests/test_identity_constraints.py-209-
apps/api/tests/test_identity_constraints.py-210-def
test_platform_role_assignment_rejects_workspace_scoped_roles():
```

```
apps/api/tests/test_identity_constraints.py-211-    email =
f"wrong-scope-{uuid4().hex}@example.test"
apps/api/tests/test_identity_constraints.py-212-    with connect() as conn, conn.cursor()
as cur:
--
apps/api/tests/test_identity_constraints.py-252-            "INSERT INTO auth_sessions
(user_id, refresh_token_hash, expires_at) "
apps/api/tests/test_identity_constraints.py-253-            "VALUES (%s, %s, %s)",
apps/api/tests/test_identity_constraints.py-254-            (user_id, digest,
session_expiry(5)),
apps/api/tests/test_identity_constraints.py-255-        )
apps/api/tests/test_identity_constraints.py-256-
apps/api/tests/test_identity_constraints.py-257-    try:
apps/api/tests/test_identity_constraints.py-258-        with TestClient(app) as client:
apps/api/tests/test_identity_constraints.py-259-            first_response = client.get(
apps/api/tests/test_identity_constraints.py:260:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-261-                cookies={SESSION_COOKIE:
token},
apps/api/tests/test_identity_constraints.py-262-            )
apps/api/tests/test_identity_constraints.py-263-            assert
first_response.status_code == 200
apps/api/tests/test_identity_constraints.py-264-            assert
first_response.json()["data"]["effective_permissions"] == [
apps/api/tests/test_identity_constraints.py-265-                "platform.admin"
apps/api/tests/test_identity_constraints.py-266-            ]
apps/api/tests/test_identity_constraints.py-267-
apps/api/tests/test_identity_constraints.py-268-            # Revoke this test user's role
without changing shared role permissions.
apps/api/tests/test_identity_constraints.py-269-            with connect() as conn,
conn.cursor() as cur:
apps/api/tests/test_identity_constraints.py-270-                cur.execute(
apps/api/tests/test_identity_constraints.py-271-                    "UPDATE users SET
platform_role_id = NULL WHERE user_id = %s",
apps/api/tests/test_identity_constraints.py-272-                    (user_id,),
apps/api/tests/test_identity_constraints.py-273-                )
apps/api/tests/test_identity_constraints.py-274-
apps/api/tests/test_identity_constraints.py-275-            next_response = client.get(
apps/api/tests/test_identity_constraints.py:276:
"/api/v1/admin/authorization/capabilities",
```

```
apps/api/tests/test_identity_constraints.py-277-            cookies={SESSION_COOKIE:
token},
apps/api/tests/test_identity_constraints.py-278-        )
apps/api/tests/test_identity_constraints.py-279-        assert
next_response.status_code == 403
apps/api/tests/test_identity_constraints.py-280-        assert next_response.json() ==
{"detail": "Permission denied"}
apps/api/tests/test_identity_constraints.py-281-    finally:
apps/api/tests/test_identity_constraints.py-282-        with connect() as conn,
conn.cursor() as cur:
apps/api/tests/test_identity_constraints.py-283-            cur.execute("DELETE FROM
users WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py-284-
apps/api/tests/test_identity_constraints.py-285-
apps/api/tests/test_identity_constraints.py-286-# def
test_capabilities_request_id_is_canonical_and_correlation_is_preserved():
apps/api/tests/test_identity_constraints.py-287-#     with connect() as conn,
conn.cursor() as cur:
apps/api/tests/test_identity_constraints.py-288-#         cur.execute(
apps/api/tests/test_identity_constraints.py-289-#         """
apps/api/tests/test_identity_constraints.py-290-#         SELECT u.user_id,
s.refresh_token_hash
apps/api/tests/test_identity_constraints.py-291-#         FROM users u
--
apps/api/tests/test_identity_constraints.py-308-#     # The database stores only the
token digest, so use a dedicated test
apps/api/tests/test_identity_constraints.py-309-#     # session instead of attempting to
recover an existing session token.
apps/api/tests/test_identity_constraints.py-310-#     pytest.skip("Use the
dedicated-session integration fixture for this assertion")
apps/api/tests/test_identity_constraints.py-311-
apps/api/tests/test_identity_constraints.py-312-# def
test_capabilities_rejects_malformed_request_id():
apps/api/tests/test_identity_constraints.py-313-#     # Authentication runs before the
endpoint validates its header.
apps/api/tests/test_identity_constraints.py-314-#     # An unauthenticated request must
not be mistaken for a header-validation test.
apps/api/tests/test_identity_constraints.py-315-#     response = TestClient(app).get(
apps/api/tests/test_identity_constraints.py:316:#
"/api/v1/admin/authorization/capabilities",
```

```
apps/api/tests/test_identity_constraints.py-317-#     headers={"request_id": "x" * 129},
apps/api/tests/test_identity_constraints.py-318-#     )
apps/api/tests/test_identity_constraints.py-319-#     assert response.status_code == 401
--
apps/api/tests/test_auth_unit.py-133-
apps/api/tests/test_auth_unit.py-134-
apps/api/tests/test_auth_unit.py-135-@pytest.mark.parametrize(
apps/api/tests/test_auth_unit.py-136-    ("method", "path", "kwargs"),
apps/api/tests/test_auth_unit.py-137-    [
apps/api/tests/test_auth_unit.py-138-        ("get", "/auth/me", {"cookies":
{"finsight_session": "synthetic-token"}}),
apps/api/tests/test_auth_unit.py-139-        ("post", "/auth/logout", {"cookies":
{"finsight_session": "synthetic-token"}}),
apps/api/tests/test_auth_unit.py-140-        ("get",
apps/api/tests/test_auth_unit.py:141:            "/api/v1/admin/authorization/capabilities",
apps/api/tests/test_auth_unit.py-142-            {"cookies": {"finsight_session":
"synthetic-token"}},
apps/api/tests/test_auth_unit.py-143-        ),
apps/api/tests/test_auth_unit.py-144-    ],
apps/api/tests/test_auth_unit.py-145-)
apps/api/tests/test_auth_unit.py-146-def
test_session_database_outages_return_controlled_error(monkeypatch, method, path,
kwargs):
apps/api/tests/test_auth_unit.py-147-    import app.main as main
apps/api/tests/test_auth_unit.py-148-    import psycopg
apps/api/tests/test_auth_unit.py-149-
apps/api/tests/test_auth_unit.py-150-    def unavailable_database():
apps/api/tests/test_auth_unit.py-151-        raise psycopg.OperationalError("synthetic
database detail")
apps/api/tests/test_auth_unit.py-152-
apps/api/tests/test_auth_unit.py-153-    monkeypatch.setattr(main, "connect",
unavailable_database)
apps/api/tests/test_auth_unit.py-154-    response = getattr(TestClient(app),
method)(path, **kwargs)
apps/api/tests/test_auth_unit.py-155-
apps/api/tests/test_auth_unit.py-156-    assert response.status_code == 503
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
-    monkeypatch.setattr(main, "connect", unavailable_database)
apps/api/tests/test_auth_unit.py-154-    response = getattr(TestClient(app),
method)(path, **kwargs)
```

```
apps/api/tests/test_auth_unit.py-155-
apps/api/tests/test_auth_unit.py-156-    assert response.status_code == 503
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % grep -n -E
'/platform/admin/capability|/api/v1/admin/authorization/capabilities' \
  IMPLEMENTATION_STATUS.txt \
  SPRINT_1_TEST_EXECUTION_STATUS.txt \
  FUNCTION_TEST_TRACEABILITY.txt
IMPLEMENTATION_STATUS.txt:340:- Added GET /platform/admin/capability to the real
FastAPI application, protected by the database-backed platform.admin permission
dependency.This endpoint is a minimal authorization capability check and does not
mutate data.
IMPLEMENTATION_STATUS.txt:377:- Endpoint: GET
/api/v1/admin/authorization/capabilities.
SPRINT_1_TEST_EXECUTION_STATUS.txt:137:- Added integration assertions for
GET /platform/admin/capability against the real FastAPI app: anonymous request
expects 401; platform-admin session expects 200; authenticated user without
permission expects 403 even with X-Role: platform_admin.
FUNCTION_TEST_TRACEABILITY.txt:4:Function: GET
/api/v1/admin/authorization/capabilities
FUNCTION_TEST_TRACEABILITY.txt:12:Endpoint: GET
/api/v1/admin/authorization/capabilities
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % git diff --check

grep -n -E '/platform/admin/capability|/api/v1/admin/authorization/capabilities' \
  IMPLEMENTATION_STATUS.txt \
  SPRINT_1_TEST_EXECUTION_STATUS.txt \
  FUNCTION_TEST_TRACEABILITY.txt

git status --short
IMPLEMENTATION_STATUS.txt:340:- Added GET /platform/admin/capability to the real
FastAPI application, protected by the database-backed platform.admin permission
dependency.This endpoint is a minimal authorization capability check and does not
mutate data.
IMPLEMENTATION_STATUS.txt:377:- Endpoint: GET
/api/v1/admin/authorization/capabilities.
SPRINT_1_TEST_EXECUTION_STATUS.txt:137:- Added integration assertions for
GET /platform/admin/capability against the real FastAPI app: anonymous request
expects 401; platform-admin session expects 200; authenticated user without
permission expects 403 even with X-Role: platform_admin.
```

```
FUNCTION_TEST_TRACEABILITY.txt:4:Function: GET
/api/v1/admin/authorization/capabilities
FUNCTION_TEST_TRACEABILITY.txt:12:Endpoint: GET
/api/v1/admin/authorization/capabilities
 M .github/workflows/ci.yml
 M IMPLEMENTATION_STATUS.txt
 M README.md
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/app/provision_user.py
 M apps/api/tests/test_auth_unit.py
 M apps/api/tests/test_identity_constraints.py
 M apps/api/tests/test_provision_user.py
 M apps/web/tests/ui/auth.spec.js
 M conversation.txt
 M docs/architecture/README.md
 M docs/requirements/traceability-requirements.csv
 M docs/requirements/traceability-stories.csv
 M scripts/package_release.py
?? FUNCTION_TEST_TRACEABILITY.txt
?? FinSight_AI_API_Design_Pack/
?? apps/api/migrations/0002_platform_authorization.sql
?? apps/web/proxy.js
?? update_fin15_tracking.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
FIN-15 remains PARTIAL. The route mismatch is understood, but frontend protection,
the broader role/operation matrix, and the system smoke test still need evidence before
the story can be accepted.tching test request paths.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
echo "===== FRONTEND ROUTING / AUTH FILES ====="
find apps/web -maxdepth 4 -type f \
  \( -iname '*auth*' -o -iname '*proxy*' -o -iname '*middleware*' \
     -o -iname '*route*' -o -iname '*layout*' \) \
  -not -path '*/node_modules/*' | sort

echo
echo "===== AUTH UI TESTS ====="
sed -n '1,260p' apps/web/tests/ui/auth.spec.js

echo
```

```
echo "===== FRONTEND PACKAGE SCRIPTS ====="
node -e '
const fs = require("fs");
const p = "apps/web/package.json";
if (fs.existsSync(p)) {
  const j = JSON.parse(fs.readFileSync(p, "utf8"));
  console.log(JSON.stringify(j.scripts || {}, null, 2));
} else {
  console.log("apps/web/package.json not found");
}
'

echo
echo "===== AUTH / PROTECTED ROUTE REFERENCES ====="
grep -RInE
'require_permission|platform\.admin|finsight_session|protected|redirect|middleware|prox
y' \
  apps/api/app apps/web \
  --exclude-dir=node_modules --exclude-dir=.next \
  --exclude='*.lock' | head -n 180

===== FRONTEND ROUTING / AUTH FILES =====
apps/web/.next/app-path-routes-manifest.json
apps/web/.next/dev/routes-manifest.json
apps/web/.next/dev/server/interception-route-rewrite-manifest.js
apps/web/.next/dev/server/middleware-build-manifest.js
apps/web/.next/dev/server/middleware-manifest.json
apps/web/.next/dev/types/routes.d.ts
apps/web/.next/diagnostics/route-bundle-stats.json
apps/web/.next/routes-manifest.json
apps/web/.next/server/chunks/_next-internal_server_app_api_runtime-config_route_acti
ons_1gyxn3ezyv1q9.js
apps/web/.next/server/chunks/_next-internal_server_app_api_runtime-config_route_acti
ons_1gyxn3ezyv1q9.js.map
apps/web/.next/server/interception-route-rewrite-manifest.js
apps/web/.next/server/middleware-build-manifest.js
apps/web/.next/server/middleware-manifest.json
apps/web/.next/standalone/.next/app-path-routes-manifest.json
apps/web/.next/standalone/.next/routes-manifest.json
apps/web/.next/static/local/_clientMiddlewareManifest.js
```

apps/web/.next/types/routes.d.ts
apps/web/app/api/runtime-config/route.js
apps/web/app/layout.js
apps/web/proxy.js
apps/web/tests/e2e/auth.spec.js
apps/web/tests/ui/auth.spec.js

===== AUTH UI TESTS =====
```
const { test, expect } = require("@playwright/test");

test.beforeEach(async ({ page }) => {
  let signedIn = false;
  await page.route("**/api/runtime-config", (route) => route.fulfill({
    status: 200,
    contentType: "application/json",
    body: JSON.stringify({ apiBaseUrl: "http://127.0.0.1:8100" }),
  }));
  await page.route("http://127.0.0.1:8100/auth/me", (route) => route.fulfill({
    status: signedIn ? 200 : 401,
    contentType: "application/json",
    body: signedIn
      ? JSON.stringify({ user_id: "11111111-1111-4111-8111-111111111111", email:
"analyst@example.test", display_name: "Synthetic Analyst" })
      : JSON.stringify({ detail: "Authentication required" }),
  }));
  await page.route("http://127.0.0.1:8100/auth/login", async (route) => {
    const payload = route.request().postDataJSON();
    if (payload.password !== "correct synthetic password") {
      return route.fulfill({ status: 401, contentType: "application/json", body:
JSON.stringify({ detail: "Email or password is incorrect." }) });
    }
    signedIn = true;
    return route.fulfill({ status: 200, contentType: "application/json", body: JSON.stringify({
user_id: "11111111-1111-4111-8111-111111111111", email: payload.email, display_name:
"Synthetic Analyst" }) });
  });
  await page.route("http://127.0.0.1:8100/auth/logout", async (route) => {
    signedIn = false;
    await route.fulfill({ status: 204, body: "" });
  });
```

```
});

test("login reports a generic error and success enables logout", async ({ page }) => {
  await page.goto("/");
  await expect(page.getByRole("heading", { name: "Sign in" })).toBeVisible();
  await page.getByLabel("Email").fill("analyst@example.test");
  await page.getByLabel("Password").fill("wrong synthetic password");
  await page.getByRole("button", { name: "Sign in" }).click();
  await expect(page.getByText("Email or password is incorrect.")).toBeVisible();

  await page.getByLabel("Password").fill("correct synthetic password");
  await page.getByRole("button", { name: "Sign in" }).click();
  await expect(page.getByRole("status").filter({ hasText: "Signed in as Synthetic Analyst"
})).toBeVisible();
  await page.getByRole("button", { name: "Sign out" }).click();
  await expect(page.getByRole("heading", { name: "Sign in" })).toBeVisible();
});

test("service outage is visible and the form remains usable", async ({ page }) => {
  await page.route("**/api/runtime-config", (route) => route.fulfill({ status: 503,
contentType: "application/json", body: "{}" }));
  await page.goto("/");
  await expect(page.getByText("Authentication service is unavailable.")).toBeVisible();
  await expect(page.getByLabel("Email")).toBeVisible();
});

test("protected app routes redirect anonymous browsers while sign-in stays public",
async ({ page }) => {
  const response = await page.goto("/app/private-report");
  expect(response.status()).toBe(200);
  await expect(page).toHaveURL(/\/\?next=/);
  await expect(page.getByRole("heading", { name: "Sign in" })).toBeVisible();

  await page.goto("/");
  await expect(page.getByRole("heading", { name: "Sign in" })).toBeVisible();
});

===== FRONTEND PACKAGE SCRIPTS =====
{
  "dev": "next dev",
```

```
  "build": "next build",
  "start": "next start",
  "test:ui": "playwright test --config playwright.ui.config.js",
  "test:e2e": "playwright test --config playwright.e2e.config.js"
}

===== AUTH / PROTECTED ROUTE REFERENCES =====
apps/api/app/main.py:15:from fastapi.middleware.cors import CORSMiddleware
apps/api/app/main.py:26:app.add_middleware(
apps/api/app/main.py:34:SESSION_COOKIE = "finsight_session"
apps/api/app/main.py:109:def require_permission(permission_key: str):
apps/api/app/main.py:187:    _authorized: Annotated[dict[str, Any],
Depends(require_permission("platform.admin"))],
apps/api/app/main.py:204:        if "platform.admin" in permissions
apps/web/proxy.js:3:// This is an optimistic page redirect only. Protected APIs must
validate the
apps/web/proxy.js:5:export function proxy(request) {
apps/web/proxy.js:6:  if (!request.cookies.has("finsight_session")) {
apps/web/proxy.js:9:    return NextResponse.redirect(signIn);
apps/web/tests/ui/auth.spec.js:53:test("protected app routes redirect anonymous
browsers while sign-in stays public", async ({ page }) => {
apps/web/tests/e2e/auth.spec.js:3:test("browser login, generic rejection, protected
session, and logout use the live API", async ({ page }) => {
apps/web/tests/e2e/auth.spec.js:20:  expect(await page.evaluate(() =>
document.cookie)).not.toContain("finsight_session");
apps/web/test-results/e2e/results.xml:3:<testcase name="browser login, generic
rejection, protected session, and logout use the live API" classname="auth.spec.js"
time="0.814">
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd apps/web
npx playwright test tests/ui/auth.spec.js

Running 3 tests using 1 worker

  ✘  1 tests/ui/auth.spec.js:31:1 › login reports a generic error and success enables
logout (225ms)
  ✘  2 tests/ui/auth.spec.js:46:1 › service outage is visible and the form remains usable
(60ms)
  ✘  3 tests/ui/auth.spec.js:53:1 › protected app routes redirect anonymous browsers
while sign-in stays public (55ms)
```

```
  1) tests/ui/auth.spec.js:31:1 › login reports a generic error and success enables logout
─────────────

    Error: page.goto: Protocol error (Page.navigate): Cannot navigate to invalid URL
    Call log:
      - navigating to "/", waiting until "load"


      30 |
      31 | test("login reports a generic error and success enables logout", async ({ page })
=> {
    > 32 |   await page.goto("/");
         |              ^
      33 |   await expect(page.getByRole("heading", { name: "Sign in" })).toBeVisible();
      34 |   await page.getByLabel("Email").fill("analyst@example.test");
      35 |   await page.getByLabel("Password").fill("wrong synthetic password");
        at
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/apps/web/tests/ui/auth.sp
ec.js:32:14

    Error Context:
test-results/tests-ui-auth-login-report-6c96b--and-success-enables-logout/error-context.
md

  2) tests/ui/auth.spec.js:46:1 › service outage is visible and the form remains usable
─────────────────

    Error: page.goto: Protocol error (Page.navigate): Cannot navigate to invalid URL
    Call log:
      - navigating to "/", waiting until "load"


      46 | test("service outage is visible and the form remains usable", async ({ page }) =>
{
      47 |   await page.route("**/api/runtime-config", (route) => route.fulfill({ status: 503,
contentType: "application/json", body: "{}" }));
    > 48 |   await page.goto("/");
         |              ^
```

```
      49 |   await expect(page.getByText("Authentication service is
unavailable.")).toBeVisible();
      50 |   await expect(page.getByLabel("Email")).toBeVisible();
      51 | });
        at
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/apps/web/tests/ui/auth.sp
ec.js:48:14

    Error Context:
test-results/tests-ui-auth-service-outa-f6410-and-the-form-remains-usable/error-context.
md

  3) tests/ui/auth.spec.js:53:1 › protected app routes redirect anonymous browsers while
sign-in stays public

    Error: page.goto: Protocol error (Page.navigate): Cannot navigate to invalid URL
    Call log:
      - navigating to "/app/private-report", waiting until "load"


      52 |
      53 | test("protected app routes redirect anonymous browsers while sign-in stays
public", async ({ page }) => {
    > 54 |   const response = await page.goto("/app/private-report");
         |                              ^
      55 |   expect(response.status()).toBe(200);
      56 |   await expect(page).toHaveURL(/\/\?next=/);
      57 |   await expect(page.getByRole("heading", { name: "Sign in" })).toBeVisible();
        at
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/apps/web/tests/ui/auth.sp
ec.js:54:31

    Error Context:
test-results/tests-ui-auth-protected-ap-67b14--while-sign-in-stays-public/error-context.m
d

  3 failed
    tests/ui/auth.spec.js:31:1 › login reports a generic error and success enables logout
————————————
```

```
  tests/ui/auth.spec.js:46:1 › service outage is visible and the form remains usable
──────────────────

  tests/ui/auth.spec.js:53:1 › protected app routes redirect anonymous browsers while
sign-in stays public
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro web % npm run test:ui --
tests/ui/auth.spec.js

> finsight-web@0.1.0 test:ui
> playwright test --config playwright.ui.config.js tests/ui/auth.spec.js

Error: Timed out waiting 60000ms from config.webServer.

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro web %
(base) sandeeph@Sandeeps-MacBook-Pro NVDIA_Play_Book %  source
/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/bin/activate
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro NVDIA_Play_Book % cd
FinSight_AI
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd apps/api
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % python -m pytest
tests/test_identity_constraints.py -q
FFFFF
[100%]
================================================================
============== FAILURES
================================================================
===============

_____________________________________________________
test_identity_schema_rejects_duplicate_email_and_cascades_sessions
_____________________________________________________

  def test_identity_schema_rejects_duplicate_email_and_cascades_sessions():
      email = f"constraint-{uuid4().hex}@example.com"
>       with connect() as conn, conn.cursor() as cur:
          ^^^^^^^^^

tests/test_identity_constraints.py:16:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

  def connect() -> psycopg.Connection:
```

```
        database_url = os.environ.get("DATABASE_URL")
        if not database_url:
>           raise RuntimeError("DATABASE_URL is required")
E           RuntimeError: DATABASE_URL is required

app/database.py:17: RuntimeError
_____ test_platform_permission_is_resolved_from_database_role_and_session _____

    def test_platform_permission_is_resolved_from_database_role_and_session():
        admin_email = f"platform-admin-{uuid4().hex}@example.test"
        user_email = f"unprivileged-{uuid4().hex}@example.test"
        tokens = []
>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^

tests/test_identity_constraints.py:49:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

    def connect() -> psycopg.Connection:
        database_url = os.environ.get("DATABASE_URL")
        if not database_url:
>           raise RuntimeError("DATABASE_URL is required")
E           RuntimeError: DATABASE_URL is required

app/database.py:17: RuntimeError
_____ test_permission_grant_revocation_takes_effect_on_next_request _____

    def test_permission_grant_revocation_takes_effect_on_next_request():
        suffix = uuid4().hex
        role_key = f"test-admin-{suffix}"
        email = f"permission-revocation-{suffix}@example.test"
        user_id = None
        role_id = None
        permission_id = None
```

```
        try:
>           with connect() as conn, conn.cursor() as cur:
                ^^^^^^^^

tests/test_identity_constraints.py:138:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

    def connect() -> psycopg.Connection:
        database_url = os.environ.get("DATABASE_URL")
        if not database_url:
>           raise RuntimeError("DATABASE_URL is required")
E           RuntimeError: DATABASE_URL is required

app/database.py:17: RuntimeError

During handling of the above exception, another exception occurred:

    def test_permission_grant_revocation_takes_effect_on_next_request():
        suffix = uuid4().hex
        role_key = f"test-admin-{suffix}"
        email = f"permission-revocation-{suffix}@example.test"
        user_id = None
        role_id = None
        permission_id = None

        try:
            with connect() as conn, conn.cursor() as cur:
                cur.execute(
                    """
                    INSERT INTO roles (role_key, display_name, scope, description)
                    VALUES (%s, %s, 'platform', %s)
                    RETURNING role_id
                    """,
                    (role_key, "Synthetic Permission Revocation Role", "Test-only role"),
                )
                role_id = cur.fetchone()[0]

                cur.execute(
```

```
            "SELECT permission_id FROM permissions WHERE permission_key =
%s",
            ("platform.admin",),
        )
        row = cur.fetchone()
        assert row is not None, "platform.admin permission must exist"
        permission_id = row[0]

        cur.execute(
            "INSERT INTO role_permissions (role_id, permission_id) VALUES (%s,
%s)",
            (role_id, permission_id),
        )
        cur.execute(
            """
            INSERT INTO users (email, display_name, platform_role_id)
            VALUES (%s, %s, %s)
            RETURNING user_id
            """,
            (email, "Synthetic Permission Revocation User", role_id),
        )
        user_id = cur.fetchone()[0]

        token, digest = new_session_token()
        cur.execute(
            """
            INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at)
            VALUES (%s, %s, %s)
            """,
            (user_id, digest, session_expiry(5)),
        )

    with TestClient(app) as client:
        first_response = client.get(
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: token},
        )
        assert first_response.status_code == 200
        assert first_response.json()["data"]["effective_permissions"] == [
            "platform.admin"
```

```
            ]

            with connect() as conn, conn.cursor() as cur:
                cur.execute(
                    "DELETE FROM role_permissions WHERE role_id = %s AND
permission_id = %s",
                    (role_id, permission_id),
                )

            next_response = client.get(
                "/api/v1/admin/authorization/capabilities",
                cookies={SESSION_COOKIE: token},
            )
            assert next_response.status_code == 403
            assert next_response.json() == {"detail": "Permission denied"}

        finally:
>           with connect() as conn, conn.cursor() as cur:
              ^^^^^^^^^

tests/test_identity_constraints.py:204:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

    def connect() -> psycopg.Connection:
        database_url = os.environ.get("DATABASE_URL")
        if not database_url:
>           raise RuntimeError("DATABASE_URL is required")
E           RuntimeError: DATABASE_URL is required

app/database.py:17: RuntimeError
_____________________________________________________________
test_platform_role_assignment_rejects_workspace_scoped_roles
_____________________________________________________________

    def test_platform_role_assignment_rejects_workspace_scoped_roles():
        email = f"wrong-scope-{uuid4().hex}@example.test"
>       with connect() as conn, conn.cursor() as cur:
          ^^^^^^^^^

/.env.example
```

```
===== CURRENT DATABASE_URL STATE =====
DATABASE_URL is not set in this terminal.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % echo "===== POSTGRESQL
READINESS ====="
pg_isready -h 127.0.0.1 -p 55439

echo
echo "===== TEST DATABASE CONNECTION ====="
psql "postgresql:///finsight_test?host=127.0.0.1&port=55439" \
  -X -v ON_ERROR_STOP=1 -c \
  "SELECT current_database() AS database, current_user AS db_user;"
===== POSTGRESQL READINESS =====
127.0.0.1:55439 - accepting connections

===== TEST DATABASE CONNECTION =====
psql: error: connection to server at "127.0.0.1", port 55439 failed: FATAL:  database
"finsight_test" does not exist
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
DATABASE_URL="postgresql:///finsight_test?host=127.0.0.1&port=55439" \
python -m pytest tests/test_identity_constraints.py -q
FFFFF
[100%]
================================================================================
=============================== FAILURES ===============================
================================================================================

_____
test_identity_schema_rejects_duplicate_email_and_cascades_sessions
_____

    def test_identity_schema_rejects_duplicate_email_and_cascades_sessions():
        email = f"constraint-{uuid4().hex}@example.com"
>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

tests/test_identity_constraints.py:16:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _
app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
```

```
        ^^^
```

```
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
>                raise last_ex.with_traceback(None)
E                psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  database "finsight_test" does not exist

../../../.venv/lib/python3.12/site-packages/psycopg/connection.py:120: OperationalError
_____ test_platform_permission_is_resolved_from_database_role_and_session _____

    def test_platform_permission_is_resolved_from_database_role_and_session():
        admin_email = f"platform-admin-{uuid4().hex}@example.test"
        user_email = f"unprivileged-{uuid4().hex}@example.test"
        tokens = []
>       with connect() as conn, conn.cursor() as cur:
             ^^^^^^^^^

tests/test_identity_constraints.py:49:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _
app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _

cls = <class 'psycopg.Connection'>, conninfo = 'dbname=finsight_test host=127.0.0.1
port=55439 connect_timeout=3 hostaddr=127.0.0.1', autocommit = False, prepare_threshold =
5, context = None, row_factory = None
cursor_factory = None, kwargs = {'connect_timeout': 3}, params = {'connect_timeout': 3,
'dbname': 'finsight_test', 'host': '127.0.0.1', 'port': '55439'}, timeout = 3

    @classmethod
    def connect(
        cls,
```

```
        conninfo: str = "",
        *,
        autocommit: bool = False,
        prepare_threshold: int | None = 5,
        context: AdaptContext | None = None,
        row_factory: RowFactory[Row] | None = None,
        cursor_factory: type[Cursor[Row]] | None = None,
        **kwargs: ConnParam,
    ) -> Self:
        """
        Connect to a database server and return a new `Connection` instance.
        """

        params = cls._get_connection_params(conninfo, **kwargs)
        timeout = timeout_from_conninfo(params)
        rv = None
        attempts = conninfo_attempts(params)
        conn_errors: list[tuple[e.Error, str]] = []
        for attempt in attempts:
            tdescr = (attempt.get("host"), attempt.get("port"), attempt.get("hostaddr"))
            descr = "host: %r, port: %r, hostaddr: %r" % tdescr
            logger.debug("connection attempt: %s", descr)
            try:
                conninfo = make_conninfo("", **attempt)
                gen = cls._connect_gen(conninfo, timeout=timeout)
                rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
            except e.Error as ex:
                logger.debug("connection failed: %s: %s", descr, str(ex))
                conn_errors.append((ex, descr))
            except e._NO_TRACEBACK as ex:
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
>               raise last_ex.with_traceback(None)
E               psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  database "finsight_test" does not exist

../../../.venv/lib/python3.12/site-packages/psycopg/connection.py:120: OperationalError
```

```
_____ test_permission_grant_revocation_takes_effect_on_next_request _____

    def test_permission_grant_revocation_takes_effect_on_next_request():
        suffix = uuid4().hex
        role_key = f"test-admin-{suffix}"
        email = f"permission-revocation-{suffix}@example.test"
        user_id = None
        role_id = None
        permission_id = None

        try:
>           with connect() as conn, conn.cursor() as cur:
                ^^^^^^^^^

tests/test_identity_constraints.py:138:
_ _ _ _
```

```
        **kwargs: ConnParam,
    ) -> Self:
        """
        Connect to a database server and return a new `Connection` instance.
        """

        params = cls._get_connection_params(conninfo, **kwargs)
        timeout = timeout_from_conninfo(params)
        rv = None
        attempts = conninfo_attempts(params)
        conn_errors: list[tuple[e.Error, str]] = []
        for attempt in attempts:
            tdescr = (attempt.get("host"), attempt.get("port"), attempt.get("hostaddr"))
            descr = "host: %r, port: %r, hostaddr: %r" % tdescr
            logger.debug("connection attempt: %s", descr)
            try:
                conninfo = make_conninfo("", **attempt)
                gen = cls._connect_gen(conninfo, timeout=timeout)
                rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
            except e.Error as ex:
                logger.debug("connection failed: %s: %s", descr, str(ex))
                conn_errors.append((ex, descr))
            except e._NO_TRACEBACK as ex:
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
>               raise last_ex.with_traceback(None)
E               psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  database "finsight_test" does not exist

../../../.venv/lib/python3.12/site-packages/psycopg/connection.py:120: OperationalError

During handling of the above exception, another exception occurred:

    def test_permission_grant_revocation_takes_effect_on_next_request():
        suffix = uuid4().hex
        role_key = f"test-admin-{suffix}"
        email = f"permission-revocation-{suffix}@example.test"
        user_id = None
```

```
role_id = None
permission_id = None

try:
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO roles (role_key, display_name, scope, description)
            VALUES (%s, %s, 'platform', %s)
            RETURNING role_id
            """,
            (role_key, "Synthetic Permission Revocation Role", "Test-only role"),
        )
        role_id = cur.fetchone()[0]

        cur.execute(
            "SELECT permission_id FROM permissions WHERE permission_key = %s",
            ("platform.admin",),
        )
        row = cur.fetchone()
        assert row is not None, "platform.admin permission must exist"
        permission_id = row[0]

        cur.execute(
            "INSERT INTO role_permissions (role_id, permission_id) VALUES (%s, %s)",
            (role_id, permission_id),
        )
        cur.execute(
            """
            INSERT INTO users (email, display_name, platform_role_id)
            VALUES (%s, %s, %s)
            RETURNING user_id
            """,
            (email, "Synthetic Permission Revocation User", role_id),
        )
        user_id = cur.fetchone()[0]

        token, digest = new_session_token()
        cur.execute(
            """
            INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at)
            VALUES (%s, %s, %s)
            """,
            (user_id, digest, session_expiry(5)),
```

```
            )

        with TestClient(app) as client:
            first_response = client.get(
                "/api/v1/admin/authorization/capabilities",
                cookies={SESSION_COOKIE: token},
            )
            assert first_response.status_code == 200
            assert first_response.json()["data"]["effective_permissions"] == [
                "platform.admin"
            ]

            with connect() as conn, conn.cursor() as cur:
                cur.execute(
                    "DELETE FROM role_permissions WHERE role_id = %s AND permission_id =
%s",
                    (role_id, permission_id),
                )

            next_response = client.get(
                "/api/v1/admin/authorization/capabilities",
                cookies={SESSION_COOKIE: token},
            )
            assert next_response.status_code == 403
            assert next_response.json() == {"detail": "Permission denied"}

    finally:
>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

tests/test_identity_constraints.py:204:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _
app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _

cls = <class 'psycopg.Connection'>, conninfo = 'dbname=finsight_test host=127.0.0.1
port=55439 connect_timeout=3 hostaddr=127.0.0.1', autocommit = False, prepare_threshold =
5, context = None, row_factory = None
```

cursor_factory = None, kwargs = {'connect_timeout': 3}, params = {'connect_timeout': 3, 'dbname': 'finsight_test', 'host': '127.0.0.1', 'port': '55439'}, timeout = 3

```
    @classmethod
    def connect(
        cls,
        conninfo: str = "",
        *,
        autocommit: bool = False,
        prepare_threshold: int | None = 5,
        context: AdaptContext | None = None,
        row_factory: RowFactory[Row] | None = None,
        cursor_factory: type[Cursor[Row]] | None = None,
        **kwargs: ConnParam,
    ) -> Self:
        """
        Connect to a database server and return a new `Connection` instance.
        """

        params = cls._get_connection_params(conninfo, **kwargs)
        timeout = timeout_from_conninfo(params)
        rv = None
        attempts = conninfo_attempts(params)
        conn_errors: list[tuple[e.Error, str]] = []
        for attempt in attempts:
            tdescr = (attempt.get("host"), attempt.get("port"), attempt.get("hostaddr"))
            descr = "host: %r, port: %r, hostaddr: %r" % tdescr
            logger.debug("connection attempt: %s", descr)
            try:
                conninfo = make_conninfo("", **attempt)
                gen = cls._connect_gen(conninfo, timeout=timeout)
                rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
            except e.Error as ex:
                logger.debug("connection failed: %s: %s", descr, str(ex))
                conn_errors.append((ex, descr))
            except e._NO_TRACEBACK as ex:
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
```

```
>               raise last_ex.with_traceback(None)
E               psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  database "finsight_test" does not exist

../../../.venv/lib/python3.12/site-packages/psycopg/connection.py:120: OperationalError
_____________________________________________________________________________
_ test_platform_role_assignment_rejects_workspace_scoped_roles
_____________________________________________________________________________
__

    def test_platform_role_assignment_rejects_workspace_scoped_roles():
        email = f"wrong-scope-{uuid4().hex}@example.test"
>       with connect() as conn, conn.cursor() as cur:
             ^^^^^^^^^

tests/test_identity_constraints.py:212:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _
app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _

cls = <class 'psycopg.Connection'>, conninfo = 'dbname=finsight_test host=127.0.0.1
port=55439 connect_timeout=3 hostaddr=127.0.0.1', autocommit = False, prepare_threshold =
5, context = None, row_factory = None
cursor_factory = None, kwargs = {'connect_timeout': 3}, params = {'connect_timeout': 3,
'dbname': 'finsight_test', 'host': '127.0.0.1', 'port': '55439'}, timeout = 3

    @classmethod
    def connect(
        cls,
        conninfo: str = "",
        *,
        autocommit: bool = False,
        prepare_threshold: int | None = 5,
        context: AdaptContext | None = None,
        row_factory: RowFactory[Row] | None = None,
        cursor_factory: type[Cursor[Row]] | None = None,
        **kwargs: ConnParam,
    ) -> Self:
```

```
        """
        Connect to a database server and return a new `Connection` instance.
        """

        params = cls._get_connection_params(conninfo, **kwargs)
        timeout = timeout_from_conninfo(params)
        rv = None
        attempts = conninfo_attempts(params)
        conn_errors: list[tuple[e.Error, str]] = []
        for attempt in attempts:
            tdescr = (attempt.get("host"), attempt.get("port"), attempt.get("hostaddr"))
            descr = "host: %r, port: %r, hostaddr: %r" % tdescr
            logger.debug("connection attempt: %s", descr)
            try:
                conninfo = make_conninfo("", **attempt)
                gen = cls._connect_gen(conninfo, timeout=timeout)
                rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
            except e.Error as ex:
                logger.debug("connection failed: %s: %s", descr, str(ex))
                conn_errors.append((ex, descr))
            except e._NO_TRACEBACK as ex:
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
>               raise last_ex.with_traceback(None)
E               psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  database "finsight_test" does not exist

../../../.venv/lib/python3.12/site-packages/psycopg/connection.py:120: OperationalError
_____
__ test_platform_role_revocation_takes_effect_on_next_request
_____
___

    def test_platform_role_revocation_takes_effect_on_next_request():
        email = f"revoked-platform-admin-{uuid4().hex}@example.test"
>       with connect() as conn, conn.cursor() as cur:
             ^^^^^^^^^
```

tests/test_identity_constraints.py:237:

_ _ _ _

```
                gen = cls._connect_gen(conninfo, timeout=timeout)
                rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
            except e.Error as ex:
                logger.debug("connection failed: %s: %s", descr, str(ex))
                conn_errors.append((ex, descr))
            except e._NO_TRACEBACK as ex:
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
>               raise last_ex.with_traceback(None)
E               psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  database "finsight_test" does not exist

../../../.venv/lib/python3.12/site-packages/psycopg/connection.py:120: OperationalError
================================================================================
============================ warnings summary
================================================================================
============================
../../../.venv/lib/python3.12/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/fastapi/test
client.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated;
install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

../../../.venv/lib/python3.12/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
stclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use
anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================================================================
======================== short test summary info
================================================================================
========================
FAILED
tests/test_identity_constraints.py::test_identity_schema_rejects_duplicate_email_and_cascades
```

```
_sessions - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1",
port 55439 failed: FATAL:  database "finsight_test" does not exist
FAILED
tests/test_identity_constraints.py::test_platform_permission_is_resolved_from_database_role_a
nd_session - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1",
port 55439 failed: FATAL:  database "finsight_test" does not exist
FAILED
tests/test_identity_constraints.py::test_permission_grant_revocation_takes_effect_on_next_req
uest - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  database "finsight_test" does not exist
FAILED
tests/test_identity_constraints.py::test_platform_role_assignment_rejects_workspace_scoped_r
oles - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  database "finsight_test" does not exist
FAILED
tests/test_identity_constraints.py::test_platform_role_revocation_takes_effect_on_next_request
- psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439
failed: FATAL:  database "finsight_test" does not exist
5 failed, 2 warnings in 0.35s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % psql
"postgresql:///postgres?host=127.0.0.1&port=55439" \
  -X -c "SELECT datname FROM pg_database WHERE datistemplate = false ORDER BY
datname;"
    datname
----------------
 finsight
 finsight_fin75
 postgres
(3 rows)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % psql
"postgresql:///finsight?host=127.0.0.1&port=55439" \
  -X -c "SELECT current_database(), current_user;"
 current_database | current_user
------------------+--------------
 finsight         | sandeeph
(1 row)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
tname FROM pg_database WHERE datistemplate = false ORDER BY datname;"
    datname
----------------
```

```
 finsight
 finsight_fin75
 postgres
(3 rows)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % psql
"postgresql:///finsight?host=127.0.0.1&port=55439" \
  -X -c "SELECT current_database(), current_user;"
 current_database | current_user
------------------+--------------
 finsight         | sandeeph
(1 row)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % echo "===== CHECK DATABASE
CONNECTION ====="
psql "postgresql:///finsight_fin75?host=127.0.0.1&port=55439" \
  -X -v ON_ERROR_STOP=1 -c \
  "SELECT current_database(), current_user;"

echo
echo "===== CHECK REQUIRED TABLES ====="
psql "postgresql:///finsight_fin75?host=127.0.0.1&port=55439" \
  -X -c \
  "SELECT tablename FROM pg_tables WHERE schemaname = 'public' ORDER BY
tablename;"
===== CHECK DATABASE CONNECTION =====
 current_database | current_user
------------------+--------------
 finsight_fin75   | sandeeph
(1 row)


===== CHECK REQUIRED TABLES =====
   tablename
---------------
 auth_sessions
 roles
 users
(3 rows)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %

 roles
 users
```

```
(3 rows)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % psql
"postgresql:///finsight_fin75?host=127.0.0.1&port=55439" \
  -X -v ON_ERROR_STOP=1 -c "
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
  AND table_type = 'BASE TABLE'
ORDER BY table_name;
"

echo "===== AUTHORIZATION TABLE COLUMNS ====="

psql "postgresql:///finsight_fin75?host=127.0.0.1&port=55439" \
  -X -c "
SELECT table_name, column_name
FROM information_schema.columns
WHERE table_schema = 'public'
  AND table_name IN (
    'users',
    'roles',
    'auth_sessions',
    'permissions',
    'role_permissions'
  )
ORDER BY table_name, ordinal_position;
"
  table_name
---------------
 auth_sessions
 roles
 users
(3 rows)

===== AUTHORIZATION TABLE COLUMNS =====
  table_name   |    column_name
---------------+--------------------
 auth_sessions | session_id
 auth_sessions | user_id
 auth_sessions | refresh_token_hash
 auth_sessions | ip_address
 auth_sessions | user_agent
 auth_sessions | created_at
```

```
 auth_sessions | last_seen_at
 auth_sessions | expires_at
 auth_sessions | revoked_at
 auth_sessions | revoke_reason
 roles         | role_id
 roles         | role_key
 roles         | display_name
 roles         | scope
 roles         | description
 roles         | created_at
 users         | user_id
 users         | email
 users         | password_hash
 users         | display_name
 users         | account_status
 users         | platform_role_id
 users         | email_verified_at
 users         | mfa_required
 users         | last_login_at
 users         | created_at
 users         | updated_at
 users         | deleted_at
(28 rows)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
    'roles',
    'auth_sessions',
    'permissions',
    'role_permissions'
  )
ORDER BY table_name, ordinal_position;
"
  table_name
---------------
 auth_sessions
 roles
 users
(3 rows)

===== AUTHORIZATION TABLE COLUMNS =====
  table_name   |    column_name
---------------+--------------------
 auth_sessions | session_id
 auth_sessions | user_id
```

```
 auth_sessions | refresh_token_hash
 auth_sessions | ip_address
 auth_sessions | user_agent
 auth_sessions | created_at
 auth_sessions | last_seen_at
 auth_sessions | expires_at
 auth_sessions | revoked_at
 auth_sessions | revoke_reason
 roles         | role_id
 roles         | role_key
 roles         | display_name
 roles         | scope
 roles         | description
 roles         | created_at
 users         | user_id
 users         | email
 users         | password_hash
 users         | display_name
 users         | account_status
 users         | platform_role_id
 users         | email_verified_at
 users         | mfa_required
 users         | last_login_at
 users         | created_at
 users         | updated_at
 users         | deleted_at
(28 rows)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % echo "===== MIGRATION FILES
====="
ls -l migrations/

echo
echo "===== PLATFORM AUTHORIZATION MIGRATION ====="
sed -n '1,240p' migrations/0002_platform_authorization.sql

echo
echo "===== MIGRATION TRACKING TABLES ====="
psql "postgresql:///finsight_fin75?host=127.0.0.1&port=55439" \
  -X -c "
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
  AND table_name ILIKE '%migration%'
```

```
ORDER BY table_name;
"
===== MIGRATION FILES =====
total 0
-rw-r--r--@ 1 sandeeph  staff  1728 Oct 18 22:05 0001_auth_identity.sql
-rw-r--r--@ 1 sandeeph  staff  2580 Oct 19 11:20 0002_platform_authorization.sql

===== PLATFORM AUTHORIZATION MIGRATION =====
-- FIN-15: only the platform-scoped administrator permission is implemented.
-- Workspace roles and workspace permissions remain unavailable until workspace
membership exists.
BEGIN;

CREATE TABLE permissions (
   permission_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   permission_key VARCHAR(120) NOT NULL UNIQUE,
   description TEXT NOT NULL
);

CREATE TABLE role_permissions (
   role_id UUID NOT NULL REFERENCES roles(role_id) ON DELETE CASCADE,
   permission_id UUID NOT NULL REFERENCES permissions(permission_id) ON DELETE
CASCADE,
   PRIMARY KEY (role_id, permission_id)
);

INSERT INTO roles (role_key, display_name, scope, description)
VALUES ('platform_admin', 'Platform Administrator', 'platform',
       'Administrator privileges for currently implemented platform operations only.')
ON CONFLICT (role_key) DO NOTHING;

DO $$
BEGIN
   IF NOT EXISTS (SELECT 1 FROM roles WHERE role_key = 'platform_admin' AND scope =
'platform') THEN
       RAISE EXCEPTION 'Existing platform_admin role has non-platform scope';
   END IF;
END
$$;

INSERT INTO permissions (permission_key, description)
VALUES ('platform.admin', 'Access operations explicitly assigned platform administrator
permission.')
ON CONFLICT (permission_key) DO NOTHING;
```

```
INSERT INTO role_permissions (role_id, permission_id)
SELECT r.role_id, p.permission_id
FROM roles r CROSS JOIN permissions p
WHERE r.role_key = 'platform_admin' AND r.scope = 'platform'
  AND p.permission_key = 'platform.admin'
ON CONFLICT DO NOTHING;

CREATE FUNCTION enforce_platform_role_scope() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
   IF NEW.platform_role_id IS NOT NULL AND NOT EXISTS (
      SELECT 1 FROM roles WHERE role_id = NEW.platform_role_id AND scope = 'platform'
   ) THEN
      RAISE EXCEPTION 'platform_role_id must reference a platform-scoped role'
         USING ERRCODE = '23514';
   END IF;
   RETURN NEW;
END
$$;

CREATE TRIGGER users_platform_role_scope_check
BEFORE INSERT OR UPDATE OF platform_role_id ON users
FOR EACH ROW EXECUTE FUNCTION enforce_platform_role_scope();

CREATE FUNCTION prevent_platform_role_scope_drift() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
   IF OLD.scope = 'platform' AND NEW.scope <> 'platform' AND EXISTS (
      SELECT 1 FROM users WHERE platform_role_id = OLD.role_id
   ) THEN
      RAISE EXCEPTION 'Cannot change scope of an assigned platform role';
   END IF;
   RETURN NEW;
END
$$;

CREATE TRIGGER roles_prevent_scope_drift
BEFORE UPDATE OF scope ON roles
FOR EACH ROW EXECUTE FUNCTION prevent_platform_role_scope_drift();

COMMIT;

===== MIGRATION TRACKING TABLES =====
```

```
 table_name
------------
(0 rows)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %

------------
(0 rows)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % echo "===== CHECK FIRST
MIGRATION ====="
sed -n '1,220p' migrations/0001_auth_identity.sql

echo
echo "===== CHECK DATABASE CREATION PRIVILEGE ====="
psql "postgresql:///postgres?host=127.0.0.1&port=55439" \
  -X -c "SELECT rolcreatedb FROM pg_roles WHERE rolname = current_user;"
===== CHECK FIRST MIGRATION =====
-- FIN-13 identity/session subset. Derived from Database/schema (1).sql.
-- Apply only to a new development/test database; this is not the full target schema.
BEGIN;

CREATE TABLE roles (
    role_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    role_key VARCHAR(50) NOT NULL UNIQUE,
    display_name VARCHAR(100) NOT NULL,
    scope VARCHAR(20) NOT NULL CHECK (scope IN ('platform','workspace')),
    description TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(320) NOT NULL,
    password_hash TEXT,
    display_name VARCHAR(160) NOT NULL,
    account_status VARCHAR(20) NOT NULL DEFAULT 'active'
        CHECK (account_status IN ('invited','active','locked','disabled','deleted')),
    platform_role_id UUID REFERENCES roles(role_id) ON DELETE SET NULL,
    email_verified_at TIMESTAMPTZ,
    mfa_required BOOLEAN NOT NULL DEFAULT false,
    last_login_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at TIMESTAMPTZ
);
CREATE UNIQUE INDEX uq_users_email_lower ON users (lower(email));

CREATE TABLE auth_sessions (
    session_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    refresh_token_hash TEXT UNIQUE,
    ip_address INET,
    user_agent TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_seen_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at TIMESTAMPTZ NOT NULL,
    revoked_at TIMESTAMPTZ,
    revoke_reason VARCHAR(80)
);
CREATE INDEX ix_auth_sessions_active ON auth_sessions(user_id, expires_at) WHERE
revoked_at IS NULL;

COMMIT;

===== CHECK DATABASE CREATION PRIVILEGE =====
 rolcreatedb
-------------
 t
(1 row)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %

CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(320) NOT NULL,
    password_hash TEXT,
    display_name VARCHAR(160) NOT NULL,
    account_status VARCHAR(20) NOT NULL DEFAULT 'active'
        CHECK (account_status IN ('invited','active','locked','disabled','deleted')),
    platform_role_id UUID REFERENCES roles(role_id) ON DELETE SET NULL,
    email_verified_at TIMESTAMPTZ,
    mfa_required BOOLEAN NOT NULL DEFAULT false,
    last_login_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at TIMESTAMPTZ
```

```
);
CREATE UNIQUE INDEX uq_users_email_lower ON users (lower(email));

CREATE TABLE auth_sessions (
   session_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
   refresh_token_hash TEXT UNIQUE,
   ip_address INET,
   user_agent TEXT,
   created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
   last_seen_at TIMESTAMPTZ NOT NULL DEFAULT now(),
   expires_at TIMESTAMPTZ NOT NULL,
   revoked_at TIMESTAMPTZ,
   revoke_reason VARCHAR(80)
);
CREATE INDEX ix_auth_sessions_active ON auth_sessions(user_id, expires_at) WHERE
revoked_at IS NULL;

COMMIT;

===== CHECK DATABASE CREATION PRIVILEGE =====
 rolcreatedb
-------------
 t
(1 row)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % set -e

DB_URL="postgresql:///finsight_fin15_test?host=127.0.0.1&port=55439"

echo "===== CHECK DATABASE DOES NOT ALREADY EXIST ====="
EXISTS=$(psql "postgresql:///postgres?host=127.0.0.1&port=55439" \
  -X -tA -c "SELECT 1 FROM pg_database WHERE datname = 'finsight_fin15_test';")

if [ "$EXISTS" = "1" ]; then
  echo "STOP: finsight_fin15_test already exists. No changes made."
  exit 1
fi

echo "===== CREATE DEDICATED TEST DATABASE ====="
createdb -h 127.0.0.1 -p 55439 finsight_fin15_test

echo "===== APPLY IDENTITY MIGRATION ====="
```

```
psql "$DB_URL" -X -v ON_ERROR_STOP=1 \
  -f migrations/0001_auth_identity.sql

echo "===== APPLY PLATFORM AUTHORIZATION MIGRATION ====="
psql "$DB_URL" -X -v ON_ERROR_STOP=1 \
  -f migrations/0002_platform_authorization.sql

echo "===== VERIFY TABLES AND SEEDED PERMISSION ====="
psql "$DB_URL" -X -v ON_ERROR_STOP=1 -c "
SELECT tablename
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY tablename;
"

psql "$DB_URL" -X -v ON_ERROR_STOP=1 -c "
SELECT r.role_key, r.scope, p.permission_key
FROM roles r
JOIN role_permissions rp ON rp.role_id = r.role_id
JOIN permissions p ON p.permission_id = rp.permission_id
WHERE r.role_key = 'platform_admin';
"
===== CHECK DATABASE DOES NOT ALREADY EXIST =====
===== CREATE DEDICATED TEST DATABASE =====
===== APPLY IDENTITY MIGRATION =====
BEGIN
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE TABLE
CREATE INDEX
COMMIT
===== APPLY PLATFORM AUTHORIZATION MIGRATION =====
BEGIN
CREATE TABLE
CREATE TABLE
INSERT 0 1
DO
INSERT 0 1
INSERT 0 1
CREATE FUNCTION
CREATE TRIGGER
CREATE FUNCTION
CREATE TRIGGER
```

```
COMMIT
===== VERIFY TABLES AND SEEDED PERMISSION =====
   tablename
------------------
 auth_sessions
 permissions
 role_permissions
 roles
 users
(5 rows)

   role_key    |  scope   | permission_key
---------------+----------+----------------
 platform_admin | platform | platform.admin
(1 row)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
DATABASE_URL="postgresql:///finsight_fin15_test?host=127.0.0.1&port=55439" \
APP_ENV=test COOKIE_SECURE=false \
python -m pytest tests/test_identity_constraints.py -q
.....
[100%]
=============================================================================
===== warnings summary
=============================================================================
======
../../../.venv/lib/python3.12/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/fastapi/test
client.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated;
install `httpx2` instead.
   from starlette.testclient import TestClient as TestClient  # noqa

../../../.venv/lib/python3.12/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
stclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use
anyio.from_thread.BlockingPortal instead.
   _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_identity_constraints.py::test_platform_permission_is_resolved_from_databas
e_role_and_session
apps/api/tests/test_identity_constraints.py::test_platform_permission_is_resolved_from_databas
e_role_and_session
```

```
apps/api/tests/test_identity_constraints.py::test_platform_permission_is_resolved_from_databas
e_role_and_session
apps/api/tests/test_identity_constraints.py::test_platform_permission_is_resolved_from_databas
e_role_and_session
apps/api/tests/test_identity_constraints.py::test_platform_permission_is_resolved_from_databas
e_role_and_session
apps/api/tests/test_identity_constraints.py::test_permission_grant_revocation_takes_effect_on_
next_request
apps/api/tests/test_identity_constraints.py::test_permission_grant_revocation_takes_effect_on_
next_request
apps/api/tests/test_identity_constraints.py::test_platform_role_revocation_takes_effect_on_next
_request
apps/api/tests/test_identity_constraints.py::test_platform_role_revocation_takes_effect_on_next
_request

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
stclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being deprecated,
because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on
the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
5 passed, 11 warnings in 4.40s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
nstraints.py::test_platform_role_revocation_takes_effect_on_next_request
apps/api/tests/test_identity_constraints.py::test_platform_role_revocation_takes_effect_on_next
_request

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
stclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being deprecated,
because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on
the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
5 passed, 11 warnings in 4.40s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
DATABASE_URL="postgresql:///finsight_fin15_test?host=127.0.0.1&port=55439" \
APP_ENV=test COOKIE_SECURE=false \
python -m pytest -q
.....................................................
[100%]
=================================================================
===== warnings summary
```

```
============================================================================
======
../../../.venv/lib/python3.12/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/fastapi/test
client.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated;
install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

../../../.venv/lib/python3.12/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
stclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use
anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py: 4 warnings
apps/api/tests/test_auth_unit.py: 3 warnings
apps/api/tests/test_identity_constraints.py: 9 warnings

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
stclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being deprecated,
because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on
the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
58 passed, 18 warnings in 9.17s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
...                                                                      [100%]
============================================================================
===== warnings summary
============================================================================
======
../../../.venv/lib/python3.12/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/fastapi/test
client.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated;
install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

../../../.venv/lib/python3.12/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
```

stclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py: 4 warnings
apps/api/tests/test_auth_unit.py: 3 warnings
apps/api/tests/test_identity_constraints.py: 9 warnings

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on the client instance instead.
  return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
58 passed, 18 warnings in 9.17s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % cd ../..

echo "===== REPOSITORY INSTRUCTIONS ====="
sed -n '1,220p' AGENTS.md

echo
echo "===== API QUALITY CONFIGURATION ====="
ls -la apps/api
===== REPOSITORY INSTRUCTIONS =====
FinSight AI — Mandatory Repository Rules for Codex

Status: Project-wide engineering instructions
Applies to: The entire repository, unless a more-specific `AGENTS.md` exists in a subdirectory.
Primary objective: Implement FinSight AI incrementally and safely against the approved project artifacts, with verifiable tests and an honest record of progress.

Codex must read and follow this file at the beginning of every task and reread the relevant source artifacts before changing code in that domain. These instructions remain active across the project; do not treat them as a one-time checklist.

---

1. Role and operating principles

Act as a coordinated technical leadership and engineering team: system architect, technical lead, backend/frontend/data/AI engineer, QA engineer, and DevSecOps reviewer. Make

implementation decisions consistent with the supplied project artifacts and the existing repository.

1. **Inspect before changing.** Establish the current repository state, actual stack, current branch, existing conventions, available tests, and Jira issue/sprint mapping before editing files. Do not assume the repository is empty or that a documented feature already exists.
2. **Preserve existing work.** Prefer small, reviewable changes. Do not overwrite working code, discard user changes, regenerate the whole project, or replace the existing architecture just because another approach is familiar.
3. **Follow the active scope.** Work only on the issue/story or bounded task the user has authorized. The existence of 72 planned stories does not authorize implementing all 72 at once.
4. **Test the behavior, not just the build.** Add or update tests that map to acceptance criteria. Report actual command results; do not claim a test passed unless it was run and passed.
5. **Use the smallest sound design.** Keep the backend modular. Do not introduce microservices, infrastructure, dependencies, abstractions, or frameworks without a concreterequirement and a clear reason.
6. **Keep the user involved in material decisions.** Ask before making a decision that changes product scope, architecture, data model meaning, security posture, release boundaries, external spending, or irreversible repository/Jira state.
7. **Never fabricate progress.** Distinguish `NOT STARTED`, `IN PROGRESS`, `BLOCKED`, `PARTIAL`, `FAILED`, and `VERIFIED COMPLETE`. A story is not complete merely because code was written or a task was attempted.

2. Source-of-truth hierarchy

Read the actual files present in the repository. Filenames and directories may vary; the screenshot showed a project root containing `Component_diagrams/` and a directory spelled `Dcos/`, so search for the documents rather than assuming a corrected folder name. Do not copy, rename, or “clean up” these source artifacts without authorization.

Use the sources in this order, according to the question being answered:

1. **Software Requirements Specification (SRS):** authoritative functional and non-functional requirements, security expectations, page responsibilities, and observable user behavior.
2. **Core SQL schema and Database Schema/ERD Reference:** authoritative target relational entities, column types, PK/FK relationships, constraints, provenance, and domain model. Treat the SQL files supplied as reference inputs, not disposable migration scripts.
3. **Technical Design and Implementation Plan:** architectural boundaries, contracts, configuration patterns, testing/release gates, operational design, and implementation approach.
4. **The 72-story Jira backlog with acceptance criteria:** story goals, acceptance criteria, priority, dependencies, proposed role-based ownership, implementation subtasks,and original story aliases.

5. **Full Project Four-Sprint Plan:** four-sprint allocation and intended sequencing for the 72 catalog entries. It is a plan, not proof that work has been completed and not permission to bypass release guards.
6. **Sprint 1 Test Plan PDF and Sprint 1 Test Plan Management TXT:** Sprint 1 test inventory, story-to-test traceability, setup and execution rules. The original files are read-only reference documents; record actual execution results in a new status/execution file, not by editing the source register.
7. **Component diagrams and their companion/explanation document:** clarify components and relationships. They do not overrule explicit SRS requirements or schema definitions.
8. **Live repository, tests, configuration, and Jira state:** evidence of what exists now and what has actually been executed. The repository is authoritative about implementation state; Jira is authoritative about current issue keys, assignments, status, and sprint membership. Neither silently changes approved product requirements.

If two sources genuinely conflict, do not silently choose, rewrite a source file, or invent a resolution. Record the conflict and its impact in `conversation.txt`, cite therelevant filenames/sections/pages, recommend the safest interpretation, and ask the user when the decision affects implementation or scope. You may continue independent, reversible inspection work while waiting.

3. Product architecture baseline

Unless the source documents or current repository establish a documented, user-approved variation, follow this baseline:

- **Frontend:** Next.js / React.
- **Backend:** modular FastAPI application with clear domain boundaries and typed API contracts.
- **Structured database:** PostgreSQL; structured relational records are the source of truth.
- **Vector retrieval:** pgvector in the PostgreSQL deployment for the initial RAG design; document/chunk IDs and permissions remain relationally authoritative.
- **LLM inference:** Ollama behind a provider/adapter interface, so model access can be configured, mocked in tests, or replaced without changing business logic.
- **Embedding and market-data providers:** typed adapters with validation, timeouts, useful error mapping, and deterministic test doubles.
- **Long-running operations:** persisted job/run states, stable IDs, bounded retries, idempotency, and safe recovery. Redis is optional and must not become the authoritative record unless an approved design change says so.
- **Financial calculations:** deterministic, tested backend functions/services own numeric outputs. The LLM may explain those outputs and assumptions; it must not be the authoritative calculator.
- **Authorization:** validate authentication and resource/operation permissions on the backend. Filter document permissions before any content is added to an LLM context.

First inspect the actual repository. Do not create a second frontend/backend, replace a working framework, or invent directory structures until the existing structure and gaps have been mapped.

4. Scope, planning, and Jira rules

4.1 Four-sprint baseline

The active project plan is the **four-sprint** plan for 72 story/task entries. Do not resurrect the superseded seven-day plan or use it to reorder the project. Preserve the`FS-*` aliases as traceability labels and map them to the real Jira issue keys found in the live project.

The source allocation describes Sprint 1 as foundation, identity, secure configuration, and delivery controls (23 entries, 63 source story points), with a total of 72 entries and 251 source story points across four sprints. These are source planning estimates, not evidence of actual team velocity or a promise that every acceptance criterion can be delivered on the stated calendar dates.

4.2 Verify Jira before relying on it

Before changing Jira, inspect current issue keys, stories, epics, status, assignees, dependencies, and sprint membership. A provided screenshot showed a `FIN Sprint 1` section with **0 work items**, while the surrounding backlog showed many issues in `To Do`. Treat that as a discrepancy to verify, not as proof that the sprint is populated.

If Sprint 1's actual Jira membership conflicts with the four-sprint plan:

- Record the discrepancy in `conversation.txt` with the relevant issue keys and counts.
- Tell the user precisely what needs confirming.
- Do not bulk-move stories, change estimates, alter priorities, assign people, start a sprint, or create/close issues without explicit authorization.
- Continue repository inspection and safe, independent Sprint 1 preparation when possible.

Do not claim that Jira has been updated unless the change was actually made and verified. If no Jira integration/tool is available, prepare instructions or an import artifact instead of claiming direct modification.

4.3 Scope gates

- Implement only authorized work, starting with the current Sprint 1 scope unless the user explicitly directs otherwise.
- Preserve original priorities, estimates, release labels, acceptance criteria, and dependencies unless the user approves a change.

- Do not mark a grouped story complete while any required acceptance criterion remains unimplemented or unverified. Use subtasks and status notes to show partial progress.
- A story can be marked complete only when its acceptance criteria, appropriate automated tests, code review, relevant security/data-isolation checks, and required traceability/evidence satisfy the Definition of Done.

5. Release and database safety rules

5.1 Release 1 is read-only financial research

The Release 1 guardrail is mandatory:

- No live broker order submission, modification, or cancellation endpoints.
- No enabled broker execution flow or live-trading credentials.
- Do not apply `schema_optional_trading.sql` or otherwise create its optional 15-table trading extension in Release 1.
- The trading-related stories identified in the backlog as Future / Release 2 remain gated even if they appear in a sprint allocation.
- A read-only connection or analytical feature must not silently expand into trading permission.

Any future change to this boundary requires explicit user approval and the separate security/operational review specified in the project documents.

5.2 PostgreSQL schema handling

- The supplied core `schema.sql` describes a 158-table target-state schema. It is **not** an instruction to implement all tables or all features immediately.
- Inspect the current database, migrations, and code before proposing changes. Do not run the supplied full schema directly against an existing or non-disposable database without explicit approval and a verified backup/recovery plan.
- Create new, version-controlled migrations in the repository's established migration system. Never edit the source SQL/schema/reference PDFs to show implementation progress.
- Apply migrations only to a confirmed development/test database. Verify a clean install and, where supported, a forward migration from the previous schema version.
- Preserve declared types, PK/FK constraints, uniqueness, checks, ownership, source lineage, timestamps, and financial precision. Use `NUMERIC` rather than binary floating point for stored money/financial quantities in accordance with the schema and design.
- Treat `seed_reference_data.sql` as optional development reference seed data, not production market data. Check that target tables and conflict behavior are compatible before use.
- Use PostgreSQL 16+ and pgvector as the design baseline unless the existing supported environment documents otherwise. Confirm required extensions are available before depending on them.
- Never drop tables, truncate user data, reset a database, rewrite migration history, or run destructive SQL without the user's explicit approval and a clear recovery plan.

6. Immutable project artifacts

The following supplied items are reference inputs and must remain unchanged unless the user specifically authorizes a revision:

- SRS PDF and source requirements documents.
- Technical Design and Implementation Plan PDF.
- Database Schema/ERD Reference PDF and source SQL files, including `schema.sql`, `seed_reference_data.sql`, and `schema_optional_trading.sql`.
- Original Jira backlog PDFs, 72-story backlog/acceptance-criteria document, and four-sprint plan.
- Sprint 1 Test Plan PDF and its existing management TXT.
- Component architecture PNG/SVG/Mermaid files and companion/explanation PDF.

Do not edit their contents, overwrite them with generated text, rename them, regenerate them, or alter their dates/claims. Use code, migrations, tests, configuration, ADRs,and new coordination/status files for implementation outputs. If an approved design change is needed, write a separate addendum/ADR rather than silently rewriting a source baseline.

Before and after a task that touches project files, inspect `git status` and the diff. Do not discard uncommitted user changes. Do not run `git reset --hard`, `git clean -fd`, forced pushes, or equivalent destructive Git commands. Do not commit, push, merge, create a pull request, or alter remote branches unless the user explicitly asks for that action.

7. Required coordination files

Use small root-level coordination files so implementation progress and questions are visible without altering the source documents. If a file already exists, read it first and preserve existing entries.

7.1 `conversation.txt` — user/Codex question and decision log

Create this file in the repository root if it does not exist. Treat it as a shared conversation/decision log, not as source code or a place to store secrets. It should contain:

- Open questions, each with an ID, why the answer matters, choices if useful, a recommended choice, and the exact next action needed from the user.
- Requests for missing provider/service configuration, with the **environment variable names only**, purpose, whether required now or later, and step-by-step safe setup instructions.
- User-provided non-secret answers/decisions, decision date, and the task/story affected.
- Blocking issues, source conflicts, and a `WAITING FOR USER` or `ANSWERED` marker.

When a question requires user action, explain exact steps: which application/provider to open, where to find/create the setting or credential, where to configure it locally, and what non-secret confirmation to add to `conversation.txt`. Then pause any work dependent on the answer. Continue only work that is independent and safe.

Secrets policy: never ask the user to paste API keys, passwords, tokens, private keys, database passwords, or session secrets into `conversation.txt`, source files, terminal output, screenshots, Jira descriptions, or Git. Never print, echo, or commit actual secret values. Request that the user add secret values directly to a local ignored `.env` file or approved secret manager and return only a confirmation such as `Configured locally; value not shared`. Ensure `.env`, credentials, tokens, local database dumps, and other sensitive artifacts are ignored by Git. `.env.example` may contain variable names and safe placeholders only.

If Codex discovers a secret already tracked or exposed, stop further exposure, avoid repeating its value, notify the user, recommend revocation/rotation, and follow the repository's established secret-removal procedure only with authorization.

7.2 `IMPLEMENTATION_STATUS.txt` — completion markers

Create this file if missing and update it after meaningful work. Include the task/story alias and real Jira key when known, status, concise implementation summary, changed file paths, commands actually run, tests and observed outcomes, unresolved acceptance criteria, blockers, and evidence paths/links if available.

Use these states only:

- `NOT STARTED`
- `IN PROGRESS`
- `BLOCKED`
- `PARTIAL`
- `FAILED`
- `VERIFIED COMPLETE`

Only use `VERIFIED COMPLETE` when the applicable Definition of Done is met and evidence is recorded. Otherwise use `PARTIAL`, `FAILED`, or `BLOCKED` as appropriate. Never infer success from an exit code that was not observed, from a planned test, or from code inspection alone when runtime verification is required.

7.3 Test-execution record

Do not edit the supplied test-plan files. If actual Sprint 1 test results need recording, create a separate root file such as `SPRINT_1_TEST_EXECUTION_STATUS.txt` with eachtest ID, initial status `NOT RUN`, execution date, actual result, tester/owner, evidence, and linked defect if any. Update statuses only after executing the case. Use `PASS`, `FAIL`, `BLOCKED`, and `NOT

RUN`; `N/A` requires documented justification. Do not represent a test as passed if its dependency/component does not exist.

Do not create unnecessary coordination files beyond these needs. Keep records concise and factual.

8. Clarification and stop conditions

Ask a clear, focused user question before proceeding when any of the following is true:

- A conflict in source documents changes behavior, schema meaning, scope, or release eligibility.
- A decision is irreversible/destructive or could lose user work/data.
- A required external credential, licensed data feed, paid service, or permission is unavailable.
- The Jira issue/sprint mapping or scope is ambiguous and changing it would affect commitments.
- An acceptance criterion cannot be met without choosing between materially different product behaviors.
- An architecture/security decision is not established by the source documents or current repository.

Do not ask questions whose answers can be determined safely from the repository or supplied documents. Before asking, investigate the available evidence and report what youfound. When blocked, write the question and next action to `conversation.txt`, summarize the blocker to the user, and stop only the dependent work.

9. Credentials, API keys, and external providers

- Do not assume that any specific API key is required until a selected feature/provider actually needs it.
- Ollama may be available locally without a hosted LLM API key; inspect the approved setup and test connectivity before concluding it is configured.
- Market-data providers, embedding services, hosted models, email delivery, or other integrations may require credentials depending on the selected adapter and environment.Do not choose a paid provider or trigger spending without approval.
- Use provider interfaces and deterministic fixtures/mocks so tests can run without live API access where appropriate.
- If a key is required, add a non-secret request to `conversation.txt` containing the variable name, service, reason, whether it blocks the active story, and exact setup steps. Wait for confirmation that it has been configured safely; do not request the secret itself.
- Distinguish `missing configuration`, `provider unavailable`, `invalid credentials`, `rate limited`, and `no data` in internal error handling. Do not leak provider responses or secrets to users/logs.

10. Coding standards and engineering workflow

Apply the repository's existing formatter, linter, type checker, tests, dependency manager, and naming conventions. Do not introduce a competing toolchain without approval.

For each authorized story or task:

1. **Read:** reread relevant SRS requirements, schema entities, technical design sections, backlog acceptance criteria, dependencies, and relevant test-plan cases.
2. **Inspect:** review the current implementation, `git status`, recent relevant commits where available, and existing test patterns.
3. **Plan:** identify the smallest vertical slice, dependencies, impacted files, migration impact, security concerns, test cases, and any user decision needed. Record concise intent in the status file.
4. **Test first where practical:** add or adjust a failing test for a clearly defined behavior before implementing it (TDD). Do not claim TDD was followed if it was not.
5. **Implement:** make small, typed, maintainable changes; validate all inputs at boundaries; use explicit timeouts and bounded retries for external calls; preserve ownership and provenance; keep side effects controlled.
6. **Verify:** run focused tests first, then relevant integration/regression tests, lint/type/static/security checks and builds available in the repo. Use disposable/test resources and synthetic data.
7. **Review:** inspect the complete diff for accidental changes, secrets, unsafe defaults, unrelated formatting churn, altered source documents, schema drift, missing tests, and unmet acceptance criteria.
8. **Record:** update only the new coordination/status files with actual results. Include exact commands and concise output summaries; redact environment values and secrets.
9. **Report:** state what changed, what tests ran, what passed/failed/was blocked, remaining criteria, and the next recommended step. Do not claim deployment or Jira updates unless they happened and were verified.

Do not weaken or delete a test simply to make the build pass. If a test exposes a product/design ambiguity, stop and ask rather than encoding a convenient assumption.

General code requirements

- Prefer explicit types, cohesive modules, small functions, and clear dependency boundaries.
- Validate and normalize user/provider input at the boundary.
- Use parameterized SQL/ORM operations. Never interpolate untrusted input into SQL or permit arbitrary SQL execution through Data Explorer features.
- Use transactions for coupled database updates and enforce constraints in the database where appropriate.
- Make retried ingestion and job operations idempotent. Use stable run IDs and safe state transitions.

- Keep logs structured and useful but redact secrets and sensitive user data; never return stack traces or internal secrets to end users.
- Set realistic timeouts and bounded retries for database, network, model, and provider operations. Fail gracefully when optional dependencies are down.
- Add accessible labels, keyboard operation, visible focus, useful loading/error/empty states, and responsive behavior for UI work.
- Avoid nondeterministic unit tests and internet-dependent CI tests. Use fixtures or mocks for external integrations; separately verify real integration connections when needed.
- Keep financial calculations reproducible: document formulas/units, input snapshots, assumptions, data timestamps, precision, and formula/model versions as applicable.

11. Security, privacy, RAG, and financial correctness gates

The following are release-blocking if violated:

```
===== API QUALITY CONFIGURATION =====
total 16
drwxr-xr-x@ 9 sandeeph  staff  288 Oct 21 09:33 .
drwxr-xr-x@ 5 sandeeph  staff  160 Oct 21 13:01 ..
drwxr-xr-x@ 8 sandeeph  staff  256 Oct 21 09:33 app
drwxr-xr-x@ 4 sandeeph  staff  128 Oct 20 10:57 migrations
-rw-r--r--@ 1 sandeeph  staff  574 Oct 19 00:03 requirements-dev.lock
-rw-r--r--@ 1 sandeeph  staff   66 Oct 19 00:02 requirements-dev.txt
-rw-r--r--@ 1 sandeeph  staff  411 Oct 19 00:03 requirements.lock
-rw-r--r--@ 1 sandeeph  staff  138 Oct 19 00:02 requirements.txt
drwxr-xr-x@ 7 sandeeph  staff  224 Oct 20 16:20 tests
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
6:20 tests
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== PACKAGE
SCRIPTS ====="
node -e "for (const f of ['package.json','apps/web/package.json']) { try { const p=require('./'+f);
console.log('\n'+f); console.log(JSON.stringify(p.scripts || {}, null, 2)); } catch(e) { console.log(f+':
not found'); } }"

echo
echo "===== PYTHON QUALITY TOOL REFERENCES ====="
grep -RInE 'ruff|flake8|mypy|pyright|compileall|pip check|pytest' \
  README.md AGENTS.md apps/api/requirements-dev.txt \
  apps/api/requirements-dev.lock .github/workflows \
  2>/dev/null | head -n 120

echo
```

```
echo "===== GIT WHITESPACE CHECK ====="
git diff --check
===== PACKAGE SCRIPTS =====
package.json: not found

apps/web/package.json
{
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "test:ui": "playwright test --config playwright.ui.config.js",
  "test:e2e": "playwright test --config playwright.e2e.config.js"
}

===== PYTHON QUALITY TOOL REFERENCES =====
README.md:69:  - Backend line and branch coverage (unit tests need no database; integration tests require the identity migration): from the repository root, set `DATABASE_URL` to a disposable migrated PostgreSQL database and run `.venv/bin/python -m pytest -q --cov=app --cov-branch --cov-report=term-missing --cov-report=xml:coverage.xml --cov-report=html:htmlcov apps/api/tests`. This writes a machine-readable `coverage.xml` and browsable `htmlcov/index.html`; reports are ignored local build output.
apps/api/requirements-dev.txt:2:pytest==9.0.3
apps/api/requirements-dev.txt:4:pytest-cov==6.2.1
apps/api/requirements-dev.lock:26:pytest==9.0.3
apps/api/requirements-dev.lock:27:pytest-cov==6.2.1
.github/workflows/ci.yml:126:        run: python -m pip check
.github/workflows/ci.yml:132:        run: python -m compileall -q apps/api/app apps/api/tests
.github/workflows/ci.yml:135:          python -m pytest -q --cov=app --cov-branch --cov-report=term-missing
.github/workflows/ci.yml:140:          python -m pytest -q --cov=app --cov-branch --cov-append --cov-report=term-missing

===== GIT WHITESPACE CHECK =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

compileall exit code: 0
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
DATABASE_URL="postgresql:///finsight_fin15_test?host=127.0.0.1&port=55439" \
APP_ENV=test COOKIE_SECURE=false \
.venv/bin/python -m pytest -q \
  --cov=app \
  --cov-branch \
  --cov-report=term-missing \
  --cov-report=xml:coverage.xml \
```

```
  --cov-report=html:htmlcov \
  apps/api/tests
.....................................................
[100%]
=============================================================================
===== warnings summary
=============================================================================
======
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py: 4 warnings
apps/api/tests/test_auth_unit.py: 3 warnings
apps/api/tests/test_identity_constraints.py: 9 warnings

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=============================================================================
====== tests coverage
=============================================================================
=======
_____________________________________________________________________ coverage: platform
darwin, python 3.14.6-final-0
_____________________________________________________________________

Name                          Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------
apps/api/app/__init__.py          0      0      0      0   100%
```

```
apps/api/app/auth.py                25      0      4      0   100%
apps/api/app/database.py            11      0      2      0   100%
apps/api/app/main.py                111      0     18      0   100%
apps/api/app/provision_user.py      27      1      8      1    94%   45
-------------------------------------------------------------------------
TOTAL                               174      1     32      1    99%
Coverage HTML written to dir htmlcov
Coverage XML written to file coverage.xml
58 passed, 18 warnings in 438.92s (0:07:18)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
------------------
TOTAL                               174      1     32      1    99%
Coverage HTML written to dir htmlcov
Coverage XML written to file coverage.xml
58 passed, 18 warnings in 438.92s (0:07:18)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== FIN-15
TRACKING REFERENCES ====="
grep -n -i -E
'FIN-15|FS-01-02|TC_AUTHZ_001|TC_AUTHZ_002|TC_AUTHZ_003|TC_PRIV_002|TC_SYS_
001' \
  IMPLEMENTATION_STATUS.txt \
  SPRINT_1_TEST_EXECUTION_STATUS.txt \
  FUNCTION_TEST_TRACEABILITY.txt

echo
echo "===== CURRENT CHANGE SUMMARY ====="
git status --short

echo
echo "===== WHITESPACE VALIDATION ====="
git diff --check
===== FIN-15 TRACKING REFERENCES =====
IMPLEMENTATION_STATUS.txt:59:Reviewed requirements: SRS FR-P1-01..04, FR-P1-06,
FR-P1-08 and NFR-P1-01..03/05; canonical identity DDL `Database/schema (1).sql` (`roles`,
`users`, `auth_sessions`); backlog FS-01-01 page 2; test-plan management cases
TC_AUTHN_001..005 and TC_AUTHZ_003.
IMPLEMENTATION_STATUS.txt:92:Test execution: detailed statuses and evidence are
recorded in `SPRINT_1_TEST_EXECUTION_STATUS.txt`. TC_AUTHN_001, TC_AUTHN_002,
TC_AUTHN_003, and the shared session-expiry behavior in TC_AUTHZ_003 passed against
disposable PostgreSQL. TC_AUTHN_005 is BLOCKED because the full administrator invitation
lifecycle is not implemented. TC_AUTHN_006 passed the disposable database dump/log/API
privacy checks recorded below; TC_AUTHN_007 is BLOCKED pending deployed HTTPS
verification. TC_AUTHN_004 is BLOCKED pending the SRS-required password recovery
scope/delivery decision.
```

IMPLEMENTATION_STATUS.txt:214:- `TC_INT_001` remains BLOCKED for configuration-version/audit persistence and allowed/denied configuration flows (not implemented). `TC_SYS_001` remains BLOCKED for dashboard/settings, health and Ollama flows (not implemented). Password recovery, admin invitations and deployed HTTPS remain the previously recordedFIN-13 gaps and are untouched.
IMPLEMENTATION_STATUS.txt:319:FIN-15 / FS-01-02 — Enforce roles and protected routes (2026-10-19)
IMPLEMENTATION_STATUS.txt:321:Inspected relevant SRS access-control requirements, canonical schema and reference seeds, FS-01-02 backlog, four-sprint/test-plan mapping, architecture docs, identity/session API, migration 0001, trusted provisioning CLI, and relevant tests. Existing routes: public POST `/auth/login`, POST `/auth/logout`, protected GET `/auth/me`; no product/admin operation or protected frontend page currently exists. Existing session dependency already rejects absent, revoked, and expired sessions; prior `TC_AUTHZ_003` expiry result remains PASS as previously recorded.
IMPLEMENTATION_STATUS.txt:324:Remaining acceptance criteria: role/permission matrix, 403 behavior, trusted role resolution/provisioning assignment, protected route application, and frontend guard cannot be completed safely until scope decision and a concrete capability/resource mapping are established. TC_AUTHZ_001 also requires a protected frontend page (none exists); TC_AUTHZ_002 currently names a model/provider admin operation that does not exist; TC_PRIV_002 requires workspace membership/resources not implemented. FIN-15 is not ready for acceptance.
IMPLEMENTATION_STATUS.txt:326:FIN-15 verification update — 2026-10-20
IMPLEMENTATION_STATUS.txt:336:- FIN-15 is not yet marked complete pending route-level authorization verification and remaining acceptance checks.
IMPLEMENTATION_STATUS.txt:338:FIN-15 implementation update — 2026-10-20
IMPLEMENTATION_STATUS.txt:342:- Function/test traceability added in FUNCTION_TEST_TRACEABILITY.txt for TC_AUTHZ_001, TC_AUTHZ_002, and related existing session coverage TC_AUTHZ_003.
IMPLEMENTATION_STATUS.txt:343:- Verification status: implementation edits prepared; tests have NOT RUN after this change. Do not mark FIN-15 VERIFIED COMPLETE until test results and all acceptance criteria are checked.
IMPLEMENTATION_STATUS.txt:346:FIN-15 — VERIFIED TEST RUN ADDENDUM (2026-10-20)
IMPLEMENTATION_STATUS.txt:359:- test-results/FIN-15/20261020_141030_identity_authorization.log
IMPLEMENTATION_STATUS.txt:360:- test-results/FIN-15/20261020_141030_full_api_suite.log
IMPLEMENTATION_STATUS.txt:361:- test-results/FIN-15/run_20261020_141030_summary.txt
IMPLEMENTATION_STATUS.txt:364:the behavior exercised by those tests, not automatic completion of every FIN-15
IMPLEMENTATION_STATUS.txt:370:FIN-15 — CURRENT VERIFICATION ADDENDUM (2026-10-20)
IMPLEMENTATION_STATUS.txt:381:- TC_AUTHZ_001 remains NOT RUN as a complete test-plan case because its

IMPLEMENTATION_STATUS.txt:383:- TC_AUTHZ_002 remains NOT RUN as a complete test-plan case because its
IMPLEMENTATION_STATUS.txt:386:- TC_AUTHZ_003 retains its existing recorded status; the current 58-test
IMPLEMENTATION_STATUS.txt:388:- TC_PRIV_002 remains NOT RUN / deferred: workspace and cross-user
IMPLEMENTATION_STATUS.txt:390:- TC_SYS_001 remains NOT RUN until the required end-to-end smoke path is run.
IMPLEMENTATION_STATUS.txt:392:Overall FIN-15 status: PARTIAL. Passing API tests do not constitute
IMPLEMENTATION_STATUS.txt:397:FIN-15 TRACKING RECONCILIATION — 2026-10-20 17:38 IST
IMPLEMENTATION_STATUS.txt:404:- Evidence: test-results/FIN-15/20261020_141030_identity_authorization.log,
IMPLEMENTATION_STATUS.txt:405: test-results/FIN-15/20261020_141030_full_api_suite.log, and
IMPLEMENTATION_STATUS.txt:406: test-results/FIN-15/run_20261020_141030_summary.txt.
IMPLEMENTATION_STATUS.txt:410:FIN-15 status: PARTIAL — do not mark story accepted/complete from these test
IMPLEMENTATION_STATUS.txt:415:- TC_AUTHZ_001: PARTIAL. API anonymous/invalid-session checks are evidenced,
IMPLEMENTATION_STATUS.txt:418:- TC_AUTHZ_002: PARTIAL for the narrow platform permission behavior asserted.
IMPLEMENTATION_STATUS.txt:421:- TC_AUTHZ_003: retain the previously recorded PASS only as historical evidence
IMPLEMENTATION_STATUS.txt:425:- TC_PRIV_002: DEFERRED / NOT RUN under the approved platform-only scope.
IMPLEMENTATION_STATUS.txt:428:- TC_SYS_001: NOT RUN / NOT ESTABLISHED by the API suite alone.
IMPLEMENTATION_STATUS.txt:431:- Before marking FIN-15 complete, verify the actual route path in
IMPLEMENTATION_STATUS.txt:435:FIN-15 ROUTE RECONCILIATION — 2026-10-20
SPRINT_1_TEST_EXECUTION_STATUS.txt:51:TC_AUTHZ_003 — Expire inactive sessions according to configured policy
SPRINT_1_TEST_EXECUTION_STATUS.txt:78:TC_SYS_001 | Status: BLOCKED | Execution date: 2026-10-18 | Actual result: Browser smoke verified current sign-in/session/logout path against live frontend, FastAPI, and PostgreSQL. Full planned dashboard/settings, dependency health, Ollama available/unavailable, and correlation-log workflow cannot run because those components are not implemented | Tester/owner: Codex | Evidence: apps/web/tests/e2e/auth.spec.js; scripts/run_live_e2e.py; IMPLEMENTATION_STATUS.txt | Linked defect: Current application implements auth starter only
SPRINT_1_TEST_EXECUTION_STATUS.txt:94:TC_AUTHZ_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —

SPRINT_1_TEST_EXECUTION_STATUS.txt:95:TC_AUTHZ_002 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
SPRINT_1_TEST_EXECUTION_STATUS.txt:96:TC_AUTHZ_003 | Status: PASS | Execution date: 2026-10-18 | Actual result: Stale session rejected under configured TTL | Tester/owner: Codex | Evidence: apps/api/tests/test_auth.py | Linked defect: —
SPRINT_1_TEST_EXECUTION_STATUS.txt:115:TC_PRIV_002 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
SPRINT_1_TEST_EXECUTION_STATUS.txt:122:TC_SYS_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
SPRINT_1_TEST_EXECUTION_STATUS.txt:135:FIN-15 route-level authorization test update — 2026-10-20
SPRINT_1_TEST_EXECUTION_STATUS.txt:136:- Test mapping: TC_AUTHZ_001 / TC_AUTHZ_002 / TC_AUTHZ_003 (existing session expiry regression coverage).
SPRINT_1_TEST_EXECUTION_STATUS.txt:140:FIN-15 — VERIFIED EXECUTION ADDENDUM (2026-10-20)
SPRINT_1_TEST_EXECUTION_STATUS.txt:150: test-results/FIN-15/20261020_141030_identity_authorization.log
SPRINT_1_TEST_EXECUTION_STATUS.txt:151: test-results/FIN-15/20261020_141030_full_api_suite.log
SPRINT_1_TEST_EXECUTION_STATUS.txt:152: test-results/FIN-15/run_20261020_141030_summary.txt
SPRINT_1_TEST_EXECUTION_STATUS.txt:156:FIN-15 story is complete. Workspace-level authorization remains deferred.
SPRINT_1_TEST_EXECUTION_STATUS.txt:159:FIN-15 TEST-STATUS RECONCILIATION — 2026-10-20 17:38 IST
SPRINT_1_TEST_EXECUTION_STATUS.txt:164: test-results/FIN-15/20261020_141030_identity_authorization.log
SPRINT_1_TEST_EXECUTION_STATUS.txt:165: test-results/FIN-15/20261020_141030_full_api_suite.log
SPRINT_1_TEST_EXECUTION_STATUS.txt:166: test-results/FIN-15/run_20261020_141030_summary.txt
SPRINT_1_TEST_EXECUTION_STATUS.txt:169:- TC_AUTHZ_001: PARTIAL — API unauthenticated/invalid-session behavior was
SPRINT_1_TEST_EXECUTION_STATUS.txt:171:- TC_AUTHZ_002: PARTIAL — the narrow platform-admin/standard-user permission
SPRINT_1_TEST_EXECUTION_STATUS.txt:174:- TC_AUTHZ_003: historical PASS only — the existing expiry regression was
SPRINT_1_TEST_EXECUTION_STATUS.txt:176:- TC_PRIV_002: DEFERRED / NOT RUN — workspace/resource authorization is outside
SPRINT_1_TEST_EXECUTION_STATUS.txt:178:- TC_SYS_001: NOT RUN — a passing API suite is not a browser/system smoke test.
SPRINT_1_TEST_EXECUTION_STATUS.txt:179:Overall FIN-15 remains PARTIAL, not accepted. Do not alter FIN-14 tracking.

SPRINT_1_TEST_EXECUTION_STATUS.txt:181:FIN-15 ROUTE RECONCILIATION — 2026-10-20
FUNCTION_TEST_TRACEABILITY.txt:3:FIN-15 / FS-01-02 — platform authorization capability endpoint
FUNCTION_TEST_TRACEABILITY.txt:7:Test mapping: TC_AUTHZ_001 (unauthenticated/invalid session denial), TC_AUTHZ_002 (permission matrix: platform admin allowed, standard userdenied; client-supplied X-Role ignored), TC_AUTHZ_003 (session expiry/revocation behavior via existing session dependency).
FUNCTION_TEST_TRACEABILITY.txt:10:FIN-15 — EXECUTION EVIDENCE ADDENDUM (2026-10-20)
FUNCTION_TEST_TRACEABILITY.txt:19: test-results/FIN-15/20261020_141030_identity_authorization.log
FUNCTION_TEST_TRACEABILITY.txt:20: test-results/FIN-15/20261020_141030_full_api_suite.log
FUNCTION_TEST_TRACEABILITY.txt:21: test-results/FIN-15/run_20261020_141030_summary.txt
FUNCTION_TEST_TRACEABILITY.txt:24:- TC_AUTHZ_001: verify its exact assertions before marking fully PASS; the integration
FUNCTION_TEST_TRACEABILITY.txt:26:- TC_AUTHZ_002: PASS for the platform permission behavior explicitly asserted by the
FUNCTION_TEST_TRACEABILITY.txt:28:- TC_AUTHZ_003: existing expiry/revocation regression passed in an earlier recorded run;
FUNCTION_TEST_TRACEABILITY.txt:30:- TC_PRIV_002: BLOCKED / deferred because workspace membership and resource-level
FUNCTION_TEST_TRACEABILITY.txt:32:- TC_SYS_001: not established by the API suite alone; broader system acceptance remains
FUNCTION_TEST_TRACEABILITY.txt:39:FIN-15 TRACEABILITY RECONCILIATION — 2026-10-20 17:38 IST
FUNCTION_TEST_TRACEABILITY.txt:45:- TC_AUTHZ_001: PARTIAL; API anonymous/invalid-session assertions passed, but
FUNCTION_TEST_TRACEABILITY.txt:48:- TC_AUTHZ_002: PARTIAL; platform permission allow/deny and client-supplied
FUNCTION_TEST_TRACEABILITY.txt:51:- TC_AUTHZ_003: previously recorded PASS is historical; this run did not
FUNCTION_TEST_TRACEABILITY.txt:53:- TC_PRIV_002: DEFERRED / NOT RUN; workspace membership and resource-level
FUNCTION_TEST_TRACEABILITY.txt:55:- TC_SYS_001: NOT RUN / not established by the API suite alone.
FUNCTION_TEST_TRACEABILITY.txt:63:Story disposition: FIN-15 / FS-01-02 remains PARTIAL. The capability endpoint

===== CURRENT CHANGE SUMMARY =====
 M .github/workflows/ci.yml
 M IMPLEMENTATION_STATUS.txt

```
 M README.md
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/app/provision_user.py
 M apps/api/tests/test_auth_unit.py
 M apps/api/tests/test_identity_constraints.py
 M apps/api/tests/test_provision_user.py
 M apps/web/tests/ui/auth.spec.js
 M conversation.txt
 M docs/architecture/README.md
 M docs/requirements/traceability-requirements.csv
 M docs/requirements/traceability-stories.csv
 M scripts/package_release.py
?? FUNCTION_TEST_TRACEABILITY.txt
?? FinSight_AI_API_Design_Pack/
?? apps/api/migrations/0002_platform_authorization.sql
?? apps/web/proxy.js
?? update_fin15_tracking.py

===== WHITESPACE VALIDATION =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
el authorization
remain deferred. FIN-14 is outside this verification scope.

No commit or push was performed as part of this verification.
EOF
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % tail -n 45
IMPLEMENTATION_STATUS.txt
git diff --check
git status --short
Older references to GET /platform/admin/capability above are historical
implementation notes and do not describe the current route.
No application code or migration was changed by this documentation update.

FIN-15 — FRESH QUALITY VERIFICATION ADDENDUM (2026-10-21)

Scope: FIN-15 / FS-01-02, platform-level authorization only.
Database: dedicated local PostgreSQL database finsight_fin15_test.
Migrations: 0001_auth_identity.sql and 0002_platform_authorization.sql
applied successfully to the dedicated test database.

Observed verification results:
- Focused identity/authorization integration tests:
  5 passed, 0 failed.
```

- Full backend test suite:
  58 passed, 0 failed, 18 deprecation warnings.
- Full suite with statement and branch coverage:
  58 passed, 0 failed, 18 warnings; 99% total coverage.
- Module coverage: auth.py 100%, database.py 100%, main.py 100%,
  provision_user.py 94%.
- Python dependency check: pip check passed; no broken requirements.
- Python compilation: compileall passed, exit code 0.
- Git whitespace validation: git diff --check returned no errors.
- Coverage reports generated locally: coverage.xml and htmlcov/.

The above results are fresh local verification evidence. They do not
establish all FIN-15 acceptance criteria.

Outstanding criteria:
- TC_AUTHZ_001: PARTIAL; complete acceptance-case evidence remains
  outstanding.
- TC_AUTHZ_002: PARTIAL; platform permission behavior is tested, but
  the full originally specified protected-resource acceptance case is
  not established.
- TC_AUTHZ_003: retain the previously recorded historical PASS; the
  latest suite is not by itself a new idle-timeout-specific execution.
- TC_PRIV_002: DEFERRED / NOT RUN under the approved platform-only scope.
- TC_SYS_001: NOT RUN for the full planned system acceptance workflow.

Overall FIN-15 status: PARTIAL. Do not mark accepted or VERIFIED COMPLETE
until the remaining applicable acceptance criteria and evidence are
reconciled. Workspace roles, membership, and resource-level authorization
remain deferred. FIN-14 is outside this verification scope.

No commit or push was performed as part of this verification.
 M .github/workflows/ci.yml
 M IMPLEMENTATION_STATUS.txt
 M README.md
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/app/provision_user.py
 M apps/api/tests/test_auth_unit.py
 M apps/api/tests/test_identity_constraints.py
 M apps/api/tests/test_provision_user.py
 M apps/web/tests/ui/auth.spec.js
 M conversation.txt
 M docs/architecture/README.md
 M docs/requirements/traceability-requirements.csv

```
 M docs/requirements/traceability-stories.csv
 M scripts/package_release.py
?? FUNCTION_TEST_TRACEABILITY.txt
?? FinSight_AI_API_Design_Pack/
?? apps/api/migrations/0002_platform_authorization.sql
?? apps/web/proxy.js
?? update_fin15_tracking.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

./scripts/__pycache__/package_release.cpython-314.pyc
./scripts/__pycache__/validate_traceability.cpython-312.pyc
./scripts/package_release.py
./scripts/validate_traceability.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "=====
IMPLEMENTATION STATUS ====="
tail -n 70 IMPLEMENTATION_STATUS.txt

echo "===== SPRINT 1 EXECUTION STATUS ====="
tail -n 55 SPRINT_1_TEST_EXECUTION_STATUS.txt

echo "===== FUNCTION TEST TRACEABILITY ====="
tail -n 55 FUNCTION_TEST_TRACEABILITY.txt

echo "===== TRACEABILITY VALIDATOR HELP ====="
python scripts/validate_traceability.py --help

echo "===== WHITESPACE VALIDATION ====="
git diff --check

echo "===== TRACKING FILE DIFF SUMMARY ====="
git diff --stat -- \
  IMPLEMENTATION_STATUS.txt \
  SPRINT_1_TEST_EXECUTION_STATUS.txt \
  README.md \
  docs/requirements/traceability-requirements.csv \
  docs/requirements/traceability-stories.csv

echo "===== CURRENT TRACKING SCRIPT ====="
sed -n '1,240p' update_fin15_tracking.py
===== IMPLEMENTATION STATUS =====
  session denial; platform-admin success; standard-user denial despite an
  X-Role: platform_admin request header; platform role scope constraint.
- TC_AUTHZ_001: PARTIAL. API anonymous/invalid-session checks are evidenced,
```

but the source test case also requires a protected frontend page and broader end-to-end checks. Do not mark fully PASS until those are exercised.
- TC_AUTHZ_002: PARTIAL for the narrow platform permission behavior asserted. The planned model/provider admin operation and full operator/standard-user operation matrix are not evidenced by the capability endpoint.
- TC_AUTHZ_003: retain the previously recorded PASS only as historical evidence for the existing session-expiry regression. It was not separately rerun in run 20261020_141030; confirm the configured idle-inactivity threshold if required by the source test case.
- TC_PRIV_002: DEFERRED / NOT RUN under the approved platform-only scope. Workspace membership, workspace resources, and resource-level isolation remain outside this implementation.
- TC_SYS_001: NOT RUN / NOT ESTABLISHED by the API suite alone.
- Approved decision remains platform-level authorization only. Workspace roles, membership, and workspace tables are deferred. FIN-14 is not changed.
- Before marking FIN-15 complete, verify the actual route path in apps/api/app/main.py and its tests, align stale route references in the tracking records, and close the remaining story-level acceptance gaps.

FIN-15 ROUTE RECONCILIATION — 2026-10-20
Verified against the current FastAPI route decorator and test request paths:
GET /api/v1/admin/authorization/capabilities
Older references to GET /platform/admin/capability above are historical implementation notes and do not describe the current route.
No application code or migration was changed by this documentation update.

FIN-15 — FRESH QUALITY VERIFICATION ADDENDUM (2026-10-21)

Scope: FIN-15 / FS-01-02, platform-level authorization only.
Database: dedicated local PostgreSQL database finsight_fin15_test.
Migrations: 0001_auth_identity.sql and 0002_platform_authorization.sql applied successfully to the dedicated test database.

Observed verification results:
- Focused identity/authorization integration tests: 5 passed, 0 failed.
- Full backend test suite: 58 passed, 0 failed, 18 deprecation warnings.
- Full suite with statement and branch coverage: 58 passed, 0 failed, 18 warnings; 99% total coverage.
- Module coverage: auth.py 100%, database.py 100%, main.py 100%, provision_user.py 94%.
- Python dependency check: pip check passed; no broken requirements.
- Python compilation: compileall passed, exit code 0.

- Git whitespace validation: git diff --check returned no errors.
- Coverage reports generated locally: coverage.xml and htmlcov/.

The above results are fresh local verification evidence. They do not establish all FIN-15 acceptance criteria.

Outstanding criteria:
- TC_AUTHZ_001: PARTIAL; complete acceptance-case evidence remains outstanding.
- TC_AUTHZ_002: PARTIAL; platform permission behavior is tested, but the full originally specified protected-resource acceptance case is not established.
- TC_AUTHZ_003: retain the previously recorded historical PASS; the latest suite is not by itself a new idle-timeout-specific execution.
- TC_PRIV_002: DEFERRED / NOT RUN under the approved platform-only scope.
- TC_SYS_001: NOT RUN for the full planned system acceptance workflow.

Overall FIN-15 status: PARTIAL. Do not mark accepted or VERIFIED COMPLETE until the remaining applicable acceptance criteria and evidence are reconciled. Workspace roles, membership, and resource-level authorization remain deferred. FIN-14 is outside this verification scope.

No commit or push was performed as part of this verification.
===== SPRINT 1 EXECUTION STATUS =====
- Environment: local PostgreSQL 17.6; dedicated finsight_test database.
- Migrations applied successfully before test execution: 0001_auth_identity.sql and 0002_platform_authorization.sql.
- Warnings are Starlette/httpx deprecations; no test failures in the final full-suite run.
- Note: these results record the executed suite only; do not infer individual test-case IDs as PASS without verifying their mapping to the test inventory.

FIN-15 route-level authorization test update — 2026-10-20
- Test mapping: TC_AUTHZ_001 / TC_AUTHZ_002 / TC_AUTHZ_003 (existing session expiry regression coverage).
- Added integration assertions for GET /platform/admin/capability against the real FastAPI app: anonymous request expects 401; platform-admin session expects 200; authenticated user without permission expects 403 even with X-Role: platform_admin.
- Status after code edit: NOT RUN. Execute the mapped tests and update statuses only from observed results.

FIN-15 — VERIFIED EXECUTION ADDENDUM (2026-10-20)
Run ID: 20261020_141030
Python: 3.14.6
Database: finsight_test

Recorded Git revision: 64f2637

Results:
- Identity/authorization integration suite: 3 passed, 4 warnings; exit code 0.
- Full API suite: 55 passed, 10 warnings; exit code 0.
- Evidence files:
  test-results/FIN-15/20261020_141030_identity_authorization.log
  test-results/FIN-15/20261020_141030_full_api_suite.log
  test-results/FIN-15/run_20261020_141030_summary.txt

These results supersede earlier NOT RUN notes for the test execution performed in this run. They do not establish that every mapped test-plan case or the entire FIN-15 story is complete. Workspace-level authorization remains deferred.

FIN-15 TEST-STATUS RECONCILIATION — 2026-10-20 17:38 IST
Recorded run: 20261020_141030. This addendum does not claim a new execution.
- Identity/authorization integration suite: 3 passed, 4 warnings; exit code 0.
- Full API suite: 55 passed, 10 warnings; exit code 0.
- Evidence paths:
  test-results/FIN-15/20261020_141030_identity_authorization.log
  test-results/FIN-15/20261020_141030_full_api_suite.log
  test-results/FIN-15/run_20261020_141030_summary.txt

Conservative case disposition:
- TC_AUTHZ_001: PARTIAL — API unauthenticated/invalid-session behavior was exercised; protected frontend page and full test-plan scope remain unverified.
- TC_AUTHZ_002: PARTIAL — the narrow platform-admin/standard-user permission behavior and X-Role spoof rejection were exercised; the planned model/provider admin operation and complete role/operation matrix remain unverified.
- TC_AUTHZ_003: historical PASS only — the existing expiry regression was recorded earlier; it was not separately rerun in run 20261020_141030.
- TC_PRIV_002: DEFERRED / NOT RUN — workspace/resource authorization is outside the approved platform-only scope.
- TC_SYS_001: NOT RUN — a passing API suite is not a browser/system smoke test.
Overall FIN-15 remains PARTIAL, not accepted. Do not alter FIN-14 tracking.

FIN-15 ROUTE RECONCILIATION — 2026-10-20
The current route is GET /api/v1/admin/authorization/capabilities, verified against apps/api/app/main.py and the matching test request paths. Older route references above are historical records; preserve them as such.
===== FUNCTION TEST TRACEABILITY =====
- Both pytest commands exited with code 0.
- Evidence:

test-results/FIN-15/20261020_141030_identity_authorization.log
test-results/FIN-15/20261020_141030_full_api_suite.log
test-results/FIN-15/run_20261020_141030_summary.txt

Case-status guidance:
- TC_AUTHZ_001: verify its exact assertions before marking fully PASS; the integration test suite passed, but broader acceptance criteria must also be checked.
- TC_AUTHZ_002: PASS for the platform permission behavior explicitly asserted by the integration test; this does not imply missing model/provider operations exist.
- TC_AUTHZ_003: existing expiry/revocation regression passed in an earlier recorded run; it was not separately rerun under run ID 20261020_141030.
- TC_PRIV_002: BLOCKED / deferred because workspace membership and resource-level authorization are outside the approved platform-only scope.
- TC_SYS_001: not established by the API suite alone; broader system acceptance remains to be verified.

Do not infer full story completion from a green test suite. Historical entries are retained, and this addendum records the latest observed execution.

FIN-15 TRACEABILITY RECONCILIATION — 2026-10-20 17:38 IST
Recorded run: 20261020_141030. The existing execution record reports 3 authorization integration tests and 55 full API tests passing; this entry does not represent a new run.

Status interpretation:
- TC_AUTHZ_001: PARTIAL; API anonymous/invalid-session assertions passed, but the source test case also requires a protected frontend page and broader end-to-end verification.
- TC_AUTHZ_002: PARTIAL; platform permission allow/deny and client-supplied X-Role rejection are evidenced. A model/provider admin operation and the full role/operation matrix are not evidenced.
- TC_AUTHZ_003: previously recorded PASS is historical; this run did not separately rerun the configured idle-inactivity test.
- TC_PRIV_002: DEFERRED / NOT RUN; workspace membership and resource-level authorization are outside the approved platform-only scope.
- TC_SYS_001: NOT RUN / not established by the API suite alone.

Route reconciliation warning: historical sections in this file and other tracking files contain conflicting route names. Before changing the canonical route text, inspect the current route decorator in apps/api/app/main.py and the matching test request path, then make all tracking references agree with that source. Do not infer the route from an older historical note.

Story disposition: FIN-15 / FS-01-02 remains PARTIAL. The capability endpoint
is a narrow API authorization test seam, not proof that every protected
operation, frontend guard, or system-level acceptance criterion is complete.
Workspace roles, membership, and workspace tables remain deferred. FIN-14 is
unchanged.

ROUTE VERIFICATION — 2026-10-20
Canonical current route: GET /api/v1/admin/authorization/capabilities.
Verified against apps/api/app/main.py and the matching test request paths.
===== TRACEABILITY VALIDATOR HELP =====
SRS requirement IDs: 409; source story aliases: 72; Sprint 1 source tests: 54
Requirements with Jira/test-plan story links: 328; unmapped: 81; no linked Sprint 1 test: 305
Source tests linked to a catalog story: 54/54
Sprint 1 stories: 23; with SRS IDs: 22; process-story exception: FS-11-09
Tests lacking explicit execution rows (source default NOT RUN): TC_ENV_001, TC_ENV_002
Jira key shape and dated live-query marker checked; this offline script does not query Jira.
VALIDATION PASS: inventories, test references, evidence fields, and Release 2 labels are
consistent.
===== WHITESPACE VALIDATION =====
===== TRACKING FILE DIFF SUMMARY =====
 IMPLEMENTATION_STATUS.txt | 165
++++++

```
    ROOT / "IMPLEMENTATION_STATUS.txt",
    ROOT / "SPRINT_1_TEST_EXECUTION_STATUS.txt",
    ROOT / "FUNCTION_TEST_TRACEABILITY.txt",
]
missing = [str(p) for p in required if not p.is_file()]
if missing:
    raise SystemExit(
        "Run this script from the FinSight_AI repository root. Missing: "
        + ", ".join(missing)
    )

date = datetime.now().astimezone().strftime("%Y-%m-%d %H:%M %Z")
run_id = "20261020_141030"

implementation_addendum = f"""

FIN-15 TRACKING RECONCILIATION — {date}
Evidence source: recorded run {run_id}; this is a documentation reconciliation,
not a new test execution. Historical entries above are intentionally retained.

Recorded execution:
- Identity/authorization integration tests: 3 passed, 4 warnings; exit code 0.
- Full API suite: 55 passed, 10 warnings; exit code 0.
- Evidence: test-results/FIN-15/{run_id}_identity_authorization.log,
  test-results/FIN-15/{run_id}_full_api_suite.log, and
  test-results/FIN-15/run_{run_id}_summary.txt.
- Warnings recorded as Starlette/httpx deprecations and per-request cookie
  deprecation warnings; no test failures were recorded in that run.

FIN-15 status: PARTIAL — do not mark story accepted/complete from these test
results alone.
- Demonstrated by the recorded integration test: unauthenticated and invalid
  session denial; platform-admin success; standard-user denial despite an
  X-Role: platform_admin request header; platform role scope constraint.
- TC_AUTHZ_001: PARTIAL. API anonymous/invalid-session checks are evidenced,
  but the source test case also requires a protected frontend page and broader
  end-to-end checks. Do not mark fully PASS until those are exercised.
- TC_AUTHZ_002: PARTIAL for the narrow platform permission behavior asserted.
  The planned model/provider admin operation and full operator/standard-user
  operation matrix are not evidenced by the capability endpoint.
- TC_AUTHZ_003: retain the previously recorded PASS only as historical evidence
  for the existing session-expiry regression. It was not separately rerun in
  run {run_id}; confirm the configured idle-inactivity threshold if required by
  the source test case.
```

- TC_PRIV_002: DEFERRED / NOT RUN under the approved platform-only scope.
  Workspace membership, workspace resources, and resource-level isolation
  remain outside this implementation.
- TC_SYS_001: NOT RUN / NOT ESTABLISHED by the API suite alone.
- Approved decision remains platform-level authorization only. Workspace roles,
  membership, and workspace tables are deferred. FIN-14 is not changed.
- Before marking FIN-15 complete, verify the actual route path in
  apps/api/app/main.py and its tests, align stale route references in the
  tracking records, and close the remaining story-level acceptance gaps.
"""

test_status_addendum = f"""

FIN-15 TEST-STATUS RECONCILIATION — {date}
Recorded run: {run_id}. This addendum does not claim a new execution.
- Identity/authorization integration suite: 3 passed, 4 warnings; exit code 0.
- Full API suite: 55 passed, 10 warnings; exit code 0.
- Evidence paths:
  test-results/FIN-15/{run_id}_identity_authorization.log
  test-results/FIN-15/{run_id}_full_api_suite.log
  test-results/FIN-15/run_{run_id}_summary.txt

Conservative case disposition:
- TC_AUTHZ_001: PARTIAL — API unauthenticated/invalid-session behavior was
  exercised; protected frontend page and full test-plan scope remain unverified.
- TC_AUTHZ_002: PARTIAL — the narrow platform-admin/standard-user permission
  behavior and X-Role spoof rejection were exercised; the planned model/provider
  admin operation and complete role/operation matrix remain unverified.
- TC_AUTHZ_003: historical PASS only — the existing expiry regression was
  recorded earlier; it was not separately rerun in run {run_id}.
- TC_PRIV_002: DEFERRED / NOT RUN — workspace/resource authorization is outside
  the approved platform-only scope.
- TC_SYS_001: NOT RUN — a passing API suite is not a browser/system smoke test.
Overall FIN-15 remains PARTIAL, not accepted. Do not alter FIN-14 tracking.
"""

function_addendum = f"""

FIN-15 TRACEABILITY RECONCILIATION — {date}
Recorded run: {run_id}. The existing execution record reports 3 authorization
integration tests and 55 full API tests passing; this entry does not represent
a new run.

Status interpretation:

- TC_AUTHZ_001: PARTIAL; API anonymous/invalid-session assertions passed, but
  the source test case also requires a protected frontend page and broader
  end-to-end verification.
- TC_AUTHZ_002: PARTIAL; platform permission allow/deny and client-supplied
  X-Role rejection are evidenced. A model/provider admin operation and the full
  role/operation matrix are not evidenced.
- TC_AUTHZ_003: previously recorded PASS is historical; this run did not
  separately rerun the configured idle-inactivity test.
- TC_PRIV_002: DEFERRED / NOT RUN; workspace membership and resource-level
  authorization are outside the approved platform-only scope.
- TC_SYS_001: NOT RUN / not established by the API suite alone.

Route reconciliation warning: historical sections in this file and other
tracking files contain conflicting route names. Before changing the canonical
route text, inspect the current route decorator in apps/api/app/main.py and the
matching test request path, then make all tracking references agree with that
source. Do not infer the route from an older historical note.

Story disposition: FIN-15 / FS-01-02 remains PARTIAL. The capability endpoint
is a narrow API authorization test seam, not proof that every protected
operation, frontend guard, or system-level acceptance criterion is complete.
Workspace roles, membership, and workspace tables remain deferred. FIN-14 is
unchanged.
"""

```
updates = [
    (required[0], implementation_addendum),
    (required[1], test_status_addendum),
    (required[2], function_addendum),
]

for path, addition in updates:
    with path.open("a", encoding="utf-8", newline="") as f:
        f.write(addition)
    print(f"Updated: {path.relative_to(ROOT)}")

print("Done. Historical records were preserved; no code, CSV, Git, Jira, or FIN-14 changes
made.")
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %


print("Done. Historical records were preserved; no code, CSV, Git, Jira, or FIN-14 changes
made.")
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
```

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== CURRENT
ROUTE ====="
grep -n -B 3 -A 5 \
  "admin/authorization/capabilities" \
  apps/api/app/main.py \
  apps/api/tests/test_identity_constraints.py \
  apps/api/tests/test_auth_unit.py

echo "===== FIN-15 REFERENCES IN TRACKING ====="
grep -n -E \

"FIN-15|FS-01-02|TC_AUTHZ_001|TC_AUTHZ_002|TC_AUTHZ_003|TC_PRIV_002|TC_SYS_
001|finsight_test|finsight_fin15_test|55 passed|58 passed|99%" \
  IMPLEMENTATION_STATUS.txt \
  SPRINT_1_TEST_EXECUTION_STATUS.txt \
  FUNCTION_TEST_TRACEABILITY.txt

echo "===== VALIDATE TRACEABILITY ====="
python scripts/validate_traceability.py

echo "===== VERIFY WORKING TREE ====="
git status --short
git diff --check
===== CURRENT ROUTE =====
apps/api/app/main.py-181-    return UserView(**user)
apps/api/app/main.py-182-
apps/api/app/main.py-183-
apps/api/app/main.py:184:@app.get("/api/v1/admin/authorization/capabilities")
apps/api/app/main.py-185-def platform_admin_capabilities(
apps/api/app/main.py-186-    request: Request,
apps/api/app/main.py-187-    _authorized: Annotated[dict[str, Any],
Depends(require_permission("platform.admin"))],
apps/api/app/main.py-188-) -> dict[str, Any]:
apps/api/app/main.py-189-    """Return the authenticated caller's database-backed platform
capabilities."""
--
apps/api/tests/test_identity_constraints.py-69-
apps/api/tests/test_identity_constraints.py-70-    try:
apps/api/tests/test_identity_constraints.py-71-        with TestClient(app) as client:
apps/api/tests/test_identity_constraints.py:72:            anonymous =
client.get("/api/v1/admin/authorization/capabilities")
apps/api/tests/test_identity_constraints.py-73-            assert anonymous.status_code == 401
apps/api/tests/test_identity_constraints.py-74-            assert anonymous.json() == {"detail":
"Authentication required"}
```

```
apps/api/tests/test_identity_constraints.py-75-
apps/api/tests/test_identity_constraints.py-76-        invalid_session = client.get(
apps/api/tests/test_identity_constraints.py:77:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-78-            cookies={SESSION_COOKIE:
"invalid-session-token"},
apps/api/tests/test_identity_constraints.py-79-        )
apps/api/tests/test_identity_constraints.py-80-        assert invalid_session.status_code == 401
apps/api/tests/test_identity_constraints.py-81-
apps/api/tests/test_identity_constraints.py-82-        allowed = client.get(
apps/api/tests/test_identity_constraints.py:83:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-84-            cookies={SESSION_COOKIE:
tokens[0]},
apps/api/tests/test_identity_constraints.py-85-        )
apps/api/tests/test_identity_constraints.py-86-        #₹ The request_id header is optional, but
if supplied it must be validated and preserved as the correlation_id.
apps/api/tests/test_identity_constraints.py-87-        correlated = client.get(
apps/api/tests/test_identity_constraints.py:88:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-89-            cookies={SESSION_COOKIE:
tokens[0]},
apps/api/tests/test_identity_constraints.py-90-            headers={"request_id":
"client-correlation-123"},
apps/api/tests/test_identity_constraints.py-91-        )
apps/api/tests/test_identity_constraints.py-92-        assert correlated.status_code == 200
apps/api/tests/test_identity_constraints.py-93-        correlated_body = correlated.json()
--
apps/api/tests/test_identity_constraints.py-96-        assert correlated_body["correlation_id"] ==
"client-correlation-123"
apps/api/tests/test_identity_constraints.py-97-
apps/api/tests/test_identity_constraints.py-98-        malformed = client.get(
apps/api/tests/test_identity_constraints.py:99:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-100-            cookies={SESSION_COOKIE:
tokens[0]},
apps/api/tests/test_identity_constraints.py-101-            headers={"request_id": "x" * 129},
apps/api/tests/test_identity_constraints.py-102-        )
apps/api/tests/test_identity_constraints.py-103-        assert malformed.status_code == 400
apps/api/tests/test_identity_constraints.py-104-
--
apps/api/tests/test_identity_constraints.py-114-        assert body["correlation_id"] ==
body["request_id"]
apps/api/tests/test_identity_constraints.py-115-
```

```
apps/api/tests/test_identity_constraints.py-116-        denied = client.get(
apps/api/tests/test_identity_constraints.py:117:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-118-            cookies={SESSION_COOKIE:
tokens[1]},
apps/api/tests/test_identity_constraints.py-119-            headers={"X-Role": "platform_admin"},
apps/api/tests/test_identity_constraints.py-120-        )
apps/api/tests/test_identity_constraints.py-121-        assert denied.status_code == 403
apps/api/tests/test_identity_constraints.py-122-        assert denied.json() == {"detail":
"Permission denied"}
--
apps/api/tests/test_identity_constraints.py-179-
apps/api/tests/test_identity_constraints.py-180-    with TestClient(app) as client:
apps/api/tests/test_identity_constraints.py-181-        first_response = client.get(
apps/api/tests/test_identity_constraints.py:182:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-183-            cookies={SESSION_COOKIE: token},
apps/api/tests/test_identity_constraints.py-184-        )
apps/api/tests/test_identity_constraints.py-185-        assert first_response.status_code == 200
apps/api/tests/test_identity_constraints.py-186-        assert
first_response.json()["data"]["effective_permissions"] == [
apps/api/tests/test_identity_constraints.py-187-            "platform.admin"
--
apps/api/tests/test_identity_constraints.py-194-            )
apps/api/tests/test_identity_constraints.py-195-
apps/api/tests/test_identity_constraints.py-196-        next_response = client.get(
apps/api/tests/test_identity_constraints.py:197:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-198-            cookies={SESSION_COOKIE: token},
apps/api/tests/test_identity_constraints.py-199-        )
apps/api/tests/test_identity_constraints.py-200-        assert next_response.status_code ==
403
apps/api/tests/test_identity_constraints.py-201-        assert next_response.json() == {"detail":
"Permission denied"}
apps/api/tests/test_identity_constraints.py-202-
--
apps/api/tests/test_identity_constraints.py-257-    try:
apps/api/tests/test_identity_constraints.py-258-        with TestClient(app) as client:
apps/api/tests/test_identity_constraints.py-259-            first_response = client.get(
apps/api/tests/test_identity_constraints.py:260:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-261-                cookies={SESSION_COOKIE: token},
apps/api/tests/test_identity_constraints.py-262-            )
apps/api/tests/test_identity_constraints.py-263-            assert first_response.status_code == 200
```

```
apps/api/tests/test_identity_constraints.py-264-            assert
first_response.json()["data"]["effective_permissions"] == [
apps/api/tests/test_identity_constraints.py-265-                "platform.admin"
--
apps/api/tests/test_identity_constraints.py-273-            )
apps/api/tests/test_identity_constraints.py-274-
apps/api/tests/test_identity_constraints.py-275-            next_response = client.get(
apps/api/tests/test_identity_constraints.py:276:
"/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-277-                cookies={SESSION_COOKIE: token},
apps/api/tests/test_identity_constraints.py-278-            )
apps/api/tests/test_identity_constraints.py-279-            assert next_response.status_code ==
403
apps/api/tests/test_identity_constraints.py-280-            assert next_response.json() == {"detail":
"Permission denied"}
apps/api/tests/test_identity_constraints.py-281-    finally:
--
apps/api/tests/test_identity_constraints.py-313-#     # Authentication runs before the endpoint
validates its header.
apps/api/tests/test_identity_constraints.py-314-#     # An unauthenticated request must not be
mistaken for a header-validation test.
apps/api/tests/test_identity_constraints.py-315-#     response = TestClient(app).get(
apps/api/tests/test_identity_constraints.py:316:#     "/api/v1/admin/authorization/capabilities",
apps/api/tests/test_identity_constraints.py-317-#     headers={"request_id": "x" * 129},
apps/api/tests/test_identity_constraints.py-318-#     )
apps/api/tests/test_identity_constraints.py-319-#     assert response.status_code == 401
--
apps/api/tests/test_auth_unit.py-138-        ("get", "/auth/me", {"cookies": {"finsight_session":
"synthetic-token"}}),
apps/api/tests/test_auth_unit.py-139-        ("post", "/auth/logout", {"cookies": {"finsight_session":
"synthetic-token"}}),
apps/api/tests/test_auth_unit.py-140-        ("get",
apps/api/tests/test_auth_unit.py:141:            "/api/v1/admin/authorization/capabilities",
apps/api/tests/test_auth_unit.py-142-            {"cookies": {"finsight_session": "synthetic-token"}},
apps/api/tests/test_auth_unit.py-143-        ),
apps/api/tests/test_auth_unit.py-144-    ],
apps/api/tests/test_auth_unit.py-145-)
apps/api/tests/test_auth_unit.py-146-def
test_session_database_outages_return_controlled_error(monkeypatch, method, path, kwargs):
===== FIN-15 REFERENCES IN TRACKING =====
IMPLEMENTATION_STATUS.txt:59:Reviewed requirements: SRS FR-P1-01..04, FR-P1-06,
FR-P1-08 and NFR-P1-01..03/05; canonical identity DDL `Database/schema (1).sql` (`roles`,
`users`, `auth_sessions`); backlog FS-01-01 page 2; test-plan management cases
TC_AUTHN_001..005 and TC_AUTHZ_003.
```

IMPLEMENTATION_STATUS.txt:92:Test execution: detailed statuses and evidence are recorded in `SPRINT_1_TEST_EXECUTION_STATUS.txt`. TC_AUTHN_001, TC_AUTHN_002, TC_AUTHN_003, and the shared session-expiry behavior in TC_AUTHZ_003 passed against disposable PostgreSQL. TC_AUTHN_005 is BLOCKED because the full administrator invitation lifecycle is not implemented. TC_AUTHN_006 passed the disposable database dump/log/API privacy checks recorded below; TC_AUTHN_007 is BLOCKED pending deployed HTTPS verification. TC_AUTHN_004 is BLOCKED pending the SRS-required password recovery scope/delivery decision.
IMPLEMENTATION_STATUS.txt:214:- `TC_INT_001` remains BLOCKED for configuration-version/audit persistence and allowed/denied configuration flows (not implemented). `TC_SYS_001` remains BLOCKED for dashboard/settings, health and Ollama flows (not implemented). Password recovery, admin invitations and deployed HTTPS remain the previously recorded FIN-13 gaps and are untouched.
IMPLEMENTATION_STATUS.txt:319:FIN-15 / FS-01-02 — Enforce roles and protected routes (2026-10-19)
IMPLEMENTATION_STATUS.txt:321:Inspected relevant SRS access-control requirements, canonical schema and reference seeds, FS-01-02 backlog, four-sprint/test-plan mapping, architecture docs, identity/session API, migration 0001, trusted provisioning CLI, and relevant tests. Existing routes: public POST `/auth/login`, POST `/auth/logout`, protected GET `/auth/me`; no product/admin operation or protected frontend page currently exists. Existing session dependency already rejects absent, revoked, and expired sessions; prior `TC_AUTHZ_003` expiry result remains PASS as previously recorded.
IMPLEMENTATION_STATUS.txt:324:Remaining acceptance criteria: role/permission matrix, 403 behavior, trusted role resolution/provisioning assignment, protected routeapplication, and frontend guard cannot be completed safely until scope decision and a concrete capability/resource mapping are established. TC_AUTHZ_001 also requires a protected frontend page (none exists); TC_AUTHZ_002 currently names a model/provider admin operation that does not exist; TC_PRIV_002 requires workspace membership/resources not implemented. FIN-15 is not ready for acceptance.
IMPLEMENTATION_STATUS.txt:326:FIN-15 verification update — 2026-10-20
IMPLEMENTATION_STATUS.txt:328:- Created and used the dedicated finsight_test database.
IMPLEMENTATION_STATUS.txt:332:- Full API test suite: 55 passed, 0 failed, 9 warnings.
IMPLEMENTATION_STATUS.txt:336:- FIN-15 is not yet marked complete pending route-level authorization verification and remaining acceptance checks.
IMPLEMENTATION_STATUS.txt:338:FIN-15 implementation update — 2026-10-20
IMPLEMENTATION_STATUS.txt:342:- Function/test traceability added in FUNCTION_TEST_TRACEABILITY.txt for TC_AUTHZ_001, TC_AUTHZ_002, and related existing session coverage TC_AUTHZ_003.
IMPLEMENTATION_STATUS.txt:343:- Verification status: implementation edits prepared; tests have NOT RUN after this change. Do not mark FIN-15 VERIFIED COMPLETE until test results and all acceptance criteria are checked.
IMPLEMENTATION_STATUS.txt:346:FIN-15 — VERIFIED TEST RUN ADDENDUM (2026-10-20)
IMPLEMENTATION_STATUS.txt:350:Database: finsight_test

IMPLEMENTATION_STATUS.txt:354:- Full API suite: 55 passed, 10 warnings.
IMPLEMENTATION_STATUS.txt:359:-
test-results/FIN-15/20261020_141030_identity_authorization.log
IMPLEMENTATION_STATUS.txt:360:- test-results/FIN-15/20261020_141030_full_api_suite.log
IMPLEMENTATION_STATUS.txt:361:- test-results/FIN-15/run_20261020_141030_summary.txt
IMPLEMENTATION_STATUS.txt:364:the behavior exercised by those tests, not automatic
completion of every FIN-15
IMPLEMENTATION_STATUS.txt:370:FIN-15 — CURRENT VERIFICATION ADDENDUM
(2026-10-20)
IMPLEMENTATION_STATUS.txt:375:- Full API suite: 58 passed, 18 warnings.
IMPLEMENTATION_STATUS.txt:381:- TC_AUTHZ_001 remains NOT RUN as a complete
test-plan case because its
IMPLEMENTATION_STATUS.txt:383:- TC_AUTHZ_002 remains NOT RUN as a complete
test-plan case because its
IMPLEMENTATION_STATUS.txt:386:- TC_AUTHZ_003 retains its existing recorded status; the
current 58-test
IMPLEMENTATION_STATUS.txt:388:- TC_PRIV_002 remains NOT RUN / deferred: workspace
and cross-user
IMPLEMENTATION_STATUS.txt:390:- TC_SYS_001 remains NOT RUN until the required
end-to-end smoke path is run.
IMPLEMENTATION_STATUS.txt:392:Overall FIN-15 status: PARTIAL. Passing API tests do not
constitute
IMPLEMENTATION_STATUS.txt:397:FIN-15 TRACKING RECONCILIATION — 2026-10-20
17:38 IST
IMPLEMENTATION_STATUS.txt:403:- Full API suite: 55 passed, 10 warnings; exit code 0.
IMPLEMENTATION_STATUS.txt:404:- Evidence:
test-results/FIN-15/20261020_141030_identity_authorization.log,
IMPLEMENTATION_STATUS.txt:405: test-results/FIN-15/20261020_141030_full_api_suite.log,
and
IMPLEMENTATION_STATUS.txt:406: test-results/FIN-15/run_20261020_141030_summary.txt.
IMPLEMENTATION_STATUS.txt:410:FIN-15 status: PARTIAL — do not mark story
accepted/complete from these test
IMPLEMENTATION_STATUS.txt:415:- TC_AUTHZ_001: PARTIAL. API
anonymous/invalid-session checks are evidenced,
IMPLEMENTATION_STATUS.txt:418:- TC_AUTHZ_002: PARTIAL for the narrow platform
permission behavior asserted.
IMPLEMENTATION_STATUS.txt:421:- TC_AUTHZ_003: retain the previously recorded PASS
only as historical evidence
IMPLEMENTATION_STATUS.txt:425:- TC_PRIV_002: DEFERRED / NOT RUN under the
approved platform-only scope.
IMPLEMENTATION_STATUS.txt:428:- TC_SYS_001: NOT RUN / NOT ESTABLISHED by the
API suite alone.
IMPLEMENTATION_STATUS.txt:431:- Before marking FIN-15 complete, verify the actual route
path in

IMPLEMENTATION_STATUS.txt:435:FIN-15 ROUTE RECONCILIATION — 2026-10-20
IMPLEMENTATION_STATUS.txt:442:FIN-15 — FRESH QUALITY VERIFICATION ADDENDUM (2026-10-21)
IMPLEMENTATION_STATUS.txt:444:Scope: FIN-15 / FS-01-02, platform-level authorization only.
IMPLEMENTATION_STATUS.txt:445:Database: dedicated local PostgreSQL database finsight_fin15_test.
IMPLEMENTATION_STATUS.txt:453: 58 passed, 0 failed, 18 deprecation warnings.
IMPLEMENTATION_STATUS.txt:455: 58 passed, 0 failed, 18 warnings; 99% total coverage.
IMPLEMENTATION_STATUS.txt:464:establish all FIN-15 acceptance criteria.
IMPLEMENTATION_STATUS.txt:467:- TC_AUTHZ_001: PARTIAL; complete acceptance-case evidence remains
IMPLEMENTATION_STATUS.txt:469:- TC_AUTHZ_002: PARTIAL; platform permission behavior is tested, but
IMPLEMENTATION_STATUS.txt:472:- TC_AUTHZ_003: retain the previously recorded historical PASS; the
IMPLEMENTATION_STATUS.txt:474:- TC_PRIV_002: DEFERRED / NOT RUN under the approved platform-only scope.
IMPLEMENTATION_STATUS.txt:475:- TC_SYS_001: NOT RUN for the full planned system acceptance workflow.
IMPLEMENTATION_STATUS.txt:477:Overall FIN-15 status: PARTIAL. Do not mark accepted or VERIFIED COMPLETE
SPRINT_1_TEST_EXECUTION_STATUS.txt:51:TC_AUTHZ_003 — Expire inactive sessions according to configured policy
SPRINT_1_TEST_EXECUTION_STATUS.txt:78:TC_SYS_001 | Status: BLOCKED | Execution date: 2026-10-18 | Actual result: Browser smoke verified current sign-in/session/logout path against live frontend, FastAPI, and PostgreSQL. Full planned dashboard/settings, dependency health, Ollama available/unavailable, and correlation-log workflow cannot run because those components are not implemented | Tester/owner: Codex | Evidence: apps/web/tests/e2e/auth.spec.js; scripts/run_live_e2e.py; IMPLEMENTATION_STATUS.txt | Linked defect: Current application implements auth starter only
SPRINT_1_TEST_EXECUTION_STATUS.txt:94:TC_AUTHZ_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
SPRINT_1_TEST_EXECUTION_STATUS.txt:95:TC_AUTHZ_002 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
SPRINT_1_TEST_EXECUTION_STATUS.txt:96:TC_AUTHZ_003 | Status: PASS | Execution date: 2026-10-18 | Actual result: Stale session rejected under configured TTL | Tester/owner: Codex | Evidence: apps/api/tests/test_auth.py | Linked defect: —
SPRINT_1_TEST_EXECUTION_STATUS.txt:115:TC_PRIV_002 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
SPRINT_1_TEST_EXECUTION_STATUS.txt:122:TC_SYS_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —

SPRINT_1_TEST_EXECUTION_STATUS.txt:126:- Command: DATABASE_URL="postgresql:///finsight_test" python -m pytest -q tests/test_identity_constraints.py
SPRINT_1_TEST_EXECUTION_STATUS.txt:128:- Command: DATABASE_URL="postgresql:///finsight_test" python -m pytest -q
SPRINT_1_TEST_EXECUTION_STATUS.txt:129:- Result: 55 passed, 0 failed, 9 warnings in 9.52s.
SPRINT_1_TEST_EXECUTION_STATUS.txt:130:- Environment: local PostgreSQL 17.6; dedicated finsight_test database.
SPRINT_1_TEST_EXECUTION_STATUS.txt:135:FIN-15 route-level authorization test update — 2026-10-20
SPRINT_1_TEST_EXECUTION_STATUS.txt:136:- Test mapping: TC_AUTHZ_001 / TC_AUTHZ_002 / TC_AUTHZ_003 (existing session expiry regression coverage).
SPRINT_1_TEST_EXECUTION_STATUS.txt:140:FIN-15 — VERIFIED EXECUTION ADDENDUM (2026-10-20)
SPRINT_1_TEST_EXECUTION_STATUS.txt:143:Database: finsight_test
SPRINT_1_TEST_EXECUTION_STATUS.txt:148:- Full API suite: 55 passed, 10 warnings; exit code 0.
SPRINT_1_TEST_EXECUTION_STATUS.txt:150: test-results/FIN-15/20261020_141030_identity_authorization.log
SPRINT_1_TEST_EXECUTION_STATUS.txt:151: test-results/FIN-15/20261020_141030_full_api_suite.log
SPRINT_1_TEST_EXECUTION_STATUS.txt:152: test-results/FIN-15/run_20261020_141030_summary.txt
SPRINT_1_TEST_EXECUTION_STATUS.txt:156:FIN-15 story is complete. Workspace-level authorization remains deferred.
SPRINT_1_TEST_EXECUTION_STATUS.txt:159:FIN-15 TEST-STATUS RECONCILIATION — 2026-10-20 17:38 IST
SPRINT_1_TEST_EXECUTION_STATUS.txt:162:- Full API suite: 55 passed, 10 warnings; exit code 0.
SPRINT_1_TEST_EXECUTION_STATUS.txt:164: test-results/FIN-15/20261020_141030_identity_authorization.log
SPRINT_1_TEST_EXECUTION_STATUS.txt:165: test-results/FIN-15/20261020_141030_full_api_suite.log
SPRINT_1_TEST_EXECUTION_STATUS.txt:166: test-results/FIN-15/run_20261020_141030_summary.txt
SPRINT_1_TEST_EXECUTION_STATUS.txt:169:- TC_AUTHZ_001: PARTIAL — API unauthenticated/invalid-session behavior was
SPRINT_1_TEST_EXECUTION_STATUS.txt:171:- TC_AUTHZ_002: PARTIAL — the narrow platform-admin/standard-user permission
SPRINT_1_TEST_EXECUTION_STATUS.txt:174:- TC_AUTHZ_003: historical PASS only — the existing expiry regression was
SPRINT_1_TEST_EXECUTION_STATUS.txt:176:- TC_PRIV_002: DEFERRED / NOT RUN — workspace/resource authorization is outside

SPRINT_1_TEST_EXECUTION_STATUS.txt:178:- TC_SYS_001: NOT RUN — a passing API suite is not a browser/system smoke test.
SPRINT_1_TEST_EXECUTION_STATUS.txt:179:Overall FIN-15 remains PARTIAL, not accepted. Do not alter FIN-14 tracking.
SPRINT_1_TEST_EXECUTION_STATUS.txt:181:FIN-15 ROUTE RECONCILIATION — 2026-10-20
FUNCTION_TEST_TRACEABILITY.txt:3:FIN-15 / FS-01-02 — platform authorization capability endpoint
FUNCTION_TEST_TRACEABILITY.txt:7:Test mapping: TC_AUTHZ_001 (unauthenticated/invalid session denial), TC_AUTHZ_002 (permission matrix: platform admin allowed, standard user denied; client-supplied X-Role ignored), TC_AUTHZ_003 (session expiry/revocation behavior via existing session dependency).
FUNCTION_TEST_TRACEABILITY.txt:10:FIN-15 — EXECUTION EVIDENCE ADDENDUM (2026-10-20)
FUNCTION_TEST_TRACEABILITY.txt:16:- Full API suite: 55 passed, 10 warnings.
FUNCTION_TEST_TRACEABILITY.txt:19: test-results/FIN-15/20261020_141030_identity_authorization.log
FUNCTION_TEST_TRACEABILITY.txt:20: test-results/FIN-15/20261020_141030_full_api_suite.log
FUNCTION_TEST_TRACEABILITY.txt:21: test-results/FIN-15/run_20261020_141030_summary.txt
FUNCTION_TEST_TRACEABILITY.txt:24:- TC_AUTHZ_001: verify its exact assertions before marking fully PASS; the integration
FUNCTION_TEST_TRACEABILITY.txt:26:- TC_AUTHZ_002: PASS for the platform permission behavior explicitly asserted by the
FUNCTION_TEST_TRACEABILITY.txt:28:- TC_AUTHZ_003: existing expiry/revocation regression passed in an earlier recorded run;
FUNCTION_TEST_TRACEABILITY.txt:30:- TC_PRIV_002: BLOCKED / deferred because workspace membership and resource-level
FUNCTION_TEST_TRACEABILITY.txt:32:- TC_SYS_001: not established by the API suite alone; broader system acceptance remains
FUNCTION_TEST_TRACEABILITY.txt:39:FIN-15 TRACEABILITY RECONCILIATION — 2026-10-20 17:38 IST
FUNCTION_TEST_TRACEABILITY.txt:45:- TC_AUTHZ_001: PARTIAL; API anonymous/invalid-session assertions passed, but
FUNCTION_TEST_TRACEABILITY.txt:48:- TC_AUTHZ_002: PARTIAL; platform permission allow/deny and client-supplied
FUNCTION_TEST_TRACEABILITY.txt:51:- TC_AUTHZ_003: previously recorded PASS is historical; this run did not
FUNCTION_TEST_TRACEABILITY.txt:53:- TC_PRIV_002: DEFERRED / NOT RUN; workspace membership and resource-level
FUNCTION_TEST_TRACEABILITY.txt:55:- TC_SYS_001: NOT RUN / not established by the API suite alone.

```
FUNCTION_TEST_TRACEABILITY.txt:63:Story disposition: FIN-15 / FS-01-02 remains
PARTIAL. The capability endpoint
===== VALIDATE TRACEABILITY =====
SRS requirement IDs: 409; source story aliases: 72; Sprint 1 source tests: 54
Requirements with Jira/test-plan story links: 328; unmapped: 81; no linked Sprint 1 test: 305
Source tests linked to a catalog story: 54/54
Sprint 1 stories: 23; with SRS IDs: 22; process-story exception: FS-11-09
Tests lacking explicit execution rows (source default NOT RUN): TC_ENV_001, TC_ENV_002
Jira key shape and dated live-query marker checked; this offline script does not query Jira.
VALIDATION PASS: inventories, test references, evidence fields, and Release 2 labels are
consistent.
===== VERIFY WORKING TREE =====
 M .github/workflows/ci.yml
 M IMPLEMENTATION_STATUS.txt
 M README.md
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/app/provision_user.py
 M apps/api/tests/test_auth_unit.py
 M apps/api/tests/test_identity_constraints.py
 M apps/api/tests/test_provision_user.py
 M apps/web/tests/ui/auth.spec.js
 M conversation.txt
 M docs/architecture/README.md
 M docs/requirements/traceability-requirements.csv
 M docs/requirements/traceability-stories.csv
 M scripts/package_release.py
?? FUNCTION_TEST_TRACEABILITY.txt
?? FinSight_AI_API_Design_Pack/
?? apps/api/migrations/0002_platform_authorization.sql
?? apps/web/proxy.js
?? update_fin15_tracking.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
new = (
    "Historical disposition from the 2026-10-20 execution record: FIN-15 "
    "was recorded as complete at that point. The subsequent reconciliation "
    "below supersedes that disposition. Workspace-level authorization "
    "remains deferred."
)

text = sprint.read_text(encoding="utf-8")
if old not in text:
```

```
    raise SystemExit("STOP: expected historical sentence not found; no changes made.")
sprint.write_text(text.replace(old, new, 1), encoding="utf-8")

addendum = """

FIN-15 — FRESH QUALITY VERIFICATION ADDENDUM (2026-10-21)
Database: dedicated local PostgreSQL database finsight_fin15_test.
Migrations 0001_auth_identity.sql and 0002_platform_authorization.sql applied.

Fresh observed results:
- Focused identity/authorization integration tests: 5 passed.
- Full backend suite: 58 passed, 0 failed, 18 warnings.
- Full suite with coverage: 58 passed; 99% total coverage.
- pip check, compileall, and git diff --check passed.
- Traceability validator passed.

These results supersede neither the historical 2026-10-20 run record nor
the individual acceptance criteria. TC_AUTHZ_001 and TC_AUTHZ_002 remain
PARTIAL; TC_AUTHZ_003 is historical PASS only; TC_PRIV_002 is DEFERRED /
NOT RUN under the approved platform-only scope; TC_SYS_001 remains
NOT RUN for the full planned system acceptance workflow.

Overall FIN-15 / FS-01-02 status: PARTIAL, not accepted or VERIFIED COMPLETE.
Workspace/resource authorization remains deferred. FIN-14 is unchanged.
"""

marker = "FIN-15 — FRESH QUALITY VERIFICATION ADDENDUM (2026-10-21)"
if marker in text:
    raise SystemExit("STOP: fresh addendum already exists in Sprint status.")
sprint.write_text(sprint.read_text(encoding="utf-8") + addendum, encoding="utf-8")

trace_text = trace.read_text(encoding="utf-8")
if marker in trace_text:
    raise SystemExit("STOP: fresh addendum already exists in function traceability.")
trace.write_text(trace_text + addendum, encoding="utf-8")

print("Updated Sprint 1 and function traceability records.")
print("Historical 2026-10-20 evidence preserved; latest 2026-10-21 evidence recorded.")
PY
Updated Sprint 1 and function traceability records.
Historical 2026-10-20 evidence preserved; latest 2026-10-21 evidence recorded.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % python
scripts/validate_traceability.py
git diff --check
```

```
echo "===== REVIEW TRACKING CHANGES ====="
git diff -- SPRINT_1_TEST_EXECUTION_STATUS.txt
git diff --no-index /dev/null FUNCTION_TEST_TRACEABILITY.txt
SRS requirement IDs: 409; source story aliases: 72; Sprint 1 source tests: 54
Requirements with Jira/test-plan story links: 328; unmapped: 81; no linked Sprint 1 test: 305
Source tests linked to a catalog story: 54/54
Sprint 1 stories: 23; with SRS IDs: 22; process-story exception: FS-11-09
Tests lacking explicit execution rows (source default NOT RUN): TC_ENV_001, TC_ENV_002
Jira key shape and dated live-query marker checked; this offline script does not query Jira.
VALIDATION PASS: inventories, test references, evidence fields, and Release 2 labels are
consistent.
===== REVIEW TRACKING CHANGES =====
diff --git a/SPRINT_1_TEST_EXECUTION_STATUS.txt
b/SPRINT_1_TEST_EXECUTION_STATUS.txt
index 649e4c0..ca58f95 100644
--- a/SPRINT_1_TEST_EXECUTION_STATUS.txt
+++ b/SPRINT_1_TEST_EXECUTION_STATUS.txt
@@ -121,3 +121,85 @@ TC_DASH_002 | Status: NOT RUN | Execution date: — | Actual
result: — | Teste
 TC_INT_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — |
Evidence: — | Linked defect: —
 TC_SYS_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — |
Evidence: — | Linked defect: —
 TC_BOUNDARY_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner:
— | Evidence: — | Linked defect: —
+
+Additional test evidence — 2026-10-20
+- Command: DATABASE_URL="postgresql:///finsight_test" python -m pytest -q
tests/test_identity_constraints.py
+- Result: 3 passed, 0 failed, 3 warnings.
+- Command: DATABASE_URL="postgresql:///finsight_test" python -m pytest -q
+- Result: 55 passed, 0 failed, 9 warnings in 9.52s.
+- Environment: local PostgreSQL 17.6; dedicated finsight_test database.
+- Migrations applied successfully before test execution: 0001_auth_identity.sql and
0002_platform_authorization.sql.
+- Warnings are Starlette/httpx deprecations; no test failures in the final full-suite run.
+- Note: these results record the executed suite only; do not infer individual test-case IDs as
PASS without verifying their mapping to the test inventory.
+
+FIN-15 route-level authorization test update — 2026-10-20
+- Test mapping: TC_AUTHZ_001 / TC_AUTHZ_002 / TC_AUTHZ_003 (existing session expiry
regression coverage).
```

+- Added integration assertions for GET /platform/admin/capability against the real FastAPI app: anonymous request expects 401; platform-admin session expects 200;authenticated user without permission expects 403 even with X-Role: platform_admin.
+- Status after code edit: NOT RUN. Execute the mapped tests and update statuses only from observed results.
+
+FIN-15 — VERIFIED EXECUTION ADDENDUM (2026-10-20)
+Run ID: 20261020_141030
+Python: 3.14.6
+Database: finsight_test
+Recorded Git revision: 64f2637
+
+Results:
+- Identity/authorization integration suite: 3 passed, 4 warnings; exit code 0.
+- Full API suite: 55 passed, 10 warnings; exit code 0.
+- Evidence files:
+ test-results/FIN-15/20261020_141030_identity_authorization.log
+ test-results/FIN-15/20261020_141030_full_api_suite.log
+ test-results/FIN-15/run_20261020_141030_summary.txt
+
+These results supersede earlier NOT RUN notes for the test execution performed
+in this run. They do not establish that every mapped test-plan case or the entire
+Historical disposition from the 2026-10-20 execution record: FIN-15 was recorded as complete at that point. The subsequent reconciliation below supersedes that disposition. Workspace-level authorization remains deferred.
+
+
+FIN-15 TEST-STATUS RECONCILIATION — 2026-10-20 17:38 IST
+Recorded run: 20261020_141030. This addendum does not claim a new execution.
+- Identity/authorization integration suite: 3 passed, 4 warnings; exit code 0.
diff --git a/FUNCTION_TEST_TRACEABILITY.txt b/FUNCTION_TEST_TRACEABILITY.txt
new file mode 100644
index 0000000..3384b7d
--- /dev/null
+++ b/FUNCTION_TEST_TRACEABILITY.txt
@@ -0,0 +1,92 @@
+FinSight AI — Function/Test Traceability
+
+FIN-15 / FS-01-02 — platform authorization capability endpoint
+Function: GET /api/v1/admin/authorization/capabilities
+Authorization rule: requires a valid active session and database-resolved `platform.admin` permission; deny by default.
+Scope: platform-only. Workspace roles, membership, and workspace tables are deferred.

+Test mapping: TC_AUTHZ_001 (unauthenticated/invalid session denial), TC_AUTHZ_002 (permission matrix: platform admin allowed, standard user denied; client-supplied X-Role ignored), TC_AUTHZ_003 (session expiry/revocation behavior via existing session dependency).
+Evidence note: route integration assertions were added to apps/api/tests/test_identity_constraints.py. Mark test cases PASS only after executing the mapped tests and confirming the expected behavior.
+
+FIN-15 — EXECUTION EVIDENCE ADDENDUM (2026-10-20)
+Run ID: 20261020_141030
+Endpoint: GET /api/v1/admin/authorization/capabilities
+
+Observed execution:
+- tests/test_identity_constraints.py: 3 passed, 4 warnings.
+- Full API suite: 55 passed, 10 warnings.
+- Both pytest commands exited with code 0.
+- Evidence:
+ test-results/FIN-15/20261020_141030_identity_authorization.log
+ test-results/FIN-15/20261020_141030_full_api_suite.log
+ test-results/FIN-15/run_20261020_141030_summary.txt
+
+Case-status guidance:
+- TC_AUTHZ_001: verify its exact assertions before marking fully PASS; the integration
+ test suite passed, but broader acceptance criteria must also be checked.
+- TC_AUTHZ_002: PASS for the platform permission behavior explicitly asserted by the
+ integration test; this does not imply missing model/provider operations exist.
+- TC_AUTHZ_003: existing expiry/revocation regression passed in an earlier recorded run;
+ it was not separately rerun under run ID 20261020_141030.
+- TC_PRIV_002: BLOCKED / deferred because workspace membership and resource-level
+ authorization are outside the approved platform-only scope.
+- TC_SYS_001: not established by the API suite alone; broader system acceptance remains
+ to be verified.
+
+Do not infer full story completion from a green test suite. Historical entries are
+retained, and this addendum records the latest observed execution.
+
+
+FIN-15 TRACEABILITY RECONCILIATION — 2026-10-20 17:38 IST
+Recorded run: 20261020_141030. The existing execution record reports 3 authorization
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

+
+FIN-15 TRACEABILITY RECONCILIATION — 2026-10-20 17:38 IST
+Recorded run: 20261020_141030. The existing execution record reports 3 authorization

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== SPRINT FILE:
HISTORICAL SECTION ====="
sed -n '138,175p' SPRINT_1_TEST_EXECUTION_STATUS.txt

echo "===== SPRINT FILE: LATEST ADDENDUM ====="
grep -n -A 22 "FRESH QUALITY VERIFICATION" \
  SPRINT_1_TEST_EXECUTION_STATUS.txt

echo "===== TRACEABILITY FILE: LATEST ADDENDUM ====="
grep -n -A 22 "FRESH QUALITY VERIFICATION" \
  FUNCTION_TEST_TRACEABILITY.txt

echo "===== FILE ENDINGS ====="
tail -n 12 SPRINT_1_TEST_EXECUTION_STATUS.txt
tail -n 12 FUNCTION_TEST_TRACEABILITY.txt

echo "===== VALIDATOR ====="
python scripts/validate_traceability.py
git diff --check
===== SPRINT FILE: HISTORICAL SECTION =====
- Status after code edit: NOT RUN. Execute the mapped tests and update statuses only from
observed results.

FIN-15 — VERIFIED EXECUTION ADDENDUM (2026-10-20)
Run ID: 20261020_141030
Python: 3.14.6
Database: finsight_test
Recorded Git revision: 64f2637

Results:
- Identity/authorization integration suite: 3 passed, 4 warnings; exit code 0.
- Full API suite: 55 passed, 10 warnings; exit code 0.
- Evidence files:
  test-results/FIN-15/20261020_141030_identity_authorization.log
  test-results/FIN-15/20261020_141030_full_api_suite.log
  test-results/FIN-15/run_20261020_141030_summary.txt

These results supersede earlier NOT RUN notes for the test execution performed
in this run. They do not establish that every mapped test-plan case or the entire
Historical disposition from the 2026-10-20 execution record: FIN-15 was recorded as complete
at that point. The subsequent reconciliation below supersedes that disposition. Workspace-level
authorization remains deferred.
```

FIN-15 TEST-STATUS RECONCILIATION — 2026-10-20 17:38 IST
Recorded run: 20261020_141030. This addendum does not claim a new execution.
- Identity/authorization integration suite: 3 passed, 4 warnings; exit code 0.
- Full API suite: 55 passed, 10 warnings; exit code 0.
- Evidence paths:
  test-results/FIN-15/20261020_141030_identity_authorization.log
  test-results/FIN-15/20261020_141030_full_api_suite.log
  test-results/FIN-15/run_20261020_141030_summary.txt

Conservative case disposition:
- TC_AUTHZ_001: PARTIAL — API unauthenticated/invalid-session behavior was
  exercised; protected frontend page and full test-plan scope remain unverified.
- TC_AUTHZ_002: PARTIAL — the narrow platform-admin/standard-user permission
  behavior and X-Role spoof rejection were exercised; the planned model/provider
  admin operation and complete role/operation matrix remain unverified.
- TC_AUTHZ_003: historical PASS only — the existing expiry regression was
  recorded earlier; it was not separately rerun in run 20261020_141030.
===== SPRINT FILE: LATEST ADDENDUM =====
187:FIN-15 — FRESH QUALITY VERIFICATION ADDENDUM (2026-10-21)
188-Database: dedicated local PostgreSQL database finsight_fin15_test.
189-Migrations 0001_auth_identity.sql and 0002_platform_authorization.sql applied.
190-
191-Fresh observed results:
192-- Focused identity/authorization integration tests: 5 passed.
193-- Full backend suite: 58 passed, 0 failed, 18 warnings.
194-- Full suite with coverage: 58 passed; 99% total coverage.
195-- pip check, compileall, and git diff --check passed.
196-- Traceability validator passed.
197-
198-These results supersede neither the historical 2026-10-20 run record nor
199-the individual acceptance criteria. TC_AUTHZ_001 and TC_AUTHZ_002 remain
200-PARTIAL; TC_AUTHZ_003 is historical PASS only; TC_PRIV_002 is DEFERRED /
201-NOT RUN under the approved platform-only scope; TC_SYS_001 remains
202-NOT RUN for the full planned system acceptance workflow.
203-
204-Overall FIN-15 / FS-01-02 status: PARTIAL, not accepted or VERIFIED COMPLETE.
205-Workspace/resource authorization remains deferred. FIN-14 is unchanged.
===== TRACEABILITY FILE: LATEST ADDENDUM =====
74:FIN-15 — FRESH QUALITY VERIFICATION ADDENDUM (2026-10-21)
75-Database: dedicated local PostgreSQL database finsight_fin15_test.
76-Migrations 0001_auth_identity.sql and 0002_platform_authorization.sql applied.
77-
78-Fresh observed results:

79-- Focused identity/authorization integration tests: 5 passed.
80-- Full backend suite: 58 passed, 0 failed, 18 warnings.
81-- Full suite with coverage: 58 passed; 99% total coverage.
82-- pip check, compileall, and git diff --check passed.
83-- Traceability validator passed.
84-
85-These results supersede neither the historical 2026-10-20 run record nor
86-the individual acceptance criteria. TC_AUTHZ_001 and TC_AUTHZ_002 remain
87-PARTIAL; TC_AUTHZ_003 is historical PASS only; TC_PRIV_002 is DEFERRED /
88-NOT RUN under the approved platform-only scope; TC_SYS_001 remains
89-NOT RUN for the full planned system acceptance workflow.
90-
91-Overall FIN-15 / FS-01-02 status: PARTIAL, not accepted or VERIFIED COMPLETE.
92-Workspace/resource authorization remains deferred. FIN-14 is unchanged.
===== FILE ENDINGS =====
- Full suite with coverage: 58 passed; 99% total coverage.
- pip check, compileall, and git diff --check passed.
- Traceability validator passed.

These results supersede neither the historical 2026-10-20 run record nor
the individual acceptance criteria. TC_AUTHZ_001 and TC_AUTHZ_002 remain
PARTIAL; TC_AUTHZ_003 is historical PASS only; TC_PRIV_002 is DEFERRED /
NOT RUN under the approved platform-only scope; TC_SYS_001 remains
NOT RUN for the full planned system acceptance workflow.

Overall FIN-15 / FS-01-02 status: PARTIAL, not accepted or VERIFIED COMPLETE.
Workspace/resource authorization remains deferred. FIN-14 is unchanged.
- Full suite with coverage: 58 passed; 99% total coverage.
- pip check, compileall, and git diff --check passed.
- Traceability validator passed.

These results supersede neither the historical 2026-10-20 run record nor
the individual acceptance criteria. TC_AUTHZ_001 and TC_AUTHZ_002 remain
PARTIAL; TC_AUTHZ_003 is historical PASS only; TC_PRIV_002 is DEFERRED /
NOT RUN under the approved platform-only scope; TC_SYS_001 remains
NOT RUN for the full planned system acceptance workflow.

Overall FIN-15 / FS-01-02 status: PARTIAL, not accepted or VERIFIED COMPLETE.
Workspace/resource authorization remains deferred. FIN-14 is unchanged.
===== VALIDATOR =====
SRS requirement IDs: 409; source story aliases: 72; Sprint 1 source tests: 54
Requirements with Jira/test-plan story links: 328; unmapped: 81; no linked Sprint 1 test: 305
Source tests linked to a catalog story: 54/54
Sprint 1 stories: 23; with SRS IDs: 22; process-story exception: FS-11-09

```
Tests lacking explicit execution rows (source default NOT RUN): TC_ENV_001, TC_ENV_002
Jira key shape and dated live-query marker checked; this offline script does not query Jira.
VALIDATION PASS: inventories, test references, evidence fields, and Release 2 labels are
consistent.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % python - <<'PY'
from pathlib import Path

path = Path("SPRINT_1_TEST_EXECUTION_STATUS.txt")
text = path.read_text(encoding="utf-8")

broken = (
    "These results supersede earlier NOT RUN notes for the test execution performed\n"
    "in this run. They do not establish that every mapped test-plan case or the entire\n"
    "Historical disposition from the 2026-10-20 execution record: FIN-15 was recorded as
complete at that point. The subsequent reconciliation below supersedes that disposition.
Workspace-level authorization remains deferred."
)

fixed = (
    "These results supersede earlier NOT RUN notes for the test execution performed\n"
    "in this run. They do not establish that every mapped test-plan case or the entire\n"
    "FIN-15 story is complete. Workspace-level authorization remains deferred.\n"
    "Historical disposition note: FIN-15 was recorded as complete in an earlier entry;\n"
    "the subsequent reconciliation below supersedes that disposition."
)

if broken not in text:
    raise SystemExit(
        "STOP: exact malformed paragraph not found. No changes made."
    )

path.write_text(text.replace(broken, fixed, 1), encoding="utf-8")
print("Repaired the historical paragraph; other entries preserved.")
PY
Repaired the historical paragraph; other entries preserved.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % python
scripts/validate_traceability.py
git diff --check

echo "===== REPAIRED HISTORICAL SECTION ====="
sed -n '145,165p' SPRINT_1_TEST_EXECUTION_STATUS.txt

echo "===== CURRENT FIN-15 DISPOSITION ====="
```

tail -n 25 SPRINT_1_TEST_EXECUTION_STATUS.txt
tail -n 25 FUNCTION_TEST_TRACEABILITY.txt
SRS requirement IDs: 409; source story aliases: 72; Sprint 1 source tests: 54
Requirements with Jira/test-plan story links: 328; unmapped: 81; no linked Sprint 1 test: 305
Source tests linked to a catalog story: 54/54
Sprint 1 stories: 23; with SRS IDs: 22; process-story exception: FS-11-09
Tests lacking explicit execution rows (source default NOT RUN): TC_ENV_001, TC_ENV_002
Jira key shape and dated live-query marker checked; this offline script does not query Jira.
VALIDATION PASS: inventories, test references, evidence fields, and Release 2 labels are consistent.
===== REPAIRED HISTORICAL SECTION =====

Results:
- Identity/authorization integration suite: 3 passed, 4 warnings; exit code 0.
- Full API suite: 55 passed, 10 warnings; exit code 0.
- Evidence files:
  test-results/FIN-15/20261020_141030_identity_authorization.log
  test-results/FIN-15/20261020_141030_full_api_suite.log
  test-results/FIN-15/run_20261020_141030_summary.txt

These results supersede earlier NOT RUN notes for the test execution performed in this run. They do not establish that every mapped test-plan case or the entire FIN-15 story is complete. Workspace-level authorization remains deferred.
Historical disposition note: FIN-15 was recorded as complete in an earlier entry; the subsequent reconciliation below supersedes that disposition.

FIN-15 TEST-STATUS RECONCILIATION — 2026-10-20 17:38 IST
Recorded run: 20261020_141030. This addendum does not claim a new execution.
- Identity/authorization integration suite: 3 passed, 4 warnings; exit code 0.
- Full API suite: 55 passed, 10 warnings; exit code 0.
- Evidence paths:
===== CURRENT FIN-15 DISPOSITION =====
FIN-15 ROUTE RECONCILIATION — 2026-10-20
The current route is GET /api/v1/admin/authorization/capabilities, verified against apps/api/app/main.py and the matching test request paths. Older route references above are historical records; preserve them as such.

FIN-15 — FRESH QUALITY VERIFICATION ADDENDUM (2026-10-21)
Database: dedicated local PostgreSQL database finsight_fin15_test.
Migrations 0001_auth_identity.sql and 0002_platform_authorization.sql applied.

Fresh observed results:

- Focused identity/authorization integration tests: 5 passed.
- Full backend suite: 58 passed, 0 failed, 18 warnings.
- Full suite with coverage: 58 passed; 99% total coverage.
- pip check, compileall, and git diff --check passed.
- Traceability validator passed.

These results supersede neither the historical 2026-10-20 run record nor
the individual acceptance criteria. TC_AUTHZ_001 and TC_AUTHZ_002 remain
PARTIAL; TC_AUTHZ_003 is historical PASS only; TC_PRIV_002 is DEFERRED /
NOT RUN under the approved platform-only scope; TC_SYS_001 remains
NOT RUN for the full planned system acceptance workflow.

Overall FIN-15 / FS-01-02 status: PARTIAL, not accepted or VERIFIED COMPLETE.
Workspace/resource authorization remains deferred. FIN-14 is unchanged.

ROUTE VERIFICATION — 2026-10-20
Canonical current route: GET /api/v1/admin/authorization/capabilities.
Verified against apps/api/app/main.py and the matching test request paths.

FIN-15 — FRESH QUALITY VERIFICATION ADDENDUM (2026-10-21)
Database: dedicated local PostgreSQL database finsight_fin15_test.
Migrations 0001_auth_identity.sql and 0002_platform_authorization.sql applied.

Fresh observed results:
- Focused identity/authorization integration tests: 5 passed.
- Full backend suite: 58 passed, 0 failed, 18 warnings.
- Full suite with coverage: 58 passed; 99% total coverage.
- pip check, compileall, and git diff --check passed.
- Traceability validator passed.

These results supersede neither the historical 2026-10-20 run record nor
the individual acceptance criteria. TC_AUTHZ_001 and TC_AUTHZ_002 remain
PARTIAL; TC_AUTHZ_003 is historical PASS only; TC_PRIV_002 is DEFERRED /
NOT RUN under the approved platform-only scope; TC_SYS_001 remains
NOT RUN for the full planned system acceptance workflow.

Overall FIN-15 / FS-01-02 status: PARTIAL, not accepted or VERIFIED COMPLETE.
Workspace/resource authorization remains deferred. FIN-14 is unchanged.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
rred. FIN-14 is unchanged.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== CHECK
ADDENDUM COUNTS ====="
grep -c "FRESH QUALITY VERIFICATION ADDENDUM (2026-10-21)" \

```
  SPRINT_1_TEST_EXECUTION_STATUS.txt
grep -c "FRESH QUALITY VERIFICATION ADDENDUM (2026-10-21)" \
  FUNCTION_TEST_TRACEABILITY.txt

echo "===== CHECK FINAL WORKING TREE ====="
git status --short
git diff --check
python scripts/validate_traceability.py
===== CHECK ADDENDUM COUNTS =====
1
1
===== CHECK FINAL WORKING TREE =====
 M .github/workflows/ci.yml
 M IMPLEMENTATION_STATUS.txt
 M README.md
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/app/provision_user.py
 M apps/api/tests/test_auth_unit.py
 M apps/api/tests/test_identity_constraints.py
 M apps/api/tests/test_provision_user.py
 M apps/web/tests/ui/auth.spec.js
 M conversation.txt
 M docs/architecture/README.md
 M docs/requirements/traceability-requirements.csv
 M docs/requirements/traceability-stories.csv
 M scripts/package_release.py
?? FUNCTION_TEST_TRACEABILITY.txt
?? FinSight_AI_API_Design_Pack/
?? apps/api/migrations/0002_platform_authorization.sql
?? apps/web/proxy.js
?? update_fin15_tracking.py
SRS requirement IDs: 409; source story aliases: 72; Sprint 1 source tests: 54
Requirements with Jira/test-plan story links: 328; unmapped: 81; no linked Sprint 1 test: 305
Source tests linked to a catalog story: 54/54
Sprint 1 stories: 23; with SRS IDs: 22; process-story exception: FS-11-09
Tests lacking explicit execution rows (source default NOT RUN): TC_ENV_001, TC_ENV_002
Jira key shape and dated live-query marker checked; this offline script does not query Jira.
VALIDATION PASS: inventories, test references, evidence fields, and Release 2 labels are
consistent.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
ight_AI % echo "===== API DESIGN PACK ====="
ls -lah FinSight_AI_API_Design_Pack/
```

```
echo "===== AUTHORIZATION ENDPOINT REFERENCES ====="
grep -RniE \
  "admin/authorization/capabilities|platform/admin/capability" \
  FinSight_AI_API_Design_Pack/

echo "===== API DESIGN PACK GIT STATUS ====="
git status --short -- FinSight_AI_API_Design_Pack/
===== API DESIGN PACK =====
total 2144
drwx------@  9 sandeeph  staff   288B Oct 20 14:29 .
drwxr-xr-x@ 30 sandeeph  staff   960B Oct 21 13:59 ..
-rw-r--r--@  1 sandeeph  staff    67K Oct 20 14:20
FinSight_AI_API_Design_and_Implementation_Contract.docx
-rw-r--r--@  1 sandeeph  staff    58K Oct 20 14:18
FinSight_AI_API_Design_and_Implementation_Contract.md
-rw-r--r--@  1 sandeeph  staff   126K Oct 20 14:18 FinSight_AI_API_Endpoint_Catalog.csv
-rw-r--r--@  1 sandeeph  staff   266K Oct 20 14:18
FinSight_AI_API_Requirement_Traceability.csv
-rw-r--r--@  1 sandeeph  staff    41K Oct 20 14:18 FinSight_AI_API_Story_Coverage.csv
-rw-r--r--@  1 sandeeph  staff   498K Oct 20 14:18 FinSight_AI_OpenAPI_3.1.yaml
-rw-r--r--@  1 sandeeph  staff   2.2K Oct 20 14:10 README.md
===== AUTHORIZATION ENDPOINT REFERENCES =====
FinSight_AI_API_Design_Pack/FinSight_AI_OpenAPI_3.1.yaml:1199:
/admin/authorization/capabilities:
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:11:
**Decision A is recommended and incorporated.** Implement one minimal platform-admin-only
authorization/capability endpoint in the actual FastAPI app: `GET
/api/v1/admin/authorization/capabilities`, protected by the real
`require_permission("platform.admin")` dependency. It must return only the caller’s effective
permission/capability summary and policy version, not platform user lists or unrelated business
resources. The route is a direct test seam forSRS backend RBAC (FR-P1-05/06/07) and Sprint
1 story FS-01-02 / Jira FIN-15; it is not a substitute for implementing authorization across all
protected endpoints.
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:12
4:**GET `/api/v1/admin/authorization/capabilities`** | Required permission: `platform.admin` |
Release: Sprint 1 authorization foundation | Must be wired through the real FastAPI
dependency chain, not a test-only helper.
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:24
0:| GET | `/api/v1/admin/authorization/capabilities` | Inspect authorized platform capabilities |
platform.admin | — | SPRINT 1 PROPOSED — DECISION A |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:58
0:| 4. Decision A endpoint | Add `GET /api/v1/admin/authorization/capabilities` via actual router
and dependency; return small DTO only. | Admin 200, non-admin 403,
```

```
anonymous/expired/revoked 401, DB/RBAC unavailable 503; permission revocation and no
secret leakage. |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Endpoint_Catalog.csv:16:GET,/api/v1/admin/a
uthorization/capabilities,Identity & access,Inspect authorized platform
capabilities,platform.admin,Authenticated; require permission `platform.admin`,FS-01-01;
FS-01-02; FS-01-03; FS-01-04; FS-01-05; FS-09-01; FS-09-02; FS-09-03; FS-09-04;
FS-09-05,FR-P1-01; FR-P1-02; FR-P1-03; FR-P1-04; FR-P1-05; FR-P1-06; FR-P1-07;
FR-P1-08; FR-P1-09; FR-P1-10; FR-P1-11; FR-P1-12; FR-P1-13; FR-P1-14; FR-P1-15;
FR-P9-01; FR-P9-02; FR-P9-03; FR-P9-04; FR-P9-05; FR-P9-06; FR-P9-07; FR-P9-08;
FR-P9-09; FR-P9-10; FR-P9-11; FR-P9-12; FR-P9-13; FR-P9-14; FR-P9-15; FR-P9-16;
FR-P9-17; FR-P9-18; FR-P9-19; FR-P9-20; FR-P9-21; FR-P9-22; FR-P9-25; NFR-P1-01;
NFR-P1-02; NFR-P1-03; NFR-P1-04; NFR-P1-05; NFR-P1-06; NFR-P9-01; NFR-P9-02;
NFR-P9-03; NFR-P9-04; NFR-P9-05; NFR-P9-06; NFR-P9-07,Release 1
target,false,false,SPRINT 1 PROPOSED — DECISION A,"Decision A: narrowly scoped
real-app route protected by require_permission(""platform.admin""); no user listing or unrelated
business operations."
===== API DESIGN PACK GIT STATUS =====
?? FinSight_AI_API_Design_Pack/
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
? FinSight_AI_API_Design_Pack/
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd FinSight_AI

echo "===== API DESIGN PACK README ====="
cat FinSight_AI_API_Design_Pack/README.md

echo "===== ENDPOINT CATALOG HEADER AND AUTHORIZATION ROW ====="
python - <<'PY'
import csv
from pathlib import Path

p = Path("FinSight_AI_API_Design_Pack/FinSight_AI_API_Endpoint_Catalog.csv")
with p.open(encoding="utf-8-sig", newline="") as f:
    rows = list(csv.DictReader(f))

print("Columns:", list(rows[0].keys()) if rows else "EMPTY")
for row in rows:
    if "authorization/capabilities" in str(row):
        print(row)
PY

echo "===== OPENAPI AUTHORIZATION PATH ====="
grep -n -A 100 -B 5 \
  "/admin/authorization/capabilities:" \
```

FinSight_AI_API_Design_Pack/FinSight_AI_OpenAPI_3.1.yaml

echo "===== IMPLEMENTATION RESPONSE SHAPE ====="
sed -n '180,215p' apps/api/app/main.py
cd: no such file or directory: FinSight_AI
===== API DESIGN PACK README =====
FinSight AI API Design Pack

This pack is the proposed developer-facing API target contract based on the uploaded FinSight AI source documents. It distinguishes source-backed current behavior from proposed endpoints and preserves source release labels and implementation/test statuses.

Files

- `FinSight_AI_API_Design_and_Implementation_Contract.md` — full specification with conventions, RBAC, decision A, security/test matrix, domain contracts, complete endpoint inventory, implementation sequence and source boundaries.
- `FinSight_AI_API_Design_and_Implementation_Contract.docx` — formatted Word copy of the specification.
- `FinSight_AI_OpenAPI_3.1.yaml` — machine-readable proposed OpenAPI 3.1 contract for the Release 1 target. Request/response examples are explicitly illustrative where the final Pydantic DTO is not yet implemented.
- `FinSight_AI_API_Endpoint_Catalog.csv` — searchable catalog of every proposed operation, permission, story/requirement linkage, async/idempotency and contract status.
- `FinSight_AI_API_Story_Coverage.csv` — all 72 source story aliases with source Jira key, release, acceptance criteria, tests, implementation status and gaps preserved.
- `FinSight_AI_API_Requirement_Traceability.csv` — all 409 requirement/data-integrity rows from the source traceability matrix, retaining source statuses and test execution, with an added proposed API family mapping.

Critical guardrails

1. The original SRS remains authoritative. Source schema/ERD and diagrams are target-state, not proof all tables or routes exist in the runtime.
2. Decision A is included: add a narrow platform-admin-only capability endpoint using `require_permission("platform.admin")`, tested against real database-backed sessions.
3. Release 1 has no broker/order/paper-trading endpoints. Stories FS-07-04 / FIN-54 and FS-08-04 / FIN-60 remain Release 2 only; do not apply the optional trading SQL.
4. Proposed route names are not claims of implementation. Inspect the actual Sprint 1 FastAPI router and existing login/session paths before compatibility or migration work.
5. Traceability is not scope approval. The source RTM has unmapped requirement rows; the product owner must disposition them rather than silently infer or omit them.
===== ENDPOINT CATALOG HEADER AND AUTHORIZATION ROW =====

Columns: ['method', 'path', 'tag', 'summary', 'required_permission', 'auth_policy', 'source_stories', 'source_requirement_ids', 'release_scope', 'async_operation', 'idempotency_key_required', 'contract_status', 'design_notes']
{'method': 'GET', 'path': '/api/v1/admin/authorization/capabilities', 'tag': 'Identity & access', 'summary': 'Inspect authorized platform capabilities', 'required_permission': 'platform.admin', 'auth_policy': 'Authenticated; require permission `platform.admin`', 'source_stories': 'FS-01-01; FS-01-02; FS-01-03; FS-01-04; FS-01-05; FS-09-01; FS-09-02; FS-09-03;FS-09-04; FS-09-05', 'source_requirement_ids': 'FR-P1-01; FR-P1-02; FR-P1-03; FR-P1-04; FR-P1-05; FR-P1-06; FR-P1-07; FR-P1-08; FR-P1-09; FR-P1-10; FR-P1-11; FR-P1-12; FR-P1-13; FR-P1-14; FR-P1-15; FR-P9-01; FR-P9-02; FR-P9-03; FR-P9-04; FR-P9-05; FR-P9-06; FR-P9-07; FR-P9-08; FR-P9-09; FR-P9-10; FR-P9-11; FR-P9-12; FR-P9-13; FR-P9-14; FR-P9-15; FR-P9-16; FR-P9-17; FR-P9-18; FR-P9-19; FR-P9-20; FR-P9-21; FR-P9-22; FR-P9-25; NFR-P1-01; NFR-P1-02; NFR-P1-03; NFR-P1-04; NFR-P1-05; NFR-P1-06; NFR-P9-01; NFR-P9-02; NFR-P9-03; NFR-P9-04; NFR-P9-05; NFR-P9-06; NFR-P9-07', 'release_scope': 'Release 1 target', 'async_operation': 'false', 'idempotency_key_required': 'false', 'contract_status': 'SPRINT 1 PROPOSED — DECISION A', 'design_notes': 'Decision A: narrowly scoped real-app route protected by require_permission("platform.admin"); no user listing or unrelated business operations.'}
===== OPENAPI AUTHORIZATION PATH =====
1194-   additionalProperties: false
1195-   description: Illustrative request schema. Validate exact fields, enums and constraints against the SRS before
1196-   implementing.
1197-   example:
1198-   display_name: Analyst
1199: /admin/authorization/capabilities:
1200-   get:
1201-   operationId: get_admin_authorization_capabilities
1202-   tags:
1203-   - Identity & access
1204-   summary: Inspect authorized platform capabilities
1205-   description: 'Decision A: narrowly scoped real-app route protected by require_permission("platform.admin"); no user
1206-   listing or unrelated business operations.'
1207-   x-contract-status: SPRINT 1 PROPOSED — DECISION A
1208-   x-required-permission: platform.admin
1209-   x-source-stories:
1210-   - FS-01-01
1211-   - FS-01-02
1212-   - FS-01-03
1213-   - FS-01-04
1214-   - FS-01-05
1215-   - FS-09-01
1216-   - FS-09-02

```
1217-    - FS-09-03
1218-    - FS-09-04
1219-    - FS-09-05
1220-    x-source-requirement-ids:
1221-    - FR-P1-01
1222-    - FR-P1-02
1223-    - FR-P1-03
1224-    - FR-P1-04
1225-    - FR-P1-05
1226-    - FR-P1-06
1227-    - FR-P1-07
1228-    - FR-P1-08
1229-    - FR-P1-09
1230-    - FR-P1-10
1231-    - FR-P1-11
1232-    - FR-P1-12
1233-    - FR-P1-13
1234-    - FR-P1-14
1235-    - FR-P1-15
1236-    - FR-P9-01
1237-    - FR-P9-02
1238-    - FR-P9-03
1239-    - FR-P9-04
1240-    - FR-P9-05
1241-    - FR-P9-06
1242-    - FR-P9-07
1243-    - FR-P9-08
1244-    - FR-P9-09
1245-    - FR-P9-10
1246-    - FR-P9-11
1247-    - FR-P9-12
1248-    - FR-P9-13
1249-    - FR-P9-14
1250-    - FR-P9-15
1251-    - FR-P9-16
1252-    - FR-P9-17
1253-    - FR-P9-18
1254-    - FR-P9-19
1255-    - FR-P9-20
1256-    - FR-P9-21
1257-    - FR-P9-22
1258-    - FR-P9-25
1259-    - NFR-P1-01
1260-    - NFR-P1-02
```

```
1261-      - NFR-P1-03
1262-      - NFR-P1-04
1263-      - NFR-P1-05
1264-      - NFR-P1-06
1265-      - NFR-P9-01
1266-      - NFR-P9-02
1267-      - NFR-P9-03
1268-      - NFR-P9-04
1269-      - NFR-P9-05
1270-      - NFR-P9-06
1271-      - NFR-P9-07
1272-    x-release-scope: Release 1 target
1273-    responses:
1274-      '200':
1275-        description: Successful response; operation-specific schema must be finalized from
its Pydantic DTO
1276-        content:
1277-          application/json:
1278-            schema:
1279-              $ref: '#/components/schemas/SuccessEnvelope'
1280-      '400':
1281-        $ref: '#/components/responses/BadRequest'
1282-      '401':
1283-        $ref: '#/components/responses/Unauthorized'
1284-      '403':
1285-        $ref: '#/components/responses/Forbidden'
1286-      '404':
1287-        $ref: '#/components/responses/NotFound'
1288-      '409':
1289-        $ref: '#/components/responses/Conflict'
1290-      '422':
1291-        $ref: '#/components/responses/ValidationError'
1292-      '429':
1293-        $ref: '#/components/responses/RateLimited'
1294-      '500':
1295-        $ref: '#/components/responses/InternalError'
1296-      '503':
1297-        $ref: '#/components/responses/DependencyUnavailable'
1298-    security:
1299-    - cookieSession: []
===== IMPLEMENTATION RESPONSE SHAPE =====
def who_am_i(user: CurrentUser) -> UserView:
    return UserView(**user)
```

```
@app.get("/api/v1/admin/authorization/capabilities")
def platform_admin_capabilities(
    request: Request,
    _authorized: Annotated[dict[str, Any], Depends(require_permission("platform.admin"))],
) -> dict[str, Any]:
    """Return the authenticated caller's database-backed platform capabilities."""

    supplied_request_id = request.headers.get("request_id", "").strip()
    if supplied_request_id and (
        len(supplied_request_id) > 128
        or any(ord(char) < 32 or ord(char) == 127 for char in supplied_request_id)
    ):
        raise HTTPException(status_code=400, detail="Invalid request_id")

    # Always issue a server-generated canonical request ID.
    request_id = f"req_{secrets.token_urlsafe(12)}"
    correlation_id = supplied_request_id or request_id
    permissions = sorted(set(_authorized.get("permissions", [])))
    capabilities = (
        ["inspect_authorization_policy"]
        if "platform.admin" in permissions
        else []
    )

    return {
        "data": {
            "subject": {
                "user_id": str(_authorized["user_id"]),
                "platform_role_code": _authorized.get("role_key"),
            },
            "effective_permissions": permissions,
            "capabilities": capabilities,
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

            "capabilities": capabilities,
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== COMPLETE
IMPLEMENTED ENDPOINT ====="
sed -n '184,240p' apps/api/app/main.py

echo "===== OPENAPI SCHEMA DEFINITIONS ====="
grep -n -A 45 -B 5 \
  "SuccessEnvelope:" \
  FinSight_AI_API_Design_Pack/FinSight_AI_OpenAPI_3.1.yaml
```

```
echo "===== API CONTRACT STATUS REFERENCES ====="
grep -n -E \
  "PROPOSED|IMPLEMENTED|DECISION A|authorization/capabilities" \
  FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md \
  FinSight_AI_API_Design_Pack/README.md
===== COMPLETE IMPLEMENTED ENDPOINT =====
@app.get("/api/v1/admin/authorization/capabilities")
def platform_admin_capabilities(
    request: Request,
    _authorized: Annotated[dict[str, Any], Depends(require_permission("platform.admin"))],
) -> dict[str, Any]:
    """Return the authenticated caller's database-backed platform capabilities."""

    supplied_request_id = request.headers.get("request_id", "").strip()
    if supplied_request_id and (
        len(supplied_request_id) > 128
        or any(ord(char) < 32 or ord(char) == 127 for char in supplied_request_id)
    ):
        raise HTTPException(status_code=400, detail="Invalid request_id")

    # Always issue a server-generated canonical request ID.
    request_id = f"req_{secrets.token_urlsafe(12)}"
    correlation_id = supplied_request_id or request_id
    permissions = sorted(set(_authorized.get("permissions", [])))
    capabilities = (
        ["inspect_authorization_policy"]
        if "platform.admin" in permissions
        else []
    )

    return {
        "data": {
            "subject": {
                "user_id": str(_authorized["user_id"]),
                "platform_role_code": _authorized.get("role_key"),
            },
            "effective_permissions": permissions,
            "capabilities": capabilities,
            "policy_version": "sprint-1-platform-rbac-v1",
        },
        "request_id": request_id,
        "correlation_id": correlation_id,
    }
```

```
===== OPENAPI SCHEMA DEFINITIONS =====
18758-      type: http
18759-      scheme: bearer
18760-      bearerFormat: opaque token
18761-      description: Use only if selected deployment policy uses bearer credentials; never
store tokens in ordinary logs.
18762-  schemas:
18763:    SuccessEnvelope:
18764-      type: object
18765-      required:
18766-      - data
18767-      - request_id
18768-      properties:
18769-        data:
18770-          type: object
18771-          additionalProperties: true
18772-        request_id:
18773-          type: string
18774-        correlation_id:
18775-          type: string
18776-    ErrorResponse:
18777-      type: object
18778-      required:
18779-      - error
18780-      - request_id
18781-      properties:
18782-        error:
18783-          type: object
18784-          required:
18785-          - code
18786-          - message
18787-          properties:
18788-            code:
18789-              type: string
18790-            message:
18791-              type: string
18792-            field_errors:
18793-              type: array
18794-              items:
18795-                type: object
18796-                properties:
18797-                  field:
18798-                    type: string
18799-                  code:
```

18800-   type: string
18801-   message:
18802-   type: string
18803-   details:
18804-   type: object
18805-   additionalProperties: true
18806-   request_id:
18807-   type: string
18808-   correlation_id:
===== API CONTRACT STATUS REFERENCES =====
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:11:**Decision A is recommended and incorporated.** Implement one minimal platform-admin-only authorization/capability endpoint in the actual FastAPI app: `GET /api/v1/admin/authorization/capabilities`, protected by the real `require_permission("platform.admin")` dependency. It must return only the caller's effective permission/capability summary and policy version, not platform user lists or unrelated business resources. The route is a direct test seam forSRS backend RBAC (FR-P1-05/06/07) and Sprint 1 story FS-01-02 / Jira FIN-15; it is not a substitute for implementing authorization across all protected endpoints.
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:124:**GET `/api/v1/admin/authorization/capabilities`** | Required permission: `platform.admin` | Release: Sprint 1 authorization foundation | Must be wired through the real FastAPI dependency chain, not a test-only helper.
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:200:@admin_router.get("/authorization/capabilities", response_model=CapabilitiesResponse)
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:220:| GET | `/api/v1/health/live` | Liveness probe | Public / rate-limited | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:221:| GET | `/api/v1/health/ready` | Readiness probe | Public / rate-limited | — | PROPOSED TARGETCONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:222:| GET | `/api/v1/meta/version` | Application build and API version | Public / rate-limited | —| PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:232:| POST | `/api/v1/auth/recovery-requests` | Request password recovery | Public / rate-limited | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:233:| POST | `/api/v1/auth/password-resets` | Complete password recovery | Public / rate-limited |Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:234:| GET | `/api/v1/auth/sessions` | List the caller's active sessions | Authenticated + resourceACL | — | PROPOSED TARGET CONTRACT |

FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:235:| DELETE | `/api/v1/auth/sessions/{session_id}` | Terminate one owned session | Authenticated + resource ACL | Idempotent | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:236:| POST | `/api/v1/auth/mfa/challenges` | Begin an MFA challenge | Authenticated + resource ACL| — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:237:| POST | `/api/v1/auth/mfa/verify` | Verify an MFA challenge | Authenticated + resource ACL | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:238:| GET | `/api/v1/users/me` | Read current user profile | Authenticated + resource ACL | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:239:| PATCH | `/api/v1/users/me` | Update current user profile/preferences | Authenticated + resource ACL | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:240:| GET | `/api/v1/admin/authorization/capabilities` | Inspect authorized platform capabilities | platform.admin | — | SPRINT 1 PROPOSED — DECISION A |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:241:| GET | `/api/v1/admin/roles` | List platform roles | platform.roles.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:242:| POST | `/api/v1/admin/role-permissions` | Assign permissions to a role | platform.roles.manage | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:243:| POST | `/api/v1/admin/invitations` | Create an administrator-controlled invitation | platform.users.invite | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:244:| GET | `/api/v1/admin/audit-events` | Search security/audit events | platform.audit.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:251:| GET | `/api/v1/workspaces` | List caller workspaces | Authenticated + resource ACL | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:252:| POST | `/api/v1/workspaces` | Create a workspace | workspace.create | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:253:| GET | `/api/v1/workspaces/{workspace_id}` | Read workspace metadata | workspace.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:254:| PATCH | `/api/v1/workspaces/{workspace_id}` | Update workspace metadata | workspace.manage |— | PROPOSED TARGET CONTRACT |

FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:255:| GET | `/api/v1/workspaces/{workspace_id}/members` | List workspace members | workspace.members.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:256:| POST | `/api/v1/workspaces/{workspace_id}/members` | Add/invite a workspace member | workspace.members.manage | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:257:| PATCH | `/api/v1/workspaces/{workspace_id}/members/{user_id}` | Change a member's workspace role | workspace.members.manage | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:258:| DELETE | `/api/v1/workspaces/{workspace_id}/members/{user_id}` | Remove a workspace member |workspace.members.manage | Idempotent | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:259:| GET | `/api/v1/saved-research` | List saved research artifacts | saved-research.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:260:| POST | `/api/v1/saved-research` | Save a research artifact | saved-research.write | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:261:| GET | `/api/v1/saved-research/{artifact_id}` | Get a saved artifact | saved-research.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:262:| POST | `/api/v1/saved-research/{artifact_id}/duplicate` | Duplicate a saved artifact | saved-research.write | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:263:| DELETE | `/api/v1/saved-research/{artifact_id}` | Delete a saved artifact | saved-research.delete | Idempotent | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:270:| GET | `/api/v1/jobs` | List caller-visible jobs | jobs.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:271:| GET | `/api/v1/jobs/{job_id}` | Get job status and safe diagnostic summary | jobs.read | —|PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:272:| POST | `/api/v1/jobs/{job_id}/retry` | Retry an eligible failed job | jobs.retry | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:273:| POST | `/api/v1/jobs/{job_id}/cancel` | Request job cancellation | jobs.cancel | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:274:| GET | `/api/v1/execution-runs/{run_id}` | Inspect an execution run | runs.read | — | PROPOSED TARGET CONTRACT |

FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:275:| GET | `/api/v1/execution-runs/{run_id}/events` | List run events | runs.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:282:| GET | `/api/v1/dashboard/summary` | Get dashboard summary and freshness | dashboard.read | —| PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:283:| GET | `/api/v1/dashboard/widgets` | List saved dashboard widgets | dashboard.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:284:| PUT | `/api/v1/dashboard/layout` | Save dashboard layout | dashboard.write | Idempotency-Key| PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:285:| GET | `/api/v1/watchlists` | List watchlists | watchlists.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:286:| POST | `/api/v1/watchlists` | Create watchlist | watchlists.write | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:287:| PATCH | `/api/v1/watchlists/{watchlist_id}` | Update watchlist metadata/order | watchlists.write | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:288:| POST | `/api/v1/watchlists/{watchlist_id}/items` | Add instrument to watchlist | watchlists.write | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:289:| DELETE | `/api/v1/watchlists/{watchlist_id}/items/{instrument_id}` | Remove instrument from watchlist | watchlists.write | Idempotent | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:296:| GET | `/api/v1/instruments` | Search instruments | market.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:297:| GET | `/api/v1/instruments/{instrument_id}` | Get instrument profile | market.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:298:| GET | `/api/v1/instruments/{instrument_id}/bars` | Get historical price bars | market.read |— | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:299:| GET | `/api/v1/instruments/{instrument_id}/fundamentals` | Get point-in-time fundamentals | market.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:300:| GET | `/api/v1/instruments/{instrument_id}/corporate-actions` | Get corporate actions | market.read | — | PROPOSED TARGET CONTRACT |

FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:301:| GET | `/api/v1/instruments/compare` | Compare instruments over a common window | market.read| — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:302:| GET | `/api/v1/market/overview` | Get market overview / supported movers | market.read | — |PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:303:| GET | `/api/v1/market/news` | Search licensed/configured news and events | market.news.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:310:| GET | `/api/v1/market-data/providers` | List configured provider capabilities | providers.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:311:| GET | `/api/v1/market-data/source-registry` | List approved data sources | sources.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:312:| POST | `/api/v1/market-data/ingestion-runs` | Start a market-data ingestion run | market-data.ingest | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:313:| GET | `/api/v1/market-data/ingestion-runs` | List ingestion runs | ingestion.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:314:| GET | `/api/v1/market-data/ingestion-runs/{run_id}` | Get ingestion run details | ingestion.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:315:| POST | `/api/v1/market-data/ingestion-runs/{run_id}/reconcile` | Reconcile raw ingestion records | market-data.reconcile | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:316:| GET | `/api/v1/data-snapshots` | List data snapshots | snapshots.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:317:| GET | `/api/v1/data-snapshots/{snapshot_id}` | Get snapshot provenance and quality summary |snapshots.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:318:| GET | `/api/v1/data-quality/rules` | List data quality rules | quality.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:319:| GET | `/api/v1/data-quality/issues` | Query data quality issues | quality.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:320:| POST | `/api/v1/data-quality/issues/{issue_id}/resolve` | Resolve a quality issue | quality.resolve | Idempotency-Key | PROPOSED TARGET CONTRACT |

FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:327:| GET | `/api/v1/data-sources` | List structured data sources | data-sources.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:328:| POST | `/api/v1/data-sources` | Register a structured source | data-sources.manage | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:329:| POST | `/api/v1/data-sources/{source_id}/test` | Test source connectivity safely | data-sources.manage | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:330:| GET | `/api/v1/data-sources/{source_id}/schema` | Discover allowlisted source schema | data-sources.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:331:| POST | `/api/v1/data-queries/validate` | Validate a read-only query or query plan | data-queries.run | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:332:| POST | `/api/v1/data-queries` | Execute a bounded read-only data query | data-queries.run | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:333:| GET | `/api/v1/data-queries/{query_id}` | Get saved query definition | data-queries.read | —| PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:334:| POST | `/api/v1/datasets/imports` | Import a dataset for preview and validation | datasets.import | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:335:| POST | `/api/v1/datasets/{dataset_id}/transformations` | Apply a versioned dataset transformation | datasets.transform | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:336:| GET | `/api/v1/datasets/{dataset_id}/lineage` | Get dataset lineage | datasets.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:337:| POST | `/api/v1/datasets/{dataset_id}/exports` | Export an authorized dataset | datasets.export | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:344:| GET | `/api/v1/research/sources` | List research source connectors | research.sources.read |— | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:345:| POST | `/api/v1/research/sources` | Register a source or research feed | research.sources.manage | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:346:| GET | `/api/v1/documents` | Search/list research documents | documents.read | — | PROPOSED TARGET CONTRACT |

FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:347:| POST | `/api/v1/documents` | Upload/register a document for ingestion | documents.create | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:348:| GET | `/api/v1/documents/{document_id}` | Get document metadata/status | documents.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:349:| PATCH | `/api/v1/documents/{document_id}` | Update document metadata | documents.manage | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:350:| GET | `/api/v1/documents/{document_id}/versions` | List document versions | documents.read |— | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:351:| GET | `/api/v1/documents/{document_id}/chunks` | Preview extracted chunks | documents.read |— | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:352:| POST | `/api/v1/documents/{document_id}/reindex` | Re-process/re-index a document version | documents.reindex | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:353:| POST | `/api/v1/documents/{document_id}/retry-processing` | Retry a recoverable processing failure | documents.reprocess | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:354:| DELETE | `/api/v1/documents/{document_id}` | Delete/archive a document under lifecycle policy | documents.delete | Idempotent | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:355:| POST | `/api/v1/documents/search` | Search documents/chunks with access filters | documents.search | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:356:| GET | `/api/v1/knowledge-indexes` | List knowledge indexes | knowledge-indexes.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:357:| POST | `/api/v1/knowledge-indexes/{index_id}/reconcile` | Reconcile index/chunk/vector consistency | knowledge-indexes.manage | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:358:| POST | `/api/v1/retrieval-runs` | Run authorized retrieval and capture trace | retrieval.run| 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:359:| GET | `/api/v1/retrieval-runs/{retrieval_run_id}` | Get retrieval trace and authorized citations | retrieval.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:360:| GET | `/api/v1/documents/{document_id}/access-grants` | List document access grants | documents.acl.read | — | PROPOSED TARGET CONTRACT |

FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:361:| POST | `/api/v1/documents/{document_id}/access-grants` | Grant document access | documents.acl.manage | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:362:| DELETE | `/api/v1/documents/{document_id}/access-grants/{grant_id}` | Revoke document access| documents.acl.manage | Idempotent | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:369:| POST | `/api/v1/conversations` | Create conversation | conversations.create | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:370:| GET | `/api/v1/conversations` | List caller conversations | conversations.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:371:| GET | `/api/v1/conversations/{conversation_id}` | Get conversation and message history | conversations.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:372:| DELETE | `/api/v1/conversations/{conversation_id}` | Delete/archive conversation | conversations.delete | Idempotent | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:373:| POST | `/api/v1/conversations/{conversation_id}/messages` | Submit an analyst message | conversations.write | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:374:| POST | `/api/v1/conversations/{conversation_id}/messages/{message_id}/regenerate` | Regenerate an answer | conversations.write | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:375:| POST | `/api/v1/conversations/{conversation_id}/generation/stop` | Request generation stop |conversations.write | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:376:| POST | `/api/v1/analysis/financial-questions` | Answer a financial question with source-backed tools | analysis.run | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:377:| GET | `/api/v1/analysis/tools` | List caller-available financial tools | analysis.tools.read| — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:378:| POST | `/api/v1/analysis/tools/{tool_code}/execute` | Execute one allowlisted analytical tool | analysis.tools.execute | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:379:| POST | `/api/v1/analysis/feedback` | Record feedback on an AI answer | analysis.feedback | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:386:| GET | `/api/v1/settings/models` | List model/inference configuration | settings.models.read | — | PROPOSED TARGET CONTRACT |

FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:387:| PATCH | `/api/v1/settings/models` | Update model settings draft | settings.models.manage | —| PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:388:| POST | `/api/v1/settings/models/test` | Test model connectivity/configuration | settings.models.manage | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:389:| POST | `/api/v1/settings/models/activate` | Activate a validated model configuration | settings.models.activate | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:390:| GET | `/api/v1/settings/embeddings` | Get embedding/chunk/retrieval configuration | settings.rag.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:391:| PATCH | `/api/v1/settings/embeddings` | Update RAG configuration draft | settings.rag.manage| — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:392:| POST | `/api/v1/settings/embeddings/test` | Test embedding configuration | settings.rag.manage | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:393:| GET | `/api/v1/settings/providers` | List provider settings safely | settings.providers.read| — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:394:| PATCH | `/api/v1/settings/providers/{provider_id}` | Update provider configuration draft | settings.providers.manage | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:395:| POST | `/api/v1/settings/providers/{provider_id}/test` | Test provider credentials and coverage | settings.providers.manage | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:396:| GET | `/api/v1/settings/config-versions` | List effective configuration versions | settings.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:397:| POST | `/api/v1/settings/config-versions/{version_id}/rollback` | Roll back to a prior safe configuration | settings.manage | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:404:| GET | `/api/v1/portfolios` | List portfolios | portfolios.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:405:| POST | `/api/v1/portfolios` | Create portfolio | portfolios.write | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:406:| GET | `/api/v1/portfolios/{portfolio_id}` | Get portfolio detail | portfolios.read | — | PROPOSED TARGET CONTRACT |

FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:407:| PATCH | `/api/v1/portfolios/{portfolio_id}` | Update portfolio settings | portfolios.write |— | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:408:| POST | `/api/v1/portfolios/{portfolio_id}/transactions` | Record a portfolio transaction | portfolios.transactions.write | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:409:| GET | `/api/v1/portfolios/{portfolio_id}/snapshots` | List portfolio snapshots | portfolios.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:410:| POST | `/api/v1/portfolios/{portfolio_id}/snapshots` | Create portfolio snapshot | portfolios.calculate | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:411:| GET | `/api/v1/portfolios/{portfolio_id}/performance` | Calculate/view portfolio performance| portfolio-analytics.read | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:412:| GET | `/api/v1/portfolios/{portfolio_id}/allocation` | Analyze asset/sector/geographic allocation | portfolio-analytics.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:413:| GET | `/api/v1/portfolios/{portfolio_id}/attribution` | Calculate performance attribution | portfolio-analytics.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:420:| POST | `/api/v1/valuations/dcf-runs` | Run DCF valuation | valuation.run | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:421:| GET | `/api/v1/valuations/dcf-runs/{run_id}` | Get DCF run and result | valuation.read | —|PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:422:| GET | `/api/v1/valuations/dcf-runs/{run_id}/sensitivity` | Get valuation sensitivity matrix | valuation.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:423:| POST | `/api/v1/risk-runs` | Run quantitative portfolio risk analysis | risk.run | 202 async| PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:424:| GET | `/api/v1/risk-runs/{run_id}` | Get risk run and metrics | risk.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:425:| GET | `/api/v1/risk-runs/{run_id}/time-series` | Get risk metric time series | risk.read | —| PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:426:| POST | `/api/v1/risk-runs/{run_id}/stress-tests` | Evaluate configured/custom stress scenarios | risk.run | 202 async | PROPOSED TARGET CONTRACT |

FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:427:| GET | `/api/v1/risk-runs/{run_id}/stress-tests` | Get stress-test results | risk.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:434:| GET | `/api/v1/screens` | List saved screens | screens.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:435:| POST | `/api/v1/screens` | Create saved screen | screens.write | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:436:| POST | `/api/v1/screen-runs` | Run an instrument screen | screens.run | 202 async | PROPOSEDTARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:437:| GET | `/api/v1/screen-runs/{run_id}` | Get screening results and ranking | screens.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:438:| GET | `/api/v1/strategies` | List strategies | strategies.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:439:| POST | `/api/v1/strategies` | Create a strategy | strategies.write | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:440:| POST | `/api/v1/strategies/{strategy_id}/versions` | Create immutable strategy version | strategies.write | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:441:| POST | `/api/v1/strategies/{strategy_id}/readiness-checks` | Check strategy readiness | strategies.run | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:442:| POST | `/api/v1/signals/runs` | Generate signals for a strategy version | signals.run | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:443:| GET | `/api/v1/signals` | List signals and their status | signals.read | — | PROPOSED TARGETCONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:444:| GET | `/api/v1/workflows` | List workflows | workflows.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:445:| POST | `/api/v1/workflows` | Create workflow definition | workflows.write | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:446:| POST | `/api/v1/workflows/{workflow_id}/runs` | Run workflow | workflows.run | 202 async | PROPOSED TARGET CONTRACT |

FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:447:| POST | `/api/v1/workflows/{workflow_id}/schedules` | Create/update workflow schedule | workflows.schedule | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:448:| POST | `/api/v1/backtest-runs` | Start historical backtest | backtests.run | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:449:| GET | `/api/v1/backtest-runs/{run_id}` | Get backtest status and summary | backtests.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:450:| GET | `/api/v1/backtest-runs/{run_id}/trades` | Get simulated backtest trades | backtests.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:451:| GET | `/api/v1/backtest-runs/{run_id}/equity-curve` | Get backtest equity curve and drawdown| backtests.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:452:| POST | `/api/v1/backtest-runs/compare` | Compare authorized backtest runs | backtests.read |Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:459:| GET | `/api/v1/reports` | List reports | reports.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:460:| POST | `/api/v1/reports` | Create report specification | reports.write | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:461:| GET | `/api/v1/reports/{report_id}` | Get report metadata/content status | reports.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:462:| POST | `/api/v1/reports/{report_id}/exports` | Generate PDF/CSV export | reports.export | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:463:| GET | `/api/v1/reports/{report_id}/exports/{export_id}` | Check report export status | reports.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:464:| POST | `/api/v1/reports/{report_id}/access-grants` | Share report with user/workspace | reports.share | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:465:| DELETE | `/api/v1/reports/{report_id}/access-grants/{grant_id}` | Revoke report access | reports.share | Idempotent | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:472:| POST | `/api/v1/privacy/data-exports` | Request personal data export | privacy.export | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:473:| POST | `/api/v1/privacy/deletion-requests` | Request account/data deletion | privacy.delete | 202 async | PROPOSED TARGET CONTRACT |

FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:474:| GET | `/api/v1/privacy/requests/{request_id}` | Check privacy request status | privacy.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:475:| GET | `/api/v1/api-credentials` | List caller API credentials metadata | privacy.credentials.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:476:| POST | `/api/v1/api-credentials` | Create API credential | privacy.credentials.manage | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:477:| DELETE | `/api/v1/api-credentials/{credential_id}` | Revoke API credential | privacy.credentials.manage | Idempotent | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:580:| 4. Decision A endpoint | Add `GET /api/v1/admin/authorization/capabilities` via actual router and dependency; return small DTO only. | Admin 200, non-admin 403, anonymous/expired/revoked 401, DB/RBAC unavailable 503; permission revocation and no secret leakage. |
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % k/FinSight_AI_API_Design_and_Implementation_Contract.md:472:| POST | `/api/v1/privacy/data-exports` | Request personal data export | privacy.export | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:473:| POST | `/api/v1/privacy/deletion-requests` | Request account/data deletion | privacy.delete | 202 async | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:474:| GET | `/api/v1/privacy/requests/{request_id}` | Check privacy request status | privacy.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:475:| GET | `/api/v1/api-credentials` | List caller API credentials metadata | privacy.credentials.read | — | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:476:| POST | `/api/v1/api-credentials` | Create API credential | privacy.credentials.manage | Idempotency-Key | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:477:| DELETE | `/api/v1/api-credentials/{credential_id}` | Revoke API credential | privacy.credentials.manage | Idempotent | PROPOSED TARGET CONTRACT |
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md:580:| 4. Decision A endpoint | Add `GET /api/v1/admin/authorization/capabilities` via actual router and dependency; return small DTO only. | Admin 200, non-admin 403, anonymous/expired/revoked 401, DB/RBAC unavailable 503; permission revocation and no secret leakage. |
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== OPENAPI SERVER URL ====="

```
sed -n '1,80p' FinSight_AI_API_Design_Pack/FinSight_AI_OpenAPI_3.1.yaml

echo "===== MARKDOWN ENDPOINT SECTION ====="
sed -n '110,145p'
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md

echo "===== MARKDOWN DECISION A SECTION ====="
sed -n '570,590p'
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md
===== OPENAPI SERVER URL =====
openapi: 3.1.0
info:
  title: FinSight AI API
  version: 1.0.0-design
  description: Proposed canonical API contract aligned to the FinSight AI SRS, schema/ERD
reference and 72-story backlog.
    Not a claim that proposed route handlers are implemented. Release 1 excludes broker/order
execution.
servers:
- url: /api/v1
  description: Canonical target API base path; reconcile with actual Sprint 1 routes before rollout.
tags:
- name: Platform
- name: Identity & access
- name: Workspace & saved research
- name: Jobs & observability
- name: Dashboard & market data
- name: Market data
- name: Market ingestion & quality
- name: Data explorer
- name: Research & RAG
- name: AI Financial Analyst
- name: Settings & integrations
- name: Portfolio & analytics
- name: Valuation & risk
- name: Strategy & workflows
- name: Reporting
- name: Privacy & account lifecycle
paths:
  /health/live:
    get:
      operationId: get_health_live
      tags:
      - Platform
```

```
    summary: Liveness probe
    description: Process-only check; do not test external dependencies.
    x-contract-status: PROPOSED TARGET CONTRACT
    x-required-permission: null
    x-source-stories:
    - FS-02-08
    - FS-07-05
    - FS-11-12
    x-source-requirement-ids:
    - NFR-G-04
    - NFR-G-06
    - NFR-G-07
    - NFR-G-08
    - NFR-G-11
    - NFR-G-13
    - NFR-G-14
    - NFR-G-15
    - NFR-G-16
    - NFR-G-17
    - NFR-G-23
    - NFR-G-26
    - NFR-G-30
    - NFR-G-31
    - NFR-G-34
    - NFR-G-35
    - NFR-G-36
    - NFR-G-37
    - NFR-G-38
    x-release-scope: Release 1 target
    responses:
      '200':
        description: Successful response; operation-specific schema must be finalized from its
Pydantic DTO
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/SuccessEnvelope'
      '400':
        $ref: '#/components/responses/BadRequest'
      '401':
        $ref: '#/components/responses/Unauthorized'
      '403':
        $ref: '#/components/responses/Forbidden'
      '404':
```

```
        $ref: '#/components/responses/NotFound'
      '409':
        $ref: '#/components/responses/Conflict'
      '422':
        $ref: '#/components/responses/ValidationError'
===== MARKDOWN ENDPOINT SECTION =====
| --- | --- | --- |
| platform.admin | Inspect platform-level capabilities | Keep the minimal Sprint 1 route read-only. Grant only through explicit role_permissions mapping. |
| platform.roles.read / platform.roles.manage | Read/change platform role permission assignments | Separate read from mutation; immutable audit event required. |
| platform.users.invite / platform.audit.read | Invitation and security audit views | Future functionality requiring SRS-backed route/tests. |
| workspace.read / workspace.manage / workspace.members.manage | Workspace metadata and membership | Check workspace membership and role; prevent privilege escalation/orphaning. |
| documents.read / documents.create / documents.delete / documents.acl.manage | Documents and explicit access grants | Evaluate current grant before metadata/chunk/search/citation;revoke immediately for future retrieval. |
| market.read / market-data.ingest / providers.manage | Market data reads/provider operations | Provider management and ingestion restricted to approved operator capability. |
| analysis.run / valuation.run / risk.run / backtests.run | Analytical calculations and simulations | Tool calls authorize their own inputs and data snapshots. |
| reports.read / reports.export / reports.share | Reports/export/sharing | Fresh permission check both before artifact creation and download. |
| privacy.export / privacy.delete / privacy.credentials.manage | Account privacy and credential lifecycle | Step-up/confirmation and audit where required. |

### Minimal capability endpoint — approved design


**GET `/api/v1/admin/authorization/capabilities`** | Required permission: `platform.admin` | Release: Sprint 1 authorization foundation | Must be wired through the real FastAPI dependency chain, not a test-only helper.

Success contract (example):

```json
{
  "data": {
    "subject": {
      "user_id": "<uuid>",
      "platform_role_code": "platform_admin"
    },
```
```

```
    "effective_permissions": [
      "platform.admin",
      "platform.roles.read"
    ],
    "capabilities": [
      "inspect_authorization_policy"
    ],
    "policy_version": "<policy-version>"
  },
  "request_id": "req_01J..."
}
```


===== MARKDOWN DECISION A SECTION =====

| Stale/missing market data | Return available data with freshness/quality disclosure only when contract permits; otherwise safe unavailable/incomplete outcome. |

14. Sprint 1 implementation sequence — avoid mixing work

Step	Concrete deliverable	Acceptance evidence
1. Baseline audit	Inspect actual route declarations, app router mounting/prefixes, auth dependency/session model, migrations applied, tests, cookie/CSRF policy and CI. Record existing URLs before refactor.	Before/after route inventory; migration inventory; run current tests; no scope inference.
2. RBAC model/migration	Verify schema for permission records and role_permissions. Add only a forward migration if missing; seed stable permission codes with explicit mappings in test fixtures and controlled bootstrap process.	Migration up/down strategy; FK/unique/index checks; test DB setup; no edit to existing applied migrations.
3. Authorization dependency	Implement `require_permission(permission_code)` over actual authenticated principal + DB-backed session and role/permission joins. Fail closed on any dependency/DB error.	Unit tests for allow/deny + integration tests with real database-backed sessions.
4. Decision A endpoint	Add `GET /api/v1/admin/authorization/capabilities` via actual router and dependency; return small DTO only.	Admin 200, non-admin 403, anonymous/expired/revoked 401, DB/RBAC unavailable 503; permission revocation and no secret leakage.
5. Regression + security	Verify current `/auth/login`, `/auth/me`, `/auth/logout` behavior and frontend integration; cookie/CSRF tests as applicable; lint/type/test/secret scanning.	Run actual test commands and record outcomes; don’t alter planned test register statuses without evidence.
6. Documentation/traceability	Generate OpenAPI from FastAPI at runtime; compare against this target spec; record implemented-vs-proposed; update the requirement/story links andappend-only execution record.	`/openapi.json` route diff, test evidence, changed files, migration ID and unresolved gaps.

15. Endpoint ownership, permissions and traceability rules

The `FinSight_AI_API_Endpoint_Catalog.csv` includes every target endpoint, status, permission and derived story/requirement mapping. The `FinSight_AI_API_Story_Coverage.csv` preserves all 72 source aliases, Jira keys, acceptance summaries, test execution, implementation status and outstanding gaps from the source matrix. `FinSight_AI_API_Requirement_Traceability.csv` preserves each of the source RTM's requirement/data-integrity rows and original evidence/status/test fields, adding only a proposed API family and explicit contract note.

Route-to-story mapping is a design aid and is not a substitute for acceptance-criterion traceability. For each Jira story, engineers must connect every acceptance criterion to one or more endpoints/services/schema changes/tests and show test evidence. Requirements without mapped source stories remain gaps; do not infer scope or change Jira/SRS baselines from this document.

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % python3 temp.py
Updated endpoint catalog, OpenAPI metadata, Markdown contract, and README.
Other endpoint rows and statuses were preserved.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % git diff --check
python scripts/validate_traceability.py

git diff -- \
  FinSight_AI_API_Design_Pack/FinSight_AI_API_Endpoint_Catalog.csv \
  FinSight_AI_API_Design_Pack/FinSight_AI_OpenAPI_3.1.yaml \
  FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md \
  FinSight_AI_API_Design_Pack/README.md
SRS requirement IDs: 409; source story aliases: 72; Sprint 1 source tests: 54
Requirements with Jira/test-plan story links: 328; unmapped: 81; no linked Sprint 1 test: 305
Source tests linked to a catalog story: 54/54
Sprint 1 stories: 23; with SRS IDs: 22; process-story exception: FS-11-09
Tests lacking explicit execution rows (source default NOT RUN): TC_ENV_001, TC_ENV_002
Jira key shape and dated live-query marker checked; this offline script does not query Jira.
VALIDATION PASS: inventories, test references, evidence fields, and Release 2 labels are consistent.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % git diff --check
python scripts/validate_traceability.py

git diff -- \
  FinSight_AI_API_Design_Pack/FinSight_AI_API_Endpoint_Catalog.csv \

```
  FinSight_AI_API_Design_Pack/FinSight_AI_OpenAPI_3.1.yaml \
  FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md \
  FinSight_AI_API_Design_Pack/README.md
SRS requirement IDs: 409; source story aliases: 72; Sprint 1 source tests: 54
Requirements with Jira/test-plan story links: 328; unmapped: 81; no linked Sprint 1 test: 305
Source tests linked to a catalog story: 54/54
Sprint 1 stories: 23; with SRS IDs: 22; process-story exception: FS-11-09
Tests lacking explicit execution rows (source default NOT RUN): TC_ENV_001, TC_ENV_002
Jira key shape and dated live-query marker checked; this offline script does not query Jira.
VALIDATION PASS: inventories, test references, evidence fields, and Release 2 labels are
consistent.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== GIT STATUS
====="
git status --short -- FinSight_AI_API_Design_Pack

echo "===== ENDPOINT CATALOG ====="
grep -n -F "/api/v1/admin/authorization/capabilities" \
  FinSight_AI_API_Design_Pack/FinSight_AI_API_Endpoint_Catalog.csv

echo "===== OPENAPI CONTRACT STATUS ====="
grep -n -A 12 -B 5 -F "/admin/authorization/capabilities:" \
  FinSight_AI_API_Design_Pack/FinSight_AI_OpenAPI_3.1.yaml

echo "===== MARKDOWN STATUS AND IMPLEMENTATION NOTES ====="
grep -n -E "IMPLEMENTED — FIN-15 PARTIAL|Observed implementation
status|authorization/capabilities" \
  FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md

echo "===== README INTRODUCTION ====="
sed -n '1,35p' FinSight_AI_API_Design_Pack/README.md
===== GIT STATUS =====
?? FinSight_AI_API_Design_Pack/
===== ENDPOINT CATALOG =====
16:GET,/api/v1/admin/authorization/capabilities,Identity & access,Inspect authorized platform
capabilities,platform.admin,Authenticated; require permission `platform.admin`,FS-01-01;
FS-01-02; FS-01-03; FS-01-04; FS-01-05; FS-09-01; FS-09-02; FS-09-03; FS-09-04;
FS-09-05,FR-P1-01; FR-P1-02; FR-P1-03; FR-P1-04; FR-P1-05; FR-P1-06; FR-P1-07;
FR-P1-08; FR-P1-09;FR-P1-10; FR-P1-11; FR-P1-12; FR-P1-13; FR-P1-14; FR-P1-15;
FR-P9-01; FR-P9-02; FR-P9-03; FR-P9-04; FR-P9-05; FR-P9-06; FR-P9-07; FR-P9-08;
FR-P9-09; FR-P9-10; FR-P9-11; FR-P9-12; FR-P9-13; FR-P9-14; FR-P9-15; FR-P9-16;
FR-P9-17; FR-P9-18; FR-P9-19; FR-P9-20; FR-P9-21; FR-P9-22; FR-P9-25; NFR-P1-01;
NFR-P1-02; NFR-P1-03; NFR-P1-04; NFR-P1-05; NFR-P1-06; NFR-P9-01; NFR-P9-02;
NFR-P9-03; NFR-P9-04; NFR-P9-05; NFR-P9-06; NFR-P9-07,Release 1
```

target,false,false,IMPLEMENTED — FIN-15 PARTIAL,"Implemented in apps/api/app/main.py; protected by require_permission(""platform.admin""). Observed response: data.subject, effective_permissions, capabilities, policy_version, request_id, correlation_id. Story FIN-15 remains PARTIAL; broader acceptance criteria remain open."
===== OPENAPI CONTRACT STATUS =====
1194-   additionalProperties: false
1195-   description: Illustrative request schema. Validate exact fields, enums and constraints against the SRS before
1196-   implementing.
1197-   example:
1198-   display_name: Analyst
1199: /admin/authorization/capabilities:
1200-   get:
1201-   operationId: get_admin_authorization_capabilities
1202-   tags:
1203-   - Identity & access
1204-   summary: Inspect authorized platform capabilities
1205-   description: 'Implemented in apps/api/app/main.py and protected by require_permission("platform.admin"). FIN-15 remains PARTIAL; this endpoint does not establish full story acceptance.'
1206-   x-contract-status: IMPLEMENTED — FIN-15 PARTIAL
1207-   x-required-permission: platform.admin
1208-   x-source-stories:
1209-   - FS-01-01
1210-   - FS-01-02
1211-   - FS-01-03
===== MARKDOWN STATUS AND IMPLEMENTATION NOTES =====
11:**Decision A is recommended and incorporated.** Implement one minimal platform-admin-only authorization/capability endpoint in the actual FastAPI app: `GET /api/v1/admin/authorization/capabilities`, protected by the real `require_permission("platform.admin")` dependency. It must return only the caller’s effective permission/capability summary and policy version, not platform user lists or unrelated business resources. The route is a direct test seam for SRS backend RBAC (FR-P1-05/06/07) and Sprint 1 story FS-01-02 / Jira FIN-15; it is not a substitute for implementing authorization across all protected endpoints.
124:#### Observed implementation status — 2026-10-21
127:`GET /api/v1/admin/authorization/capabilities`. It uses
148:**GET `/api/v1/admin/authorization/capabilities`** | Required permission: `platform.admin` | Release: Sprint 1 authorization foundation | Must be wired through the real FastAPI dependency chain, not a test-only helper.
224:@admin_router.get("/authorization/capabilities", response_model=CapabilitiesResponse)
264:| GET | `/api/v1/admin/authorization/capabilities` | Inspect authorized platform capabilities | platform.admin | — | IMPLEMENTED — FIN-15 PARTIAL |

604:| 4. Decision A endpoint | Add `GET /api/v1/admin/authorization/capabilities` via actual router and dependency; return small DTO only. | Admin 200, non-admin 403, anonymous/expired/revoked 401, DB/RBAC unavailable 503; permission revocation and no secret leakage. |
===== README INTRODUCTION =====
FinSight AI API Design Pack

This developer-facing API pack combines the target API contract with implementation status. Most operations remain proposed. GET /api/v1/admin/authorization/capabilities is implemented, but FIN-15 remains PARTIAL and its broader acceptance criteria are not all satisfied. Source release labels and implementation/test statuses are preserved.

Files

- `FinSight_AI_API_Design_and_Implementation_Contract.md` — full specification with conventions, RBAC, decision A, security/test matrix, domain contracts, complete endpoint inventory, implementation sequence and source boundaries.
- `FinSight_AI_API_Design_and_Implementation_Contract.docx` — formatted Word copy of the specification.
- `FinSight_AI_OpenAPI_3.1.yaml` — machine-readable proposed OpenAPI 3.1 contract for the Release 1 target. Request/response examples are explicitly illustrative where the final Pydantic DTO is not yet implemented.
- `FinSight_AI_API_Endpoint_Catalog.csv` — searchable catalog of every proposed operation, permission, story/requirement linkage, async/idempotency and contract status.
- `FinSight_AI_API_Story_Coverage.csv` — all 72 source story aliases with source Jira key, release, acceptance criteria, tests, implementation status and gaps preserved.
- `FinSight_AI_API_Requirement_Traceability.csv` — all 409 requirement/data-integrity rows from the source traceability matrix, retaining source statuses and test execution, with an added proposed API family mapping.

Critical guardrails

1. The original SRS remains authoritative. Source schema/ERD and diagrams are target-state, not proof all tables or routes exist in the runtime.
2. Decision A is included: add a narrow platform-admin-only capability endpoint using `require_permission("platform.admin")`, tested against real database-backed sessions.
3. Release 1 has no broker/order/paper-trading endpoints. Stories FS-07-04 / FIN-54 and FS-08-04 / FIN-60 remain Release 2 only; do not apply the optional trading SQL.
4. Proposed route names are not claims of implementation. Inspect the actual Sprint 1 FastAPI router and existing login/session paths before compatibility or migration work.
5. Traceability is not scope approval. The source RTM has unmapped requirement rows; the product owner must disposition them rather than silently infer or omit them.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
ria are not all satisfied. Source release labels and implementation/test statuses are preserved.

Files

- `FinSight_AI_API_Design_and_Implementation_Contract.md` — full specification with conventions, RBAC, decision A, security/test matrix, domain contracts, complete endpoint inventory, implementation sequence and source boundaries.
- `FinSight_AI_API_Design_and_Implementation_Contract.docx` — formatted Word copy of the specification.
- `FinSight_AI_OpenAPI_3.1.yaml` — machine-readable proposed OpenAPI 3.1 contract for the Release 1 target. Request/response examples are explicitly illustrative where the final Pydantic DTO is not yet implemented.
- `FinSight_AI_API_Endpoint_Catalog.csv` — searchable catalog of every proposed operation, permission, story/requirement linkage, async/idempotency and contract status.
- `FinSight_AI_API_Story_Coverage.csv` — all 72 source story aliases with source Jira key, release, acceptance criteria, tests, implementation status and gaps preserved.
- `FinSight_AI_API_Requirement_Traceability.csv` — all 409 requirement/data-integrity rows from the source traceability matrix, retaining source statuses and test execution, with an added proposed API family mapping.

Critical guardrails

1. The original SRS remains authoritative. Source schema/ERD and diagrams are target-state, not proof all tables or routes exist in the runtime.
2. Decision A is included: add a narrow platform-admin-only capability endpoint using `require_permission("platform.admin")`, tested against real database-backed sessions.
3. Release 1 has no broker/order/paper-trading endpoints. Stories FS-07-04 / FIN-54 and FS-08-04 / FIN-60 remain Release 2 only; do not apply the optional trading SQL.
4. Proposed route names are not claims of implementation. Inspect the actual Sprint 1 FastAPI router and existing login/session paths before compatibility or migration work.
5. Traceability is not scope approval. The source RTM has unmapped requirement rows; the product owner must disposition them rather than silently infer or omit them.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== MARKDOWN OBSERVED IMPLEMENTATION ====="
sed -n '120,155p' \
  FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md

echo "===== MARKDOWN IMPLEMENTATION SEQUENCE ====="
sed -n '590,615p' \
  FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md

echo "===== DOCX GENERATION SCRIPTS ====="
grep -R -n -E 'python-docx|Document\(|FinSight_AI_API_Design_and_Implementation_Contract' \

```
  scripts FinSight_AI_API_Design_Pack/README.md \
  --exclude='*.docx' \
  --exclude='*.yaml' \
  --exclude='*.csv' \
  --exclude='*.md' 2>/dev/null || true

echo "===== WORD DOCUMENT TIMESTAMP ====="
ls -l \
  FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.docx
===== MARKDOWN OBSERVED IMPLEMENTATION =====

### Minimal capability endpoint — approved design


#### Observed implementation status — 2026-10-21

The endpoint is implemented in `apps/api/app/main.py` at
`GET /api/v1/admin/authorization/capabilities`. It uses
`require_permission("platform.admin")`.

The observed success response contains:
- `data.subject.user_id`
- `data.subject.platform_role_code`
- `data.effective_permissions`
- `data.capabilities`
- `data.policy_version`
- top-level `request_id` and `correlation_id`

A supplied `request_id` is rejected with HTTP 400 if it exceeds 128 characters
or contains disallowed control characters. Otherwise it is used as the
`correlation_id`; the server generates the canonical `request_id`.

This endpoint's implementation does not mean the full FIN-15 acceptance
criteria are complete. FIN-15 remains PARTIAL. Workspace/resource authorization
remains deferred under the approved platform-only scope.



**GET `/api/v1/admin/authorization/capabilities`**  |  Required permission: `platform.admin`  |
Release: Sprint 1 authorization foundation  |  Must be wired through the real FastAPI
dependency chain, not a test-only helper.

Success contract (example):
```

```json
{
  "data": {
    "subject": {
===== MARKDOWN IMPLEMENTATION SEQUENCE =====
| Provider/model unavailable | 503 or domain-safe failed job; no silent fabricated data. Return run/job IDs if work was accepted. |
| Validation issue | 422 with field-level safe errors; no stack trace. |
| Duplicate request | Return original idempotent result when request hash matches; otherwise 409. |
| Partial indexing/deletion | Expose nonterminal/failed state, enqueue safe reconciliation, preserve audit and lineage. |
| Stale/missing market data | Return available data with freshness/quality disclosure only when contract permits; otherwise safe unavailable/incomplete outcome. |

## 14. Sprint 1 implementation sequence — avoid mixing work


| Step | Concrete deliverable | Acceptance evidence |
| --- | --- | --- |
| 1. Baseline audit | Inspect actual route declarations, app router mounting/prefixes, auth dependency/session model, migrations applied, tests, cookie/CSRF policy and CI. Record existing URLs before refactor. | Before/after route inventory; migration inventory; run current tests; no scope inference. |
| 2. RBAC model/migration | Verify schema for permission records and role_permissions. Add only a forward migration if missing; seed stable permission codes with explicit mappings in test fixtures and controlled bootstrap process. | Migration up/down strategy; FK/unique/index checks; test DB setup; no edit to existing applied migrations. |
| 3. Authorization dependency | Implement `require_permission(permission_code)` over actual authenticated principal + DB-backed session and role/permission joins. Fail closed on any dependency/DB error. | Unit tests for allow/deny + integration tests with real database-backed sessions. |
| 4. Decision A endpoint | Add `GET /api/v1/admin/authorization/capabilities` via actual router and dependency; return small DTO only. | Admin 200, non-admin 403, anonymous/expired/revoked 401, DB/RBAC unavailable 503; permission revocation and no secret leakage. |
| 5. Regression + security | Verify current `/auth/login`, `/auth/me`, `/auth/logout` behavior and frontend integration; cookie/CSRF tests as applicable; lint/type/test/secret scanning. | Run actual test commands and record outcomes; don’t alter planned test register statuses without evidence. |
| 6. Documentation/traceability | Generate OpenAPI from FastAPI at runtime; compare against this target spec; record implemented-vs-proposed; update the requirement/story links andappend-only execution record. | `/openapi.json` route diff, test evidence, changed files, migration ID and unresolved gaps. |
```

15. Endpoint ownership, permissions and traceability rules

The `FinSight_AI_API_Endpoint_Catalog.csv` includes every target endpoint, status, permission and derived story/requirement mapping. The `FinSight_AI_API_Story_Coverage.csv` preserves all 72 source aliases, Jira keys, acceptance summaries, test execution, implementation status and outstanding gaps from the source matrix. `FinSight_AI_API_Requirement_Traceability.csv` preserves each of the source RTM's requirement/data-integrity rows and original evidence/status/test fields, adding only a proposed API family and explicit contract note.

Route-to-story mapping is a design aid and is not a substitute for acceptance-criterion traceability. For each Jira story, engineers must connect every acceptance criterion to one or more endpoints/services/schema changes/tests and show test evidence. Requirements without mapped source stories remain gaps; do not infer scope or change Jira/SRS baselines from this document.

16. Decisions required before broad implementation
===== DOCX GENERATION SCRIPTS =====
===== WORD DOCUMENT TIMESTAMP =====
-rw-r--r--@ 1 sandeeph staff 68641 Oct 20 14:20 FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.docx
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
ok-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
from zipfile import ZipFile
import xml.etree.ElementTree as ET

path = Path(
    "FinSight_AI_API_Design_Pack/"
    "FinSight_AI_API_Design_and_Implementation_Contract.docx"
)

terms = (
    "approved design",
    "must be wired",
    "add",
    "decision a endpoint",
    "implemented",
    "implementation sequence",
    "authorization/capabilities",
)

with ZipFile(path) as archive:

```
    root = ET.fromstring(archive.read("word/document.xml"))

ns = {"w": "http://schemas.openxmlformats.org/wordprocessingml/2006/main"}

paragraphs = []
for paragraph in root.findall(".//w:body/w:p", ns):
    text = "".join(node.text or "" for node in paragraph.findall(".//w:t", ns))
    if text.strip():
        paragraphs.append(text)

print(f"Read {len(paragraphs)} non-empty paragraphs from {path}")
print("\n--- Matching paragraphs and nearby context ---")

matches = [
    i for i, text in enumerate(paragraphs)
    if any(term in text.lower() for term in terms)
]

shown = set()
for i in matches:
    for j in range(max(0, i - 1), min(len(paragraphs), i + 2)):
        if j not in shown:
            print(f"P{j}: {paragraphs[j]}")
            shown.add(j)
    print("---")
PY
===== CHECK DOCX ARCHIVE =====
    testing: docProps/thumbnail.jpeg   OK
No errors detected in compressed data of
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.docx.
===== EXTRACT WORD TEXT (READ-ONLY) =====
Read 86 non-empty paragraphs from
FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.docx

--- Matching paragraphs and nearby context ---
P3: Executive decision and guardrails
P4: Decision A is recommended and incorporated. Implement one minimal platform-admin-only
authorization/capability endpoint in the actual FastAPI app: GET
/api/v1/admin/authorization/capabilities, protected by the real
require_permission("platform.admin") dependency. It must return only the caller’s effective
permission/capability summary and policy version, not platform user lists or unrelated business
resources. The route is a direct test seam for SRS backend RBAC (FR-P1-05/06/07) and Sprint
1 story FS-01-02 / Jira FIN-15; it is not asubstitute for implementing authorization across all
protected endpoints.
```

P5: Release 1 boundary: historical/read-only financial research and calculations; no broker order, paper-trading execution, broker connection, order/fill/approval or live trading paths in the Release 1 OpenAPI file. FS-07-04 / FIN-54 and FS-08-04 / FIN-60 remain Release 2 only. Do not apply the optional 15-table trading DDL. Internal portfolio ledger transactions are not broker orders and must be explicitly described as such.

P6: Truthfulness: the schema reference contains 158 core and 15 optional tables, but is a target-state design; it is not evidence those migrations have been applied. Source RTM records only a scoped subset of behaviors implemented and verified. This contract does not claim that proposed domains/routes exist in the current runtime.
P7: 1. Source baseline, known runtime and open gaps

P25: Authorization has two checks on every protected resource: (1) permission for the requested action, and (2) ownership/workspace membership/resource grant. Permission checks must be performed server-side in the dependency and service/repository layers; query results, exports, document chunks, retrieval traces, jobs and citations receive the same isolation.Do not rely on hidden UI buttons or a platform role name alone.
P26: Minimal capability endpoint — approved design
P27: GET /api/v1/admin/authorization/capabilities | Required permission: platform.admin | Release: Sprint 1 authorization foundation | Must be wired through the real FastAPI dependency chain, not a test-only helper.

P28: Success contract (example):

P29: { "data": { "subject": { "user_id": "<uuid>", "platform_role_code": "platform_admin" }, "effective_permissions": ["platform.admin", "platform.roles.read"], "capabilities": ["inspect_authorization_policy"], "policy_version": "<policy-version>" }, "request_id": "req_01J..."}
P30: Do not include user directories, workspace data, secrets, session identifiers, password state, or mutation capabilities not actually available. effective_permissions must comefrom persisted roles/permissions and role-permission relationships, not a hard-coded if role == "admin" branch. If canonical permission tables are missing in the current migration set, add a new forward-only migration for permissions, role_permissions (and any necessary constraints/indexes); inspect migration history first, and do not edit an applied migration or the reference SQL package.
P31: Required database-backed test matrix

P32: Never mark FIN-15 complete from the capability endpoint alone: it is a minimal authorization proof route. The role/permission matrix, access checks for all implemented protected APIs, test evidence, route ownership checks and outstanding source acceptance criteria still have to pass.
P33: FastAPI implementation pattern

P34: The example below is a design sketch, not a drop-in replacement for the current auth/session modules. It must use the repository’s real principal, database session dependency,role/permission tables and error conventions. The dependency should be shared and applied to route groups; domain services must still check resource ownership/workspace grants.
P35: from fastapi import APIRouter, Depends, HTTPException, statusfrom sqlalchemy.exc import SQLAlchemyErroradmin_router = APIRouter(prefix="/admin", tags=["Identity & access"])defrequire_permission(permission_code: str): async def dependency(principal = Depends(require_authenticated_user), db = Depends(get_db_session),): try: allowed = await permissions_repository.has_permission(db, user_id=principal.user_id, permission_code=permission_code) except SQLAlchemyError: # Log exception with request_id; never log credentials or raw prompt/document text. raise HTTPException(status_code=status.HTTP_503_SERVICE_UNAVAILABLE, detail={"code": "authorization_unavailable"},) if not allowed: raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail={"code": "permission_denied"},) return principal return dependency@admin_router.get("/authorization/capabilities", response_model=CapabilitiesResponse)async def get_authorization_capabilities(principal = Depends(require_permission("platform.admin")), db = Depends(get_db_session),): return await authorization_service.capabilities_for_subject(db, principal.user_id)
P36: Implement require_authenticated_user, get_db_session, repository and response types using existing project conventions; do not create duplicated independent session verification. Translate exceptions centrally into the standard safe error envelope. Catch only known database/dependency failures, and do not swallow programming errors as ordinary 403 responses. In FastAPI, register static paths such as /instruments/compare before parameterized routes such as /instruments/{instrument_id} to prevent route-shadowing.

P72: A report stores report metadata, input snapshot usage (report_data_snapshots), source citations (report_citations), access grants and export/job state. Generated files are stored in controlled object/file storage; the database records the artifact URI, MIME type, timestamps and retention metadata. Recheck owner/workspace/report permissions both before generating and before serving/download. Signed file links are short-lived and never a substitute for authorization.
P73: Privacy export/deletion and API credential routes must be implemented only with explicit SRS acceptance, retention/legal-hold policy and test coverage. Return credential secrets once at creation only; store a hash, show metadata thereafter and support immediate revocation. Account deletion is an auditable workflow that respects retention/backup and dependency relationships.
P74: 12. Non-functional requirements and security gates

P75: 13. Error and failure behavior
P76: 14. Sprint 1 implementation sequence — avoid mixing work
P77: 15. Endpoint ownership, permissions and traceability rules
---

P78: The FinSight_AI_API_Endpoint_Catalog.csv includes every target endpoint, status, permission and derived story/requirement mapping. The FinSight_AI_API_Story_Coverage.csv preserves all 72 source aliases, Jira keys, acceptance summaries, test execution, implementation status and outstanding gaps from the source matrix. FinSight_AI_API_Requirement_Traceability.csv preserves each of the source RTM's requirement/data-integrity rows and original evidence/status/test fields, adding only a proposed API family and explicit contract note.
P79: Route-to-story mapping is a design aid and is not a substitute for acceptance-criterion traceability. For each Jira story, engineers must connect every acceptance criterion toone or more endpoints/services/schema changes/tests and show test evidence. Requirements without mapped source stories remain gaps; do not infer scope or change Jira/SRS baselines from this document.

P83: 18. Source artifacts used
P84: This contract was grounded in the uploaded SRS (49 pages), Database Schema & ERD Reference (81 pages), Technical Design & Implementation Plan (30 pages), Component Diagram Companion (19 pages), enhanced Jira backlog (72 story entries), full four-sprint allocation, Sprint 1 Test Plan and its Management Register, traceability-matrix(1).md, and srs-addendum(1).md. The architecture diagrams shared in the conversation were used to align the market-data, document/RAG, overall system, portfolio, risk, reporting and Release 2 trading domains.
P85: Authority reminder: the original SRS remains authoritative. The addendum is supplemental/draft; supplied schema/ERD and architecture are target-state design, not proof of livedeployment. This contract proposes design details but does not change source files, Jira scope, source release labels, implementation status or test execution status.

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

no changes added to commit (use "git add" and/or "git commit -a")
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % git add .
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % git commit -m "implimented FIN-15"
[main a27197d] implimented FIN-15
 25 files changed, 21118 insertions(+), 18 deletions(-)
 create mode 100644 FUNCTION_TEST_TRACEABILITY.txt
 create mode 100644 FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.docx
 create mode 100644 FinSight_AI_API_Design_Pack/FinSight_AI_API_Design_and_Implementation_Contract.md
 create mode 100644 FinSight_AI_API_Design_Pack/FinSight_AI_API_Endpoint_Catalog.csv
 create mode 100644 FinSight_AI_API_Design_Pack/FinSight_AI_API_Requirement_Traceability.csv
 create mode 100644 FinSight_AI_API_Design_Pack/FinSight_AI_API_Story_Coverage.csv

```
 create mode 100644 FinSight_AI_API_Design_Pack/FinSight_AI_OpenAPI_3.1.yaml
 create mode 100644 FinSight_AI_API_Design_Pack/README.md
 create mode 100644 apps/api/migrations/0002_platform_authorization.sql
 create mode 100644 apps/web/proxy.js
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % git push origin main
Enumerating objects: 71, done.
Counting objects: 100% (71/71), done.
Delta compression using up to 14 threads
Compressing objects: 100% (39/39), done.
Writing objects: 100% (42/42), 163.38 KiB | 14.85 MiB/s, done.
Total 42 (delta 20), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (20/20), completed with 19 local objects.
To https://github.com/sandeephgowda8618/FinSight_AI.git
   64f2637..a27197d  main -> main
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== BRANCH AND
WORKING TREE ====="
git status --short --branch

echo "===== PROJECT RULES ====="
cat AGENTS.md
if [ -f apps/api/AGENTS.md ]; then
  cat apps/api/AGENTS.md
fi

echo "===== LOGIN, LOGOUT AND SESSION IMPLEMENTATION ====="
sed -n '1,260p' apps/api/app/main.py
sed -n '1,240p' apps/api/app/auth.py
sed -n '1,160p' apps/api/app/database.py

echo "===== EXISTING MIGRATIONS ====="
ls -la apps/api/migrations

echo "===== CURRENT AUTH TESTS ====="
grep -n -E 'login|logout|failed|audit|lockout|rate.limit|session|expired|revoked' \
  apps/api/tests/test_auth_unit.py \
  apps/api/tests/test_identity_constraints.py \
  apps/api/tests/test_provision_user.py

echo "===== AUTH CONFIGURATION AND DEPENDENCIES ====="
grep -n -Ei
'DATABASE_URL|SESSION_TTL|MFA|AUDIT|LOCKOUT|RATE_LIMIT|slowapi|limits|pytest|psy
copg' \
  .env.example apps/api/requirements.txt apps/api/requirements-dev.txt 2>/dev/null
```

```
echo "===== FIN-14 TRACKING ====="
grep -n -E 'FIN-14|FS-01-03|TC_AUTHSEC_00[123]' \
  IMPLEMENTATION_STATUS.txt conversation.txt SPRINT_1_TEST_EXECUTION_STATUS.txt
===== BRANCH AND WORKING TREE =====
## main...origin/main
===== PROJECT RULES =====
```
FinSight AI — Mandatory Repository Rules for Codex

Status: Project-wide engineering instructions
Applies to: The entire repository, unless a more-specific `AGENTS.md` exists in a subdirectory.
Primary objective: Implement FinSight AI incrementally and safely against the approved project artifacts, with verifiable tests and an honest record of progress.

Codex must read and follow this file at the beginning of every task and reread the relevant source artifacts before changing code in that domain. These instructions remain active across the project; do not treat them as a one-time checklist.

---

1. Role and operating principles

Act as a coordinated technical leadership and engineering team: system architect, technical lead, backend/frontend/data/AI engineer, QA engineer, and DevSecOps reviewer. Make implementation decisions consistent with the supplied project artifacts and the existing repository.

1. **Inspect before changing.** Establish the current repository state, actual stack, current branch, existing conventions, available tests, and Jira issue/sprint mapping before editing files. Do not assume the repository is empty or that a documented feature already exists.
2. **Preserve existing work.** Prefer small, reviewable changes. Do not overwrite working code, discard user changes, regenerate the whole project, or replace the existing architecture just because another approach is familiar.
3. **Follow the active scope.** Work only on the issue/story or bounded task the user has authorized. The existence of 72 planned stories does not authorize implementing all 72 at once.
4. **Test the behavior, not just the build.** Add or update tests that map to acceptance criteria. Report actual command results; do not claim a test passed unless it was run and passed.
5. **Use the smallest sound design.** Keep the backend modular. Do not introduce microservices, infrastructure, dependencies, abstractions, or frameworks without a concrete requirement and a clear reason.
6. **Keep the user involved in material decisions.** Ask before making a decision that changes product scope, architecture, data model meaning, security posture, release boundaries, external spending, or irreversible repository/Jira state.

7. **Never fabricate progress.** Distinguish `NOT STARTED`, `IN PROGRESS`, `BLOCKED`, `PARTIAL`, `FAILED`, and `VERIFIED COMPLETE`. A story is not complete merely because code was written or a task was attempted.

2. Source-of-truth hierarchy

Read the actual files present in the repository. Filenames and directories may vary; the screenshot showed a project root containing `Component_diagrams/` and a directory spelled `Dcos/`, so search for the documents rather than assuming a corrected folder name. Do not copy, rename, or “clean up” these source artifacts without authorization.

Use the sources in this order, according to the question being answered:

1. **Software Requirements Specification (SRS):** authoritative functional and non-functional requirements, security expectations, page responsibilities, and observable user behavior.
2. **Core SQL schema and Database Schema/ERD Reference:** authoritative target relational entities, column types, PK/FK relationships, constraints, provenance, and domain model. Treat the SQL files supplied as reference inputs, not disposable migration scripts.
3. **Technical Design and Implementation Plan:** architectural boundaries, contracts, configuration patterns, testing/release gates, operational design, and implementation approach.
4. **The 72-story Jira backlog with acceptance criteria:** story goals, acceptance criteria, priority, dependencies, proposed role-based ownership, implementation subtasks, and original story aliases.
5. **Full Project Four-Sprint Plan:** four-sprint allocation and intended sequencing for the 72 catalog entries. It is a plan, not proof that work has been completed and not permission to bypass release guards.
6. **Sprint 1 Test Plan PDF and Sprint 1 Test Plan Management TXT:** Sprint 1 test inventory, story-to-test traceability, setup and execution rules. The original files are read-only reference documents; record actual execution results in a new status/execution file, not by editing the source register.
7. **Component diagrams and their companion/explanation document:** clarify components and relationships. They do not overrule explicit SRS requirements or schema definitions.
8. **Live repository, tests, configuration, and Jira state:** evidence of what exists now and what has actually been executed. The repository is authoritative about implementation state; Jira is authoritative about current issue keys, assignments, status, and sprint membership. Neither silently changes approved product requirements.

If two sources genuinely conflict, do not silently choose, rewrite a source file, or invent a resolution. Record the conflict and its impact in `conversation.txt`, cite the relevant filenames/sections/pages, recommend the safest interpretation, and ask the user when the decision affects implementation or scope. You may continue independent, reversible inspection work while waiting.

3. Product architecture baseline

Unless the source documents or current repository establish a documented, user-approved variation, follow this baseline:

- **Frontend:** Next.js / React.
- **Backend:** modular FastAPI application with clear domain boundaries and typed API contracts.
- **Structured database:** PostgreSQL; structured relational records are the source of truth.
- **Vector retrieval:** pgvector in the PostgreSQL deployment for the initial RAG design; document/chunk IDs and permissions remain relationally authoritative.
- **LLM inference:** Ollama behind a provider/adapter interface, so model access can be configured, mocked in tests, or replaced without changing business logic.
- **Embedding and market-data providers:** typed adapters with validation, timeouts, useful error mapping, and deterministic test doubles.
- **Long-running operations:** persisted job/run states, stable IDs, bounded retries, idempotency, and safe recovery. Redis is optional and must not become the authoritative recordunless an approved design change says so.
- **Financial calculations:** deterministic, tested backend functions/services own numeric outputs. The LLM may explain those outputs and assumptions; it must not be the authoritative calculator.
- **Authorization:** validate authentication and resource/operation permissions on the backend. Filter document permissions before any content is added to an LLM context.

First inspect the actual repository. Do not create a second frontend/backend, replace a working framework, or invent directory structures until the existing structure and gaps havebeen mapped.

4. Scope, planning, and Jira rules

4.1 Four-sprint baseline

The active project plan is the **four-sprint** plan for 72 story/task entries. Do not resurrect the superseded seven-day plan or use it to reorder the project. Preserve the `FS-*` aliases as traceability labels and map them to the real Jira issue keys found in the live project.

The source allocation describes Sprint 1 as foundation, identity, secure configuration, and delivery controls (23 entries, 63 source story points), with a total of 72 entries and 251 source story points across four sprints. These are source planning estimates, not evidence of actual team velocity or a promise that every acceptance criterion can be delivered on the stated calendar dates.

4.2 Verify Jira before relying on it

Before changing Jira, inspect current issue keys, stories, epics, status, assignees, dependencies, and sprint membership. A provided screenshot showed a `FIN Sprint 1` section with**0 work items**, while the surrounding backlog showed many issues in `To Do`. Treat that as a discrepancy to verify, not as proof that the sprint is populated.

If Sprint 1's actual Jira membership conflicts with the four-sprint plan:

- Record the discrepancy in `conversation.txt` with the relevant issue keys and counts.
- Tell the user precisely what needs confirming.
- Do not bulk-move stories, change estimates, alter priorities, assign people, start a sprint, or create/close issues without explicit authorization.
- Continue repository inspection and safe, independent Sprint 1 preparation when possible.

Do not claim that Jira has been updated unless the change was actually made and verified. If no Jira integration/tool is available, prepare instructions or an import artifact instead of claiming direct modification.

4.3 Scope gates

- Implement only authorized work, starting with the current Sprint 1 scope unless the user explicitly directs otherwise.
- Preserve original priorities, estimates, release labels, acceptance criteria, and dependencies unless the user approves a change.
- Do not mark a grouped story complete while any required acceptance criterion remains unimplemented or unverified. Use subtasks and status notes to show partial progress.
- A story can be marked complete only when its acceptance criteria, appropriate automated tests, code review, relevant security/data-isolation checks, and required traceability/evidence satisfy the Definition of Done.

5. Release and database safety rules

5.1 Release 1 is read-only financial research

The Release 1 guardrail is mandatory:

- No live broker order submission, modification, or cancellation endpoints.
- No enabled broker execution flow or live-trading credentials.
- Do not apply `schema_optional_trading.sql` or otherwise create its optional 15-table trading extension in Release 1.
- The trading-related stories identified in the backlog as Future / Release 2 remain gated even if they appear in a sprint allocation.
- A read-only connection or analytical feature must not silently expand into trading permission.

Any future change to this boundary requires explicit user approval and the separate security/operational review specified in the project documents.

5.2 PostgreSQL schema handling

- The supplied core `schema.sql` describes a 158-table target-state schema. It is **not** an instruction to implement all tables or all features immediately.
- Inspect the current database, migrations, and code before proposing changes. Do not run the supplied full schema directly against an existing or non-disposable database without explicit approval and a verified backup/recovery plan.
- Create new, version-controlled migrations in the repository's established migration system. Never edit the source SQL/schema/reference PDFs to show implementation progress.
- Apply migrations only to a confirmed development/test database. Verify a clean install and, where supported, a forward migration from the previous schema version.
- Preserve declared types, PK/FK constraints, uniqueness, checks, ownership, source lineage, timestamps, and financial precision. Use `NUMERIC` rather than binary floating point for stored money/financial quantities in accordance with the schema and design.
- Treat `seed_reference_data.sql` as optional development reference seed data, not production market data. Check that target tables and conflict behavior are compatible before use.
- Use PostgreSQL 16+ and pgvector as the design baseline unless the existing supported environment documents otherwise. Confirm required extensions are available before depending on them.
- Never drop tables, truncate user data, reset a database, rewrite migration history, or run destructive SQL without the user's explicit approval and a clear recovery plan.

6. Immutable project artifacts

The following supplied items are reference inputs and must remain unchanged unless the user specifically authorizes a revision:

- SRS PDF and source requirements documents.
- Technical Design and Implementation Plan PDF.
- Database Schema/ERD Reference PDF and source SQL files, including `schema.sql`, `seed_reference_data.sql`, and `schema_optional_trading.sql`.
- Original Jira backlog PDFs, 72-story backlog/acceptance-criteria document, and four-sprint plan.
- Sprint 1 Test Plan PDF and its existing management TXT.
- Component architecture PNG/SVG/Mermaid files and companion/explanation PDF.

Do not edit their contents, overwrite them with generated text, rename them, regenerate them, or alter their dates/claims. Use code, migrations, tests, configuration, ADRs, and newcoordination/status files for implementation outputs. If an approved design change is needed, write a separate addendum/ADR rather than silently rewriting a source baseline.

Before and after a task that touches project files, inspect `git status` and the diff. Do not discard uncommitted user changes. Do not run `git reset --hard`, `git clean -fd`, forced pushes, or equivalent destructive Git commands. Do not commit, push, merge, create a pull request, or alter remote branches unless the user explicitly asks for that action.

7. Required coordination files

Use small root-level coordination files so implementation progress and questions are visible without altering the source documents. If a file already exists, read it first and preserve existing entries.

7.1 `conversation.txt` — user/Codex question and decision log

Create this file in the repository root if it does not exist. Treat it as a shared conversation/decision log, not as source code or a place to store secrets. It should contain:

- Open questions, each with an ID, why the answer matters, choices if useful, a recommended choice, and the exact next action needed from the user.
- Requests for missing provider/service configuration, with the **environment variable names only**, purpose, whether required now or later, and step-by-step safe setup instructions.
- User-provided non-secret answers/decisions, decision date, and the task/story affected.
- Blocking issues, source conflicts, and a `WAITING FOR USER` or `ANSWERED` marker.

When a question requires user action, explain exact steps: which application/provider to open, where to find/create the setting or credential, where to configure it locally, and what non-secret confirmation to add to `conversation.txt`. Then pause any work dependent on the answer. Continue only work that is independent and safe.

Secrets policy: never ask the user to paste API keys, passwords, tokens, private keys, database passwords, or session secrets into `conversation.txt`, source files, terminal output, screenshots, Jira descriptions, or Git. Never print, echo, or commit actual secret values. Request that the user add secret values directly to a local ignored `.env` file or approved secret manager and return only a confirmation such as `Configured locally; value not shared`. Ensure `.env`, credentials, tokens, local database dumps, and other sensitive artifacts are ignored by Git. `.env.example` may contain variable names and safe placeholders only.

If Codex discovers a secret already tracked or exposed, stop further exposure, avoid repeating its value, notify the user, recommend revocation/rotation, and follow the repository's established secret-removal procedure only with authorization.

7.2 `IMPLEMENTATION_STATUS.txt` — completion markers

Create this file if missing and update it after meaningful work. Include the task/story alias and real Jira key when known, status, concise implementation summary, changed file paths,

commands actually run, tests and observed outcomes, unresolved acceptance criteria, blockers, and evidence paths/links if available.

Use these states only:

- `NOT STARTED`
- `IN PROGRESS`
- `BLOCKED`
- `PARTIAL`
- `FAILED`
- `VERIFIED COMPLETE`

Only use `VERIFIED COMPLETE` when the applicable Definition of Done is met and evidence is recorded. Otherwise use `PARTIAL`, `FAILED`, or `BLOCKED` as appropriate. Never infer success from an exit code that was not observed, from a planned test, or from code inspection alone when runtime verification is required.

7.3 Test-execution record

Do not edit the supplied test-plan files. If actual Sprint 1 test results need recording, create a separate root file such as `SPRINT_1_TEST_EXECUTION_STATUS.txt` with each test ID, initial status `NOT RUN`, execution date, actual result, tester/owner, evidence, and linked defect if any. Update statuses only after executing the case. Use `PASS`, `FAIL`, `BLOCKED`, and `NOT RUN`; `N/A` requires documented justification. Do not represent a test as passed if its dependency/component does not exist.

Do not create unnecessary coordination files beyond these needs. Keep records concise and factual.

8. Clarification and stop conditions

Ask a clear, focused user question before proceeding when any of the following is true:

- A conflict in source documents changes behavior, schema meaning, scope, or release eligibility.
- A decision is irreversible/destructive or could lose user work/data.
- A required external credential, licensed data feed, paid service, or permission is unavailable.
- The Jira issue/sprint mapping or scope is ambiguous and changing it would affect commitments.
- An acceptance criterion cannot be met without choosing between materially different product behaviors.
- An architecture/security decision is not established by the source documents or current repository.

Do not ask questions whose answers can be determined safely from the repository or supplied documents. Before asking, investigate the available evidence and report what you found. When blocked, write the question and next action to `conversation.txt`, summarize the blocker to the user, and stop only the dependent work.

9. Credentials, API keys, and external providers

- Do not assume that any specific API key is required until a selected feature/provider actually needs it.
- Ollama may be available locally without a hosted LLM API key; inspect the approved setup and test connectivity before concluding it is configured.
- Market-data providers, embedding services, hosted models, email delivery, or other integrations may require credentials depending on the selected adapter and environment. Do not choose a paid provider or trigger spending without approval.
- Use provider interfaces and deterministic fixtures/mocks so tests can run without live API access where appropriate.
- If a key is required, add a non-secret request to `conversation.txt` containing the variable name, service, reason, whether it blocks the active story, and exact setup steps. Wait for confirmation that it has been configured safely; do not request the secret itself.
- Distinguish `missing configuration`, `provider unavailable`, `invalid credentials`, `rate limited`, and `no data` in internal error handling. Do not leak provider responses or secrets to users/logs.

10. Coding standards and engineering workflow

Apply the repository's existing formatter, linter, type checker, tests, dependency manager, and naming conventions. Do not introduce a competing toolchain without approval.

For each authorized story or task:

1. **Read:** reread relevant SRS requirements, schema entities, technical design sections, backlog acceptance criteria, dependencies, and relevant test-plan cases.
2. **Inspect:** review the current implementation, `git status`, recent relevant commits where available, and existing test patterns.
3. **Plan:** identify the smallest vertical slice, dependencies, impacted files, migration impact, security concerns, test cases, and any user decision needed. Record concise intent in the status file.
4. **Test first where practical:** add or adjust a failing test for a clearly defined behavior before implementing it (TDD). Do not claim TDD was followed if it was not.
5. **Implement:** make small, typed, maintainable changes; validate all inputs at boundaries; use explicit timeouts and bounded retries for external calls; preserve ownership and provenance; keep side effects controlled.
6. **Verify:** run focused tests first, then relevant integration/regression tests, lint/type/static/security checks and builds available in the repo. Use disposable/test resourcesand synthetic data.

7. **Review:** inspect the complete diff for accidental changes, secrets, unsafe defaults, unrelated formatting churn, altered source documents, schema drift, missing tests, and unmet acceptance criteria.
8. **Record:** update only the new coordination/status files with actual results. Include exact commands and concise output summaries; redact environment values and secrets.
9. **Report:** state what changed, what tests ran, what passed/failed/was blocked, remaining criteria, and the next recommended step. Do not claim deployment or Jira updates unlessthey happened and were verified.

Do not weaken or delete a test simply to make the build pass. If a test exposes a product/design ambiguity, stop and ask rather than encoding a convenient assumption.

General code requirements

- Prefer explicit types, cohesive modules, small functions, and clear dependency boundaries.
- Validate and normalize user/provider input at the boundary.
- Use parameterized SQL/ORM operations. Never interpolate untrusted input into SQL or permit arbitrary SQL execution through Data Explorer features.
- Use transactions for coupled database updates and enforce constraints in the database where appropriate.
- Make retried ingestion and job operations idempotent. Use stable run IDs and safe state transitions.
- Keep logs structured and useful but redact secrets and sensitive user data; never return stack traces or internal secrets to end users.
- Set realistic timeouts and bounded retries for database, network, model, and provider operations. Fail gracefully when optional dependencies are down.
- Add accessible labels, keyboard operation, visible focus, useful loading/error/empty states, and responsive behavior for UI work.
- Avoid nondeterministic unit tests and internet-dependent CI tests. Use fixtures or mocks for external integrations; separately verify real integration connections when needed.
- Keep financial calculations reproducible: document formulas/units, input snapshots, assumptions, data timestamps, precision, and formula/model versions as applicable.

11. Security, privacy, RAG, and financial correctness gates

The following are release-blocking if violated:

- No plaintext passwords or exposed secret values.
- Authorization is enforced in the backend for every protected operation; ownership/workspace isolation is tested.
- Unauthenticated requests receive an appropriate rejection; forbidden resource access does not disclose protected content.
- RAG permission filters run **before** private chunks enter model context. Retrieved document text is untrusted data and cannot override system/developer instructions or authorizetools.

- Citations must refer to retrieved evidence that really exists. Do not invent sources, pages, or support for model claims; provide a clear no-evidence path.
- User/provider/model errors do not leak internal prompt data, credentials, stack traces, or unrelated users' records.
- Financial numbers come from deterministic code and traceable inputs; unsupported metrics are marked unavailable rather than fabricated.
- Historical or delayed market data is labeled truthfully with source and as-of/freshness timestamps; never present fixture data as live market data.
- No Release 1 route, UI action, worker, permission, or configuration can submit real broker orders.
- Deletion/re-indexing does not leave stale or unauthorized vectors retrievable; ingestion retries do not silently duplicate accepted data.

12. Testing and acceptance

Use the supplied Sprint 1 test plan as the reference inventory for Sprint 1. It contains 54 detailed test cases and explicitly says all cases begin as `NOT RUN`; preserve that truth until observed execution results exist.

- Map each implemented acceptance criterion to one or more tests and, where available, test IDs and actual Jira keys.
- Use unit tests for pure validation/permission/calculation logic; integration tests for API/database/provider boundaries; and a smaller set of system/E2E tests for critical user workflows.
- Test positive and negative paths: valid/invalid input, permitted/forbidden users, empty/missing data, provider/model outage, timeout, retry, duplicate submission, and partial failure where relevant.
- Run migrations against disposable test databases and verify constraints and FK behavior. Never use production credentials or real user data in tests.
- Keep unexecuted tests `NOT RUN`; use `BLOCKED` when a genuine missing dependency prevents execution and record the dependency and next owner/action.
- A failed security, access-isolation, data-integrity, migration, or Release 1 boundary test blocks acceptance until fixed or explicitly handled under an approved exception process.
- Record the actual test command, result, and evidence. If the environment cannot run a test, say so and provide exact next steps; do not replace runtime evidence with a guess.

13. CI/CD, versions, and repository hygiene

- Inspect existing CI before adding or changing workflows. Keep or improve current checks rather than duplicating pipelines.
- The intended PR gates are dependency/lockfile validation, formatting/lint, type/static checks, unit tests, database/pgvector integration tests where relevant, security/secret/dependency checks, and frontend/backend builds.

- A check failure must be visible and must block acceptance for the branch/PR according to the configured policy. Do not hide failures or make commands pass by suppressing errors.
- Keep `.env` and private/local data out of version control. `.env.example` contains safe placeholders only. Verify `.gitignore` before creating files that could include secrets orlocal data.
- Avoid generated files, build output, caches, local DB files, uploaded private documents, and large artifacts in commits unless the repository specifically requires them.
- Do not change dependency versions, lockfiles, CI permissions, deployment configuration, or secret workflows without reviewing compatibility and documenting material effects.
- Do not deploy, publish images, provision paid infrastructure, run production migrations, or change provider entitlements unless explicitly approved.

14. Definition of Done

A task/story may be reported `VERIFIED COMPLETE` only when all applicable checks below are satisfied:

- [ ] The authorized scope and relevant source requirements were reviewed.
- [ ] Every acceptance criterion for the completed scope is implemented, not merely partially addressed.
- [ ] Automated tests cover the criteria and were run; results are recorded honestly.
- [ ] Relevant lint/type/static/build checks pass, or any unavoidable blocker is documented without falsely claiming completion.
- [ ] Security, privacy, ownership, provenance, and Release 1 boundaries were checked where relevant.
- [ ] Database migrations are versioned and safely validated when database changes exist.
- [ ] Code and diff were reviewed; no source reference documents were modified accidentally.
- [ ] Documentation/traceability is updated in new implementation documents or status files as appropriate.
- [ ] `IMPLEMENTATION_STATUS.txt` records the Jira/FS alias, files changed, actual commands/results, and evidence.
- [ ] The final user report clearly states remaining work and does not overclaim deployment, test, or Jira status.

15. Required first-run procedure

At the start of a fresh Codex session, perform these steps before implementation:

1. Read this `AGENTS.md` completely and identify any more-specific `AGENTS.md` files.
2. Inspect the project root, relevant subdirectories, Git branch/status/diff, package manifests, environment templates, migration setup, tests, and CI workflows. Do not reveal secret values while inspecting configuration.
3. Locate and read the applicable SRS, technical design, schema reference and core SQL, 72-story backlog/acceptance criteria, full four-sprint plan, Sprint 1 test plan, component diagrams, and any existing ADR/README files. Do not edit these source artifacts.

4. Read or create `conversation.txt` and `IMPLEMENTATION_STATUS.txt`; preserve existing contents. Check `.gitignore` before writing any configuration examples.
5. Inspect live Jira state if an authorized Jira tool/connection exists. Map `FS-*` aliases to actual issue keys and verify Sprint 1 membership. Explicitly flag the previously observed mismatch that the screenshot showed `FIN Sprint 1` with 0 work items. Do not change Jira to resolve it without the user's approval.
6. Produce a concise baseline report: repository structure, detected technologies, current branch and clean/dirty state, which required docs were found/missing, existing test/CI commands, Sprint 1 Jira mapping, risks, and the safest next authorized task.
7. If critical information is missing, append a numbered question and exact action to `conversation.txt`. Never put secrets in the file. Ask the user for the non-secret confirmation/decision and pause only dependent work.
8. Begin implementation only after the baseline is understood and any required user decision is resolved. Prefer the highest-priority, dependency-ready task in the currently authorized Sprint 1 scope; do not start all 23 entries indiscriminately.

Final rule: The source documents define what the system must be. This file governs how Codex works. New code, migrations, tests, and separate status/decision records may evolve; the original project artifacts remain read-only. Verify everything that can be verified, ask about consequential ambiguity, and never present an assumption as an observed fact.
===== LOGIN, LOGOUT AND SESSION IMPLEMENTATION =====
"""FinSight AI API with PostgreSQL-backed browser sessions."""

```
from __future__ import annotations

import os
import secrets
from datetime import UTC, datetime
from pathlib import Path
from typing import Annotated, Any
from uuid import UUID

import psycopg
from dotenv import load_dotenv
from fastapi import Cookie, Depends, FastAPI, HTTPException, Request, Response, status
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, EmailStr, Field
from psycopg.rows import dict_row

from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
from app.database import connect

load_dotenv(Path(__file__).resolve().parents[3] / ".env")
```

```
app = FastAPI(title="FinSight AI API", version="0.2.0")
web_origin = os.environ.get("WEB_ORIGIN", "http://localhost:3000")
app.add_middleware(
    CORSMiddleware,
    allow_origins=[web_origin],
    allow_credentials=True,
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)

SESSION_COOKIE = "finsight_session"
GENERIC_LOGIN_ERROR = "Email or password is incorrect."
DUMMY_PASSWORD_HASH = hash_password(secrets.token_urlsafe(32))


class LoginRequest(BaseModel):
    email: EmailStr
    password: str = Field(min_length=1, max_length=1024)


class UserView(BaseModel):
    user_id: UUID
    email: str
    display_name: str


def _ttl_minutes() -> int:
    try:
        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
        session_expiry(value)
        return value
    except ValueError as exc:
        raise HTTPException(status_code=500, detail="Session configuration is invalid") from exc


def _secure_cookie() -> bool:
    configured = os.environ.get("COOKIE_SECURE")
    if configured is not None:
        return configured.lower() in {"1", "true", "yes"}
    return os.environ.get("APP_ENV", "development").lower() == "production"


def _check_origin(request: Request) -> None:
```

```
    origin = request.headers.get("origin")
    if origin and origin != web_origin:
        raise HTTPException(status_code=403, detail="Request origin is not allowed")


def get_current_user(
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> dict[str, Any]:
    if not session_token:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Authentication required")
    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT u.user_id, u.email, u.display_name, r.role_key,
                       COALESCE(array_agg(DISTINCT p.permission_key)
                           FILTER (WHERE p.permission_key IS NOT NULL), ARRAY[]::varchar[])
AS permissions
                  FROM auth_sessions s JOIN users u ON u.user_id = s.user_id
                  LEFT JOIN roles r ON r.role_id = u.platform_role_id AND r.scope = 'platform'
                  LEFT JOIN role_permissions rp ON rp.role_id = r.role_id
                  LEFT JOIN permissions p ON p.permission_id = rp.permission_id
                  WHERE s.refresh_token_hash = %s AND s.revoked_at IS NULL
                    AND s.expires_at > %s AND u.account_status = 'active'
                    AND s.last_seen_at > %s - make_interval(mins => %s)
                    AND u.deleted_at IS NULL
                  GROUP BY u.user_id, u.email, u.display_name, r.role_key""",
                (token_digest(session_token), datetime.now(UTC), datetime.now(UTC),
_ttl_minutes()),
            )
            user = cur.fetchone()
            if not user:
                raise HTTPException(status_code=401, detail="Authentication required")
            cur.execute(
                "UPDATE auth_sessions SET last_seen_at = %s WHERE refresh_token_hash = %s",
                (datetime.now(UTC), token_digest(session_token)),
            )
            return user
    except psycopg.Error as exc:
        raise HTTPException(status_code=503, detail="Authentication service unavailable") from
exc


CurrentUser = Annotated[dict[str, Any], Depends(get_current_user)]
```

```
def require_permission(permission_key: str):
    """Build a deny-by-default backend permission dependency."""
    def dependency(user: CurrentUser) -> dict[str, Any]:
        if permission_key not in user.get("permissions", []):
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Permission
denied")
        return user

    return dependency


@app.post("/auth/login", response_model=UserView)
def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
    _check_origin(request)
    email = payload.email.strip().lower()
    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT user_id, email, display_name, password_hash
                   FROM users WHERE lower(email) = %s AND account_status = 'active'
                     AND deleted_at IS NULL""",
                (email,),
            )
            user = cur.fetchone()
            if not user or not user["password_hash"]:
                verify_password(DUMMY_PASSWORD_HASH, payload.password)
                raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)
            if not verify_password(user["password_hash"], payload.password):
                raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)
            token, digest = new_session_token()
            expires_at = session_expiry(_ttl_minutes())
            cur.execute(
                """INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at)
                   VALUES (%s, %s, %s)""",
                (user["user_id"], digest, expires_at),
            )
            cur.execute("UPDATE users SET last_login_at = %s, updated_at = %s WHERE user_id
= %s",
                        (datetime.now(UTC), datetime.now(UTC), user["user_id"]))
        response.set_cookie(
            SESSION_COOKIE,
            token,
```

```
            httponly=True,
            secure=_secure_cookie(),
            samesite="lax",
            max_age=_ttl_minutes() * 60,
            path="/",
        )
        return UserView(user_id=user["user_id"], email=user["email"],
display_name=user["display_name"])
    except psycopg.Error as exc:
        raise HTTPException(status_code=503, detail="Authentication service unavailable") from
exc


@app.post("/auth/logout", status_code=204)
def logout(request: Request, response: Response,
           session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None) ->
Response:
    _check_origin(request)
    if session_token:
        try:
            with connect() as conn, conn.cursor() as cur:
                cur.execute(
                    """UPDATE auth_sessions SET revoked_at = %s, revoke_reason = 'logout'
                       WHERE refresh_token_hash = %s AND revoked_at IS NULL""",
                    (datetime.now(UTC), token_digest(session_token)),
                )
        except psycopg.Error as exc:
            raise HTTPException(status_code=503, detail="Authentication service unavailable")
from exc
    response.delete_cookie(SESSION_COOKIE, path="/", httponly=True,
secure=_secure_cookie(), samesite="lax")
    response.status_code = status.HTTP_204_NO_CONTENT
    return response


@app.get("/auth/me", response_model=UserView)
def who_am_i(user: CurrentUser) -> UserView:
    return UserView(**user)


@app.get("/api/v1/admin/authorization/capabilities")
def platform_admin_capabilities(
    request: Request,
    _authorized: Annotated[dict[str, Any], Depends(require_permission("platform.admin"))],
```

```
) -> dict[str, Any]:
    """Return the authenticated caller's database-backed platform capabilities."""

    supplied_request_id = request.headers.get("request_id", "").strip()
    if supplied_request_id and (
        len(supplied_request_id) > 128
        or any(ord(char) < 32 or ord(char) == 127 for char in supplied_request_id)
    ):
        raise HTTPException(status_code=400, detail="Invalid request_id")

    # Always issue a server-generated canonical request ID.
    request_id = f"req_{secrets.token_urlsafe(12)}"
    correlation_id = supplied_request_id or request_id
    permissions = sorted(set(_authorized.get("permissions", [])))
    capabilities = (
        ["inspect_authorization_policy"]
        if "platform.admin" in permissions
        else []
    )

    return {
        "data": {
            "subject": {
                "user_id": str(_authorized["user_id"]),
                "platform_role_code": _authorized.get("role_key"),
            },
            "effective_permissions": permissions,
            "capabilities": capabilities,
            "policy_version": "sprint-1-platform-rbac-v1",
        },
        "request_id": request_id,
        "correlation_id": correlation_id,
    }
"""Password and opaque session helpers for the authentication API."""

from __future__ import annotations

import hashlib
import secrets
from datetime import UTC, datetime, timedelta

from argon2 import PasswordHasher
from argon2.exceptions import InvalidHashError, VerificationError
```

```
PASSWORD_HASHER = PasswordHasher()


def hash_password(password: str) -> str:
    return PASSWORD_HASHER.hash(password)


def verify_password(encoded_hash: str | None, password: str) -> bool:
    if not encoded_hash:
        return False
    try:
        return PASSWORD_HASHER.verify(encoded_hash, password)
    except (InvalidHashError, VerificationError):
        return False


def new_session_token() -> tuple[str, str]:
    token = secrets.token_urlsafe(32)
    return token, hashlib.sha256(token.encode("utf-8")).hexdigest()


def session_expiry(ttl_minutes: int, now: datetime | None = None) -> datetime:
    if ttl_minutes < 1 or ttl_minutes > 10080:
        raise ValueError("SESSION_TTL_MINUTES must be between 1 and 10080")
    return (now or datetime.now(UTC)) + timedelta(minutes=ttl_minutes)


def token_digest(token: str) -> str:
    return hashlib.sha256(token.encode("utf-8")).hexdigest()
"""Small synchronous PostgreSQL boundary for the auth endpoints."""

from __future__ import annotations

import os
from pathlib import Path

import psycopg
from dotenv import load_dotenv

load_dotenv(Path(__file__).resolve().parents[3] / ".env")


def connect() -> psycopg.Connection:
    database_url = os.environ.get("DATABASE_URL")
```

```
    if not database_url:
        raise RuntimeError("DATABASE_URL is required")
    return psycopg.connect(database_url, connect_timeout=3)
===== EXISTING MIGRATIONS =====
total 16
drwxr-xr-x@ 4 sandeeph  staff   128 Oct 20 10:57 .
drwxr-xr-x@ 9 sandeeph  staff   288 Oct 21 13:45 ..
-rw-r--r--@ 1 sandeeph  staff  1728 Oct 18 22:05 0001_auth_identity.sql
-rw-r--r--@ 1 sandeeph  staff  2580 Oct 19 11:20 0002_platform_authorization.sql
===== CURRENT AUTH TESTS =====
apps/api/tests/test_auth_unit.py:9:from app.auth import hash_password, new_session_token,
session_expiry, token_digest, verify_password
apps/api/tests/test_auth_unit.py:23:def test_session_expiry_uses_utc_and_requested_ttl():
apps/api/tests/test_auth_unit.py:26:    assert session_expiry(15, now) == now +
timedelta(minutes=15)
apps/api/tests/test_auth_unit.py:30:def test_session_expiry_rejects_out_of_range_ttl(ttl: int):
apps/api/tests/test_auth_unit.py:32:        session_expiry(ttl)
apps/api/tests/test_auth_unit.py:53:def
test_session_tokens_are_opaque_and_only_digest_is_stable():
apps/api/tests/test_auth_unit.py:54:    token, digest = new_session_token()
apps/api/tests/test_auth_unit.py:55:    another_token, another_digest = new_session_token()
apps/api/tests/test_auth_unit.py:63:def
test_login_maps_database_outage_to_controlled_error(monkeypatch):
apps/api/tests/test_auth_unit.py:73:        "/auth/login",
apps/api/tests/test_auth_unit.py:82:def
test_protected_route_rejects_missing_session_before_database_access(monkeypatch):
apps/api/tests/test_auth_unit.py:86:        raise AssertionError("database must not be accessed
without a session cookie")
apps/api/tests/test_auth_unit.py:138:        ("get", "/auth/me", {"cookies": {"finsight_session":
"synthetic-token"}}),
apps/api/tests/test_auth_unit.py:139:        ("post", "/auth/logout", {"cookies": {"finsight_session":
"synthetic-token"}}),
apps/api/tests/test_auth_unit.py:142:            {"cookies": {"finsight_session": "synthetic-token"}},
apps/api/tests/test_auth_unit.py:146:def
test_session_database_outages_return_controlled_error(monkeypatch, method, path, kwargs):
apps/api/tests/test_auth_unit.py:170:def
test_login_rejects_invalid_input_before_database_access():
apps/api/tests/test_auth_unit.py:172:        "/auth/login",
apps/api/tests/test_identity_constraints.py:9:from app.auth import new_session_token,
session_expiry
apps/api/tests/test_identity_constraints.py:14:def
test_identity_schema_rejects_duplicate_email_and_cascades_sessions():
apps/api/tests/test_identity_constraints.py:31:        _, digest = new_session_token()
```

apps/api/tests/test_identity_constraints.py:34:   "INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at) VALUES (%s, %s, %s)",
apps/api/tests/test_identity_constraints.py:35:   (user_id, digest, session_expiry(5)),
apps/api/tests/test_identity_constraints.py:38:   cur.execute("SELECT count(*) FROM auth_sessions WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py:45:def test_platform_permission_is_resolved_from_database_role_and_session():
apps/api/tests/test_identity_constraints.py:63:   token, digest = new_session_token()
apps/api/tests/test_identity_constraints.py:66:   "INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at) VALUES (%s, %s, %s)",
apps/api/tests/test_identity_constraints.py:67:   (owner_id, digest, session_expiry(5)),
apps/api/tests/test_identity_constraints.py:76:   invalid_session = client.get(
apps/api/tests/test_identity_constraints.py:78:   cookies={SESSION_COOKIE: "invalid-session-token"},
apps/api/tests/test_identity_constraints.py:80:   assert invalid_session.status_code == 401
apps/api/tests/test_identity_constraints.py:171:   token, digest = new_session_token()
apps/api/tests/test_identity_constraints.py:174:   INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at)
apps/api/tests/test_identity_constraints.py:177:   (user_id, digest, session_expiry(5)),
apps/api/tests/test_identity_constraints.py:236:   email = f"revoked-platform-admin-{uuid4().hex}@example.test"
apps/api/tests/test_identity_constraints.py:250:   token, digest = new_session_token()
apps/api/tests/test_identity_constraints.py:252:   "INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at) "
apps/api/tests/test_identity_constraints.py:254:   (user_id, digest, session_expiry(5)),
apps/api/tests/test_identity_constraints.py:295:#   JOIN auth_sessions s ON s.user_id = u.user_id
apps/api/tests/test_identity_constraints.py:298:#   AND s.revoked_at IS NULL
apps/api/tests/test_identity_constraints.py:306:#   pytest.skip("No active platform-admin test session is available")
apps/api/tests/test_identity_constraints.py:309:#   # session instead of attempting to recover an existing session token.
apps/api/tests/test_identity_constraints.py:310:#   pytest.skip("Use the dedicated-session integration fixture for this assertion")
===== AUTH CONFIGURATION AND DEPENDENCIES =====
.env.example:1:# Local development placeholders only. Replace DATABASE_URL locally; never commit it.
.env.example:2:DATABASE_URL=postgresql://USER:PASSWORD@127.0.0.1:5432/finsight
.env.example:4:SESSION_TTL_MINUTES=60
apps/api/requirements.txt:5:psycopg[binary]==3.2.13
apps/api/requirements-dev.txt:2:pytest==9.0.3
apps/api/requirements-dev.txt:4:pytest-cov==6.2.1
===== FIN-14 TRACKING =====

IMPLEMENTATION_STATUS.txt:239:Scope references: detailed backlog FS-11-06 (P0, 3 SP) and four-sprint plan (3 SP). Sprint 1 management plan maps this story to TC_ENV_002, TC_SEC_001, TC_SEC_002, TC_AUTHSEC_002, TC_AUTHN_006, TC_CFG_001, and TC_EXEC_002. Original references remain unchanged.
IMPLEMENTATION_STATUS.txt:430: membership, and workspace tables are deferred. FIN-14 is not changed.
IMPLEMENTATION_STATUS.txt:480:remain deferred. FIN-14 is outside this verification scope.
SPRINT_1_TEST_EXECUTION_STATUS.txt:97:TC_AUTHSEC_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
SPRINT_1_TEST_EXECUTION_STATUS.txt:98:TC_AUTHSEC_002 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
SPRINT_1_TEST_EXECUTION_STATUS.txt:99:TC_AUTHSEC_003 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
SPRINT_1_TEST_EXECUTION_STATUS.txt:181:Overall FIN-15 remains PARTIAL, not accepted. Do not alter FIN-14 tracking.
SPRINT_1_TEST_EXECUTION_STATUS.txt:207:Workspace/resource authorization remains deferred. FIN-14 is unchanged.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
all FIN-15 remains PARTIAL, not accepted. Do not alter FIN-14 tracking.
SPRINT_1_TEST_EXECUTION_STATUS.txt:207:Workspace/resource authorization remains deferred. FIN-14 is unchanged.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== EXISTING IDENTITY MIGRATION ====="
cat apps/api/migrations/0001_auth_identity.sql

echo "===== EXISTING RBAC MIGRATION ====="
cat apps/api/migrations/0002_platform_authorization.sql

echo "===== AUTH TESTS AND FIXTURES ====="
sed -n '1,230p' apps/api/tests/test_auth_unit.py
sed -n '1,290p' apps/api/tests/test_identity_constraints.py

echo "===== TEST CONFIGURATION ====="
find apps/api -maxdepth 2 -type f \
  $-$ $name$'$conftest.py$' $-$ o $-$ $name$'$pytest.ini$' $-$ o $-$ $name$'$pyproject.toml$' -print

echo "===== DATABASE TEST FIXTURES AND MIGRATION RUNNER ====="
grep -R -n -E 'finsight_fin15_test|DATABASE_URL|0001_auth_identity|0002_platform_authorization|CREATE TABLE.*audit_events|audit_events' \
  apps/api/tests apps/api/scripts scripts 2>/dev/null

echo "===== LOCAL CONFIGURATION NAMES (VALUES HIDDEN) ====="
grep -E '^[A-Za-z_][A-Za-z0-9_]*=' .env.example | cut -d= -f1

```
===== EXISTING IDENTITY MIGRATION =====
-- FIN-13 identity/session subset. Derived from Database/schema (1).sql.
-- Apply only to a new development/test database; this is not the full target schema.
BEGIN;

CREATE TABLE roles (
    role_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    role_key VARCHAR(50) NOT NULL UNIQUE,
    display_name VARCHAR(100) NOT NULL,
    scope VARCHAR(20) NOT NULL CHECK (scope IN ('platform','workspace')),
    description TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(320) NOT NULL,
    password_hash TEXT,
    display_name VARCHAR(160) NOT NULL,
    account_status VARCHAR(20) NOT NULL DEFAULT 'active'
        CHECK (account_status IN ('invited','active','locked','disabled','deleted')),
    platform_role_id UUID REFERENCES roles(role_id) ON DELETE SET NULL,
    email_verified_at TIMESTAMPTZ,
    mfa_required BOOLEAN NOT NULL DEFAULT false,
    last_login_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at TIMESTAMPTZ
);
CREATE UNIQUE INDEX uq_users_email_lower ON users (lower(email));

CREATE TABLE auth_sessions (
    session_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    refresh_token_hash TEXT UNIQUE,
    ip_address INET,
    user_agent TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_seen_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at TIMESTAMPTZ NOT NULL,
    revoked_at TIMESTAMPTZ,
    revoke_reason VARCHAR(80)
);
```

```
CREATE INDEX ix_auth_sessions_active ON auth_sessions(user_id, expires_at) WHERE
revoked_at IS NULL;

COMMIT;
===== EXISTING RBAC MIGRATION =====
-- FIN-15: only the platform-scoped administrator permission is implemented.
-- Workspace roles and workspace permissions remain unavailable until workspace
membership exists.
BEGIN;

CREATE TABLE permissions (
   permission_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   permission_key VARCHAR(120) NOT NULL UNIQUE,
   description TEXT NOT NULL
);

CREATE TABLE role_permissions (
   role_id UUID NOT NULL REFERENCES roles(role_id) ON DELETE CASCADE,
   permission_id UUID NOT NULL REFERENCES permissions(permission_id) ON DELETE
CASCADE,
   PRIMARY KEY (role_id, permission_id)
);

INSERT INTO roles (role_key, display_name, scope, description)
VALUES ('platform_admin', 'Platform Administrator', 'platform',
       'Administrator privileges for currently implemented platform operations only.')
ON CONFLICT (role_key) DO NOTHING;

DO $$
BEGIN
   IF NOT EXISTS (SELECT 1 FROM roles WHERE role_key = 'platform_admin' AND scope =
'platform') THEN
       RAISE EXCEPTION 'Existing platform_admin role has non-platform scope';
   END IF;
END
$$;

INSERT INTO permissions (permission_key, description)
VALUES ('platform.admin', 'Access operations explicitly assigned platform administrator
permission.')
ON CONFLICT (permission_key) DO NOTHING;

INSERT INTO role_permissions (role_id, permission_id)
SELECT r.role_id, p.permission_id
```

```
FROM roles r CROSS JOIN permissions p
WHERE r.role_key = 'platform_admin' AND r.scope = 'platform'
  AND p.permission_key = 'platform.admin'
ON CONFLICT DO NOTHING;

CREATE FUNCTION enforce_platform_role_scope() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
   IF NEW.platform_role_id IS NOT NULL AND NOT EXISTS (
      SELECT 1 FROM roles WHERE role_id = NEW.platform_role_id AND scope = 'platform'
   ) THEN
      RAISE EXCEPTION 'platform_role_id must reference a platform-scoped role'
         USING ERRCODE = '23514';
   END IF;
   RETURN NEW;
END
$$;

CREATE TRIGGER users_platform_role_scope_check
BEFORE INSERT OR UPDATE OF platform_role_id ON users
FOR EACH ROW EXECUTE FUNCTION enforce_platform_role_scope();

CREATE FUNCTION prevent_platform_role_scope_drift() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
   IF OLD.scope = 'platform' AND NEW.scope <> 'platform' AND EXISTS (
      SELECT 1 FROM users WHERE platform_role_id = OLD.role_id
   ) THEN
      RAISE EXCEPTION 'Cannot change scope of an assigned platform role';
   END IF;
   RETURN NEW;
END
$$;

CREATE TRIGGER roles_prevent_scope_drift
BEFORE UPDATE OF scope ON roles
FOR EACH ROW EXECUTE FUNCTION prevent_platform_role_scope_drift();

COMMIT;
===== AUTH TESTS AND FIXTURES =====
from __future__ import annotations

from datetime import UTC, datetime, timedelta
```

```
import pytest
from fastapi import HTTPException
from fastapi.testclient import TestClient

from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
from app.main import _secure_cookie, _ttl_minutes, app


def test_password_helpers_accept_only_the_original_password():
    encoded = hash_password("synthetic-password")

    assert encoded.startswith("argon2id")
    assert verify_password(encoded, "synthetic-password")
    assert not verify_password(encoded, "different-password")
    assert not verify_password("not-an-argon2-hash", "synthetic-password")
    assert not verify_password(None, "synthetic-password")


def test_session_expiry_uses_utc_and_requested_ttl():
    now = datetime(2026, 1, 1, tzinfo=UTC)

    assert session_expiry(15, now) == now + timedelta(minutes=15)


@pytest.mark.parametrize("ttl", [0, -1, 10081])
def test_session_expiry_rejects_out_of_range_ttl(ttl: int):
    with pytest.raises(ValueError, match="SESSION_TTL_MINUTES"):
        session_expiry(ttl)


@pytest.mark.parametrize("configured, expected", [("60", 60), ("1", 1), ("10080", 10080)])
def test_ttl_configuration_accepts_only_supported_values(monkeypatch, configured: str,
expected: int):
    monkeypatch.setenv("SESSION_TTL_MINUTES", configured)

    assert _ttl_minutes() == expected


@pytest.mark.parametrize("configured", ["0", "10081", "not-a-number"])
def test_ttl_configuration_fails_closed(monkeypatch, configured: str):
    monkeypatch.setenv("SESSION_TTL_MINUTES", configured)

    with pytest.raises(HTTPException) as error:
```

```
        _ttl_minutes()

    assert error.value.status_code == 500
    assert error.value.detail == "Session configuration is invalid"


def test_session_tokens_are_opaque_and_only_digest_is_stable():
    token, digest = new_session_token()
    another_token, another_digest = new_session_token()

    assert token != another_token
    assert digest == token_digest(token)
    assert digest != token
    assert digest != another_digest


def test_login_maps_database_outage_to_controlled_error(monkeypatch):
    import app.main as main

    def unavailable_database():
        import psycopg

        raise psycopg.OperationalError("synthetic database detail")

    monkeypatch.setattr(main, "connect", unavailable_database)
    response = TestClient(app).post(
        "/auth/login",
        json={"email": "synthetic@example.com", "password": "synthetic-password"},
    )

    assert response.status_code == 503
    assert response.json() == {"detail": "Authentication service unavailable"}
    assert "synthetic database detail" not in response.text


def test_protected_route_rejects_missing_session_before_database_access(monkeypatch):
    import app.main as main

    def unexpected_database_access():
        raise AssertionError("database must not be accessed without a session cookie")

    monkeypatch.setattr(main, "connect", unexpected_database_access)
    response = TestClient(app).get("/auth/me")
```

```
    assert response.status_code == 401
    assert response.json() == {"detail": "Authentication required"}


@pytest.mark.parametrize(
    ("user", "expected_status"),
    [({"permissions": ["platform.admin"]}, 200), ({"permissions": []}, 403), ({}, 403)],
)
def test_permission_dependency_allows_only_explicit_permission(user, expected_status):
    from fastapi import Depends, FastAPI
    from fastapi.testclient import TestClient

    import app.main as main

    protected_app = FastAPI()
    protected_app.dependency_overrides[main.get_current_user] = lambda: user
    protected_app.add_api_route(
        "/platform-admin",
        lambda _authorized=Depends(main.require_permission("platform.admin")): {"allowed":
True},
    )

    response = TestClient(protected_app).get("/platform-admin")
    assert response.status_code == expected_status
    if expected_status == 403:
        assert response.json() == {"detail": "Permission denied"}


def test_permission_dependency_ignores_client_supplied_role_header():
    from fastapi import Depends, FastAPI
    from fastapi.testclient import TestClient

    import app.main as main

    protected_app = FastAPI()
    protected_app.dependency_overrides[main.get_current_user] = lambda: {"permissions": []}
    protected_app.add_api_route(
        "/platform-admin",
        lambda _authorized=Depends(main.require_permission("platform.admin")): {"allowed":
True},
    )

    response = TestClient(protected_app).get("/platform-admin", headers={"X-Role":
"platform_admin"})
```

```
    assert response.status_code == 403


@pytest.mark.parametrize(
    ("method", "path", "kwargs"),
    [
        ("get", "/auth/me", {"cookies": {"finsight_session": "synthetic-token"}}),
        ("post", "/auth/logout", {"cookies": {"finsight_session": "synthetic-token"}}),
        ("get",
            "/api/v1/admin/authorization/capabilities",
            {"cookies": {"finsight_session": "synthetic-token"}},
        ),
    ],
)
def test_session_database_outages_return_controlled_error(monkeypatch, method, path,
kwargs):
    import app.main as main
    import psycopg

    def unavailable_database():
        raise psycopg.OperationalError("synthetic database detail")

    monkeypatch.setattr(main, "connect", unavailable_database)
    response = getattr(TestClient(app), method)(path, **kwargs)

    assert response.status_code == 503
    assert response.json() == {"detail": "Authentication service unavailable"}
    assert "synthetic database detail" not in response.text


def test_database_configuration_is_required(monkeypatch):
    import app.database as database

    monkeypatch.delenv("DATABASE_URL", raising=False)

    with pytest.raises(RuntimeError, match="DATABASE_URL is required"):
        database.connect()


def test_login_rejects_invalid_input_before_database_access():
    response = TestClient(app).post(
        "/auth/login",
        json={"email": "not-an-email", "password": ""},
    )
```

```
    assert response.status_code == 422
    assert "synthetic database detail" not in response.text


@pytest.mark.parametrize("configured", ["1", "true", "yes"])
def test_secure_cookie_accepts_explicit_secure_values(monkeypatch, configured: str):
    monkeypatch.setenv("COOKIE_SECURE", configured)

    assert _secure_cookie()


@pytest.mark.parametrize("configured", ["0", "false", "no", "invalid"])
def test_secure_cookie_treats_other_explicit_values_as_insecure(monkeypatch, configured:
str):
    monkeypatch.setenv("COOKIE_SECURE", configured)

    assert not _secure_cookie()


@pytest.mark.parametrize(("app_env", "expected"), [("production", True), ("development",
False), ("test", False)])
def test_secure_cookie_defaults_by_environment(monkeypatch, app_env: str, expected: bool):
    monkeypatch.delenv("COOKIE_SECURE", raising=False)
    monkeypatch.setenv("APP_ENV", app_env)

    assert _secure_cookie() is expected
from __future__ import annotations

from uuid import uuid4

import psycopg
import pytest
from fastapi.testclient import TestClient

from app.auth import new_session_token, session_expiry
from app.database import connect
from app.main import SESSION_COOKIE, app, require_permission


def test_identity_schema_rejects_duplicate_email_and_cascades_sessions():
    email = f"constraint-{uuid4().hex}@example.com"
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
```

```
            "INSERT INTO users (email, password_hash, display_name) VALUES (%s, %s, %s)
RETURNING user_id",
            (email, "synthetic-hash", "Synthetic Constraint Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with pytest.raises(psycopg.errors.UniqueViolation):
            with connect() as conn, conn.cursor() as cur:
                cur.execute(
                    "INSERT INTO users (email, password_hash, display_name) VALUES (%s, %s,
%s)",
                    (email.upper(), "synthetic-hash", "Duplicate Synthetic Test"),
                )

        _, digest = new_session_token()
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at) VALUES
(%s, %s, %s)",
                (user_id, digest, session_expiry(5)),
            )
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
            cur.execute("SELECT count(*) FROM auth_sessions WHERE user_id = %s", (user_id,))
            assert cur.fetchone()[0] == 0
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_platform_permission_is_resolved_from_database_role_and_session():
    admin_email = f"platform-admin-{uuid4().hex}@example.test"
    user_email = f"unprivileged-{uuid4().hex}@example.test"
    tokens = []
    with connect() as conn, conn.cursor() as cur:
        cur.execute("SELECT role_id FROM roles WHERE role_key = 'platform_admin' AND
scope = 'platform'")
        admin_role_id = cur.fetchone()[0]
        cur.execute(
            "INSERT INTO users (email, display_name, platform_role_id) VALUES (%s, %s, %s)
RETURNING user_id",
            (admin_email, "Synthetic Platform Admin", admin_role_id),
        )
        admin_id = cur.fetchone()[0]
```

```
        cur.execute(
            "INSERT INTO users (email, display_name) VALUES (%s, %s) RETURNING user_id",
            (user_email, "Synthetic Standard User"),
        )
        user_id = cur.fetchone()[0]
        for owner_id in (admin_id, user_id):
            token, digest = new_session_token()
            tokens.append(token)
            cur.execute(
                "INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at) VALUES
(%s, %s, %s)",
                (owner_id, digest, session_expiry(5)),
            )

    try:
        with TestClient(app) as client:
            anonymous = client.get("/api/v1/admin/authorization/capabilities")
            assert anonymous.status_code == 401
            assert anonymous.json() == {"detail": "Authentication required"}

            invalid_session = client.get(
                "/api/v1/admin/authorization/capabilities",
                cookies={SESSION_COOKIE: "invalid-session-token"},
            )
            assert invalid_session.status_code == 401

            allowed = client.get(
                "/api/v1/admin/authorization/capabilities",
                cookies={SESSION_COOKIE: tokens[0]},
            )
            #₹ The request_id header is optional, but if supplied it must be validated and preserved
as the correlation_id.
            correlated = client.get(
                "/api/v1/admin/authorization/capabilities",
                cookies={SESSION_COOKIE: tokens[0]},
                headers={"request_id": "client-correlation-123"},
            )
            assert correlated.status_code == 200
            correlated_body = correlated.json()
            assert correlated_body["request_id"].startswith("req_")
            assert correlated_body["request_id"] != "client-correlation-123"
            assert correlated_body["correlation_id"] == "client-correlation-123"

            malformed = client.get(
```

```
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: tokens[0]},
            headers={"request_id": "x" * 129},
        )
        assert malformed.status_code == 400

        # The platform-admin user should be allowed to inspect the authorization policy.
        assert allowed.status_code == 200
        body = allowed.json()
        assert body["data"]["subject"]["user_id"] == str(admin_id)
        assert body["data"]["subject"]["platform_role_code"] == "platform_admin"
        assert body["data"]["effective_permissions"] == ["platform.admin"]
        assert body["data"]["capabilities"] == ["inspect_authorization_policy"]
        assert body["data"]["policy_version"] == "sprint-1-platform-rbac-v1"
        assert body["request_id"].startswith("req_")
        assert body["correlation_id"] == body["request_id"]

        denied = client.get(
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: tokens[1]},
            headers={"X-Role": "platform_admin"},
        )
        assert denied.status_code == 403
        assert denied.json() == {"detail": "Permission denied"}
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id IN (%s, %s)", (admin_id, user_id))



def test_permission_grant_revocation_takes_effect_on_next_request():
    suffix = uuid4().hex
    role_key = f"test-admin-{suffix}"
    email = f"permission-revocation-{suffix}@example.test"
    user_id = None
    role_id = None
    permission_id = None

    try:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
                INSERT INTO roles (role_key, display_name, scope, description)
                VALUES (%s, %s, 'platform', %s)
```

```
        RETURNING role_id
        """,
        (role_key, "Synthetic Permission Revocation Role", "Test-only role"),
    )
    role_id = cur.fetchone()[0]

    cur.execute(
        "SELECT permission_id FROM permissions WHERE permission_key = %s",
        ("platform.admin",),
    )
    row = cur.fetchone()
    assert row is not None, "platform.admin permission must exist"
    permission_id = row[0]

    cur.execute(
        "INSERT INTO role_permissions (role_id, permission_id) VALUES (%s, %s)",
        (role_id, permission_id),
    )
    cur.execute(
        """
        INSERT INTO users (email, display_name, platform_role_id)
        VALUES (%s, %s, %s)
        RETURNING user_id
        """,
        (email, "Synthetic Permission Revocation User", role_id),
    )
    user_id = cur.fetchone()[0]

    token, digest = new_session_token()
    cur.execute(
        """
        INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at)
        VALUES (%s, %s, %s)
        """,
        (user_id, digest, session_expiry(5)),
    )

with TestClient(app) as client:
    first_response = client.get(
        "/api/v1/admin/authorization/capabilities",
        cookies={SESSION_COOKIE: token},
    )
    assert first_response.status_code == 200
    assert first_response.json()["data"]["effective_permissions"] == [
```

```
            "platform.admin"
        ]

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM role_permissions WHERE role_id = %s AND permission_id =
%s",
                (role_id, permission_id),
            )

        next_response = client.get(
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: token},
        )
        assert next_response.status_code == 403
        assert next_response.json() == {"detail": "Permission denied"}

    finally:
        with connect() as conn, conn.cursor() as cur:
            if user_id is not None:
                cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
            if role_id is not None:
                cur.execute("DELETE FROM roles WHERE role_id = %s", (role_id,))

def test_platform_role_assignment_rejects_workspace_scoped_roles():
    email = f"wrong-scope-{uuid4().hex}@example.test"
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO roles (role_key, display_name, scope) VALUES (%s, %s, 'workspace')
RETURNING role_id",
            (f"ws-{uuid4().hex[:12]}", "Synthetic Workspace Role"),
        )
        workspace_role_id = cur.fetchone()[0]
        cur.execute(
            "INSERT INTO users (email, display_name) VALUES (%s, 'Synthetic Scope User')
RETURNING user_id",
            (email,),
        )
        user_id = cur.fetchone()[0]
    try:
        with pytest.raises(psycopg.errors.CheckViolation):
            with connect() as conn, conn.cursor() as cur:
                cur.execute("UPDATE users SET platform_role_id = %s WHERE user_id = %s",
(workspace_role_id, user_id))
```

```
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
            cur.execute("DELETE FROM roles WHERE role_id = %s", (workspace_role_id,))




def test_platform_role_revocation_takes_effect_on_next_request():
    email = f"revoked-platform-admin-{uuid4().hex}@example.test"
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "SELECT role_id FROM roles WHERE role_key = 'platform_admin' AND scope =
'platform'"
        )
        row = cur.fetchone()
        assert row is not None, "Platform-admin role must exist"
        admin_role_id = row[0]
        cur.execute(
            "INSERT INTO users (email, display_name, platform_role_id) "
            "VALUES (%s, %s, %s) RETURNING user_id",
            (email, "Synthetic Revocation Test", admin_role_id),
        )
        user_id = cur.fetchone()[0]
        token, digest = new_session_token()
        cur.execute(
            "INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at) "
            "VALUES (%s, %s, %s)",
            (user_id, digest, session_expiry(5)),
        )

    try:
        with TestClient(app) as client:
            first_response = client.get(
                "/api/v1/admin/authorization/capabilities",
                cookies={SESSION_COOKIE: token},
            )
            assert first_response.status_code == 200
            assert first_response.json()["data"]["effective_permissions"] == [
                "platform.admin"
            ]

            # Revoke this test user's role without changing shared role permissions.
            with connect() as conn, conn.cursor() as cur:
```

```
            cur.execute(
                "UPDATE users SET platform_role_id = NULL WHERE user_id = %s",
                (user_id,),
            )

        next_response = client.get(
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: token},
        )
        assert next_response.status_code == 403
        assert next_response.json() == {"detail": "Permission denied"}
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


# def test_capabilities_request_id_is_canonical_and_correlation_is_preserved():
#     with connect() as conn, conn.cursor() as cur:
#         cur.execute(
#         """
#         SELECT u.user_id, s.refresh_token_hash
===== TEST CONFIGURATION =====
===== DATABASE TEST FIXTURES AND MIGRATION RUNNER =====
Binary file apps/api/tests/__pycache__/test_auth_unit.cpython-312-pytest-9.0.3.pyc matches
Binary file apps/api/tests/__pycache__/test_provision_user.cpython-312.pyc matches
Binary file apps/api/tests/__pycache__/test_provision_user.cpython-314-pytest-8.4.2.pyc
matches
Binary file apps/api/tests/__pycache__/test_provision_user.cpython-314-pytest-9.0.3.pyc
matches
Binary file apps/api/tests/__pycache__/test_provision_user.cpython-314.pyc matches
Binary file apps/api/tests/__pycache__/test_auth_unit.cpython-314.pyc matches
Binary file apps/api/tests/__pycache__/test_auth_unit.cpython-314-pytest-8.4.2.pyc matches
Binary file apps/api/tests/__pycache__/test_auth_unit.cpython-314-pytest-9.0.3.pyc matches
Binary file apps/api/tests/__pycache__/test_provision_user.cpython-312-pytest-9.0.3.pyc
matches
Binary file apps/api/tests/__pycache__/test_auth_unit.cpython-312.pyc matches
apps/api/tests/test_auth_unit.py:164:    monkeypatch.delenv("DATABASE_URL", raising=False)
apps/api/tests/test_auth_unit.py:166:    with pytest.raises(RuntimeError,
match="DATABASE_URL is required"):
apps/api/tests/test_provision_user.py:23:    assert "DATABASE_URL" not in result.stderr
Binary file scripts/__pycache__/run_live_e2e.cpython-314.pyc matches
Binary file scripts/__pycache__/package_release.cpython-312.pyc matches
Binary file scripts/__pycache__/package_release.cpython-314.pyc matches
scripts/run_live_e2e.py:51:    if not os.environ.get("DATABASE_URL"):
```

```
scripts/run_live_e2e.py:52:        raise SystemExit("DATABASE_URL must point to a disposable
PostgreSQL test database")
scripts/package_release.py:54:            "database_migrations": ["0001_auth_identity.sql",
"0002_platform_authorization.sql"],
===== LOCAL CONFIGURATION NAMES (VALUES HIDDEN) =====
DATABASE_URL
WEB_ORIGIN
SESSION_TTL_MINUTES
COOKIE_SECURE
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % pytest -q tests/test_audit_service.py

==========================================================================
============== ERRORS
==========================================================================
===============
_____________________________________________________________________ ERROR
collecting apps/api/tests/test_audit_service.py
_____________________________________________________________________
ImportError while importing test module
'/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/apps/api/tests/test_audit_service.
py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/importlib/__init__.py:90: in
import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests/test_audit_service.py:8: in <module>
    from app.audit import record_auth_event
E   ModuleNotFoundError: No module named 'app.audit'
==========================================================================
====== short test summary info
==========================================================================
======
ERROR tests/test_audit_service.py
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
1 error in 0.05s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
```

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % >....
    outcome: str,
    actor_user_id: UUID | None,
    details: dict[str, Any] | None = None,
    correlation_id: UUID | None = None,
    occurred_at: datetime | None = None,
) -> None:
    """Persist one authentication audit event on the caller's transaction.

    Persistence errors intentionally propagate to the caller. The caller must
    roll back and apply the documented fail-closed policy; events are never
    reported as recorded when this insert fails.
    """
    if outcome not in _ALLOWED_OUTCOMES:
        raise ValueError("Unsupported audit outcome")
    if not action_key or len(action_key) > 120:
        raise ValueError("action_key must contain 1 to 120 characters")

    redacted_details = _redact(details or {})
    payload = json.dumps(redacted_details, separators=(",", ":"), sort_keys=True)

    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO audit_events (
                workspace_id,
                actor_user_id,
                action_key,
                resource_type,
                resource_id,
                outcome,
                correlation_id,
                details_redacted,
                occurred_at
            )
            VALUES (NULL, %s, %s, 'authentication', NULL, %s, %s, %s::jsonb, %s)
            """,
            (
                actor_user_id,
                action_key,
                outcome,
                correlation_id,
                payload,
```

```
            occurred_at or datetime.now(UTC),
        ),
    )
PY

pytest -q tests/test_audit_service.py
.......
[100%]
7 passed in 0.01s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % echo "===== CHECK NEXT
MIGRATION NAME ====="
if [ -e migrations/0003_auth_audit_and_login_throttling.sql ]; then
  echo "EXISTS: inspect before changing"
else
  echo "AVAILABLE: migrations/0003_auth_audit_and_login_throttling.sql"
fi

echo "===== RELEASE MIGRATION MANIFEST ====="
cd ../..
grep -n -C 4 'database_migrations' scripts/package_release.py

echo "===== TEST DATABASE CONFIGURATION STATUS ====="
if [ -n "${DATABASE_URL:-}" ]; then
  echo "DATABASE_URL is set (value intentionally hidden)"
else
  echo "DATABASE_URL is not set in this shell"
fi

echo "===== CHECK AUDIT TABLE IN THE CONFIGURED DATABASE ====="
if [ -n "${DATABASE_URL:-}" ]; then
  psql "$DATABASE_URL" -X -v ON_ERROR_STOP=1 -Atc \
    "SELECT to_regclass('public.audit_events');"
else
  echo "SKIPPED: no DATABASE_URL in this shell"
fi
===== CHECK NEXT MIGRATION NAME =====
AVAILABLE: migrations/0003_auth_audit_and_login_throttling.sql
===== RELEASE MIGRATION MANIFEST =====
50-            "version": version,
51-            "commit_sha": commit_sha,
52-            "frontend": {"runtime": "Node.js 22", "entrypoint": "node frontend/server.js"},
53-            "backend": {"runtime": "Python 3.12", "entrypoint": "python -m uvicorn
app.main:app"},
```

```
54:            "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql"],
55-            "configuration": "Runtime values are supplied through environment variables; see
RELEASE_README.md.",
56-        }
57-        (payload / "RELEASE-METADATA.json").write_text(json.dumps(metadata, indent=2) +
"\n", encoding="utf-8")
58-        release_readme = ROOT / "docs/release/RELEASE_README.md"
===== TEST DATABASE CONFIGURATION STATUS =====
DATABASE_URL is not set in this shell
===== CHECK AUDIT TABLE IN THE CONFIGURED DATABASE =====
SKIPPED: no DATABASE_URL in this shell
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== CURRENT
DIRECTORY ====="
pwd

echo "===== DATABASE CONFIG REFERENCES (VALUES HIDDEN) ====="
grep -nE 'DATABASE_URL|finsight_fin15_test|55439|AUTH_LOGIN|THROTTL' \
  .env .env.example apps/api/.env apps/api/.env.example 2>/dev/null \
  | sed -E 's/(DATABASE_URL=).*/\1[HIDDEN]/; s/(PASSWORD=).*/\1[HIDDEN]/'

echo "===== LOCAL POSTGRESQL READINESS ====="
pg_isready -h localhost -p 55439 2>&1 || true

echo "===== MIGRATION FILES ====="
ls -1 apps/api/migrations
===== CURRENT DIRECTORY =====
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI
===== DATABASE CONFIG REFERENCES (VALUES HIDDEN) =====
.env.example:1:# Local development placeholders only. Replace DATABASE_URL locally; never
commit it.
.env.example:2:DATABASE_URL=[HIDDEN]
===== LOCAL POSTGRESQL READINESS =====
localhost:55439 - accepting connections
===== MIGRATION FILES =====
0001_auth_identity.sql
0002_platform_authorization.sql
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== ROOT .ENV
====="
if [ -f .env ]; then
  echo ".env exists"
else
  echo ".env is absent"
```

```
fi

echo "===== API DATABASE CONNECTION CODE ====="
sed -n '1,160p' apps/api/app/database.py

echo "===== LOCAL DATABASE NAMES ====="
psql -h localhost -p 55439 -d postgres -X -Atc \
  "SELECT datname FROM pg_database
   WHERE datistemplate = false
   ORDER BY datname;"
===== ROOT .ENV =====
.env is absent
===== API DATABASE CONNECTION CODE =====
"""Small synchronous PostgreSQL boundary for the auth endpoints."""

from __future__ import annotations

import os
from pathlib import Path

import psycopg
from dotenv import load_dotenv

load_dotenv(Path(__file__).resolve().parents[3] / ".env")


def connect() -> psycopg.Connection:
    database_url = os.environ.get("DATABASE_URL")
    if not database_url:
        raise RuntimeError("DATABASE_URL is required")
    return psycopg.connect(database_url, connect_timeout=3)
===== LOCAL DATABASE NAMES =====
finsight
finsight_fin15_test
finsight_fin75
postgres
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

postgres
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== TEST
DATABASE CONNECTION ====="
psql -h localhost -p 55439 -d finsight_fin15_test \
  -X -v ON_ERROR_STOP=1 -c \
  "SELECT current_database() AS database_name,
```

```
        current_user AS database_user,
        current_setting('transaction_read_only') AS read_only_mode;"

echo "===== EXISTING MIGRATION TRACKING TABLES ====="
psql -h localhost -p 55439 -d finsight_fin15_test \
  -X -v ON_ERROR_STOP=1 -c \
  "SELECT table_schema, table_name
   FROM information_schema.tables
   WHERE table_schema = 'public'
     AND (table_name ILIKE '%migration%'
          OR table_name IN ('users','auth_sessions','roles',
                            'permissions','audit_events'))
   ORDER BY table_name;"

echo "===== EXISTING AUTH TABLE COLUMNS ====="
psql -h localhost -p 55439 -d finsight_fin15_test \
  -X -v ON_ERROR_STOP=1 -c \
  "SELECT table_name, column_name, data_type
   FROM information_schema.columns
   WHERE table_schema = 'public'
     AND table_name IN ('users','auth_sessions','audit_events')
   ORDER BY table_name, ordinal_position;"
===== TEST DATABASE CONNECTION =====
    database_name    | database_user | read_only_mode
---------------------+---------------+----------------
 finsight_fin15_test | sandeeph      | off
(1 row)

===== EXISTING MIGRATION TRACKING TABLES =====
 table_schema |  table_name
--------------+---------------
 public       | auth_sessions
 public       | permissions
 public       | roles
 public       | users
(4 rows)

===== EXISTING AUTH TABLE COLUMNS =====
  table_name   |    column_name     |        data_type
---------------+--------------------+-------------------------
 auth_sessions | session_id         | uuid
 auth_sessions | user_id            | uuid
 auth_sessions | refresh_token_hash | text
 auth_sessions | ip_address         | inet
```

```
 auth_sessions | user_agent         | text
 auth_sessions | created_at         | timestamp with time zone
 auth_sessions | last_seen_at       | timestamp with time zone
 auth_sessions | expires_at         | timestamp with time zone
 auth_sessions | revoked_at         | timestamp with time zone
 auth_sessions | revoke_reason      | character varying
 users         | user_id            | uuid
 users         | email              | character varying
 users         | password_hash      | text
 users         | display_name       | character varying
 users         | account_status     | character varying
 users         | platform_role_id   | uuid
 users         | email_verified_at  | timestamp with time zone
 users         | mfa_required       | boolean
 users         | last_login_at      | timestamp with time zone
 users         | created_at         | timestamp with time zone
 users         | updated_at         | timestamp with time zone
 users         | deleted_at         | timestamp with time zone
(22 rows)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %


(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
    actor_user_id UUID NULL REFERENCES users(user_id) ON DELETE SET NULL,
    action_key VARCHAR(120) NOT NULL,
    resource_type VARCHAR(80) NOT NULL,
    resource_id UUID NULL,
    outcome VARCHAR(20) NOT NULL
        CHECK (outcome IN ('success', 'failure', 'denied')),
    correlation_id UUID NULL,
    details_redacted JSONB NOT NULL DEFAULT '{}'::jsonb,
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_events_occurred_at
    ON audit_events (occurred_at DESC);

CREATE INDEX idx_audit_events_actor_occurred
    ON audit_events (actor_user_id, occurred_at DESC);

CREATE INDEX idx_audit_events_action_occurred
    ON audit_events (action_key, occurred_at DESC);
```

```
-- Persistent, shared login-failure counters.
-- key_hash must be a keyed digest generated by the application;
-- raw email addresses, IP addresses, passwords and tokens do not belong here.
CREATE TABLE auth_login_throttle (
    scope_type VARCHAR(16) NOT NULL
        CHECK (scope_type IN ('account', 'ip')),
    key_hash CHAR(64) NOT NULL,
    failed_attempts INTEGER NOT NULL DEFAULT 0
        CHECK (failed_attempts >= 0),
    window_started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    blocked_until TIMESTAMPTZ NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (scope_type, key_hash)
);

CREATE INDEX idx_auth_login_throttle_blocked_until
    ON auth_login_throttle (blocked_until)
    WHERE blocked_until IS NOT NULL;

COMMIT;
SQL

echo "===== VERIFY MIGRATION FILE ====="
sed -n '1,220p' apps/api/migrations/0003_auth_audit_and_login_throttling.sql

echo "===== CHECK PATCH SCOPE ====="
git status --short

===== VERIFY MIGRATION FILE =====
BEGIN;

-- Authentication/security audit trail.
-- workspace_id remains nullable because the current application schema
-- does not yet contain the workspaces table.
CREATE TABLE audit_events (
    audit_event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NULL,
    actor_user_id UUID NULL REFERENCES users(user_id) ON DELETE SET NULL,
    action_key VARCHAR(120) NOT NULL,
    resource_type VARCHAR(80) NOT NULL,
    resource_id UUID NULL,
    outcome VARCHAR(20) NOT NULL
        CHECK (outcome IN ('success', 'failure', 'denied')),
    correlation_id UUID NULL,
```

```
    details_redacted JSONB NOT NULL DEFAULT '{}'::jsonb,
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_events_occurred_at
    ON audit_events (occurred_at DESC);

CREATE INDEX idx_audit_events_actor_occurred
    ON audit_events (actor_user_id, occurred_at DESC);

CREATE INDEX idx_audit_events_action_occurred
    ON audit_events (action_key, occurred_at DESC);

-- Persistent, shared login-failure counters.
-- key_hash must be a keyed digest generated by the application;
-- raw email addresses, IP addresses, passwords and tokens do not belong here.
CREATE TABLE auth_login_throttle (
    scope_type VARCHAR(16) NOT NULL
        CHECK (scope_type IN ('account', 'ip')),
    key_hash CHAR(64) NOT NULL,
    failed_attempts INTEGER NOT NULL DEFAULT 0
        CHECK (failed_attempts >= 0),
    window_started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    blocked_until TIMESTAMPTZ NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (scope_type, key_hash)
);

CREATE INDEX idx_auth_login_throttle_blocked_until
    ON auth_login_throttle (blocked_until)
    WHERE blocked_until IS NOT NULL;

COMMIT;
===== CHECK PATCH SCOPE =====
?? apps/api/app/audit.py
?? apps/api/migrations/0003_auth_audit_and_login_throttling.sql
?? apps/api/tests/test_audit_service.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % set -e

echo "===== PREFLIGHT ====="
psql -h localhost -p 55439 -d finsight_fin15_test \
  -X -v ON_ERROR_STOP=1 -c \
```

```
  "SELECT current_database(),
          to_regclass('public.audit_events') AS audit_events_before,
          to_regclass('public.auth_login_throttle') AS throttle_before,
          gen_random_uuid() AS uuid_function_check;"

echo "===== APPLY MIGRATION 0003 TO TEST DATABASE ====="
psql -h localhost -p 55439 -d finsight_fin15_test \
  -X -v ON_ERROR_STOP=1 \
  -f apps/api/migrations/0003_auth_audit_and_login_throttling.sql

echo "===== VERIFY CREATED TABLES AND INDEXES ====="
psql -h localhost -p 55439 -d finsight_fin15_test \
  -X -v ON_ERROR_STOP=1 -c \
  "SELECT tablename
   FROM pg_tables
   WHERE schemaname = 'public'
     AND tablename IN ('audit_events', 'auth_login_throttle')
   ORDER BY tablename;"

psql -h localhost -p 55439 -d finsight_fin15_test \
  -X -v ON_ERROR_STOP=1 -c \
  "SELECT indexname, tablename
   FROM pg_indexes
   WHERE schemaname = 'public'
     AND tablename IN ('audit_events', 'auth_login_throttle')
   ORDER BY tablename, indexname;"
===== PREFLIGHT =====
  current_database   | audit_events_before | throttle_before |         uuid_function_check
---------------------+---------------------+-----------------+--------------------------------------
 finsight_fin15_test |                     |                 | 9bbb85e0-1e1f-4ee5-9f05-9c29edb9695d
(1 row)

===== APPLY MIGRATION 0003 TO TEST DATABASE =====
BEGIN
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE TABLE
CREATE INDEX
COMMIT
===== VERIFY CREATED TABLES AND INDEXES =====
      tablename
---------------------
```

```
 audit_events
 auth_login_throttle
(2 rows)

              indexname               |      tablename
--------------------------------------+---------------------
 audit_events_pkey                    | audit_events
 idx_audit_events_action_occurred     | audit_events
 idx_audit_events_actor_occurred      | audit_events
 idx_audit_events_occurred_at         | audit_events
 auth_login_throttle_pkey             | auth_login_throttle
 idx_auth_login_throttle_blocked_until | auth_login_throttle
(6 rows)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %


(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== LOGIN /
LOGOUT HANDLERS ====="
grep -n -A180 -B20 'def login\|def logout\|@app.post("/auth/login")\|@app.post("/auth/logout")' \
  apps/api/app/main.py

echo "===== AUTH IMPORTS AND CONFIGURATION ====="
sed -n '1,150p' apps/api/app/main.py

echo "===== AUDIT SERVICE ====="
sed -n '1,220p' apps/api/app/audit.py

echo "===== AUDIT SERVICE TESTS ====="
sed -n '1,260p' apps/api/tests/test_audit_service.py

echo "===== ENVIRONMENT EXAMPLE ====="
cat .env.example

echo "===== RELEASE MIGRATION MANIFEST ====="
sed -n '35,70p' scripts/package_release.py
===== LOGIN / LOGOUT HANDLERS =====
100-            )
101-            return user
102-    except psycopg.Error as exc:
103-        raise HTTPException(status_code=503, detail="Authentication service unavailable")
from exc
104-
105-
```

```
106-CurrentUser = Annotated[dict[str, Any], Depends(get_current_user)]
107-
108-
109-def require_permission(permission_key: str):
110-    """Build a deny-by-default backend permission dependency."""
111-    def dependency(user: CurrentUser) -> dict[str, Any]:
112-        if permission_key not in user.get("permissions", []):
113-            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Permission denied")
114-        return user
115-
116-    return dependency
117-
118-
119-@app.post("/auth/login", response_model=UserView)
120:def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
121-    _check_origin(request)
122-    email = payload.email.strip().lower()
123-    try:
124-        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
125-            cur.execute(
126-                """SELECT user_id, email, display_name, password_hash
127-                   FROM users WHERE lower(email) = %s AND account_status = 'active'
128-                     AND deleted_at IS NULL""",
129-                (email,),
130-            )
131-            user = cur.fetchone()
132-            if not user or not user["password_hash"]:
133-                verify_password(DUMMY_PASSWORD_HASH, payload.password)
134-                raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)
135-            if not verify_password(user["password_hash"], payload.password):
136-                raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)
137-            token, digest = new_session_token()
138-            expires_at = session_expiry(_ttl_minutes())
139-            cur.execute(
140-                """INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at)
141-                   VALUES (%s, %s, %s)""",
142-                (user["user_id"], digest, expires_at),
143-            )
144-            cur.execute("UPDATE users SET last_login_at = %s, updated_at = %s WHERE
user_id = %s",
145-                        (datetime.now(UTC), datetime.now(UTC), user["user_id"]))
146-        response.set_cookie(
147-            SESSION_COOKIE,
```

```
148-            token,
149-            httponly=True,
150-            secure=_secure_cookie(),
151-            samesite="lax",
152-            max_age=_ttl_minutes() * 60,
153-            path="/",
154-        )
155-        return UserView(user_id=user["user_id"], email=user["email"],
display_name=user["display_name"])
156-    except psycopg.Error as exc:
157-        raise HTTPException(status_code=503, detail="Authentication service unavailable")
from exc
158-
159-
160-@app.post("/auth/logout", status_code=204)
161:def logout(request: Request, response: Response,
162-           session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None) ->
Response:
163-    _check_origin(request)
164-    if session_token:
165-        try:
166-            with connect() as conn, conn.cursor() as cur:
167-                cur.execute(
168-                    """UPDATE auth_sessions SET revoked_at = %s, revoke_reason = 'logout'
169-                       WHERE refresh_token_hash = %s AND revoked_at IS NULL""",
170-                    (datetime.now(UTC), token_digest(session_token)),
171-                )
172-        except psycopg.Error as exc:
173-            raise HTTPException(status_code=503, detail="Authentication service unavailable")
from exc
174-    response.delete_cookie(SESSION_COOKIE, path="/", httponly=True,
secure=_secure_cookie(), samesite="lax")
175-    response.status_code = status.HTTP_204_NO_CONTENT
176-    return response
177-
178-
179-@app.get("/auth/me", response_model=UserView)
180-def who_am_i(user: CurrentUser) -> UserView:
181-    return UserView(**user)
182-
183-
184-@app.get("/api/v1/admin/authorization/capabilities")
185-def platform_admin_capabilities(
186-    request: Request,
```

```
187-    _authorized: Annotated[dict[str, Any], Depends(require_permission("platform.admin"))],
188-) -> dict[str, Any]:
189-    """Return the authenticated caller's database-backed platform capabilities."""
190-
191-    supplied_request_id = request.headers.get("request_id", "").strip()
192-    if supplied_request_id and (
193-        len(supplied_request_id) > 128
194-        or any(ord(char) < 32 or ord(char) == 127 for char in supplied_request_id)
195-    ):
196-        raise HTTPException(status_code=400, detail="Invalid request_id")
197-
198-    # Always issue a server-generated canonical request ID.
199-    request_id = f"req_{secrets.token_urlsafe(12)}"
200-    correlation_id = supplied_request_id or request_id
201-    permissions = sorted(set(_authorized.get("permissions", [])))
202-    capabilities = (
203-        ["inspect_authorization_policy"]
204-        if "platform.admin" in permissions
205-        else []
206-    )
207-
208-    return {
209-        "data": {
210-            "subject": {
211-                "user_id": str(_authorized["user_id"]),
212-                "platform_role_code": _authorized.get("role_key"),
213-            },
214-            "effective_permissions": permissions,
215-            "capabilities": capabilities,
216-            "policy_version": "sprint-1-platform-rbac-v1",
217-        },
218-        "request_id": request_id,
219-        "correlation_id": correlation_id,
220-    }
===== AUTH IMPORTS AND CONFIGURATION =====
"""FinSight AI API with PostgreSQL-backed browser sessions."""

from __future__ import annotations

import os
import secrets
from datetime import UTC, datetime
from pathlib import Path
from typing import Annotated, Any
```

```
from uuid import UUID

import psycopg
from dotenv import load_dotenv
from fastapi import Cookie, Depends, FastAPI, HTTPException, Request, Response, status
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, EmailStr, Field
from psycopg.rows import dict_row

from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
from app.database import connect

load_dotenv(Path(__file__).resolve().parents[3] / ".env")

app = FastAPI(title="FinSight AI API", version="0.2.0")
web_origin = os.environ.get("WEB_ORIGIN", "http://localhost:3000")
app.add_middleware(
    CORSMiddleware,
    allow_origins=[web_origin],
    allow_credentials=True,
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)

SESSION_COOKIE = "finsight_session"
GENERIC_LOGIN_ERROR = "Email or password is incorrect."
DUMMY_PASSWORD_HASH = hash_password(secrets.token_urlsafe(32))


class LoginRequest(BaseModel):
    email: EmailStr
    password: str = Field(min_length=1, max_length=1024)


class UserView(BaseModel):
    user_id: UUID
    email: str
    display_name: str


def _ttl_minutes() -> int:
    try:
        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
```

```
        session_expiry(value)
        return value
    except ValueError as exc:
        raise HTTPException(status_code=500, detail="Session configuration is invalid") from exc


def _secure_cookie() -> bool:
    configured = os.environ.get("COOKIE_SECURE")
    if configured is not None:
        return configured.lower() in {"1", "true", "yes"}
    return os.environ.get("APP_ENV", "development").lower() == "production"


def _check_origin(request: Request) -> None:
    origin = request.headers.get("origin")
    if origin and origin != web_origin:
        raise HTTPException(status_code=403, detail="Request origin is not allowed")


def get_current_user(
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> dict[str, Any]:
    if not session_token:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Authentication required")
    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT u.user_id, u.email, u.display_name, r.role_key,
                        COALESCE(array_agg(DISTINCT p.permission_key)
                            FILTER (WHERE p.permission_key IS NOT NULL), ARRAY[]::varchar[])
AS permissions
                    FROM auth_sessions s JOIN users u ON u.user_id = s.user_id
                    LEFT JOIN roles r ON r.role_id = u.platform_role_id AND r.scope = 'platform'
                    LEFT JOIN role_permissions rp ON rp.role_id = r.role_id
                    LEFT JOIN permissions p ON p.permission_id = rp.permission_id
                    WHERE s.refresh_token_hash = %s AND s.revoked_at IS NULL
                      AND s.expires_at > %s AND u.account_status = 'active'
                      AND s.last_seen_at > %s - make_interval(mins => %s)
                      AND u.deleted_at IS NULL
                    GROUP BY u.user_id, u.email, u.display_name, r.role_key""",
                (token_digest(session_token), datetime.now(UTC), datetime.now(UTC),
_ttl_minutes()),
            )
```

```
            user = cur.fetchone()
            if not user:
                raise HTTPException(status_code=401, detail="Authentication required")
            cur.execute(
                "UPDATE auth_sessions SET last_seen_at = %s WHERE refresh_token_hash = %s",
                (datetime.now(UTC), token_digest(session_token)),
            )
            return user
    except psycopg.Error as exc:
        raise HTTPException(status_code=503, detail="Authentication service unavailable") from
exc


CurrentUser = Annotated[dict[str, Any], Depends(get_current_user)]


def require_permission(permission_key: str):
    """Build a deny-by-default backend permission dependency."""
    def dependency(user: CurrentUser) -> dict[str, Any]:
        if permission_key not in user.get("permissions", []):
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Permission
denied")
        return user

    return dependency


@app.post("/auth/login", response_model=UserView)
def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
    _check_origin(request)
    email = payload.email.strip().lower()
    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT user_id, email, display_name, password_hash
                   FROM users WHERE lower(email) = %s AND account_status = 'active'
                     AND deleted_at IS NULL""",
                (email,),
            )
            user = cur.fetchone()
            if not user or not user["password_hash"]:
                verify_password(DUMMY_PASSWORD_HASH, payload.password)
                raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)
            if not verify_password(user["password_hash"], payload.password):
```

```
            raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)
        token, digest = new_session_token()
        expires_at = session_expiry(_ttl_minutes())
        cur.execute(
            """INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at)
               VALUES (%s, %s, %s)""",
            (user["user_id"], digest, expires_at),
        )
        cur.execute("UPDATE users SET last_login_at = %s, updated_at = %s WHERE user_id
= %s",
                    (datetime.now(UTC), datetime.now(UTC), user["user_id"]))
    response.set_cookie(
        SESSION_COOKIE,
        token,
        httponly=True,
        secure=_secure_cookie(),
===== AUDIT SERVICE =====
"""Reusable, redacted audit persistence for authentication security events."""

from __future__ import annotations

import json
import re
from datetime import UTC, datetime
from typing import Any
from uuid import UUID

_ALLOWED_OUTCOMES = frozenset({"success", "failure", "denied"})
_SECRET_KEY = re.compile(
    r"(password|passwd|token|cookie|authorization|api[_-]?key|secret|credential)",
    re.IGNORECASE,
)


def _redact(value: Any) -> Any:
    """Return JSON-safe audit details without values under secret-like keys."""
    if isinstance(value, dict):
        return {
            str(key): (
                "[REDACTED]" if _SECRET_KEY.search(str(key)) else _redact(item)
            )
            for key, item in value.items()
        }
    if isinstance(value, (list, tuple)):
```

```
        return [_redact(item) for item in value]
    if isinstance(value, UUID):
        return str(value)
    if isinstance(value, datetime):
        return value.astimezone(UTC).isoformat()
    if value is None or isinstance(value, (str, int, float, bool)):
        return value
    return str(value)


def record_auth_event(
    conn: Any,
    *,
    action_key: str,
    outcome: str,
    actor_user_id: UUID | None,
    details: dict[str, Any] | None = None,
    correlation_id: UUID | None = None,
    occurred_at: datetime | None = None,
) -> None:
    """Persist one authentication audit event on the caller's transaction.

    Persistence errors intentionally propagate to the caller. The caller must
    roll back and apply the documented fail-closed policy; events are never
    reported as recorded when this insert fails.
    """
    if outcome not in _ALLOWED_OUTCOMES:
        raise ValueError("Unsupported audit outcome")
    if not action_key or len(action_key) > 120:
        raise ValueError("action_key must contain 1 to 120 characters")

    redacted_details = _redact(details or {})
    payload = json.dumps(redacted_details, separators=(",", ":"), sort_keys=True)

    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO audit_events (
                workspace_id,
                actor_user_id,
                action_key,
                resource_type,
                resource_id,
                outcome,
```

```
            correlation_id,
            details_redacted,
            occurred_at
        )
        VALUES (NULL, %s, %s, 'authentication', NULL, %s, %s, %s::jsonb, %s)
        """,
        (
            actor_user_id,
            action_key,
            outcome,
            correlation_id,
            payload,
            occurred_at or datetime.now(UTC),
        ),
    )
===== AUDIT SERVICE TESTS =====
from __future__ import annotations

from datetime import UTC, datetime
from uuid import uuid4

import pytest

from app.audit import record_auth_event


class FakeCursor:
    def __init__(self):
        self.calls = []

    def execute(self, query, params):
        self.calls.append((query, params))

    def __enter__(self):
        return self

    def __exit__(self, *args):
        return False


class FakeConnection:
    def __init__(self):
        self.cursor_instance = FakeCursor()
```

```
    def cursor(self):
        return self.cursor_instance


def test_auth_audit_uses_canonical_columns_and_parameterized_sql():
    conn = FakeConnection()
    actor_id = uuid4()
    occurred_at = datetime(2026, 10, 21, tzinfo=UTC)

    record_auth_event(
        conn,
        action_key="auth.login",
        outcome="success",
        actor_user_id=actor_id,
        occurred_at=occurred_at,
        details={"method": "password"},
    )

    assert len(conn.cursor_instance.calls) == 1
    query, params = conn.cursor_instance.calls[0]

    assert "INSERT INTO audit_events" in query
    for column in (
        "workspace_id",
        "actor_user_id",
        "action_key",
        "resource_type",
        "resource_id",
        "outcome",
        "correlation_id",
        "details_redacted",
        "occurred_at",
    ):
        assert column in query

    assert actor_id in params
    assert "auth.login" in params
    assert "success" in params
    assert occurred_at in params
    assert "password" not in query.lower()


def test_unknown_account_failure_can_be_audited_without_an_actor():
    conn = FakeConnection()
```

```
    record_auth_event(
        conn,
        action_key="auth.login",
        outcome="failure",
        actor_user_id=None,
        details={"reason": "invalid_credentials"},
    )

    _, params = conn.cursor_instance.calls[0]
    assert None in params
    assert "invalid_credentials" in str(params)


@pytest.mark.parametrize(
    "secret",
    [
        "raw-password-value",
        "raw-session-token-value",
        "raw-api-key-value",
        "cookie-secret-value",
    ],
)
def test_auth_audit_redacts_secret_values(secret):
    conn = FakeConnection()

    record_auth_event(
        conn,
        action_key="auth.login",
        outcome="failure",
        actor_user_id=None,
        details={
            "reason": "invalid_credentials",
            "password": secret,
            "session_token": secret,
            "api_key": secret,
            "cookie": secret,
        },
    )

    query, params = conn.cursor_instance.calls[0]
    persisted_values = f"{query} {params}"

    assert secret not in persisted_values
```

```
    assert "invalid_credentials" in persisted_values


def test_auth_audit_rejects_unknown_outcome_before_database_write():
    conn = FakeConnection()

    with pytest.raises(ValueError, match="outcome"):
        record_auth_event(
            conn,
            action_key="auth.login",
            outcome="maybe",
            actor_user_id=None,
        )

    assert conn.cursor_instance.calls == []
===== ENVIRONMENT EXAMPLE =====
# Local development placeholders only. Replace DATABASE_URL locally; never commit it.
DATABASE_URL=postgresql://USER:PASSWORD@127.0.0.1:5432/finsight
WEB_ORIGIN=http://localhost:3000
SESSION_TTL_MINUTES=60
COOKIE_SECURE=false
===== RELEASE MIGRATION MANIFEST =====
    output_dir.mkdir(parents=True, exist_ok=True)
    artifact_base = f"finsight-{version}" + (f"-{commit_sha[:12]}" if commit_sha else "")
    with tempfile.TemporaryDirectory(prefix="finsight-release-") as temp_dir:
        payload = Path(temp_dir) / "finsight"
        frontend = payload / "frontend"
        shutil.copytree(frontend_dir, frontend, symlinks=True)
        shutil.copytree(web_static, frontend / ".next/static", symlinks=True)

        backend = payload / "backend"
        shutil.copytree(ROOT / "apps/api/app", backend / "app",
ignore=shutil.ignore_patterns("__pycache__", "*.pyc"))
        shutil.copytree(ROOT / "apps/api/migrations", backend / "migrations")
        shutil.copy2(ROOT / "apps/api/requirements.lock", backend / "requirements.lock")

        metadata = {
            "artifact": artifact_base,
            "version": version,
            "commit_sha": commit_sha,
            "frontend": {"runtime": "Node.js 22", "entrypoint": "node frontend/server.js"},
            "backend": {"runtime": "Python 3.12", "entrypoint": "python -m uvicorn app.main:app"},
            "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql"],
```

```
        "configuration": "Runtime values are supplied through environment variables; see
RELEASE_README.md.",
    }
    (payload / "RELEASE-METADATA.json").write_text(json.dumps(metadata, indent=2) + "\n",
encoding="utf-8")
    release_readme = ROOT / "docs/release/RELEASE_README.md"
    shutil.copy2(release_readme, payload / "RELEASE_README.md")

    artifact_path = output_dir / f"{artifact_base}.tar.gz"
    def normalized_member(member: tarfile.TarInfo) -> tarfile.TarInfo:
        member.mtime = 0
        member.uid = 0
        member.gid = 0
        member.uname = ""
        member.gname = ""
        return member

    with artifact_path.open("wb") as raw:
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.open("wb") as raw:
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== AUTH TEST
FILES ====="
find apps/api/tests -maxdepth 2 -type f \
  -iname '*auth*' -o -iname '*login*' -o -iname '*session*' \
  -print

echo "===== LOGIN TESTS AND DATABASE MOCKS ====="
grep -R -n -E \
  'auth/login|def test_.*login|GENERIC_LOGIN_ERROR|new_session_token|def
connect|monkeypatch|DATABASE_URL' \
  apps/api/tests

echo "===== AUTH UNIT TESTS ====="
sed -n '1,280p' apps/api/tests/test_auth_unit.py

echo "===== API TEST FILE LIST ====="
find apps/api/tests -maxdepth 1 -type f -print | sort
===== AUTH TEST FILES =====
apps/api/tests/test_auth.py
apps/api/tests/__pycache__/test_auth_unit.cpython-312-pytest-9.0.3.pyc
apps/api/tests/__pycache__/test_auth.cpython-314.pyc
apps/api/tests/__pycache__/test_auth.cpython-312.pyc
apps/api/tests/__pycache__/test_auth.cpython-314-pytest-9.0.3.pyc
apps/api/tests/__pycache__/test_auth.cpython-314-pytest-8.4.2.pyc
```

apps/api/tests/__pycache__/test_auth_unit.cpython-314.pyc
apps/api/tests/__pycache__/test_auth_unit.cpython-314-pytest-8.4.2.pyc
apps/api/tests/__pycache__/test_auth_unit.cpython-314-pytest-9.0.3.pyc
apps/api/tests/__pycache__/test_auth_unit.cpython-312.pyc
apps/api/tests/__pycache__/test_auth.cpython-312-pytest-9.0.3.pyc
apps/api/tests/test_auth_unit.py
===== LOGIN TESTS AND DATABASE MOCKS =====
apps/api/tests/test_identity_constraints.py:9:from app.auth import new_session_token,
session_expiry
apps/api/tests/test_identity_constraints.py:31:   _, digest = new_session_token()
apps/api/tests/test_identity_constraints.py:63:   token, digest = new_session_token()
apps/api/tests/test_identity_constraints.py:171:   token, digest = new_session_token()
apps/api/tests/test_identity_constraints.py:250:   token, digest = new_session_token()
apps/api/tests/test_auth.py:35:def client(monkeypatch):
apps/api/tests/test_auth.py:36:   monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
apps/api/tests/test_auth.py:37:   monkeypatch.setenv("COOKIE_SECURE", "false")
apps/api/tests/test_auth.py:38:   monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
apps/api/tests/test_auth.py:51:def
test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
apps/api/tests/test_auth.py:53:   response = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:76:   known = client.post("/auth/login", json={"email": email,
"password": "wrong-password"})
apps/api/tests/test_auth.py:77:   absent = client.post("/auth/login", json={"email":
"absent@example.com", "password": "wrong-password"})
apps/api/tests/test_auth.py:85:   login = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:106:   login = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:126:def
test_secure_cookie_can_be_required_by_deployment(client, identity, monkeypatch):
apps/api/tests/test_auth.py:128:   monkeypatch.setenv("COOKIE_SECURE", "true")
apps/api/tests/test_auth.py:129:   response = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
Binary file apps/api/tests/__pycache__/test_auth_unit.cpython-312-pytest-9.0.3.pyc matches
Binary file apps/api/tests/__pycache__/test_provision_user.cpython-312.pyc matches
Binary file apps/api/tests/__pycache__/test_auth.cpython-314.pyc matches
Binary file apps/api/tests/__pycache__/test_provision_user.cpython-314-pytest-8.4.2.pyc
matches
Binary file apps/api/tests/__pycache__/test_provision_user.cpython-314-pytest-9.0.3.pyc
matches
Binary file apps/api/tests/__pycache__/test_provision_user.cpython-314.pyc matches
Binary file apps/api/tests/__pycache__/test_auth.cpython-312.pyc matches

Binary file apps/api/tests/__pycache__/test_identity_constraints.cpython-314-pytest-8.4.2.pyc matches
Binary file apps/api/tests/__pycache__/test_identity_constraints.cpython-314-pytest-9.0.3.pyc matches
Binary file apps/api/tests/__pycache__/test_auth.cpython-314-pytest-9.0.3.pyc matches
Binary file apps/api/tests/__pycache__/test_auth.cpython-314-pytest-8.4.2.pyc matches
Binary file apps/api/tests/__pycache__/test_identity_constraints.cpython-312-pytest-9.0.3.pyc matches
Binary file apps/api/tests/__pycache__/test_auth_unit.cpython-314.pyc matches
Binary file apps/api/tests/__pycache__/test_identity_constraints.cpython-314.pyc matches
Binary file apps/api/tests/__pycache__/test_auth_unit.cpython-314-pytest-8.4.2.pyc matches
Binary file apps/api/tests/__pycache__/test_auth_unit.cpython-314-pytest-9.0.3.pyc matches
Binary file apps/api/tests/__pycache__/test_provision_user.cpython-312-pytest-9.0.3.pyc matches
Binary file apps/api/tests/__pycache__/test_auth_unit.cpython-312.pyc matches
Binary file apps/api/tests/__pycache__/test_identity_constraints.cpython-312.pyc matches
Binary file apps/api/tests/__pycache__/test_auth.cpython-312-pytest-9.0.3.pyc matches
apps/api/tests/test_auth_unit.py:9:from app.auth import hash_password, new_session_token, session_expiry, token_digest, verify_password
apps/api/tests/test_auth_unit.py:36:def test_ttl_configuration_accepts_only_supported_values(monkeypatch, configured: str, expected: int):
apps/api/tests/test_auth_unit.py:37:   monkeypatch.setenv("SESSION_TTL_MINUTES", configured)
apps/api/tests/test_auth_unit.py:43:def test_ttl_configuration_fails_closed(monkeypatch, configured: str):
apps/api/tests/test_auth_unit.py:44:   monkeypatch.setenv("SESSION_TTL_MINUTES", configured)
apps/api/tests/test_auth_unit.py:54:   token, digest = new_session_token()
apps/api/tests/test_auth_unit.py:55:   another_token, another_digest = new_session_token()
apps/api/tests/test_auth_unit.py:63:def test_login_maps_database_outage_to_controlled_error(monkeypatch):
apps/api/tests/test_auth_unit.py:71:   monkeypatch.setattr(main, "connect", unavailable_database)
apps/api/tests/test_auth_unit.py:73:   "/auth/login",
apps/api/tests/test_auth_unit.py:82:def test_protected_route_rejects_missing_session_before_database_access(monkeypatch):
apps/api/tests/test_auth_unit.py:88:   monkeypatch.setattr(main, "connect", unexpected_database_access)
apps/api/tests/test_auth_unit.py:146:def test_session_database_outages_return_controlled_error(monkeypatch, method, path, kwargs):
apps/api/tests/test_auth_unit.py:153:   monkeypatch.setattr(main, "connect", unavailable_database)

apps/api/tests/test_auth_unit.py:161:def
test_database_configuration_is_required(monkeypatch):
apps/api/tests/test_auth_unit.py:164:   monkeypatch.delenv("DATABASE_URL", raising=False)
apps/api/tests/test_auth_unit.py:166:   with pytest.raises(RuntimeError,
match="DATABASE_URL is required"):
apps/api/tests/test_auth_unit.py:170:def
test_login_rejects_invalid_input_before_database_access():
apps/api/tests/test_auth_unit.py:172:   "/auth/login",
apps/api/tests/test_auth_unit.py:181:def
test_secure_cookie_accepts_explicit_secure_values(monkeypatch, configured: str):
apps/api/tests/test_auth_unit.py:182:   monkeypatch.setenv("COOKIE_SECURE", configured)
apps/api/tests/test_auth_unit.py:188:def
test_secure_cookie_treats_other_explicit_values_as_insecure(monkeypatch, configured: str):
apps/api/tests/test_auth_unit.py:189:   monkeypatch.setenv("COOKIE_SECURE", configured)
apps/api/tests/test_auth_unit.py:195:def
test_secure_cookie_defaults_by_environment(monkeypatch, app_env: str, expected: bool):
apps/api/tests/test_auth_unit.py:196:   monkeypatch.delenv("COOKIE_SECURE",
raising=False)
apps/api/tests/test_auth_unit.py:197:   monkeypatch.setenv("APP_ENV", app_env)
apps/api/tests/test_provision_user.py:23:   assert "DATABASE_URL" not in result.stderr
apps/api/tests/test_provision_user.py:26:def
test_provision_user_requires_email_and_display_arguments(monkeypatch, capsys):
apps/api/tests/test_provision_user.py:27:   monkeypatch.setattr(sys, "argv", ["provision_user"])
apps/api/tests/test_provision_user.py:33:def
test_provision_user_platform_admin_is_explicit_opt_in(monkeypatch, capsys):
apps/api/tests/test_provision_user.py:34:   monkeypatch.setattr(sys, "argv", ["provision_user",
"admin@example.test", "Synthetic Admin", "--platform-admin"])
apps/api/tests/test_provision_user.py:36:   monkeypatch.setattr(provision_user.getpass,
"getpass", lambda _prompt: next(values))
apps/api/tests/test_provision_user.py:53:   monkeypatch.setattr(provision_user, "connect",
Connection)
apps/api/tests/test_provision_user.py:61:def
test_provision_user_rejects_unknown_role_argument(monkeypatch, capsys):
apps/api/tests/test_provision_user.py:62:   monkeypatch.setattr(sys, "argv", ["provision_user",
"analyst@example.test", "Synthetic User", "--operator"])
apps/api/tests/test_provision_user.py:76:def
test_provision_user_rejects_invalid_identity_fields(monkeypatch, capsys, email, display_name):
apps/api/tests/test_provision_user.py:77:   monkeypatch.setattr(sys, "argv", ["provision_user",
email, display_name])
apps/api/tests/test_provision_user.py:78:   monkeypatch.setattr(provision_user, "connect",
lambda: pytest.fail("database must not be opened"))
apps/api/tests/test_provision_user.py:79:   monkeypatch.setattr(provision_user.getpass,
"getpass", lambda _prompt: pytest.fail("password must not be requested"))

apps/api/tests/test_provision_user.py:86:def
test_provision_user_rejects_empty_or_mismatched_passwords(monkeypatch, capsys,
passwords):
apps/api/tests/test_provision_user.py:87:   monkeypatch.setattr(sys, "argv", ["provision_user",
"analyst@example.com", "Synthetic Analyst"])
apps/api/tests/test_provision_user.py:89:   monkeypatch.setattr(provision_user.getpass,
"getpass", lambda _prompt: next(values))
apps/api/tests/test_provision_user.py:90:   monkeypatch.setattr(provision_user, "connect",
lambda: pytest.fail("invalid password must not change the database"))
apps/api/tests/test_provision_user.py:98:def
test_provision_user_hashes_and_upserts_without_printing_password(monkeypatch, capsys):
apps/api/tests/test_provision_user.py:99:   monkeypatch.setattr(sys, "argv", ["provision_user", "
Analyst@Example.com ", " Synthetic Analyst "])
apps/api/tests/test_provision_user.py:101:   monkeypatch.setattr(provision_user.getpass,
"getpass", lambda _prompt: next(values))
apps/api/tests/test_provision_user.py:129:   monkeypatch.setattr(provision_user, "connect",
Connection)
===== AUTH UNIT TESTS =====
```
from __future__ import annotations

from datetime import UTC, datetime, timedelta

import pytest
from fastapi import HTTPException
from fastapi.testclient import TestClient

from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
from app.main import _secure_cookie, _ttl_minutes, app


def test_password_helpers_accept_only_the_original_password():
    encoded = hash_password("synthetic-password")

    assert encoded.startswith("argon2id")
    assert verify_password(encoded, "synthetic-password")
    assert not verify_password(encoded, "different-password")
    assert not verify_password("not-an-argon2-hash", "synthetic-password")
    assert not verify_password(None, "synthetic-password")


def test_session_expiry_uses_utc_and_requested_ttl():
    now = datetime(2026, 1, 1, tzinfo=UTC)
```

```
    assert session_expiry(15, now) == now + timedelta(minutes=15)


@pytest.mark.parametrize("ttl", [0, -1, 10081])
def test_session_expiry_rejects_out_of_range_ttl(ttl: int):
    with pytest.raises(ValueError, match="SESSION_TTL_MINUTES"):
        session_expiry(ttl)


@pytest.mark.parametrize("configured, expected", [("60", 60), ("1", 1), ("10080", 10080)])
def test_ttl_configuration_accepts_only_supported_values(monkeypatch, configured: str,
expected: int):
    monkeypatch.setenv("SESSION_TTL_MINUTES", configured)

    assert _ttl_minutes() == expected


@pytest.mark.parametrize("configured", ["0", "10081", "not-a-number"])
def test_ttl_configuration_fails_closed(monkeypatch, configured: str):
    monkeypatch.setenv("SESSION_TTL_MINUTES", configured)

    with pytest.raises(HTTPException) as error:
        _ttl_minutes()

    assert error.value.status_code == 500
    assert error.value.detail == "Session configuration is invalid"


def test_session_tokens_are_opaque_and_only_digest_is_stable():
    token, digest = new_session_token()
    another_token, another_digest = new_session_token()

    assert token != another_token
    assert digest == token_digest(token)
    assert digest != token
    assert digest != another_digest


def test_login_maps_database_outage_to_controlled_error(monkeypatch):
    import app.main as main

    def unavailable_database():
        import psycopg
```

```
        raise psycopg.OperationalError("synthetic database detail")

    monkeypatch.setattr(main, "connect", unavailable_database)
    response = TestClient(app).post(
        "/auth/login",
        json={"email": "synthetic@example.com", "password": "synthetic-password"},
    )

    assert response.status_code == 503
    assert response.json() == {"detail": "Authentication service unavailable"}
    assert "synthetic database detail" not in response.text


def test_protected_route_rejects_missing_session_before_database_access(monkeypatch):
    import app.main as main

    def unexpected_database_access():
        raise AssertionError("database must not be accessed without a session cookie")

    monkeypatch.setattr(main, "connect", unexpected_database_access)
    response = TestClient(app).get("/auth/me")

    assert response.status_code == 401
    assert response.json() == {"detail": "Authentication required"}


@pytest.mark.parametrize(
    ("user", "expected_status"),
    [({"permissions": ["platform.admin"]}, 200), ({"permissions": []}, 403), ({}, 403)],
)
def test_permission_dependency_allows_only_explicit_permission(user, expected_status):
    from fastapi import Depends, FastAPI
    from fastapi.testclient import TestClient

    import app.main as main

    protected_app = FastAPI()
    protected_app.dependency_overrides[main.get_current_user] = lambda: user
    protected_app.add_api_route(
        "/platform-admin",
        lambda _authorized=Depends(main.require_permission("platform.admin")): {"allowed":
True},
    )
```

```
    response = TestClient(protected_app).get("/platform-admin")
    assert response.status_code == expected_status
    if expected_status == 403:
        assert response.json() == {"detail": "Permission denied"}


def test_permission_dependency_ignores_client_supplied_role_header():
    from fastapi import Depends, FastAPI
    from fastapi.testclient import TestClient

    import app.main as main

    protected_app = FastAPI()
    protected_app.dependency_overrides[main.get_current_user] = lambda: {"permissions": []}
    protected_app.add_api_route(
        "/platform-admin",
        lambda _authorized=Depends(main.require_permission("platform.admin")): {"allowed":
True},
    )

    response = TestClient(protected_app).get("/platform-admin", headers={"X-Role":
"platform_admin"})
    assert response.status_code == 403


@pytest.mark.parametrize(
    ("method", "path", "kwargs"),
    [
        ("get", "/auth/me", {"cookies": {"finsight_session": "synthetic-token"}}),
        ("post", "/auth/logout", {"cookies": {"finsight_session": "synthetic-token"}}),
        ("get",
            "/api/v1/admin/authorization/capabilities",
            {"cookies": {"finsight_session": "synthetic-token"}},
        ),
    ],
)
def test_session_database_outages_return_controlled_error(monkeypatch, method, path,
kwargs):
    import app.main as main
    import psycopg

    def unavailable_database():
        raise psycopg.OperationalError("synthetic database detail")
```

```
    monkeypatch.setattr(main, "connect", unavailable_database)
    response = getattr(TestClient(app), method)(path, **kwargs)

    assert response.status_code == 503
    assert response.json() == {"detail": "Authentication service unavailable"}
    assert "synthetic database detail" not in response.text


def test_database_configuration_is_required(monkeypatch):
    import app.database as database

    monkeypatch.delenv("DATABASE_URL", raising=False)

    with pytest.raises(RuntimeError, match="DATABASE_URL is required"):
        database.connect()


def test_login_rejects_invalid_input_before_database_access():
    response = TestClient(app).post(
        "/auth/login",
        json={"email": "not-an-email", "password": ""},
    )

    assert response.status_code == 422
    assert "synthetic database detail" not in response.text


@pytest.mark.parametrize("configured", ["1", "true", "yes"])
def test_secure_cookie_accepts_explicit_secure_values(monkeypatch, configured: str):
    monkeypatch.setenv("COOKIE_SECURE", configured)

    assert _secure_cookie()


@pytest.mark.parametrize("configured", ["0", "false", "no", "invalid"])
def test_secure_cookie_treats_other_explicit_values_as_insecure(monkeypatch, configured:
str):
    monkeypatch.setenv("COOKIE_SECURE", configured)

    assert not _secure_cookie()


@pytest.mark.parametrize(("app_env", "expected"), [("production", True), ("development",
False), ("test", False)])
```

```
def test_secure_cookie_defaults_by_environment(monkeypatch, app_env: str, expected: bool):
    monkeypatch.delenv("COOKIE_SECURE", raising=False)
    monkeypatch.setenv("APP_ENV", app_env)

    assert _secure_cookie() is expected
===== API TEST FILE LIST =====
apps/api/tests/test_audit_service.py
apps/api/tests/test_auth.py
apps/api/tests/test_auth_unit.py
apps/api/tests/test_identity_constraints.py
apps/api/tests/test_provision_user.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
ook-Pro NVDIA_Play_Book % cd FinSight_AI
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== AUTH
INTEGRATION TESTS ====="
sed -n '1,240p' apps/api/tests/test_auth.py

echo "===== TEST DATABASE FIXTURES ====="
grep -n -A45 -B10 'def identity\|def client\|TEST_PASSWORD' \
  apps/api/tests/test_auth.py

echo "===== CURRENT THROTTLE CONFIG REFERENCES ====="
grep -R -n --exclude-dir=__pycache__ \
  -E 'AUTH_THROTTLE|AUTH_LOGIN_MAX_FAILURES|auth_login_throttle' \
  apps/api .env.example scripts/package_release.py
===== AUTH INTEGRATION TESTS =====
from __future__ import annotations

import secrets
from uuid import uuid4

import pytest
from fastapi.testclient import TestClient
from psycopg.rows import dict_row

from app.auth import hash_password, token_digest, verify_password
from app.database import connect
from app.main import SESSION_COOKIE, app

TEST_PASSWORD = secrets.token_urlsafe(32)


@pytest.fixture
```

```
def identity():
    email = f"{uuid4().hex}@example.com"
    encoded = hash_password(TEST_PASSWORD)
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO users (email, password_hash, display_name, account_status) VALUES
(%s, %s, %s, 'active') RETURNING user_id",
            (email, encoded, "Synthetic Analyst"),
        )
        user_id = cur.fetchone()[0]
    try:
        yield user_id, email, encoded
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


@pytest.fixture
def client(monkeypatch):
    monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
    monkeypatch.setenv("COOKIE_SECURE", "false")
    monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
    with TestClient(app) as test_client:
        yield test_client


def test_password_hash_is_argon2_and_never_plaintext():
    encoded = hash_password(TEST_PASSWORD)
    assert encoded.startswith("argon2id")
    assert encoded != TEST_PASSWORD
    assert verify_password(encoded, TEST_PASSWORD)
    assert not verify_password(encoded, "wrong")


def test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
    user_id, email, encoded = identity
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
    assert response.json() == {"user_id": str(user_id), "email": email, "display_name": "Synthetic
Analyst"}
    assert TEST_PASSWORD not in response.text and encoded not in response.text
    cookie = response.cookies.get(SESSION_COOKIE)
    assert cookie
    set_cookie = response.headers["set-cookie"].lower()
```

```
    assert "httponly" in set_cookie and "samesite=lax" in set_cookie and "path=/" in set_cookie
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute("SELECT refresh_token_hash, expires_at, revoked_at FROM auth_sessions
WHERE user_id = %s", (user_id,))
        stored = cur.fetchone()
    assert stored["refresh_token_hash"] == token_digest(cookie)
    assert stored["refresh_token_hash"] != cookie
    assert stored["expires_at"] is not None and stored["revoked_at"] is None
    protected = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
    assert protected.status_code == 200 and protected.json()["user_id"] == str(user_id)
    captured = capsys.readouterr()
    assert TEST_PASSWORD not in caplog.text + captured.out + captured.err
    assert cookie not in caplog.text + captured.out + captured.err


def test_invalid_credentials_have_same_generic_response(client, identity):
    _, email, _ = identity
    known = client.post("/auth/login", json={"email": email, "password": "wrong-password"})
    absent = client.post("/auth/login", json={"email": "absent@example.com", "password":
"wrong-password"})
    assert (known.status_code, known.json()) == (absent.status_code, absent.json())
    assert known.status_code == 401 and known.json() == {"detail": "Email or password is
incorrect."}
    assert SESSION_COOKIE not in known.cookies and SESSION_COOKIE not in
absent.cookies


def test_logout_revokes_session_and_replay_fails(client, identity):
    _, email, _ = identity
    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    cookie = login.cookies.get(SESSION_COOKIE)
    assert cookie
    logout = client.post("/auth/logout", cookies={SESSION_COOKIE: cookie})
    assert logout.status_code == 204
    replay = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
    assert replay.status_code == 401
    with connect() as conn, conn.cursor() as cur:
        cur.execute("SELECT revoked_at FROM auth_sessions WHERE refresh_token_hash =
%s", (token_digest(cookie),))
        assert cur.fetchone()[0] is not None


def test_logout_without_session_is_idempotent(client):
    response = client.post("/auth/logout")
```

```
    assert response.status_code == 204
    assert response.headers["set-cookie"].startswith(f"{SESSION_COOKIE}=")


def test_expired_session_is_rejected(client, identity):
    user_id, email, _ = identity
    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    cookie = login.cookies.get(SESSION_COOKIE)
    assert cookie
    with connect() as conn, conn.cursor() as cur:
        cur.execute("UPDATE auth_sessions SET last_seen_at = now() - interval '61 minutes'
WHERE user_id = %s", (user_id,))
    assert client.get("/auth/me", cookies={SESSION_COOKIE: cookie}).status_code == 401


def test_public_registration_is_not_available(client):
    response = client.post("/auth/register", json={"email": "public@example.com", "password":
TEST_PASSWORD})
    assert response.status_code == 404


def test_auth_mutation_rejects_foreign_origin(client):
    response = client.post(
        "/auth/logout", headers={"Origin": "https://attacker.invalid"},
    )
    assert response.status_code == 403


def test_secure_cookie_can_be_required_by_deployment(client, identity, monkeypatch):
    _, email, _ = identity
    monkeypatch.setenv("COOKIE_SECURE", "true")
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
    assert "secure" in response.headers["set-cookie"].lower()
===== TEST DATABASE FIXTURES =====
4-from uuid import uuid4
5-
6-import pytest
7-from fastapi.testclient import TestClient
8-from psycopg.rows import dict_row
9-
10-from app.auth import hash_password, token_digest, verify_password
11-from app.database import connect
```

```
12-from app.main import SESSION_COOKIE, app
13-
14:TEST_PASSWORD = secrets.token_urlsafe(32)
15-
16-
17-@pytest.fixture
18:def identity():
19-    email = f"{uuid4().hex}@example.com"
20:    encoded = hash_password(TEST_PASSWORD)
21-    with connect() as conn, conn.cursor() as cur:
22-        cur.execute(
23-            "INSERT INTO users (email, password_hash, display_name, account_status)
VALUES (%s, %s, %s, 'active') RETURNING user_id",
24-            (email, encoded, "Synthetic Analyst"),
25-        )
26-        user_id = cur.fetchone()[0]
27-    try:
28-        yield user_id, email, encoded
29-    finally:
30-        with connect() as conn, conn.cursor() as cur:
31-            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
32-
33-
34-@pytest.fixture
35:def client(monkeypatch):
36-    monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
37-    monkeypatch.setenv("COOKIE_SECURE", "false")
38-    monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
39-    with TestClient(app) as test_client:
40-        yield test_client
41-
42-
43-def test_password_hash_is_argon2_and_never_plaintext():
44:    encoded = hash_password(TEST_PASSWORD)
45-    assert encoded.startswith("argon2id")
46:    assert encoded != TEST_PASSWORD
47:    assert verify_password(encoded, TEST_PASSWORD)
48-    assert not verify_password(encoded, "wrong")
49-
50-
51-def test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
52-    user_id, email, encoded = identity
53:    response = client.post("/auth/login", json={"email": email, "password":
TEST_PASSWORD})
```

```
54-    assert response.status_code == 200
55-    assert response.json() == {"user_id": str(user_id), "email": email, "display_name":
"Synthetic Analyst"}
56:    assert TEST_PASSWORD not in response.text and encoded not in response.text
57-    cookie = response.cookies.get(SESSION_COOKIE)
58-    assert cookie
59-    set_cookie = response.headers["set-cookie"].lower()
60-    assert "httponly" in set_cookie and "samesite=lax" in set_cookie and "path=/" in
set_cookie
61-    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
62-        cur.execute("SELECT refresh_token_hash, expires_at, revoked_at FROM
auth_sessions WHERE user_id = %s", (user_id,))
63-        stored = cur.fetchone()
64-    assert stored["refresh_token_hash"] == token_digest(cookie)
65-    assert stored["refresh_token_hash"] != cookie
66-    assert stored["expires_at"] is not None and stored["revoked_at"] is None
67-    protected = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
68-    assert protected.status_code == 200 and protected.json()["user_id"] == str(user_id)
69-    captured = capsys.readouterr()
70:    assert TEST_PASSWORD not in caplog.text + captured.out + captured.err
71-    assert cookie not in caplog.text + captured.out + captured.err
72-
73-
74-def test_invalid_credentials_have_same_generic_response(client, identity):
75-    _, email, _ = identity
76-    known = client.post("/auth/login", json={"email": email, "password": "wrong-password"})
77-    absent = client.post("/auth/login", json={"email": "absent@example.com", "password":
"wrong-password"})
78-    assert (known.status_code, known.json()) == (absent.status_code, absent.json())
79-    assert known.status_code == 401 and known.json() == {"detail": "Email or password is
incorrect."}
80-    assert SESSION_COOKIE not in known.cookies and SESSION_COOKIE not in
absent.cookies
81-
82-
83-def test_logout_revokes_session_and_replay_fails(client, identity):
84-    _, email, _ = identity
85:    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
86-    cookie = login.cookies.get(SESSION_COOKIE)
87-    assert cookie
88-    logout = client.post("/auth/logout", cookies={SESSION_COOKIE: cookie})
89-    assert logout.status_code == 204
90-    replay = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
91-    assert replay.status_code == 401
```

```
92-    with connect() as conn, conn.cursor() as cur:
93-        cur.execute("SELECT revoked_at FROM auth_sessions WHERE refresh_token_hash =
%s", (token_digest(cookie),))
94-        assert cur.fetchone()[0] is not None
95-
96-
97-def test_logout_without_session_is_idempotent(client):
98-    response = client.post("/auth/logout")
99-
100-    assert response.status_code == 204
101-    assert response.headers["set-cookie"].startswith(f"{SESSION_COOKIE}=")
102-
103-
104-def test_expired_session_is_rejected(client, identity):
105-    user_id, email, _ = identity
106:    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
107-    cookie = login.cookies.get(SESSION_COOKIE)
108-    assert cookie
109-    with connect() as conn, conn.cursor() as cur:
110-        cur.execute("UPDATE auth_sessions SET last_seen_at = now() - interval '61 minutes'
WHERE user_id = %s", (user_id,))
111-    assert client.get("/auth/me", cookies={SESSION_COOKIE: cookie}).status_code == 401
112-
113-
114-def test_public_registration_is_not_available(client):
115:    response = client.post("/auth/register", json={"email": "public@example.com",
"password": TEST_PASSWORD})
116-    assert response.status_code == 404
117-
118-
119-def test_auth_mutation_rejects_foreign_origin(client):
120-    response = client.post(
121-        "/auth/logout", headers={"Origin": "https://attacker.invalid"},
122-    )
123-    assert response.status_code == 403
124-
125-
126-def test_secure_cookie_can_be_required_by_deployment(client, identity, monkeypatch):
127-    _, email, _ = identity
128-    monkeypatch.setenv("COOKIE_SECURE", "true")
129:    response = client.post("/auth/login", json={"email": email, "password":
TEST_PASSWORD})
130-    assert response.status_code == 200
131-    assert "secure" in response.headers["set-cookie"].lower()
```

```
===== CURRENT THROTTLE CONFIG REFERENCES =====
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:32:CREATE TABLE
auth_login_throttle (
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:44:CREATE INDEX
idx_auth_login_throttle_blocked_until
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:45:    ON auth_login_throttle
(blocked_until)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "===== PYTHON
VERSION ====="
apps/api/.venv/bin/python --version 2>/dev/null || python --version

echo "===== CRYPTOGRAPHY AND TEST DEPENDENCIES ====="
grep -n -E 'cryptography|pytest|psycopg|argon2|fastapi' \
  apps/api/requirements.txt \
  apps/api/requirements-dev.txt \
  apps/api/requirements.lock 2>/dev/null

echo "===== RELEASE CONFIGURATION DOCUMENTATION ====="
grep -n -A30 -B5 -E 'SESSION_TTL_MINUTES|COOKIE_SECURE|DATABASE_URL' \
  docs/release/RELEASE_README.md
===== PYTHON VERSION =====
Python 3.12.8
===== CRYPTOGRAPHY AND TEST DEPENDENCIES =====
apps/api/requirements.txt:1:fastapi==0.133.0
apps/api/requirements.txt:4:argon2-cffi==25.1.0
apps/api/requirements.txt:5:psycopg[binary]==3.2.13
apps/api/requirements-dev.txt:2:pytest==9.0.3
apps/api/requirements-dev.txt:4:pytest-cov==6.2.1
apps/api/requirements.lock:4:argon2-cffi==25.1.0
apps/api/requirements.lock:5:argon2-cffi-bindings==26.1.0
apps/api/requirements.lock:10:fastapi==0.133.0
apps/api/requirements.lock:13:psycopg==3.2.13
apps/api/requirements.lock:14:psycopg-binary==3.2.13
===== RELEASE CONFIGURATION DOCUMENTATION =====
13-
14-On macOS, use `shasum -a 256` and compare its output with the `.sha256` file.
15-
16-## Configure and start
17-
18:Extract the verified archive into a new, versioned release directory. Install a supported
Node.js 22 runtime and Python 3.12 runtime. Configure the frontend process with
`API_BASE_URL=https://<api-origin>` and the backend process with `DATABASE_URL`,
`WEB_ORIGIN`, `SESSION_TTL_MINUTES`, `COOKIE_SECURE=true`, and
```

`APP_ENV=production`. Values containing credentials belong only in the deployment secret manager. Set the web origin exactly to the browser-facing frontend origin; terminate TLS at an approved proxy/load balancer and forward HTTPS safely.

For the frontend, from the extracted bundle root:

```sh
cd frontend
HOSTNAME=0.0.0.0 PORT=3000 node server.js
```

For the backend, from the extracted bundle root, create a virtual environment and install the bundled pinned runtime requirements:

```sh
cd backend
python3.12 -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.lock
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Run database migrations as a separate, explicitly approved deployment step with a migration-owner role. The bundle includes the current versioned migration under `backend/migrations`; it does not include or apply the supplied target schema wholesale. Use a separate least-privileged application database role at runtime.

Promotion and rollback

Retain each verified archive, checksum, metadata, migration result, smoke-test report, and deployment record. Promote the same versioned bundle through environments; configure `API_BASE_URL` and backend settings per environment at runtime. Record which artifact version and full commit SHA is active.

To roll back application code, stop the affected processes and point the deployment supervisor to the previous known-good versioned release directory by its metadata version and commit SHA. Verify its checksum before reuse, start it with the appropriate environment configuration, then run the deployment's readiness and authentication smoke checks. Keep the database at its current migrated version unless the migration has a reviewed, tested, explicitly approved down migration. Do not restore an older application against a schema it cannot safely read.

Database migration policy for safe application rollback:

47-1. Prefer additive expand migrations that preserve existing columns/records and remain compatible with both old and new application versions.
48-2. Deploy code that can operate with both schema shapes before any contract/removal migration.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
 that preserve existing columns/records and remain compatible with both old and new application versions.
48-2. Deploy code that can operate with both schema shapes before any contract/removal migration.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....

```
                ELSE auth_login_throttle.window_started_at
            END,
            blocked_until = CASE
                WHEN CASE
                    WHEN auth_login_throttle.window_started_at
                        <= %s - make_interval(secs => %s)
                    THEN 1
                    ELSE auth_login_throttle.failed_attempts + 1
                END >= %s
                THEN %s + make_interval(secs => %s)
                WHEN auth_login_throttle.blocked_until > %s
                THEN auth_login_throttle.blocked_until
                ELSE NULL
            END,
            updated_at = %s
        RETURNING blocked_until
        """,
        (
            scope_type, key_hash, current_time, current_time,
            current_time, config.window_seconds,
            current_time, config.window_seconds, current_time,
            current_time, config.window_seconds, config.max_failures,
            current_time, config.block_seconds, current_time,
            current_time,
        ),
    )
    row = cur.fetchone()

if row and row[0] and row[0] > current_time:
    return row[0]
return None
```

```
def reset_bucket(conn: Any, *, scope_type: str, key_hash: str) -> None:
    """Clear a successful login's corresponding failure counter."""
    with conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = %s AND key_hash = %s",
            (scope_type, key_hash),
        )
PY

echo "===== COMPILE THROTTLE SERVICE ====="
python -m py_compile apps/api/app/throttle.py

echo "===== VERIFY CHANGE SCOPE ====="
git status --short

===== COMPILE THROTTLE SERVICE =====
===== VERIFY CHANGE SCOPE =====
?? apps/api/app/audit.py
?? apps/api/app/throttle.py
?? apps/api/migrations/0003_auth_audit_and_login_throttling.sql
?? apps/api/tests/test_audit_service.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
ps/api/tests/test_audit_service.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....

    assert is_blocked(
        conn, scope_type="account", key_hash="a" * 64, now=now
    )
    query, params = conn.cursor_instance.calls[0]
    assert "auth_login_throttle" in query
    assert params == (now, "account", "a" * 64)


def test_is_blocked_returns_false_for_expired_or_missing_bucket():
    now = datetime(2026, 10, 21, tzinfo=UTC)

    assert not is_blocked(
        FakeConnection((now - timedelta(seconds=1),)),
        scope_type="ip",
        key_hash="b" * 64,
        now=now,
    )
    assert not is_blocked(
        FakeConnection(None),
```

```
        scope_type="ip",
        key_hash="b" * 64,
        now=now,
    )


def test_throttle_config_defaults_are_documented_values(monkeypatch):
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", "x" * 32)
    for name in (
        "AUTH_LOGIN_MAX_FAILURES",
        "AUTH_LOGIN_WINDOW_SECONDS",
        "AUTH_LOGIN_BLOCK_SECONDS",
    ):
        monkeypatch.delenv(name, raising=False)

    config = load_throttle_config()
    assert config == ThrottleConfig(
        secret=b"x" * 32,
        max_failures=5,
        window_seconds=900,
        block_seconds=900,
    )
PY

echo "===== RUN THROTTLE UNIT TESTS ====="
cd apps/api
pytest -q tests/test_throttle.py

===== RUN THROTTLE UNIT TESTS =====
.............F.
[100%]
================================================================================
===== FAILURES
================================================================================
=====
_____________________________________________________
test_is_blocked_returns_false_for_expired_or_missing_bucket
_____________________________________________________

    def test_is_blocked_returns_false_for_expired_or_missing_bucket():
        now = datetime(2026, 10, 21, tzinfo=UTC)

>       assert not is_blocked(
            FakeConnection((now - timedelta(seconds=1),)),
```

```
            scope_type="ip",
            key_hash="b" * 64,
            now=now,
        )
E       AssertionError: assert not True
E        +  where True = is_blocked(<test_throttle.FakeConnection object at 0x1034d2f30>,
scope_type='ip', key_hash=('b' * 64), now=datetime.datetime(2026, 10, 21,0, 0,
tzinfo=datetime.timezone.utc))
E        +    where <test_throttle.FakeConnection object at 0x1034d2f30> =
FakeConnection((datetime.datetime(2026, 10, 20, 23, 59, 59, tzinfo=datetime.timezone.utc),))

tests/test_throttle.py:115: AssertionError
=========================================================================
short test summary info
=========================================================================
FAILED tests/test_throttle.py::test_is_blocked_returns_false_for_expired_or_missing_bucket -
AssertionError: assert not True
1 failed, 13 passed in 0.04s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
python - <<'PY'
from pathlib import Path

path = Path("app/throttle.py")
source = path.read_text()

old = '''    return bool(row and row[0])
'''

new = '''    # The SQL returns whether blocked_until is still in the future.
    # A NULL row or expired timestamp must never count as blocked.
    return bool(row and row[0] is True)
'''

if old not in source:
    raise SystemExit("Expected code not found; no file changed.")

path.write_text(source.replace(old, new, 1))
print("Updated app/throttle.py")
PY

echo "===== RERUN THROTTLE TESTS ====="
pytest -q tests/test_throttle.py
```

```
Updated app/throttle.py
===== RERUN THROTTLE TESTS =====
.....F..
[100%]
================================================================================
===== FAILURES
================================================================================
=====

_____________________________________________________________
test_is_blocked_returns_true_only_for_future_expiry
_____________________________________________________________

    def test_is_blocked_returns_true_only_for_future_expiry():
        now = datetime(2026, 10, 21, tzinfo=UTC)
        conn = FakeConnection((now + timedelta(minutes=5),))

>       assert is_blocked(
            conn, scope_type="account", key_hash="a" * 64, now=now
        )
E       AssertionError: assert False
E        +  where False = is_blocked(<test_throttle.FakeConnection object at 0x106b0af30>,
scope_type='account', key_hash=('a' * 64), now=datetime.datetime(2026, 10, 21, 0, 0,
tzinfo=datetime.timezone.utc))

tests/test_throttle.py:104: AssertionError
================================================================================
short test summary info
================================================================================
FAILED tests/test_throttle.py::test_is_blocked_returns_true_only_for_future_expiry -
AssertionError: assert False
1 failed, 13 passed in 0.04s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % python - <<'PY'
from pathlib import Path

path = Path("app/throttle.py")
source = path.read_text()

old = '''    # The SQL returns whether blocked_until is still in the future.
    # A NULL row or expired timestamp must never count as blocked.
    return bool(row and row[0] is True)
'''

new = '''    # Accept the timestamp result used by the database query, and Boolean
```

```
    # results from adapters that evaluate the comparison in SQL.
    if not row or row[0] is None:
        return False
    value = row[0]
    if isinstance(value, bool):
        return value
    return value > current_time
'''

if old not in source:
    raise SystemExit("Expected code not found; no file changed.")

path.write_text(source.replace(old, new, 1))
print("Updated app/throttle.py")
PY

pytest -q tests/test_throttle.py
Updated app/throttle.py
..............
[100%]
14 passed in 0.01s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
h@Sandeeps-MacBook-Pro api % python - <<'PY'
import hashlib
import psycopg

key_hash = hashlib.sha256(b"synthetic-fin14-throttle-test").hexdigest()

with psycopg.connect(
    host="localhost",
    port=55439,
    dbname="finsight_fin15_test",
    user="sandeeph",
) as conn:
    with conn.cursor() as cur:
        cur.execute(
            """
            DELETE FROM auth_login_throttle
            WHERE scope_type = 'account' AND key_hash = %s
            RETURNING key_hash
            """,
            (key_hash,),
        )
        removed = cur.fetchone()
```

```
        cur.execute(
            """
            SELECT count(*)
            FROM auth_login_throttle
            WHERE scope_type = 'account' AND key_hash = %s
            """,
            (key_hash,),
        )
        remaining = cur.fetchone()[0]

assert remaining == 0
print("PASS: synthetic throttle test row is absent")
print("Rows removed:", 1 if removed else 0)
PY
PASS: synthetic throttle test row is absent
Rows removed: 1

Rows removed: 1
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % echo "=== Authentication imports and
constants ==="
sed -n '1,100p' app/main.py

echo "=== Login and logout endpoints ==="
grep -n -A150 -B15 'def login\|def logout' app/main.py

echo "=== Current throttle implementation ==="
sed -n '1,260p' app/throttle.py

echo "=== Current audit service ==="
sed -n '1,220p' app/audit.py
=== Authentication imports and constants ===
"""FinSight AI API with PostgreSQL-backed browser sessions."""

from __future__ import annotations

import os
import secrets
from datetime import UTC, datetime
from pathlib import Path
from typing import Annotated, Any
from uuid import UUID

import psycopg
from dotenv import load_dotenv
```

```
from fastapi import Cookie, Depends, FastAPI, HTTPException, Request, Response, status
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, EmailStr, Field
from psycopg.rows import dict_row

from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
from app.database import connect

load_dotenv(Path(__file__).resolve().parents[3] / ".env")

app = FastAPI(title="FinSight AI API", version="0.2.0")
web_origin = os.environ.get("WEB_ORIGIN", "http://localhost:3000")
app.add_middleware(
    CORSMiddleware,
    allow_origins=[web_origin],
    allow_credentials=True,
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)

SESSION_COOKIE = "finsight_session"
GENERIC_LOGIN_ERROR = "Email or password is incorrect."
DUMMY_PASSWORD_HASH = hash_password(secrets.token_urlsafe(32))


class LoginRequest(BaseModel):
    email: EmailStr
    password: str = Field(min_length=1, max_length=1024)


class UserView(BaseModel):
    user_id: UUID
    email: str
    display_name: str


def _ttl_minutes() -> int:
    try:
        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
        session_expiry(value)
        return value
    except ValueError as exc:
        raise HTTPException(status_code=500, detail="Session configuration is invalid") from exc
```

```
def _secure_cookie() -> bool:
    configured = os.environ.get("COOKIE_SECURE")
    if configured is not None:
        return configured.lower() in {"1", "true", "yes"}
    return os.environ.get("APP_ENV", "development").lower() == "production"


def _check_origin(request: Request) -> None:
    origin = request.headers.get("origin")
    if origin and origin != web_origin:
        raise HTTPException(status_code=403, detail="Request origin is not allowed")


def get_current_user(
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> dict[str, Any]:
    if not session_token:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Authentication required")
    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT u.user_id, u.email, u.display_name, r.role_key,
                       COALESCE(array_agg(DISTINCT p.permission_key)
                           FILTER (WHERE p.permission_key IS NOT NULL), ARRAY[]::varchar[])
AS permissions
                  FROM auth_sessions s JOIN users u ON u.user_id = s.user_id
                  LEFT JOIN roles r ON r.role_id = u.platform_role_id AND r.scope = 'platform'
                  LEFT JOIN role_permissions rp ON rp.role_id = r.role_id
                  LEFT JOIN permissions p ON p.permission_id = rp.permission_id
                  WHERE s.refresh_token_hash = %s AND s.revoked_at IS NULL
                    AND s.expires_at > %s AND u.account_status = 'active'
                    AND s.last_seen_at > %s - make_interval(mins => %s)
                    AND u.deleted_at IS NULL
                  GROUP BY u.user_id, u.email, u.display_name, r.role_key""",
                (token_digest(session_token), datetime.now(UTC), datetime.now(UTC),
_ttl_minutes()),
            )
            user = cur.fetchone()
            if not user:
                raise HTTPException(status_code=401, detail="Authentication required")
            cur.execute(
```

```
            "UPDATE auth_sessions SET last_seen_at = %s WHERE refresh_token_hash = %s",
            (datetime.now(UTC), token_digest(session_token)),
        )
=== Login and logout endpoints ===
105-
106-CurrentUser = Annotated[dict[str, Any], Depends(get_current_user)]
107-
108-
109-def require_permission(permission_key: str):
110-    """Build a deny-by-default backend permission dependency."""
111-    def dependency(user: CurrentUser) -> dict[str, Any]:
112-        if permission_key not in user.get("permissions", []):
113-            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Permission denied")
114-        return user
115-
116-    return dependency
117-
118-
119-@app.post("/auth/login", response_model=UserView)
120:def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
121-    _check_origin(request)
122-    email = payload.email.strip().lower()
123-    try:
124-        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
125-            cur.execute(
126-                """SELECT user_id, email, display_name, password_hash
127-                   FROM users WHERE lower(email) = %s AND account_status = 'active'
128-                     AND deleted_at IS NULL""",
129-                (email,),
130-            )
131-            user = cur.fetchone()
132-            if not user or not user["password_hash"]:
133-                verify_password(DUMMY_PASSWORD_HASH, payload.password)
134-                raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)
135-            if not verify_password(user["password_hash"], payload.password):
136-                raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)
137-            token, digest = new_session_token()
138-            expires_at = session_expiry(_ttl_minutes())
139-            cur.execute(
140-                """INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at)
141-                   VALUES (%s, %s, %s)""",
142-                (user["user_id"], digest, expires_at),
143-            )
```

```
144-            cur.execute("UPDATE users SET last_login_at = %s, updated_at = %s WHERE
user_id = %s",
145-                        (datetime.now(UTC), datetime.now(UTC), user["user_id"]))
146-        response.set_cookie(
147-            SESSION_COOKIE,
148-            token,
149-            httponly=True,
150-            secure=_secure_cookie(),
151-            samesite="lax",
152-            max_age=_ttl_minutes() * 60,
153-            path="/",
154-        )
155-        return UserView(user_id=user["user_id"], email=user["email"],
display_name=user["display_name"])
156-    except psycopg.Error as exc:
157-        raise HTTPException(status_code=503, detail="Authentication service unavailable")
from exc
158-
159-
160-@app.post("/auth/logout", status_code=204)
161:def logout(request: Request, response: Response,
162-           session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None) ->
Response:
163-    _check_origin(request)
164-    if session_token:
165-        try:
166-            with connect() as conn, conn.cursor() as cur:
167-                cur.execute(
168-                    """UPDATE auth_sessions SET revoked_at = %s, revoke_reason = 'logout'
169-                       WHERE refresh_token_hash = %s AND revoked_at IS NULL""",
170-                    (datetime.now(UTC), token_digest(session_token)),
171-                )
172-        except psycopg.Error as exc:
173-            raise HTTPException(status_code=503, detail="Authentication service unavailable")
from exc
174-    response.delete_cookie(SESSION_COOKIE, path="/", httponly=True,
secure=_secure_cookie(), samesite="lax")
175-    response.status_code = status.HTTP_204_NO_CONTENT
176-    return response
177-
178-
179-@app.get("/auth/me", response_model=UserView)
180-def who_am_i(user: CurrentUser) -> UserView:
181-    return UserView(**user)
```

```
182-
183-
184-@app.get("/api/v1/admin/authorization/capabilities")
185-def platform_admin_capabilities(
186-    request: Request,
187-    _authorized: Annotated[dict[str, Any], Depends(require_permission("platform.admin"))],
188-) -> dict[str, Any]:
189-    """Return the authenticated caller's database-backed platform capabilities."""
190-
191-    supplied_request_id = request.headers.get("request_id", "").strip()
192-    if supplied_request_id and (
193-        len(supplied_request_id) > 128
194-        or any(ord(char) < 32 or ord(char) == 127 for char in supplied_request_id)
195-    ):
196-        raise HTTPException(status_code=400, detail="Invalid request_id")
197-
198-    # Always issue a server-generated canonical request ID.
199-    request_id = f"req_{secrets.token_urlsafe(12)}"
200-    correlation_id = supplied_request_id or request_id
201-    permissions = sorted(set(_authorized.get("permissions", [])))
202-    capabilities = (
203-        ["inspect_authorization_policy"]
204-        if "platform.admin" in permissions
205-        else []
206-    )
207-
208-    return {
209-        "data": {
210-            "subject": {
211-                "user_id": str(_authorized["user_id"]),
212-                "platform_role_code": _authorized.get("role_key"),
213-            },
214-            "effective_permissions": permissions,
215-            "capabilities": capabilities,
216-            "policy_version": "sprint-1-platform-rbac-v1",
217-        },
218-        "request_id": request_id,
219-        "correlation_id": correlation_id,
220-    }
=== Current throttle implementation ===
"""PostgreSQL-backed login throttling with secret-keyed identifiers."""

from __future__ import annotations
```

```
import hashlib
import hmac
import os
from dataclasses import dataclass
from datetime import UTC, datetime
from typing import Any


@dataclass(frozen=True)
class ThrottleConfig:
    secret: bytes
    max_failures: int
    window_seconds: int
    block_seconds: int


def load_throttle_config() -> ThrottleConfig:
    """Load validated throttle settings; never use a weak fallback secret."""
    secret = os.environ.get("AUTH_THROTTLE_SECRET", "")
    if len(secret.encode("utf-8")) < 32:
        raise ValueError("AUTH_THROTTLE_SECRET must contain at least 32 bytes")

    try:
        max_failures = int(os.environ.get("AUTH_LOGIN_MAX_FAILURES", "5"))
        window_seconds = int(os.environ.get("AUTH_LOGIN_WINDOW_SECONDS", "900"))
        block_seconds = int(os.environ.get("AUTH_LOGIN_BLOCK_SECONDS", "900"))
    except ValueError as exc:
        raise ValueError("Login throttle configuration is invalid") from exc

    if not 1 <= max_failures <= 100:
        raise ValueError("AUTH_LOGIN_MAX_FAILURES must be between 1 and 100")
    if not 1 <= window_seconds <= 86400:
        raise ValueError("AUTH_LOGIN_WINDOW_SECONDS must be between 1 and 86400")
    if not 1 <= block_seconds <= 86400:
        raise ValueError("AUTH_LOGIN_BLOCK_SECONDS must be between 1 and 86400")

    return ThrottleConfig(
        secret=secret.encode("utf-8"),
        max_failures=max_failures,
        window_seconds=window_seconds,
        block_seconds=block_seconds,
    )
```

```
def make_throttle_key(secret: bytes, scope_type: str, value: str) -> str:
    """Return a stable HMAC-SHA256 identifier without storing the raw value."""
    if scope_type not in {"account", "ip"}:
        raise ValueError("Unsupported throttle scope")
    if not value:
        raise ValueError("Throttle key value cannot be empty")

    message = f"finsight-login:{scope_type}:{value}".encode("utf-8")
    return hmac.new(secret, message, hashlib.sha256).hexdigest()


def is_blocked(
    conn: Any,
    *,
    scope_type: str,
    key_hash: str,
    now: datetime | None = None,
) -> bool:
    current_time = now or datetime.now(UTC)
    with conn.cursor() as cur:
        cur.execute(
            """
            SELECT blocked_until > %s
            FROM auth_login_throttle
            WHERE scope_type = %s AND key_hash = %s
            """,
            (current_time, scope_type, key_hash),
        )
        row = cur.fetchone()
    # Accept the timestamp result used by the database query, and Boolean
    # results from adapters that evaluate the comparison in SQL.
    if not row or row[0] is None:
        return False
    value = row[0]
    if isinstance(value, bool):
        return value
    return value > current_time


def record_failure(
    conn: Any,
    *,
    scope_type: str,
    key_hash: str,
```

```
    config: ThrottleConfig,
    now: datetime | None = None,
) -> datetime | None:
    """Atomically increment a bucket and return its active block expiry, if any."""
    current_time = now or datetime.now(UTC)
    expired_window = current_time.timestamp()  # Used only to validate a timestamp exists.
    del expired_window

    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO auth_login_throttle (
                scope_type, key_hash, failed_attempts,
                window_started_at, blocked_until, updated_at
            )
            VALUES (%s, %s, 1, %s, NULL, %s)
            ON CONFLICT (scope_type, key_hash) DO UPDATE SET
                failed_attempts = CASE
                    WHEN auth_login_throttle.window_started_at
                        <= %s - make_interval(secs => %s)
                    THEN 1
                    ELSE auth_login_throttle.failed_attempts + 1
                END,
                window_started_at = CASE
                    WHEN auth_login_throttle.window_started_at
                        <= %s - make_interval(secs => %s)
                    THEN %s
                    ELSE auth_login_throttle.window_started_at
                END,
                blocked_until = CASE
                    WHEN CASE
                        WHEN auth_login_throttle.window_started_at
                            <= %s - make_interval(secs => %s)
                        THEN 1
                        ELSE auth_login_throttle.failed_attempts + 1
                    END >= %s
                    THEN %s + make_interval(secs => %s)
                    WHEN auth_login_throttle.blocked_until > %s
                    THEN auth_login_throttle.blocked_until
                    ELSE NULL
                END,
                updated_at = %s
            RETURNING blocked_until
            """,
```

```
            (
                scope_type, key_hash, current_time, current_time,
                current_time, config.window_seconds,
                current_time, config.window_seconds, current_time,
                current_time, config.window_seconds, config.max_failures,
                current_time, config.block_seconds, current_time,
                current_time,
            ),
        )
        row = cur.fetchone()

    if row and row[0] and row[0] > current_time:
        return row[0]
    return None


def reset_bucket(conn: Any, *, scope_type: str, key_hash: str) -> None:
    """Clear a successful login's corresponding failure counter."""
    with conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = %s AND key_hash = %s",
            (scope_type, key_hash),
        )
=== Current audit service ===
"""Reusable, redacted audit persistence for authentication security events."""

from __future__ import annotations

import json
import re
from datetime import UTC, datetime
from typing import Any
from uuid import UUID

_ALLOWED_OUTCOMES = frozenset({"success", "failure", "denied"})
_SECRET_KEY = re.compile(
    r"(password|passwd|token|cookie|authorization|api[_-]?key|secret|credential)",
    re.IGNORECASE,
)


def _redact(value: Any) -> Any:
    """Return JSON-safe audit details without values under secret-like keys."""
    if isinstance(value, dict):
```

```
        return {
            str(key): (
                "[REDACTED]" if _SECRET_KEY.search(str(key)) else _redact(item)
            )
            for key, item in value.items()
        }
    if isinstance(value, (list, tuple)):
        return [_redact(item) for item in value]
    if isinstance(value, UUID):
        return str(value)
    if isinstance(value, datetime):
        return value.astimezone(UTC).isoformat()
    if value is None or isinstance(value, (str, int, float, bool)):
        return value
    return str(value)


def record_auth_event(
    conn: Any,
    *,
    action_key: str,
    outcome: str,
    actor_user_id: UUID | None,
    details: dict[str, Any] | None = None,
    correlation_id: UUID | None = None,
    occurred_at: datetime | None = None,
) -> None:
    """Persist one authentication audit event on the caller's transaction.

    Persistence errors intentionally propagate to the caller. The caller must
    roll back and apply the documented fail-closed policy; events are never
    reported as recorded when this insert fails.
    """
    if outcome not in _ALLOWED_OUTCOMES:
        raise ValueError("Unsupported audit outcome")
    if not action_key or len(action_key) > 120:
        raise ValueError("action_key must contain 1 to 120 characters")

    redacted_details = _redact(details or {})
    payload = json.dumps(redacted_details, separators=(",", ":"), sort_keys=True)

    with conn.cursor() as cur:
        cur.execute(
            """
```

```
            INSERT INTO audit_events (
                workspace_id,
                actor_user_id,
                action_key,
                resource_type,
                resource_id,
                outcome,
                correlation_id,
                details_redacted,
                occurred_at
            )
            VALUES (NULL, %s, %s, 'authentication', NULL, %s, %s, %s::jsonb, %s)
            """,
            (
                actor_user_id,
                action_key,
                outcome,
                correlation_id,
                payload,
                occurred_at or datetime.now(UTC),
            ),
        )
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
            VALUES (NULL, %s, %s, 'authentication', NULL, %s, %s, %s::jsonb, %s)
            """,
            (
                actor_user_id,
                action_key,
                outcome,
                correlation_id,
                payload,
                occurred_at or datetime.now(UTC),
            ),
        )
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % python - <<'PY'
from pathlib import Path

path = Path("app/throttle.py")
source = path.read_text()

old = """    expired_window = current_time.timestamp()  # Used only to validate a timestamp
exists.
    del expired_window
```

```
"""
if old not in source:
    raise SystemExit("Expected code not found; no changes made.")

path.write_text(source.replace(old, "", 1))
print("Removed unused timestamp assignment from app/throttle.py")
PY

pytest -q tests/test_throttle.py tests/test_audit_service.py
Removed unused timestamp assignment from app/throttle.py
.....................
[100%]
21 passed in 0.02s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
ignment from app/throttle.py
.....................
[100%]
21 passed in 0.02s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % echo "=== Login integration test
fixtures ==="
sed -n '1,240p' tests/test_auth.py

echo "=== Authentication unit tests ==="
sed -n '1,260p' tests/test_auth_unit.py

echo "=== Project configuration ==="
cat requirements.txt
=== Login integration test fixtures ===
from __future__ import annotations

import secrets
from uuid import uuid4

import pytest
from fastapi.testclient import TestClient
from psycopg.rows import dict_row

from app.auth import hash_password, token_digest, verify_password
from app.database import connect
from app.main import SESSION_COOKIE, app

TEST_PASSWORD = secrets.token_urlsafe(32)
```

```
@pytest.fixture
def identity():
    email = f"{uuid4().hex}@example.com"
    encoded = hash_password(TEST_PASSWORD)
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO users (email, password_hash, display_name, account_status) VALUES
(%s, %s, %s, 'active') RETURNING user_id",
            (email, encoded, "Synthetic Analyst"),
        )
        user_id = cur.fetchone()[0]
    try:
        yield user_id, email, encoded
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


@pytest.fixture
def client(monkeypatch):
    monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
    monkeypatch.setenv("COOKIE_SECURE", "false")
    monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
    with TestClient(app) as test_client:
        yield test_client


def test_password_hash_is_argon2_and_never_plaintext():
    encoded = hash_password(TEST_PASSWORD)
    assert encoded.startswith("argon2id")
    assert encoded != TEST_PASSWORD
    assert verify_password(encoded, TEST_PASSWORD)
    assert not verify_password(encoded, "wrong")


def test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
    user_id, email, encoded = identity
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
    assert response.json() == {"user_id": str(user_id), "email": email, "display_name": "Synthetic
Analyst"}
    assert TEST_PASSWORD not in response.text and encoded not in response.text
    cookie = response.cookies.get(SESSION_COOKIE)
```

```
    assert cookie
    set_cookie = response.headers["set-cookie"].lower()
    assert "httponly" in set_cookie and "samesite=lax" in set_cookie and "path=/" in set_cookie
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute("SELECT refresh_token_hash, expires_at, revoked_at FROM auth_sessions
WHERE user_id = %s", (user_id,))
        stored = cur.fetchone()
    assert stored["refresh_token_hash"] == token_digest(cookie)
    assert stored["refresh_token_hash"] != cookie
    assert stored["expires_at"] is not None and stored["revoked_at"] is None
    protected = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
    assert protected.status_code == 200 and protected.json()["user_id"] == str(user_id)
    captured = capsys.readouterr()
    assert TEST_PASSWORD not in caplog.text + captured.out + captured.err
    assert cookie not in caplog.text + captured.out + captured.err


def test_invalid_credentials_have_same_generic_response(client, identity):
    _, email, _ = identity
    known = client.post("/auth/login", json={"email": email, "password": "wrong-password"})
    absent = client.post("/auth/login", json={"email": "absent@example.com", "password":
"wrong-password"})
    assert (known.status_code, known.json()) == (absent.status_code, absent.json())
    assert known.status_code == 401 and known.json() == {"detail": "Email or password is
incorrect."}
    assert SESSION_COOKIE not in known.cookies and SESSION_COOKIE not in
absent.cookies


def test_logout_revokes_session_and_replay_fails(client, identity):
    _, email, _ = identity
    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    cookie = login.cookies.get(SESSION_COOKIE)
    assert cookie
    logout = client.post("/auth/logout", cookies={SESSION_COOKIE: cookie})
    assert logout.status_code == 204
    replay = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
    assert replay.status_code == 401
    with connect() as conn, conn.cursor() as cur:
        cur.execute("SELECT revoked_at FROM auth_sessions WHERE refresh_token_hash =
%s", (token_digest(cookie),))
        assert cur.fetchone()[0] is not None
```

```
def test_logout_without_session_is_idempotent(client):
    response = client.post("/auth/logout")

    assert response.status_code == 204
    assert response.headers["set-cookie"].startswith(f"{SESSION_COOKIE}=")


def test_expired_session_is_rejected(client, identity):
    user_id, email, _ = identity
    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    cookie = login.cookies.get(SESSION_COOKIE)
    assert cookie
    with connect() as conn, conn.cursor() as cur:
        cur.execute("UPDATE auth_sessions SET last_seen_at = now() - interval '61 minutes'
WHERE user_id = %s", (user_id,))
    assert client.get("/auth/me", cookies={SESSION_COOKIE: cookie}).status_code == 401


def test_public_registration_is_not_available(client):
    response = client.post("/auth/register", json={"email": "public@example.com", "password":
TEST_PASSWORD})
    assert response.status_code == 404


def test_auth_mutation_rejects_foreign_origin(client):
    response = client.post(
        "/auth/logout", headers={"Origin": "https://attacker.invalid"},
    )
    assert response.status_code == 403


def test_secure_cookie_can_be_required_by_deployment(client, identity, monkeypatch):
    _, email, _ = identity
    monkeypatch.setenv("COOKIE_SECURE", "true")
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
    assert "secure" in response.headers["set-cookie"].lower()
=== Authentication unit tests ===
from __future__ import annotations

from datetime import UTC, datetime, timedelta

import pytest
from fastapi import HTTPException
```

```
from fastapi.testclient import TestClient

from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
from app.main import _secure_cookie, _ttl_minutes, app


def test_password_helpers_accept_only_the_original_password():
    encoded = hash_password("synthetic-password")

    assert encoded.startswith("argon2id")
    assert verify_password(encoded, "synthetic-password")
    assert not verify_password(encoded, "different-password")
    assert not verify_password("not-an-argon2-hash", "synthetic-password")
    assert not verify_password(None, "synthetic-password")


def test_session_expiry_uses_utc_and_requested_ttl():
    now = datetime(2026, 1, 1, tzinfo=UTC)

    assert session_expiry(15, now) == now + timedelta(minutes=15)


@pytest.mark.parametrize("ttl", [0, -1, 10081])
def test_session_expiry_rejects_out_of_range_ttl(ttl: int):
    with pytest.raises(ValueError, match="SESSION_TTL_MINUTES"):
        session_expiry(ttl)


@pytest.mark.parametrize("configured, expected", [("60", 60), ("1", 1), ("10080", 10080)])
def test_ttl_configuration_accepts_only_supported_values(monkeypatch, configured: str,
expected: int):
    monkeypatch.setenv("SESSION_TTL_MINUTES", configured)

    assert _ttl_minutes() == expected


@pytest.mark.parametrize("configured", ["0", "10081", "not-a-number"])
def test_ttl_configuration_fails_closed(monkeypatch, configured: str):
    monkeypatch.setenv("SESSION_TTL_MINUTES", configured)

    with pytest.raises(HTTPException) as error:
        _ttl_minutes()
```

```
    assert error.value.status_code == 500
    assert error.value.detail == "Session configuration is invalid"


def test_session_tokens_are_opaque_and_only_digest_is_stable():
    token, digest = new_session_token()
    another_token, another_digest = new_session_token()

    assert token != another_token
    assert digest == token_digest(token)
    assert digest != token
    assert digest != another_digest


def test_login_maps_database_outage_to_controlled_error(monkeypatch):
    import app.main as main

    def unavailable_database():
        import psycopg

        raise psycopg.OperationalError("synthetic database detail")

    monkeypatch.setattr(main, "connect", unavailable_database)
    response = TestClient(app).post(
        "/auth/login",
        json={"email": "synthetic@example.com", "password": "synthetic-password"},
    )

    assert response.status_code == 503
    assert response.json() == {"detail": "Authentication service unavailable"}
    assert "synthetic database detail" not in response.text


def test_protected_route_rejects_missing_session_before_database_access(monkeypatch):
    import app.main as main

    def unexpected_database_access():
        raise AssertionError("database must not be accessed without a session cookie")

    monkeypatch.setattr(main, "connect", unexpected_database_access)
    response = TestClient(app).get("/auth/me")

    assert response.status_code == 401
    assert response.json() == {"detail": "Authentication required"}
```

```
@pytest.mark.parametrize(
    ("user", "expected_status"),
    [({"permissions": ["platform.admin"]}, 200), ({"permissions": []}, 403), ({}, 403)],
)
def test_permission_dependency_allows_only_explicit_permission(user, expected_status):
    from fastapi import Depends, FastAPI
    from fastapi.testclient import TestClient

    import app.main as main

    protected_app = FastAPI()
    protected_app.dependency_overrides[main.get_current_user] = lambda: user
    protected_app.add_api_route(
        "/platform-admin",
        lambda _authorized=Depends(main.require_permission("platform.admin")): {"allowed":
True},
    )

    response = TestClient(protected_app).get("/platform-admin")
    assert response.status_code == expected_status
    if expected_status == 403:
        assert response.json() == {"detail": "Permission denied"}


def test_permission_dependency_ignores_client_supplied_role_header():
    from fastapi import Depends, FastAPI
    from fastapi.testclient import TestClient

    import app.main as main

    protected_app = FastAPI()
    protected_app.dependency_overrides[main.get_current_user] = lambda: {"permissions": []}
    protected_app.add_api_route(
        "/platform-admin",
        lambda _authorized=Depends(main.require_permission("platform.admin")): {"allowed":
True},
    )

    response = TestClient(protected_app).get("/platform-admin", headers={"X-Role":
"platform_admin"})
    assert response.status_code == 403
```

```
@pytest.mark.parametrize(
    ("method", "path", "kwargs"),
    [
        ("get", "/auth/me", {"cookies": {"finsight_session": "synthetic-token"}}),
        ("post", "/auth/logout", {"cookies": {"finsight_session": "synthetic-token"}}),
        ("get",
            "/api/v1/admin/authorization/capabilities",
            {"cookies": {"finsight_session": "synthetic-token"}},
        ),
    ],
)
def test_session_database_outages_return_controlled_error(monkeypatch, method, path,
kwargs):
    import app.main as main
    import psycopg

    def unavailable_database():
        raise psycopg.OperationalError("synthetic database detail")

    monkeypatch.setattr(main, "connect", unavailable_database)
    response = getattr(TestClient(app), method)(path, **kwargs)

    assert response.status_code == 503
    assert response.json() == {"detail": "Authentication service unavailable"}
    assert "synthetic database detail" not in response.text


def test_database_configuration_is_required(monkeypatch):
    import app.database as database

    monkeypatch.delenv("DATABASE_URL", raising=False)

    with pytest.raises(RuntimeError, match="DATABASE_URL is required"):
        database.connect()


def test_login_rejects_invalid_input_before_database_access():
    response = TestClient(app).post(
        "/auth/login",
        json={"email": "not-an-email", "password": ""},
    )

    assert response.status_code == 422
```

```
    assert "synthetic database detail" not in response.text


@pytest.mark.parametrize("configured", ["1", "true", "yes"])
def test_secure_cookie_accepts_explicit_secure_values(monkeypatch, configured: str):
    monkeypatch.setenv("COOKIE_SECURE", configured)

    assert _secure_cookie()


@pytest.mark.parametrize("configured", ["0", "false", "no", "invalid"])
def test_secure_cookie_treats_other_explicit_values_as_insecure(monkeypatch, configured:
str):
    monkeypatch.setenv("COOKIE_SECURE", configured)

    assert not _secure_cookie()


@pytest.mark.parametrize(("app_env", "expected"), [("production", True), ("development",
False), ("test", False)])
def test_secure_cookie_defaults_by_environment(monkeypatch, app_env: str, expected: bool):
    monkeypatch.delenv("COOKIE_SECURE", raising=False)
    monkeypatch.setenv("APP_ENV", app_env)

    assert _secure_cookie() is expected
=== Project configuration ===
fastapi==0.133.0
starlette==1.3.1
uvicorn==0.40.0
argon2-cffi==25.1.0
psycopg[binary]==3.2.13
python-dotenv==1.2.2
email-validator==2.3.0
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
venv) (base) sandeeph@Sandeeps-MacBook-Pro api % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % cp app/main.py
/tmp/finsight_main_before_fin14.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % pytest -q tests/test_throttle.py
tests/test_audit_service.py tests/test_auth_unit.py
.....................................................
[100%]
=============================================================================
= warnings summary
```

```
============================================================================
=
../../../.venv/lib/python3.12/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/fastapi/test
client.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated;
install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

../../../.venv/lib/python3.12/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
stclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use
anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[get-/a
uth/me-kwargs0]
apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[post-/
auth/logout-kwargs1]
apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[get-/a
pi/v1/admin/authorization/capabilities-kwargs2]

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
stclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being deprecated,
because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on
the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
54 passed, 5 warnings in 0.42s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
ps://docs.pytest.org/en/stable/how-to/capture-warnings.html
54 passed, 5 warnings in 0.42s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % echo "=== Check FIN-14 imports
==="
grep -n -E 'app\.audit|app\.throttle|uuid4' app/main.py

echo "=== Check integrated login behavior ==="
grep -n -A125 'def login' app/main.py

echo "=== Check test throttle secret ==="
grep -n -A8 'def client' tests/test_auth.py
=== Check FIN-14 imports ===
```

```
12:from uuid import UUID, uuid4
14:from app.audit import record_auth_event
15:from app.throttle import (
145:    correlation_id = uuid4()
=== Check integrated login behavior ===
131:def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
132-    _check_origin(request)
133-    email = payload.email.strip().lower()
134-    try:
135-        config = load_throttle_config()
136-    except ValueError as exc:
137-        raise HTTPException(
138-            status_code=503,
139-            detail="Authentication service unavailable",
140-        ) from exc
141-
142-    client_ip = request.client.host if request.client else "unknown"
143-    account_key = make_throttle_key(config.secret, "account", email)
144-    ip_key = make_throttle_key(config.secret, "ip", client_ip)
145-    correlation_id = uuid4()
146-    login_succeeded = False
147-    user = None
148-
149-    try:
150-        with connect() as conn:
151-            account_blocked = is_blocked(
152-                conn, scope_type="account", key_hash=account_key
153-            )
154-            ip_blocked = is_blocked(
155-                conn, scope_type="ip", key_hash=ip_key
156-            )
157-
158-            if account_blocked or ip_blocked:
159-                record_auth_event(
160-                    conn,
161-                    action_key="auth.login",
162-                    outcome="denied",
163-                    actor_user_id=None,
164-                    details={"reason": "throttled"},
165-                    correlation_id=correlation_id,
166-                )
167-            else:
168-                with conn.cursor(row_factory=dict_row) as cur:
169-                    cur.execute(
```

```
                """SELECT user_id, email, display_name, password_hash
                   FROM users
                   WHERE lower(email) = %s
                     AND account_status = 'active'
                     AND deleted_at IS NULL""",
                (email,),
            )
            user = cur.fetchone()

        password_hash = (
            user["password_hash"]
            if user and user["password_hash"]
            else DUMMY_PASSWORD_HASH
        )
        password_valid = verify_password(
            password_hash, payload.password
        )
        login_succeeded = bool(
            user and user["password_hash"] and password_valid
        )

        if not login_succeeded:
            record_failure(
                conn,
                scope_type="account",
                key_hash=account_key,
                config=config,
            )
            record_failure(
                conn,
                scope_type="ip",
                key_hash=ip_key,
                config=config,
            )
            record_auth_event(
                conn,
                action_key="auth.login",
                outcome="failure",
                actor_user_id=user["user_id"] if user else None,
                details={"reason": "invalid_credentials"},
                correlation_id=correlation_id,
            )
        else:
            # Clear account failures only. Preserve IP-wide history.
```

```
214-                reset_bucket(
215-                    conn,
216-                    scope_type="account",
217-                    key_hash=account_key,
218-                )
219-
220-                token, digest = new_session_token()
221-                expires_at = session_expiry(_ttl_minutes())
222-
223-                with conn.cursor() as cur:
224-                    cur.execute(
225-                        """INSERT INTO auth_sessions
226-                               (user_id, refresh_token_hash, expires_at)
227-                           VALUES (%s, %s, %s)""",
228-                        (user["user_id"], digest, expires_at),
229-                    )
230-                    now = datetime.now(UTC)
231-                    cur.execute(
232-                        """UPDATE users
233-                           SET last_login_at = %s, updated_at = %s
234-                           WHERE user_id = %s""",
235-                        (now, now, user["user_id"]),
236-                    )
237-
238-                record_auth_event(
239-                    conn,
240-                    action_key="auth.login",
241-                    outcome="success",
242-                    actor_user_id=user["user_id"],
243-                    details={"method": "password"},
244-                    correlation_id=correlation_id,
245-                )
246-
247-    except psycopg.Error as exc:
248-        raise HTTPException(
249-            status_code=503,
250-            detail="Authentication service unavailable",
251-        ) from exc
252-
253-    if not login_succeeded:
254-        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)
255-
256-    response.set_cookie(
=== Check test throttle secret ===
```

```
35:def client(monkeypatch):
36-    monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
37-    monkeypatch.setenv("COOKIE_SECURE", "false")
38-    monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
39-    monkeypatch.setenv(
40-        "AUTH_THROTTLE_SECRET",
41-        "test-only-secret-material-at-least-32-bytes",)
42-    with TestClient(app) as test_client:
43-        yield test_client
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % export
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test'
export AUTH_THROTTLE_SECRET="$(python -c 'import secrets;
print(secrets.token_urlsafe(48))')"

pytest -q tests/test_auth.py
.........
[100%]
=============================================================================
= warnings summary
=============================================================================
=
../../../.venv/lib/python3.12/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/fastapi/test
client.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated;
install `httpx2` instead.
   from starlette.testclient import TestClient as TestClient  # noqa

../../../.venv/lib/python3.12/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
stclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use
anyio.from_thread.BlockingPortal instead.
   _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
stclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being deprecated,
```

```
because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on
the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
9 passed, 6 warnings in 0.95s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % >....
        with TestClient(app) as client:
            first = client.post(
                "/auth/login", json={"email": email, "password": "wrong-password"}
            )
            second = client.post(
                "/auth/login", json={"email": email, "password": "wrong-password"}
            )
            blocked = client.post(
                "/auth/login", json={"email": email, "password": password}
            )

        assert first.status_code == 401
        assert second.status_code == 401
        assert blocked.status_code == 401
        assert first.json() == second.json() == blocked.json()

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT outcome, details_redacted::text
                    FROM audit_events
                    WHERE action_key = 'auth.login' AND actor_user_id = %s
                    ORDER BY occurred_at DESC LIMIT 2""",
                (user_id,),
            )
            events = cur.fetchall()

        assert len(events) == 2
        assert all(event[0] == "failure" for event in events)
        assert all("wrong-password" not in event[1] for event in events)
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM auth_login_throttle WHERE key_hash IN ("
                        "SELECT %s UNION SELECT %s)",
                        ("unused-account-key", "unused-ip-key"))
```

```
        cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
PY

pytest -q tests/test_fin14_auth_security.py

..
[100%]
=============================================================================
= warnings summary
=============================================================================
=
../../../.venv/lib/python3.12/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/fastapi/test
client.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated;
install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

../../../.venv/lib/python3.12/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
stclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use
anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
2 passed, 2 warnings in 0.53s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
passed, 2 warnings in 0.53s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % echo "=== Changed and untracked
files ==="
git status --short

echo "=== Migration packaging metadata ==="
grep -n -A8 -B3 'database_migrations' ../../scripts/package_release.py

echo "=== Release environment documentation ==="
grep -n -A20 -B3 'DATABASE_URL\|SESSION_TTL_MINUTES\|COOKIE_SECURE'
../../docs/release/RELEASE_README.md

echo "=== Environment template ==="
cat ../../.env.example
=== Changed and untracked files ===
 M app/main.py
```

```
 M tests/test_auth.py
?? app/audit.py
?? app/throttle.py
?? migrations/0003_auth_audit_and_login_throttling.sql
?? tests/test_audit_service.py
?? tests/test_fin14_auth_security.py
?? tests/test_throttle.py
=== Migration packaging metadata ===
51-            "commit_sha": commit_sha,
52-            "frontend": {"runtime": "Node.js 22", "entrypoint": "node frontend/server.js"},
53-            "backend": {"runtime": "Python 3.12", "entrypoint": "python -m uvicorn
app.main:app"},
54:            "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql"],
55-            "configuration": "Runtime values are supplied through environment variables; see
RELEASE_README.md.",
56-        }
57-        (payload / "RELEASE-METADATA.json").write_text(json.dumps(metadata, indent=2) +
"\n", encoding="utf-8")
58-        release_readme = ROOT / "docs/release/RELEASE_README.md"
59-        shutil.copy2(release_readme, payload / "RELEASE_README.md")
60-
61-        artifact_path = output_dir / f"{artifact_base}.tar.gz"
62-        def normalized_member(member: tarfile.TarInfo) -> tarfile.TarInfo:
=== Release environment documentation ===
15-
16-## Configure and start
17-
18:Extract the verified archive into a new, versioned release directory. Install a supported
Node.js 22 runtime and Python 3.12 runtime. Configure the frontend process with
`API_BASE_URL=https://<api-origin>` and the backend process with `DATABASE_URL`,
`WEB_ORIGIN`, `SESSION_TTL_MINUTES`, `COOKIE_SECURE=true`, and
`APP_ENV=production`. Values containing credentials belong only in the deployment secret
manager. Set the web origin exactly to the browser-facing frontend origin; terminate TLS at an
approved proxy/load balancer and forward HTTPS safely.
19-
20-For the frontend, from the extracted bundle root:
21-
22-```sh
23-cd frontend
24-HOSTNAME=0.0.0.0 PORT=3000 node server.js
25-```
26-
27-For the backend, from the extracted bundle root, create a virtual environment and install the
bundled pinned runtime requirements:
```

28-
29-```sh
30-cd backend
31-python3.12 -m venv .venv
32-. .venv/bin/activate
33-python -m pip install -r requirements.lock
34-python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
35-```
36-
37-Run database migrations as a separate, explicitly approved deployment step with a migration-owner role. The bundle includes the current versioned migration under `backend/migrations`; it does not include or apply the supplied target schema wholesale. Use a separate least-privileged application database role at runtime.
38-
=== Environment template ===
Local development placeholders only. Replace DATABASE_URL locally; never commit it.
DATABASE_URL=postgresql://USER:PASSWORD@127.0.0.1:5432/finsight
WEB_ORIGIN=http://localhost:3000
SESSION_TTL_MINUTES=60
COOKIE_SECURE=false
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % cd ../..

python - <<'PY'
from pathlib import Path

path = Path("scripts/package_release.py")
source = path.read_text()

old = '"database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql"],'
new = '"database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql", "0003_auth_audit_and_login_throttling.sql"],'

if old not in source:
    raise SystemExit("Expected migration manifest not found; no changes made.")

path.write_text(source.replace(old, new, 1))
print("Updated release metadata to include migration 0003.")
PY
Updated release metadata to include migration 0003.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
cBook-Pro api % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % cd ../..

```
python - <<'PY'
from pathlib import Path

path = Path("scripts/package_release.py")
source = path.read_text()

old = '"database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql"],'
new = '"database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql",
"0003_auth_audit_and_login_throttling.sql"],'

if old not in source:
    raise SystemExit("Expected migration manifest not found; no changes made.")

path.write_text(source.replace(old, new, 1))
print("Updated release metadata to include migration 0003.")
PY
Updated release metadata to include migration 0003.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % python - <<'PY'
from pathlib import Path

path = Path(".env.example")
source = path.read_text()

if "AUTH_THROTTLE_SECRET=" in source:
    raise SystemExit("Throttle configuration already exists; no changes made.")

source += """
# Generate a unique secret locally; do not commit the actual secret.
# Example: python -c 'import secrets; print(secrets.token_urlsafe(48))'
AUTH_THROTTLE_SECRET=replace-with-a-random-secret-of-at-least-32-bytes
AUTH_LOGIN_MAX_FAILURES=5
AUTH_LOGIN_WINDOW_SECONDS=900
AUTH_LOGIN_BLOCK_SECONDS=900
"""
path.write_text(source)
print("Added throttle configuration placeholders to .env.example.")
PY
Added throttle configuration placeholders to .env.example.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % python - <<'PY'
from pathlib import Path

path = Path("docs/release/RELEASE_README.md")
source = path.read_text()
```

```
old = (
    "Configure the frontend process with `API_BASE_URL=https://<api-origin>` "
    "and the backend process with `DATABASE_URL`, `WEB_ORIGIN`, "
    "`SESSION_TTL_MINUTES`, `COOKIE_SECURE=true`, and `APP_ENV=production`."
)
new = (
    "Configure the frontend process with `API_BASE_URL=https://<api-origin>` "
    "and the backend process with `DATABASE_URL`, `WEB_ORIGIN`, "
    "`SESSION_TTL_MINUTES`, `COOKIE_SECURE=true`, `APP_ENV=production`, "
    "`AUTH_THROTTLE_SECRET`, `AUTH_LOGIN_MAX_FAILURES`, "
    "`AUTH_LOGIN_WINDOW_SECONDS`, and `AUTH_LOGIN_BLOCK_SECONDS`. "
    "Generate a unique random `AUTH_THROTTLE_SECRET` of at least 32 bytes "
    "and keep it in the deployment secret manager. Rotating this secret "
    "changes throttle identifiers and makes existing throttle buckets inaccessible."
)

if old not in source:
    raise SystemExit("Expected release configuration paragraph not found; no changes made.")

path.write_text(source.replace(old, new, 1))
print("Updated release configuration documentation.")
PY
Updated release configuration documentation.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % grep -n -A2
'database_migrations' scripts/package_release.py
grep -n 'AUTH_THROTTLE' .env.example
grep -n -A8 'Configure the frontend' docs/release/RELEASE_README.md
54:            "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql",
"0003_auth_audit_and_login_throttling.sql"],
55-            "configuration": "Runtime values are supplied through environment variables; see
RELEASE_README.md.",
56-        }
9:AUTH_THROTTLE_SECRET=replace-with-a-random-secret-of-at-least-32-bytes
18:Extract the verified archive into a new, versioned release directory. Install a supported
Node.js 22 runtime and Python 3.12 runtime. Configure the frontend process with
`API_BASE_URL=https://<api-origin>` and the backend process with `DATABASE_URL`,
`WEB_ORIGIN`, `SESSION_TTL_MINUTES`, `COOKIE_SECURE=true`,
`APP_ENV=production`, `AUTH_THROTTLE_SECRET`, `AUTH_LOGIN_MAX_FAILURES`,
`AUTH_LOGIN_WINDOW_SECONDS`, and `AUTH_LOGIN_BLOCK_SECONDS`. Generate a
unique random `AUTH_THROTTLE_SECRET` of at least 32 bytes and keep it in the
deployment secret manager. Rotating this secret changes throttle identifiers and makes existing
throttle buckets inaccessible. Values containing credentials belong only in the deployment secret
manager. Set the web origin exactly to the browser-facing frontend origin; terminate TLS at an
approved proxy/load balancer and forward HTTPS safely.
```

19-
20-For the frontend, from the extracted bundle root:
21-
22-```sh
23-cd frontend
24-HOSTNAME=0.0.0.0 PORT=3000 node server.js
25-```
26-
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

26-
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd apps/api

```
sed -n '250,310p' app/main.py
sed -n '1,240p' tests/test_fin14_auth_security.py
            detail="Authentication service unavailable",
        ) from exc

    if not login_succeeded:
        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

    response.set_cookie(
        SESSION_COOKIE,
        token,
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
        max_age=_ttl_minutes() * 60,
        path="/",
    )
    return UserView(
        user_id=user["user_id"],
        email=user["email"],
        display_name=user["display_name"],
    )


@app.post("/auth/logout", status_code=204)
def logout(request: Request, response: Response,
           session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None) ->
Response:
    _check_origin(request)
    if session_token:
        try:
```

```
            with connect() as conn, conn.cursor() as cur:
                cur.execute(
                    """UPDATE auth_sessions SET revoked_at = %s, revoke_reason = 'logout'
                       WHERE refresh_token_hash = %s AND revoked_at IS NULL""",
                    (datetime.now(UTC), token_digest(session_token)),
                )
        except psycopg.Error as exc:
            raise HTTPException(status_code=503, detail="Authentication service unavailable")
from exc
    response.delete_cookie(SESSION_COOKIE, path="/", httponly=True,
secure=_secure_cookie(), samesite="lax")
    response.status_code = status.HTTP_204_NO_CONTENT
    return response


@app.get("/auth/me", response_model=UserView)
def who_am_i(user: CurrentUser) -> UserView:
    return UserView(**user)


@app.get("/api/v1/admin/authorization/capabilities")
def platform_admin_capabilities(
    request: Request,
    _authorized: Annotated[dict[str, Any], Depends(require_permission("platform.admin"))],
) -> dict[str, Any]:
    """Return the authenticated caller's database-backed platform capabilities."""

    supplied_request_id = request.headers.get("request_id", "").strip()
    if supplied_request_id and (
        len(supplied_request_id) > 128
        or any(ord(char) < 32 or ord(char) == 127 for char in supplied_request_id)
    ):
        raise HTTPException(status_code=400, detail="Invalid request_id")

    # Always issue a server-generated canonical request ID.
from __future__ import annotations

from uuid import uuid4

from fastapi.testclient import TestClient

from app.auth import hash_password
from app.database import connect
from app.main import app
```

```
def test_successful_login_is_audited_without_secrets(monkeypatch):
    monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Password-Only"
    correlation = None

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """INSERT INTO users (email, password_hash, display_name, account_status)
               VALUES (%s, %s, %s, 'active') RETURNING user_id""",
            (email, hash_password(password), "FIN14 Audit Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with TestClient(app) as client:
            response = client.post(
                "/auth/login", json={"email": email, "password": password}
            )
        assert response.status_code == 200

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT action_key, outcome, actor_user_id, details_redacted::text
                   FROM audit_events
                   WHERE action_key = 'auth.login' AND actor_user_id = %s
                   ORDER BY occurred_at DESC LIMIT 1""",
                (user_id,),
            )
            event = cur.fetchone()

        assert event is not None
        assert event[0] == "auth.login"
        assert event[1] == "success"
        assert event[2] == user_id
        assert password not in event[3]
        assert response.cookies.get("finsight_session") not in event[3]
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
```

```
def test_failed_login_is_audited_and_repeated_failures_are_blocked(monkeypatch):
    monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
    monkeypatch.setenv("AUTH_LOGIN_MAX_FAILURES", "2")
    monkeypatch.setenv("AUTH_LOGIN_WINDOW_SECONDS", "600")
    monkeypatch.setenv("AUTH_LOGIN_BLOCK_SECONDS", "60")

    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Password-Only"

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """INSERT INTO users (email, password_hash, display_name, account_status)
               VALUES (%s, %s, %s, 'active') RETURNING user_id""",
            (email, hash_password(password), "FIN14 Throttle Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with TestClient(app) as client:
            first = client.post(
                "/auth/login", json={"email": email, "password": "wrong-password"}
            )
            second = client.post(
                "/auth/login", json={"email": email, "password": "wrong-password"}
            )
            blocked = client.post(
                "/auth/login", json={"email": email, "password": password}
            )

        assert first.status_code == 401
        assert second.status_code == 401
        assert blocked.status_code == 401
        assert first.json() == second.json() == blocked.json()

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT outcome, details_redacted::text
                   FROM audit_events
                   WHERE action_key = 'auth.login' AND actor_user_id = %s
                   ORDER BY occurred_at DESC LIMIT 2""",
                (user_id,),
            )
```

```
            events = cur.fetchall()

        assert len(events) == 2
        assert all(event[0] == "failure" for event in events)
        assert all("wrong-password" not in event[1] for event in events)
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM auth_login_throttle WHERE key_hash IN ("
                        "SELECT %s UNION SELECT %s)",
                        ("unused-account-key", "unused-ip-key"))
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % nano temp.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % python3 temp.py
Updated logout to persist a redacted authentication audit event.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % pytest -q tests/test_auth.py
tests/test_auth_unit.py tests/test_audit_service.py tests/test_fin14_auth_security.py
.....................................................F
[100%]
================================================================================
===== FAILURES
================================================================================
=====

_____________________________________________________
test_failed_login_is_audited_and_repeated_failures_are_blocked
_____________________________________________________

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x10ce43740>

    def test_failed_login_is_audited_and_repeated_failures_are_blocked(monkeypatch):
        monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
        monkeypatch.setenv("AUTH_LOGIN_MAX_FAILURES", "2")
        monkeypatch.setenv("AUTH_LOGIN_WINDOW_SECONDS", "600")
        monkeypatch.setenv("AUTH_LOGIN_BLOCK_SECONDS", "60")

        email = f"{uuid4().hex}@example.com"
        password = "Synthetic-FIN14-Password-Only"

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """INSERT INTO users (email, password_hash, display_name, account_status)
                   VALUES (%s, %s, %s, 'active') RETURNING user_id""",
                (email, hash_password(password), "FIN14 Throttle Test"),
```

```
            )
            user_id = cur.fetchone()[0]

        try:
            with TestClient(app) as client:
                first = client.post(
                    "/auth/login", json={"email": email, "password": "wrong-password"}
                )
                second = client.post(
                    "/auth/login", json={"email": email, "password": "wrong-password"}
                )
                blocked = client.post(
                    "/auth/login", json={"email": email, "password": password}
                )

            assert first.status_code == 401
            assert second.status_code == 401
            assert blocked.status_code == 401
            assert first.json() == second.json() == blocked.json()

            with connect() as conn, conn.cursor() as cur:
                cur.execute(
                    """SELECT outcome, details_redacted::text
                       FROM audit_events
                       WHERE action_key = 'auth.login' AND actor_user_id = %s
                       ORDER BY occurred_at DESC LIMIT 2""",
                    (user_id,),
                )
                events = cur.fetchall()

>           assert len(events) == 2
E           assert 1 == 2
E            +  where 1 = len([('failure', '{"reason": "invalid_credentials"}')])

tests/test_fin14_auth_security.py:98: AssertionError
=============================================================================
= warnings summary
=============================================================================
=
../../../.venv/lib/python3.12/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/fastapi/test
client.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated;
install `httpx2` instead.
```

```
    from starlette.testclient import TestClient as TestClient  # noqa

../../../.venv/lib/python3.12/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
stclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use
anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected
apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[get-/a
uth/me-kwargs0]
apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[post-/
auth/logout-kwargs1]
apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[get-/a
pi/v1/admin/authorization/capabilities-kwargs2]

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
stclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being deprecated,
because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on
the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=========================================================================
short test summary info
=========================================================================
FAILED
tests/test_fin14_auth_security.py::test_failed_login_is_audited_and_repeated_failures_are_bloc
ked - assert 1 == 2
1 failed, 50 passed, 9 warnings in 1.20s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
4 failed, 47 passed, 6 warnings in 1.09s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % >....

new = '''    throttle_secret = "test-only-secret-material-at-least-32-bytes"
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
    ip_key = make_throttle_key(
        throttle_secret.encode("utf-8"), "ip", "testclient"
    )
```

```
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
        )

    try:
        with TestClient(app) as test_client:
            yield test_client
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
                (ip_key,),
            )
'''

if old not in source:
    raise SystemExit("Expected client fixture not found; no changes made.")

path.write_text(source.replace(old, new, 1))
print("Updated auth fixture to isolate the synthetic IP throttle bucket.")
PY
Updated auth fixture to isolate the synthetic IP throttle bucket.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % >....
    '''                     ORDER BY occurred_at DESC LIMIT 3"""
            )
''',
    '''                     AND occurred_at >= %s
                    ORDER BY occurred_at DESC LIMIT 3""",
                (test_started_at,),
            )
''',
    1,
)

# Replace placeholder cleanup with cleanup of the actual IP bucket.
after = after.replace(
    '''            cur.execute("DELETE FROM auth_login_throttle WHERE key_hash IN ("
                    "SELECT %s UNION SELECT %s)",
                    ("unused-account-key", "unused-ip-key"))
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
''',
    '''            cur.execute(
```

```
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
        )
        cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
'''.
    1,
)

path.write_text(before + after)
print("Updated throttle test isolation, audit time filter, and cleanup.")
PY
Updated throttle test isolation, audit time filter, and cleanup.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % pytest -q tests/test_auth.py
tests/test_auth_unit.py tests/test_audit_service.py tests/test_fin14_auth_security.py
.....................................................
[100%]
=============================================================================
= warnings summary
=============================================================================
=
../../../.venv/lib/python3.12/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/fastapi/test
client.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated;
install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

../../../.venv/lib/python3.12/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/te
stclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use
anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected
apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[get-/a
uth/me-kwargs0]
apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[post-/
auth/logout-kwargs1]
apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[get-/a
pi/v1/admin/authorization/capabilities-kwargs2]
```

```
/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on the client instance instead.
  return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
51 passed, 9 warnings in 1.29s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
51 passed, 9 warnings in 1.29s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % python - <<'PY'
from app.throttle import make_throttle_key
from app.database import connect

secret = "test-only-secret-material-at-least-32-bytes"
ip_key = make_throttle_key(secret.encode(), "ip", "testclient")

with connect() as conn, conn.cursor() as cur:
  cur.execute(
    """
    SELECT outcome, count(*)
    FROM audit_events
    WHERE action_key = 'auth.logout'
    GROUP BY outcome
    ORDER BY outcome
    """
  )
  print("Logout audit totals:", cur.fetchall())

  cur.execute(
    """
    SELECT count(*)
    FROM auth_login_throttle
    WHERE scope_type = 'ip' AND key_hash = %s
    """,
    (ip_key,),
  )
  print("Test fixture IP buckets remaining:", cur.fetchone()[0])
PY
Logout audit totals: [('denied', 3), ('success', 2)]
Test fixture IP buckets remaining: 0
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %
Test fixture IP buckets remaining: 0
```

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % >....
        assert session_token

        logout = client.post(
            "/auth/logout",
            cookies={"finsight_session": session_token},
        )
        assert logout.status_code == 204

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """SELECT outcome, actor_user_id, details_redacted::text
               FROM audit_events
               WHERE action_key = 'auth.logout'
                 AND actor_user_id = %s
               ORDER BY occurred_at DESC LIMIT 1""",
            (user_id,),
        )
        event = cur.fetchone()

    assert event is not None
    assert event[0] == "success"
    assert event[1] == user_id
    assert session_token not in event[2]
    assert password not in event[2]
finally:
    with connect() as conn, conn.cursor() as cur:
        cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
PY

pytest -q tests/test_fin14_auth_security.py tests/test_auth.py
.....
[100%]
=============================================================================
= warnings summary
=============================================================================
=
../../../.venv/lib/python3.12/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/fastapi/test
client.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated;
install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient  # noqa
```

../../../.venv/lib/python3.12/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_token
apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected

/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/lib/python3.12/site-packages/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on the client instance instead.
  return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
12 passed, 7 warnings in 1.29s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api % cd ../..

echo "=== Compile release builder ==="
python -m py_compile scripts/package_release.py

echo "=== Inspect release builder inputs and output options ==="
sed -n '1,180p' scripts/package_release.py

echo "=== Confirm migration is included in the release source tree ==="
ls -l apps/api/migrations/0003_auth_audit_and_login_throttling.sql

echo "=== Review current working tree ==="
git status --short
=== Compile release builder ===
=== Inspect release builder inputs and output options ===
#!/usr/bin/env python3
"""Build a versioned FinSight application bundle from locked/build outputs."""

from __future__ import annotations

```
import argparse
import hashlib
import json
import os
import re
import shutil
import tarfile
import tempfile
from datetime import UTC, datetime
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
VERSION_PATTERN = re.compile(r"\A[0-9]+\.[0-9]+\.[0-9]+(?:[-+][0-9A-Za-z.-]+)?\Z")
SHA_PATTERN = re.compile(r"\A[0-9a-f]{40}\Z")


def package(frontend_dir: Path, version: str, commit_sha: str | None, output_dir: Path) ->
tuple[Path, Path, Path]:
    if not VERSION_PATTERN.fullmatch(version):
        raise ValueError("version must be a numeric semantic version, optionally with a
prerelease/build suffix")
    if commit_sha and not SHA_PATTERN.fullmatch(commit_sha):
        raise ValueError("commit SHA must be a full 40-character lowercase hexadecimal SHA")
    if not frontend_dir.is_dir() or not (frontend_dir / "server.js").is_file():
        raise FileNotFoundError("frontend standalone build is missing server.js")

    web_static = ROOT / "apps/web/.next/static"
    runtime_config = ROOT / "apps/web/app/api/runtime-config/route.js"
    if not web_static.is_dir() or not runtime_config.is_file():
        raise FileNotFoundError("build the frontend before packaging")

    output_dir.mkdir(parents=True, exist_ok=True)
    artifact_base = f"finsight-{version}" + (f"-{commit_sha[:12]}" if commit_sha else "")
    with tempfile.TemporaryDirectory(prefix="finsight-release-") as temp_dir:
        payload = Path(temp_dir) / "finsight"
        frontend = payload / "frontend"
        shutil.copytree(frontend_dir, frontend, symlinks=True)
        shutil.copytree(web_static, frontend / ".next/static", symlinks=True)

        backend = payload / "backend"
        shutil.copytree(ROOT / "apps/api/app", backend / "app",
ignore=shutil.ignore_patterns("__pycache__", "*.pyc"))
        shutil.copytree(ROOT / "apps/api/migrations", backend / "migrations")
        shutil.copy2(ROOT / "apps/api/requirements.lock", backend / "requirements.lock")
```

```
        metadata = {
            "artifact": artifact_base,
            "version": version,
            "commit_sha": commit_sha,
            "frontend": {"runtime": "Node.js 22", "entrypoint": "node frontend/server.js"},
            "backend": {"runtime": "Python 3.12", "entrypoint": "python -m uvicorn app.main:app"},
            "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql",
"0003_auth_audit_and_login_throttling.sql"],
            "configuration": "Runtime values are supplied through environment variables; see
RELEASE_README.md.",
        }
        (payload / "RELEASE-METADATA.json").write_text(json.dumps(metadata, indent=2) + "\n",
encoding="utf-8")
        release_readme = ROOT / "docs/release/RELEASE_README.md"
        shutil.copy2(release_readme, payload / "RELEASE_README.md")

        artifact_path = output_dir / f"{artifact_base}.tar.gz"
        def normalized_member(member: tarfile.TarInfo) -> tarfile.TarInfo:
            member.mtime = 0
            member.uid = 0
            member.gid = 0
            member.uname = ""
            member.gname = ""
            return member

        with artifact_path.open("wb") as raw:
            import gzip

            with gzip.GzipFile(fileobj=raw, mode="wb", mtime=0, filename="") as compressed:
                with tarfile.open(fileobj=compressed, mode="w", format=tarfile.PAX_FORMAT) as
archive:
                    for item in sorted(payload.rglob("*")):
                        archive.add(
                            item,
                            arcname=(Path("finsight") / item.relative_to(payload)).as_posix(),
                            recursive=False,
                            filter=normalized_member,
                        )

    digest_builder = hashlib.sha256()
    with artifact_path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1024 * 1024), b""):
            digest_builder.update(chunk)
```

```
    digest = digest_builder.hexdigest()
    checksum_path = output_dir / f"{artifact_path.name}.sha256"
    checksum_path.write_text(f"{digest}  {artifact_path.name}\n", encoding="ascii")
    build_info_path = output_dir / f"{artifact_path.name}.build-info.json"
    build_info = {
        **metadata,
        "built_at_utc": datetime.now(UTC).replace(microsecond=0).isoformat(),
        "sha256": digest,
    }
    build_info_path.write_text(json.dumps(build_info, indent=2) + "\n", encoding="utf-8")
    return artifact_path, checksum_path, build_info_path


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--frontend-dir", type=Path, required=True, help="Next standalone
output directory")
    parser.add_argument("--version", required=True)
    parser.add_argument("--commit-sha", default=os.environ.get("GITHUB_SHA"))
    parser.add_argument("--output-dir", type=Path, default=ROOT / "dist")
    args = parser.parse_args()
    artifact, checksum, build_info = package(args.frontend_dir, args.version, args.commit_sha,
args.output_dir)
    print(f"Created {artifact}")
    print(f"Created {checksum}")
    print(f"Created {build_info}")


if __name__ == "__main__":
    main()
=== Confirm migration is included in the release source tree ===
-rw-r--r--@ 1 sandeeph  staff  1738 Oct 21 15:02
apps/api/migrations/0003_auth_audit_and_login_throttling.sql
=== Review current working tree ===
 M .env.example
 M apps/api/app/main.py
 M apps/api/tests/test_auth.py
 M docs/release/RELEASE_README.md
 M scripts/package_release.py
?? apps/api/app/audit.py
?? apps/api/app/throttle.py
?? apps/api/migrations/0003_auth_audit_and_login_throttling.sql
?? apps/api/temp.py
?? apps/api/tests/test_audit_service.py
```

```
?? apps/api/tests/test_fin14_auth_security.py
?? apps/api/tests/test_throttle.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
tle.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "=== Inspect temporary
helper before removing it ==="
sed -n '1,100p' apps/api/temp.py

echo "=== Remove only the temporary helper ==="
rm apps/api/temp.py

echo "=== Check required release build inputs ==="
test -f apps/web/.next/standalone/server.js && echo "Frontend standalone server exists" || echo
"Frontend standalone server missing"
test -d apps/web/.next/static && echo "Frontend static assets exist" || echo "Frontend static
assets missing"
test -f apps/web/app/api/runtime-config/route.js && echo "Runtime config route exists" || echo
"Runtime config route missing"

echo "=== Check release output directory before packaging ==="
ls -ld dist 2>/dev/null || echo "No dist directory currently exists"
=== Inspect temporary helper before removing it ===

from pathlib import Path

path = Path("tests/test_fin14_auth_security.py")
source = path.read_text()

old = '''        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT outcome, details_redacted::text
                    FROM audit_events
                    WHERE action_key = 'auth.login' AND actor_user_id = %s
                    ORDER BY occurred_at DESC LIMIT 2""",
                (user_id,),
            )
            events = cur.fetchall()

        assert len(events) == 2
        assert all(event[0] == "failure" for event in events)
        assert all("wrong-password" not in event[1] for event in events)
'''

new = '''        with connect() as conn, conn.cursor() as cur:
```

```
        cur.execute(
            """SELECT outcome, actor_user_id, details_redacted::text
               FROM audit_events
               WHERE action_key = 'auth.login'
                 AND details_redacted->>'reason' IN (
                     'invalid_credentials', 'throttled'
                 )
               ORDER BY occurred_at DESC LIMIT 3"""
        )
        events = cur.fetchall()

    assert len(events) == 3
    assert sum(event[0] == "failure" for event in events) == 2
    assert sum(event[0] == "denied" for event in events) == 1
    assert all(
        "wrong-password" not in event[2]
        for event in events
    )
    assert any(event[1] == user_id for event in events)
'''

if old not in source:
    raise SystemExit("Expected test assertion block not found; no changes made.")

path.write_text(source.replace(old, new, 1))
print("Updated the test to verify failed and throttled audit events separately.")
=== Remove only the temporary helper ===
=== Check required release build inputs ===
Frontend standalone server exists
Frontend static assets exist
Runtime config route exists
=== Check release output directory before packaging ===
drwxr-xr-x@ 5 sandeeph  staff  160 Oct 20 13:30 dist
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
r-xr-x@ 5 sandeeph  staff  160 Oct 20 13:30 dist
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % VALIDATION_DIR="
(mktemp − d"{TMPDIR:-/tmp}/finsight-fin14-release-check.XXXXXX")"

echo "Validating release in a temporary directory."

python scripts/package_release.py \
  --frontend-dir apps/web/.next/standalone \
  --version 0.0.0-fin14check \
  --output-dir "$VALIDATION_DIR"
```

```
echo "=== Check generated archive ==="
tar -tzf "$VALIDATION_DIR/finsight-0.0.0-fin14check.tar.gz" \
  | grep -E
'RELEASE-METADATA.json|backend/migrations/0003_auth_audit_and_login_throttling.sql|back
end/app/(audit|throttle)\.py'

echo "=== Verify metadata inside the archive ==="
tar -xOzf "$VALIDATION_DIR/finsight-0.0.0-fin14check.tar.gz" \
  finsight/RELEASE-METADATA.json \
  | python -c '
import json, sys
metadata = json.load(sys.stdin)
expected = "0003_auth_audit_and_login_throttling.sql"
assert expected in metadata["database_migrations"], metadata["database_migrations"]
print("PASS: migration 0003 is declared in release metadata")
print("Packaged migrations:", ", ".join(metadata["database_migrations"]))
'

echo "=== Validation directory ==="
echo "$VALIDATION_DIR"
Validating release in a temporary directory.
Created
/var/folders/s5/b1gbwdg93h943rt8m90xg6rh0000gn/T/finsight-fin14-release-check.vjgqOY/finsig
ht-0.0.0-fin14check.tar.gz
Created
/var/folders/s5/b1gbwdg93h943rt8m90xg6rh0000gn/T/finsight-fin14-release-check.vjgqOY/finsig
ht-0.0.0-fin14check.tar.gz.sha256
Created
/var/folders/s5/b1gbwdg93h943rt8m90xg6rh0000gn/T/finsight-fin14-release-check.vjgqOY/finsig
ht-0.0.0-fin14check.tar.gz.build-info.json
=== Check generated archive ===
finsight/RELEASE-METADATA.json
finsight/backend/app/audit.py
finsight/backend/app/throttle.py
finsight/backend/migrations/0003_auth_audit_and_login_throttling.sql
=== Verify metadata inside the archive ===
PASS: migration 0003 is declared in release metadata
Packaged migrations: 0001_auth_identity.sql, 0002_platform_authorization.sql,
0003_auth_audit_and_login_throttling.sql
=== Validation directory ===
/var/folders/s5/b1gbwdg93h943rt8m90xg6rh0000gn/T//finsight-fin14-release-check.vjgqOY
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
```

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
VALIDATION_DIR="/var/folders/s5/b1gbwdg93h943rt8m90xg6rh0000gn/T/finsight-fin14-release
-check.vjgqOY"

cd "$VALIDATION_DIR" || exit 1

shasum -a 256 -c finsight-0.0.0-fin14check.tar.gz.sha256

echo "=== Build information ==="
cat finsight-0.0.0-fin14check.tar.gz.build-info.json
finsight-0.0.0-fin14check.tar.gz: OK
=== Build information ===
{
  "artifact": "finsight-0.0.0-fin14check",
  "version": "0.0.0-fin14check",
  "commit_sha": null,
  "frontend": {
    "runtime": "Node.js 22",
    "entrypoint": "node frontend/server.js"
  },
  "backend": {
    "runtime": "Python 3.12",
    "entrypoint": "python -m uvicorn app.main:app"
  },
  "database_migrations": [
    "0001_auth_identity.sql",
    "0002_platform_authorization.sql",
    "0003_auth_audit_and_login_throttling.sql"
  ],
  "configuration": "Runtime values are supplied through environment variables; see
RELEASE_README.md.",
  "built_at_utc": "2026-10-21T10:41:04+00:00",
  "sha256": "b076d664d10291c721c56e16384d932ffa489052fd52864b3accc18a3857a3d8"
}
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro finsight-fin14-release-check.vjgqOY %
(base) sandeeph@Sandeeps-MacBook-Pro NVDIA_Play_Book %  source
/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/bin/activate
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro NVDIA_Play_Book % cd FinSight_AI
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

echo "=== 1. Check patch formatting ==="
=== 1. Check patch formatting ===
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % git diff --check
```

```
apps/api/tests/test_auth.py:62: trailing whitespace.
+
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % .venv/bin/python -m pytest -q
\
  apps/api/tests/test_auth.py \
  apps/api/tests/test_auth_unit.py \
  apps/api/tests/test_audit_service.py \
  apps/api/tests/test_throttle.py \
  apps/api/tests/test_fin14_auth_security.py
.EEEEEEEE.....................................................FFF
[100%]
==========================================================================
============== ERRORS
==========================================================================
===============
_____________________________________________________ ERROR at setup of
test_login_creates_protected_hashed_expiring_session
_____________________________________________________

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x10338dba0>

    @pytest.fixture
    def client(monkeypatch):
        monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
        monkeypatch.setenv("COOKIE_SECURE", "false")
        monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
        throttle_secret = "test-only-secret-material-at-least-32-bytes"
        monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
        ip_key = make_throttle_key(
            throttle_secret.encode("utf-8"), "ip", "testclient"
        )

>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_auth.py:46:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

    def connect() -> psycopg.Connection:
        database_url = os.environ.get("DATABASE_URL")
        if not database_url:
>           raise RuntimeError("DATABASE_URL is required")
E           RuntimeError: DATABASE_URL is required
```

```
apps/api/app/database.py:17: RuntimeError
_____ ERROR at setup of test_invalid_credentials_have_same_generic_response _____

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x105d81ba0>

    @pytest.fixture
    def client(monkeypatch):
        monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
        monkeypatch.setenv("COOKIE_SECURE", "false")
        monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
        throttle_secret = "test-only-secret-material-at-least-32-bytes"
        monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
        ip_key = make_throttle_key(
            throttle_secret.encode("utf-8"), "ip", "testclient"
        )

>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_auth.py:46:
_ _ _ _
```

```
        throttle_secret = "test-only-secret-material-at-least-32-bytes"
        monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
        ip_key = make_throttle_key(
            throttle_secret.encode("utf-8"), "ip", "testclient"
        )

>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_auth.py:46:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

    def connect() -> psycopg.Connection:
        database_url = os.environ.get("DATABASE_URL")
        if not database_url:
>           raise RuntimeError("DATABASE_URL is required")
E           RuntimeError: DATABASE_URL is required

apps/api/app/database.py:17: RuntimeError
_____________________________________________________________________ ERROR at setup of
test_logout_without_session_is_idempotent
_____________________________________________________________________

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x105d7a470>

    @pytest.fixture
    def client(monkeypatch):
        monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
        monkeypatch.setenv("COOKIE_SECURE", "false")
        monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
        throttle_secret = "test-only-secret-material-at-least-32-bytes"
        monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
        ip_key = make_throttle_key(
            throttle_secret.encode("utf-8"), "ip", "testclient"
        )

>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_auth.py:46:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
```

```
    def connect() -> psycopg.Connection:
        database_url = os.environ.get("DATABASE_URL")
        if not database_url:
>           raise RuntimeError("DATABASE_URL is required")
E           RuntimeError: DATABASE_URL is required

apps/api/app/database.py:17: RuntimeError
```

_____ ERROR at setup of test_expired_session_is_rejected _____

```
monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x105d7abe0>

    @pytest.fixture
    def client(monkeypatch):
        monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
        monkeypatch.setenv("COOKIE_SECURE", "false")
        monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
        throttle_secret = "test-only-secret-material-at-least-32-bytes"
        monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
        ip_key = make_throttle_key(
            throttle_secret.encode("utf-8"), "ip", "testclient"
        )

>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_auth.py:46:
_ _ _ _
```

```
    @pytest.fixture
    def client(monkeypatch):
        monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
        monkeypatch.setenv("COOKIE_SECURE", "false")
        monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
        throttle_secret = "test-only-secret-material-at-least-32-bytes"
        monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
        ip_key = make_throttle_key(
            throttle_secret.encode("utf-8"), "ip", "testclient"
        )

>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_auth.py:46:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

    def connect() -> psycopg.Connection:
        database_url = os.environ.get("DATABASE_URL")
        if not database_url:
>           raise RuntimeError("DATABASE_URL is required")
E           RuntimeError: DATABASE_URL is required

apps/api/app/database.py:17: RuntimeError
_____________________________________________________________________ ERROR at setup of
test_auth_mutation_rejects_foreign_origin
_____________________________________________________________________

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x105e80750>

    @pytest.fixture
    def client(monkeypatch):
        monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
        monkeypatch.setenv("COOKIE_SECURE", "false")
        monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
        throttle_secret = "test-only-secret-material-at-least-32-bytes"
        monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
        ip_key = make_throttle_key(
            throttle_secret.encode("utf-8"), "ip", "testclient"
        )

>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^
```

```
apps/api/tests/test_auth.py:46:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

    def connect() -> psycopg.Connection:
        database_url = os.environ.get("DATABASE_URL")
        if not database_url:
>           raise RuntimeError("DATABASE_URL is required")
E           RuntimeError: DATABASE_URL is required

apps/api/app/database.py:17: RuntimeError
_____________________________________________________________ ERROR at setup of
test_secure_cookie_can_be_required_by_deployment
_____________________________________________________________

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x1058fff20>

    @pytest.fixture
    def client(monkeypatch):
        monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
        monkeypatch.setenv("COOKIE_SECURE", "false")
        monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
        throttle_secret = "test-only-secret-material-at-least-32-bytes"
        monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
        ip_key = make_throttle_key(
            throttle_secret.encode("utf-8"), "ip", "testclient"
        )

>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_auth.py:46:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

    def connect() -> psycopg.Connection:
        database_url = os.environ.get("DATABASE_URL")
        if not database_url:
>           raise RuntimeError("DATABASE_URL is required")
E           RuntimeError: DATABASE_URL is required

apps/api/app/database.py:17: RuntimeError
```

```
=================================================================================== FAILURES ===================================================================================
_____ test_successful_login_is_audited_without_secrets _____

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x105dbcfa0>

    def test_successful_login_is_audited_without_secrets(monkeypatch):
        monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
        email = f"{uuid4().hex}@example.com"
        password = "Synthetic-FIN14-Password-Only"
        correlation = None

>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_fin14_auth_security.py:20:
_ _ _ _
```

```
        email = f"{uuid4().hex}@example.com"
        password = "Synthetic-FIN14-Password-Only"
        test_started_at = datetime.now(UTC)
        ip_key = make_throttle_key(
            b"integration-test-secret-material-32-bytes",
            "ip",
            "testclient",
        )

>       with connect() as conn, conn.cursor() as cur:
             ^^^^^^^^^

apps/api/tests/test_fin14_auth_security.py:71:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

    def connect() -> psycopg.Connection:
        database_url = os.environ.get("DATABASE_URL")
        if not database_url:
>           raise RuntimeError("DATABASE_URL is required")
E           RuntimeError: DATABASE_URL is required

apps/api/app/database.py:17: RuntimeError
_____________________________________________________________________
test_logout_is_audited_without_recording_session_token
_____________________________________________________________________

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x105d47c40>

    def test_logout_is_audited_without_recording_session_token(monkeypatch):
        monkeypatch.setenv(
            "AUTH_THROTTLE_SECRET",
            "integration-test-secret-material-32-bytes",
        )
        email = f"{uuid4().hex}@example.com"
        password = "Synthetic-FIN14-Logout-Password"

>       with connect() as conn, conn.cursor() as cur:
             ^^^^^^^^^

apps/api/tests/test_fin14_auth_security.py:137:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
```

```
    def connect() -> psycopg.Connection:
        database_url = os.environ.get("DATABASE_URL")
        if not database_url:
>           raise RuntimeError("DATABASE_URL is required")
E           RuntimeError: DATABASE_URL is required

apps/api/app/database.py:17: RuntimeError
=============================================================================
========= warnings summary
=============================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[get-/a
uth/me-kwargs0]
apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[post-/
auth/logout-kwargs1]
apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[get-/a
pi/v1/admin/authorization/capabilities-kwargs2]

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=============================================================================
====== short test summary info
=============================================================================
======
```

```
FAILED
apps/api/tests/test_fin14_auth_security.py::test_successful_login_is_audited_without_secrets -
RuntimeError: DATABASE_URL is required
FAILED
apps/api/tests/test_fin14_auth_security.py::test_failed_login_is_audited_and_repeated_failures_
are_blocked - RuntimeError: DATABASE_URL is required
FAILED
apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken - RuntimeError: DATABASE_URL is required
ERROR apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session -
RuntimeError: DATABASE_URL is required
ERROR apps/api/tests/test_auth.py::test_invalid_credentials_have_same_generic_response -
RuntimeError: DATABASE_URL is required
ERROR apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails -
RuntimeError: DATABASE_URL is required
ERROR apps/api/tests/test_auth.py::test_logout_without_session_is_idempotent -
RuntimeError: DATABASE_URL is required
ERROR apps/api/tests/test_auth.py::test_expired_session_is_rejected - RuntimeError:
DATABASE_URL is required
ERROR apps/api/tests/test_auth.py::test_public_registration_is_not_available - RuntimeError:
DATABASE_URL is required
ERROR apps/api/tests/test_auth.py::test_auth_mutation_rejects_foreign_origin - RuntimeError:
DATABASE_URL is required
ERROR apps/api/tests/test_auth.py::test_secure_cookie_can_be_required_by_deployment -
RuntimeError: DATABASE_URL is required
3 failed, 55 passed, 5 warnings, 8 errors in 465.08s (0:07:45)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
5 files changed, 220 insertions(+), 48 deletions(-)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

# Configure the dedicated local test database for this terminal session.
export DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test'
export AUTH_THROTTLE_SECRET="$(.venv/bin/python -c 'import secrets;
print(secrets.token_urlsafe(48))')"

# Remove trailing whitespace from the reported line only.
.venv/bin/python -c '
from pathlib import Path
p = Path("apps/api/tests/test_auth.py")
lines = p.read_text().splitlines()
if len(lines) >= 62:
    lines[61] = lines[61].rstrip()
    p.write_text("\n".join(lines) + "\n")
```

```
print("Fixed trailing whitespace on line 62.")
'

echo "=== Verify patch formatting ==="
git diff --check || exit 1

echo "=== Verify test database connection ==="
.venv/bin/python -c '
from apps.api.app.database import connect
with connect() as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT current_database()")
        print("Connected database:", cur.fetchone()[0])
'

echo "=== Run FIN-14 authentication/security tests ==="
.venv/bin/python -m pytest -q \
  apps/api/tests/test_auth.py \
  apps/api/tests/test_auth_unit.py \
  apps/api/tests/test_audit_service.py \
  apps/api/tests/test_throttle.py \
  apps/api/tests/test_fin14_auth_security.py
zsh: command not found: #
zsh: command not found: #
Fixed trailing whitespace on line 62.
=== Verify patch formatting ===
=== Verify test database connection ===
Connected database: finsight_fin15_test
=== Run FIN-14 authentication/security tests ===
.....................................................................
[100%]
=============================================================================
========= warnings summary
=============================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53
```

```
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-packages/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected
apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[get-/auth/me-kwargs0]
apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[post-/auth/logout-kwargs1]
apps/api/tests/test_auth_unit.py::test_session_database_outages_return_controlled_error[get-/api/v1/admin/authorization/capabilities-kwargs2]
apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_token

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-packages/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
66 passed, 10 warnings in 3.83s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
rg -n -C 3 'MFA|multi.factor|active.session|FIN-14|AUTHSEC_003' \
  conversation.txt
=== FIN-14 implementation entries ===
zsh: command not found: rg
=== FIN-14 test execution entries ===
zsh: command not found: rg
=== Function/test traceability entries ===
zsh: command not found: rg
=== Relevant conversation decisions ===
zsh: command not found: rg
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

echo "=== FIN-14 implementation entries ==="
rg -n -C 5 'FIN-14|AUTHSEC_001|AUTHSEC_002|AUTHSEC_003' \
  IMPLEMENTATION_STATUS.txt
```

```
echo "=== FIN-14 test execution entries ==="
rg -n -C 4 'AUTHSEC_001|AUTHSEC_002|AUTHSEC_003|FIN-14' \
  SPRINT_1_TEST_EXECUTION_STATUS.txt

echo "=== Function/test traceability entries ==="
rg -n -C 3 'AUTHSEC_001|AUTHSEC_002|AUTHSEC_003|FIN-14' \
  FUNCTION_TEST_TRACEABILITY.txt

echo "=== Relevant conversation decisions ==="
rg -n -C 3 'MFA|multi.factor|active.session|FIN-14|AUTHSEC_003' \
  conversation.txt
=== FIN-14 implementation entries ===
zsh: command not found: rg
=== FIN-14 test execution entries ===
zsh: command not found: rg
=== Function/test traceability entries ===
zsh: command not found: rg
=== Relevant conversation decisions ===
zsh: command not found: rg
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

echo "=== FIN-14 implementation entries ==="
grep -n -E -B 5 -A 5 'FIN-14|AUTHSEC_001|AUTHSEC_002|AUTHSEC_003' \
  IMPLEMENTATION_STATUS.txt || true

echo "=== FIN-14 test execution entries ==="
grep -n -E -B 4 -A 4 'AUTHSEC_001|AUTHSEC_002|AUTHSEC_003|FIN-14' \
  SPRINT_1_TEST_EXECUTION_STATUS.txt || true

echo "=== Function/test traceability entries ==="
grep -n -E -B 3 -A 3 'AUTHSEC_001|AUTHSEC_002|AUTHSEC_003|FIN-14' \
  FUNCTION_TEST_TRACEABILITY.txt || true

echo "=== Relevant conversation decisions ==="
grep -n -i -E -B 3 -A 3 'MFA|multi.factor|active.session|FIN-14|AUTHSEC_003' \
  conversation.txt || true
=== FIN-14 implementation entries ===
234-- Dependency advisory follow-up: previous npm install reported 3 advisories (2 high, 1
critical). Package-level triage was not performed here; recorded for FIN-77 in `conversation.txt`.
235-Remaining acceptance criteria/blockers: FIN-76 remains PARTIAL pending GitHub Actions
confirmation that the coverage artifact is uploaded/retained and a decision on whether frontend
```

source-coverage instrumentation is required for acceptance. No Git, Jira, source artifact, or dependency changes were made in this task.
236-
237-FIN-77 / FS-11-06 — Security validation and dependency checks (2026-10-19)
238-Status: PARTIAL
239:Scope references: detailed backlog FS-11-06 (P0, 3 SP) and four-sprint plan (3 SP). Sprint 1 management plan maps this story to TC_ENV_002, TC_SEC_001, TC_SEC_002, TC_AUTHSEC_002, TC_AUTHN_006, TC_CFG_001, and TC_EXEC_002. Original references remain unchanged.
240-Npm baseline and remediation:
241-- Before: `npm audit --json` reported 3 vulnerable dependency nodes: 1 critical (`next@16.2.12`, direct), 2 high (`postcss@8.4.31`, transitive via Next; `sharp@0.34.5`, optional transitive via Next). npm groups severity by affected package, while each package has multiple GHSA findings.
242-- Critical Next advisories: unauthenticated RCE on Windows-hosted servers (GHSA-p293-qw3h-jr36, `>=16.0.0 <16.3.3`); Image Optimization AVIF RCE (GHSA-2xp9-vwfh-vxw4, same range); `next/og` ImageResponse RCE (GHSA-vcvr-r3jv-pc5j, `>=16.2.0 <16.3.6`). Audit also reported Next SSRF (high; GHSA-cjq9-62q9-8jv4, `<16.3.8`) and lower-severity cache/information-disclosure findings.
243-- `postcss@8.4.31` had high source-map file-read/path-traversal advisories (GHSA-6g55-p6wh-862q, `<=8.5.11`; GHSA-r28c-9q8g-f849, `<=8.5.17`) and moderate findings through `8.5.22`. `sharp@0.34.5` had three high inherited libvips/libheif/librsvg findings (GHSA-f88m-g3jw-g9cj `<0.35.0`; GHSA-rgj7-g3m4-5g8c `<0.35.4`; GHSA-wq5f-xc86-pv6w `<0.35.5`).
244-- npm recommended non-major Next `16.4.2`. Updated `apps/web/package.json` and regenerated `package-lock.json` with `npm install --package-lock-only --ignore-scripts --save-exact next@16.4.2 --no-audit --no-fund`. Locked tree resolves `next@16.4.2`, `postcss@8.5.23`, `sharp@0.35.5`. Full and production-only audits now report 0 vulnerabilities; exact CI command `npm audit --audit-level=low` passed. App entry points contain no direct `next/image`, `ImageResponse`, or sourceMappingURL usage; this did not waive framework dependency risk.
--
425-- TC_PRIV_002: DEFERRED / NOT RUN under the approved platform-only scope.
426- Workspace membership, workspace resources, and resource-level isolation
427- remain outside this implementation.
428-- TC_SYS_001: NOT RUN / NOT ESTABLISHED by the API suite alone.
429-- Approved decision remains platform-level authorization only. Workspace roles,
430: membership, and workspace tables are deferred. FIN-14 is not changed.
431-- Before marking FIN-15 complete, verify the actual route path in
432- apps/api/app/main.py and its tests, align stale route references in the
433- tracking records, and close the remaining story-level acceptance gaps.
434-
435-FIN-15 ROUTE RECONCILIATION — 2026-10-20
--

475-- TC_SYS_001: NOT RUN for the full planned system acceptance workflow.
476-
477-Overall FIN-15 status: PARTIAL. Do not mark accepted or VERIFIED COMPLETE
478-until the remaining applicable acceptance criteria and evidence are
479-reconciled. Workspace roles, membership, and resource-level authorization
480:remain deferred. FIN-14 is outside this verification scope.
481-
482-No commit or push was performed as part of this verification.
=== FIN-14 test execution entries ===
93-TC_AUTHN_005 | Status: BLOCKED | Execution date: 2026-10-18 | Actual result: Public registration is absent, but admin onboarding workflow is incomplete | Tester/owner: Codex | Evidence: apps/api/tests/test_auth.py | Linked defect: —
94-TC_AUTHZ_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
95-TC_AUTHZ_002 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
96-TC_AUTHZ_003 | Status: PASS | Execution date: 2026-10-18 | Actual result: Stale session rejected under configured TTL | Tester/owner: Codex | Evidence: apps/api/tests/test_auth.py | Linked defect: —
97:TC_AUTHSEC_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
98:TC_AUTHSEC_002 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
99:TC_AUTHSEC_003 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
100-TC_AUTHSEC_004 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
101-TC_AUTHN_006 | Status: PASS | Execution date: 2026-10-18 | Actual result: Disposable synthetic database dump contained password/session digests; plaintext password/token absent; API test checked captured logs/stdout/stderr and response | Tester/owner: Codex | Evidence: apps/api/tests/test_auth.py; disposable scan summarized in IMPLEMENTATION_STATUS.txt (dump removed) | Linked defect: —
102-TC_AUTHN_007 | Status: BLOCKED | Execution date: 2026-10-18 | Actual result: Local cookie flags checked; deployed HTTPS verification unavailable | Tester/owner: Codex | Evidence: apps/api/tests/test_auth.py | Linked defect: —
103-TC_AUTHN_008 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
--
177- recorded earlier; it was not separately rerun in run 20261020_141030.
178-- TC_PRIV_002: DEFERRED / NOT RUN — workspace/resource authorization is outside
179- the approved platform-only scope.
180-- TC_SYS_001: NOT RUN — a passing API suite is not a browser/system smoke test.
181:Overall FIN-15 remains PARTIAL, not accepted. Do not alter FIN-14 tracking.
182-

183-FIN-15 ROUTE RECONCILIATION — 2026-10-20
184-The current route is GET /api/v1/admin/authorization/capabilities, verified
185-against apps/api/app/main.py and the matching test request paths. Older route
--
203-NOT RUN under the approved platform-only scope; TC_SYS_001 remains
204-NOT RUN for the full planned system acceptance workflow.
205-
206-Overall FIN-15 / FS-01-02 status: PARTIAL, not accepted or VERIFIED COMPLETE.
207:Workspace/resource authorization remains deferred. FIN-14 is unchanged.
=== Function/test traceability entries ===
63-Story disposition: FIN-15 / FS-01-02 remains PARTIAL. The capability endpoint
64-is a narrow API authorization test seam, not proof that every protected
65-operation, frontend guard, or system-level acceptance criterion is complete.
66:Workspace roles, membership, and workspace tables remain deferred. FIN-14 is
67-unchanged.
68-
69-ROUTE VERIFICATION — 2026-10-20
--
89-NOT RUN for the full planned system acceptance workflow.
90-
91-Overall FIN-15 / FS-01-02 status: PARTIAL, not accepted or VERIFIED COMPLETE.
92:Workspace/resource authorization remains deferred. FIN-14 is unchanged.
=== Relevant conversation decisions ===
64-Post-change repository review note: `git status --short --branch && git diff --check && git diff --stat` again produced no output within 10 seconds and remained stalled. Per theseparate Git recovery task instruction, no troubleshooting or Git mutation was attempted. Therefore this task's Git diff/status review is incomplete; publication remains separatelyblocked under Q-007.
65-
66-FIN-76 / FS-11-05 — Coverage and test quality (2026-10-18)
67:Status: PARTIAL. Enabled backend branch coverage in the existing CI test commands and retained XML and HTML reports as a 14-day GitHub artifact (workflow configured; remote upload not yet observed). Updated README commands. Added focused tests for missing session access, login/session database failures, required DATABASE_URL, logout without an active session, provisioning validation/hash/upsert behavior, and the provisioning module CLI entry point. CI-equivalent local suites passed: unit 37 tests; PostgreSQL integration 10 tests; combined 47 passed. Final cumulative backend coverage: line 156/156 (100%), branch 28/28 (100%). Evidence: ignored local `coverage.xml`, `htmlcov/index.html`, and `/tmp/finsight-coverage.json`. No overall threshold was invented. Frontend coverage is not measured: existing Playwright tests cover interactions but do not instrument source code. A minimal future configuration would instrument client source modules and connect coverage reporting to the existing interaction suite; decide whether that additional frontend metric is required for acceptance.
68-Verification detail: the first PostgreSQL run from the sandbox could not connect (`Operation not permitted`); using approved access to the existing disposable `finsight_fin75` test database,

both CI-equivalent coverage commands passed. PyYAML parsed the edited workflow. No Git command was run and no GitHub Actions execution occurred.
69-
70-FIN-77 advisory triage initial note (2026-10-18; superseded by the execution below): the prior install reported 3 dependency advisories (2 high and 1 critical), with package-level detail pending FIN-77.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
Status: OPEN — acceptance decision and implementation remain outstanding.
Date recorded: 2026-10-21.

Why it matters:
FIN-14 / FS-01-03 includes conditional MFA and active-session inspection/ termination acceptance criteria. The current implementation does not include an MFA challenge/provider flow or a capability to inspect and terminate other active sessions. Logging out the current session is not equivalent to managing all sessions.

Required decision:
Approve the MFA policy (when MFA is required, the challenge method/provider, and how isolated test configuration is managed). Do not enable a fake or placeholder MFA flow. Implement session listing and revocation for authorized users within the approved scope, then test that revoked sessions are rejected.

Until the policy and functionality are resolved, TC_AUTHSEC_003 remains BLOCKED and FIN-14 remains PARTIAL. No requirement or source schema document was changed to imply acceptance.
"""
conversation.write_text(conversation_text)

print("Updated four tracking files:")
for path in (implementation, execution, traceability, conversation):
    print(f" {path}")
print("Test statuses: TC_AUTHSEC_001 PARTIAL; TC_AUTHSEC_002 PARTIAL; TC_AUTHSEC_003 BLOCKED")
print("FIN-14 remains PARTIAL. No source requirements or test-plan documents were edited.")
PY
Updated four tracking files:
  IMPLEMENTATION_STATUS.txt
  SPRINT_1_TEST_EXECUTION_STATUS.txt
  FUNCTION_TEST_TRACEABILITY.txt
  conversation.txt
Test statuses: TC_AUTHSEC_001 PARTIAL; TC_AUTHSEC_002 PARTIAL; TC_AUTHSEC_003 BLOCKED
FIN-14 remains PARTIAL. No source requirements or test-plan documents were edited.

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "=== Confirm the three test statuses ==="
grep -n -E 'TC_AUTHSEC_00[123] \| Status:' \
  SPRINT_1_TEST_EXECUTION_STATUS.txt

echo "=== Confirm FIN-14 was recorded ==="
grep -n -E 'FIN-14 / FS-01-03|Q-011|AUTHSEC_00[123]' \
  IMPLEMENTATION_STATUS.txt \
  SPRINT_1_TEST_EXECUTION_STATUS.txt \
  FUNCTION_TEST_TRACEABILITY.txt \
  conversation.txt

echo "=== Whitespace check ==="
git diff --check
=== Confirm the three test statuses ===
97:TC_AUTHSEC_001 | Status: PARTIAL | Execution date: 2026-10-21 | Actual result: Successful login, failed login, and logout auditing plus secret-redaction tests passed; end-to-endcorrelation propagation and append-only enforcement remain unverified | Tester/owner: Codex | Evidence: apps/api/tests/test_fin14_auth_security.py; apps/api/tests/test_audit_service.py | Linked defect: FIN-14 acceptance gaps
98:TC_AUTHSEC_002 | Status: PARTIAL | Execution date: 2026-10-21 | Actual result: Configurable database-backed throttling, repeated-failure blocking, configuration validation, and throttle expiry behavior are tested; the complete login-level expiry-and-successful-retry flow is not established | Tester/owner: Codex | Evidence: apps/api/tests/test_throttle.py; apps/api/tests/test_fin14_auth_security.py; PostgreSQL throttle verification recorded in IMPLEMENTATION_STATUS.txt | Linked defect: FIN-14 acceptance gap
99:TC_AUTHSEC_003 | Status: BLOCKED | Execution date: 2026-10-21 | Actual result: MFA challenge enforcement and inspection/termination of other active sessions are not implemented;current-session logout is not equivalent | Tester/owner: Codex | Evidence: apps/api/app/main.py; apps/api/tests/test_auth.py | Linked defect: MFA policy/provider decision and cross-session management
=== Confirm FIN-14 was recorded ===
IMPLEMENTATION_STATUS.txt:239:Scope references: detailed backlog FS-11-06 (P0, 3 SP) and four-sprint plan (3 SP). Sprint 1 management plan maps this story to TC_ENV_002, TC_SEC_001, TC_SEC_002, TC_AUTHSEC_002, TC_AUTHN_006, TC_CFG_001, and TC_EXEC_002. Original references remain unchanged.
IMPLEMENTATION_STATUS.txt:322:Schema blocker: migration 0001 has only nullable `users.platform_role_id`; it has no permission/role_permissions/workspace/workspace_members persistence. Canonical schema seeds `platform_admin` as platform scope and `operator`/`standard_user` as workspace scope. Converting workspace roles into global roles or silently creating a default workspace would change authorization/data semantics. See Q-011 in `conversation.txt`; user decision requested:
```

platform roles only and defer workspace roles, or authorize minimal canonical workspace membership model.
IMPLEMENTATION_STATUS.txt:485:FIN-14 / FS-01-03 — AUTHENTICATION SECURITY IMPLEMENTATION (2026-10-21)
IMPLEMENTATION_STATUS.txt:513:- TC_AUTHSEC_001 remains PARTIAL: end-to-end correlation propagation and
IMPLEMENTATION_STATUS.txt:515:- TC_AUTHSEC_002 remains PARTIAL: the complete login-level lockout-expiry
IMPLEMENTATION_STATUS.txt:517:- TC_AUTHSEC_003 remains BLOCKED: MFA policy/provider and challenge flow are
SPRINT_1_TEST_EXECUTION_STATUS.txt:97:TC_AUTHSEC_001 | Status: PARTIAL | Execution date: 2026-10-21 | Actual result: Successful login, failed login, and logout auditing plus secret-redaction tests passed; end-to-end correlation propagation and append-only enforcement remain unverified | Tester/owner: Codex | Evidence: apps/api/tests/test_fin14_auth_security.py; apps/api/tests/test_audit_service.py | Linked defect: FIN-14 acceptance gaps
SPRINT_1_TEST_EXECUTION_STATUS.txt:98:TC_AUTHSEC_002 | Status: PARTIAL | Execution date: 2026-10-21 | Actual result: Configurable database-backed throttling, repeated-failure blocking, configuration validation, and throttle expiry behavior are tested; the complete login-level expiry-and-successful-retry flow is not established | Tester/owner: Codex | Evidence: apps/api/tests/test_throttle.py; apps/api/tests/test_fin14_auth_security.py; PostgreSQL throttle verification recorded in IMPLEMENTATION_STATUS.txt | Linked defect: FIN-14 acceptance gap
SPRINT_1_TEST_EXECUTION_STATUS.txt:99:TC_AUTHSEC_003 | Status: BLOCKED | Execution date: 2026-10-21 | Actual result: MFA challenge enforcement and inspection/termination of other active sessions are not implemented; current-session logout is not equivalent | Tester/owner: Codex | Evidence: apps/api/app/main.py; apps/api/tests/test_auth.py | Linked defect: MFApolicy/provider decision and cross-session management
SPRINT_1_TEST_EXECUTION_STATUS.txt:210:FIN-14 / FS-01-03 — AUTHENTICATION SECURITY EXECUTION ADDENDUM
SPRINT_1_TEST_EXECUTION_STATUS.txt:223:- TC_AUTHSEC_001: PARTIAL. Login success/failure and logout audit events,
SPRINT_1_TEST_EXECUTION_STATUS.txt:226:- TC_AUTHSEC_002: PARTIAL. Configurable PostgreSQL-backed throttling and
SPRINT_1_TEST_EXECUTION_STATUS.txt:230:- TC_AUTHSEC_003: BLOCKED. MFA enforcement and listing/terminating other
SPRINT_1_TEST_EXECUTION_STATUS.txt:234:Overall FIN-14 / FS-01-03 status: PARTIAL, not accepted or VERIFIED COMPLETE.
FUNCTION_TEST_TRACEABILITY.txt:97:Story: FIN-14 / FS-01-03 — Protect Accounts and Audit Authentication.
FUNCTION_TEST_TRACEABILITY.txt:99:- TC_AUTHSEC_001 — PARTIAL.
FUNCTION_TEST_TRACEABILITY.txt:105:- TC_AUTHSEC_002 — PARTIAL.
FUNCTION_TEST_TRACEABILITY.txt:111:- TC_AUTHSEC_003 — BLOCKED.

conversation.txt:4:Q-011 — FIN-15 / FS-01-02 role assignment scope — WAITING FOR USER
conversation.txt:130:Q-011 — FIN-14 MFA policy/provider and active-session management
conversation.txt:135:FIN-14 / FS-01-03 includes conditional MFA and active-session inspection/
conversation.txt:147:Until the policy and functionality are resolved, TC_AUTHSEC_003 remains
=== Whitespace check ===
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
=== Confirm FIN-14 was recorded ===
IMPLEMENTATION_STATUS.txt:239:Scope references: detailed backlog FS-11-06 (P0, 3 SP) and four-sprint plan (3 SP). Sprint 1 management plan maps this story to TC_ENV_002, TC_SEC_001, TC_SEC_002, TC_AUTHSEC_002, TC_AUTHN_006, TC_CFG_001, and TC_EXEC_002. Original references remain unchanged.
IMPLEMENTATION_STATUS.txt:322:Schema blocker: migration 0001 has only nullable `users.platform_role_id`; it has no permission/role_permissions/workspace/workspace_members persistence. Canonical schema seeds `platform_admin` as platform scope and `operator`/`standard_user` as workspace scope. Converting workspace roles into global roles or silently creating a default workspace would change authorization/data semantics. See Q-011 in `conversation.txt`; user decision requested: platform roles only and defer workspace roles, or authorize minimal canonical workspace membership model.
IMPLEMENTATION_STATUS.txt:485:FIN-14 / FS-01-03 — AUTHENTICATION SECURITY IMPLEMENTATION (2026-10-21)
IMPLEMENTATION_STATUS.txt:513:- TC_AUTHSEC_001 remains PARTIAL: end-to-end correlation propagation and
IMPLEMENTATION_STATUS.txt:515:- TC_AUTHSEC_002 remains PARTIAL: the complete login-level lockout-expiry
IMPLEMENTATION_STATUS.txt:517:- TC_AUTHSEC_003 remains BLOCKED: MFA policy/provider and challenge flow are
SPRINT_1_TEST_EXECUTION_STATUS.txt:97:TC_AUTHSEC_001 | Status: PARTIAL | Execution date: 2026-10-21 | Actual result: Successful login, failed login, and logout auditing plus secret-redaction tests passed; end-to-end correlation propagation and append-only enforcement remain unverified | Tester/owner: Codex | Evidence: apps/api/tests/test_fin14_auth_security.py; apps/api/tests/test_audit_service.py | Linked defect: FIN-14 acceptance gaps
SPRINT_1_TEST_EXECUTION_STATUS.txt:98:TC_AUTHSEC_002 | Status: PARTIAL | Execution date: 2026-10-21 | Actual result: Configurable database-backed throttling, repeated-failure blocking, configuration validation, and throttle expiry behavior are tested; the complete login-level expiry-and-successful-retry flow is not established | Tester/owner: Codex | Evidence: apps/api/tests/test_throttle.py; apps/api/tests/test_fin14_auth_security.py; PostgreSQL throttle verification recorded in IMPLEMENTATION_STATUS.txt | Linked defect: FIN-14 acceptance gap
SPRINT_1_TEST_EXECUTION_STATUS.txt:99:TC_AUTHSEC_003 | Status: BLOCKED | Execution date: 2026-10-21 | Actual result: MFA challenge enforcement and inspection/termination of other active sessions are not implemented; current-session logout is not equivalent | Tester/owner: Codex | Evidence: apps/api/app/main.py;

```
apps/api/tests/test_auth.py | Linked defect: MFApolicy/provider decision and cross-session
management
SPRINT_1_TEST_EXECUTION_STATUS.txt:210:FIN-14 / FS-01-03 — AUTHENTICATION
SECURITY EXECUTION ADDENDUM
SPRINT_1_TEST_EXECUTION_STATUS.txt:223:- TC_AUTHSEC_001: PARTIAL. Login
success/failure and logout audit events,
SPRINT_1_TEST_EXECUTION_STATUS.txt:226:- TC_AUTHSEC_002: PARTIAL. Configurable
PostgreSQL-backed throttling and
SPRINT_1_TEST_EXECUTION_STATUS.txt:230:- TC_AUTHSEC_003: BLOCKED. MFA
enforcement and listing/terminating other
SPRINT_1_TEST_EXECUTION_STATUS.txt:234:Overall FIN-14 / FS-01-03 status: PARTIAL,
not accepted or VERIFIED COMPLETE.
FUNCTION_TEST_TRACEABILITY.txt:97:Story: FIN-14 / FS-01-03 — Protect Accounts and
Audit Authentication.
FUNCTION_TEST_TRACEABILITY.txt:99:- TC_AUTHSEC_001 — PARTIAL.
FUNCTION_TEST_TRACEABILITY.txt:105:- TC_AUTHSEC_002 — PARTIAL.
FUNCTION_TEST_TRACEABILITY.txt:111:- TC_AUTHSEC_003 — BLOCKED.
conversation.txt:4:Q-011 — FIN-15 / FS-01-02 role assignment scope — WAITING FOR USER
conversation.txt:130:Q-011 — FIN-14 MFA policy/provider and active-session management
conversation.txt:135:FIN-14 / FS-01-03 includes conditional MFA and active-session inspection/
conversation.txt:147:Until the policy and functionality are resolved, TC_AUTHSEC_003 remains
=== Whitespace check ===
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
    "end-to-endcorrelation propagation": "end-to-end correlation propagation",
    "not implemented;current-session logout": "not implemented; current-session logout",
    "MFApolicy/provider decision": "MFA policy/provider decision",
}
for old, new in fixes.items():
    s = s.replace(old, new)
p.write_text(s)

print("Corrected the FIN-14 decision ID and test-index spacing.")
PY

echo "=== Verify decision IDs ==="
grep -n -E '^Q-011|^Q-012' conversation.txt

echo "=== Locate authentication and audit implementation ==="
grep -n -E 'def login|def logout|record_auth_event|correlation_id|auth_login_throttle|CREATE
TABLE.*audit_events|CREATE TABLE.*auth_login_throttle' \
  apps/api/app/main.py \
  apps/api/app/audit.py \
  apps/api/app/throttle.py \
```

```
  apps/api/migrations/0003_auth_audit_and_login_throttling.sql

echo "=== Audit service ==="
sed -n '1,240p' apps/api/app/audit.py

echo "=== Throttle service ==="
sed -n '1,280p' apps/api/app/throttle.py

echo "=== Migration 0003 ==="
sed -n '1,260p' apps/api/migrations/0003_auth_audit_and_login_throttling.sql

echo "=== Login and logout route locations ==="
grep -n -B 8 -A 100 'def login\|def logout' apps/api/app/main.py

echo "=== Working-tree whitespace validation ==="
git diff --check

Corrected the FIN-14 decision ID and test-index spacing.
=== Verify decision IDs ===
4:Q-011 — FIN-15 / FS-01-02 role assignment scope — WAITING FOR USER
130:Q-012 — FIN-14 MFA policy/provider and active-session management
=== Locate authentication and audit implementation ===
apps/api/app/main.py:14:from app.audit import record_auth_event
apps/api/app/main.py:131:def login(payload: LoginRequest, request: Request, response:
Response) -> UserView:
apps/api/app/main.py:145:    correlation_id = uuid4()
apps/api/app/main.py:159:              record_auth_event(
apps/api/app/main.py:165:                  correlation_id=correlation_id,
apps/api/app/main.py:204:                  record_auth_event(
apps/api/app/main.py:210:                      correlation_id=correlation_id,
apps/api/app/main.py:238:                  record_auth_event(
apps/api/app/main.py:244:                      correlation_id=correlation_id,
apps/api/app/main.py:273:def logout(
apps/api/app/main.py:279:    correlation_id = uuid4()
apps/api/app/main.py:299:          record_auth_event(
apps/api/app/main.py:305:              correlation_id=correlation_id,
apps/api/app/main.py:345:    correlation_id = supplied_request_id or request_id
apps/api/app/main.py:364:        "correlation_id": correlation_id,
apps/api/app/audit.py:38:def record_auth_event(
apps/api/app/audit.py:45:    correlation_id: UUID | None = None,
apps/api/app/audit.py:72:              correlation_id,
apps/api/app/audit.py:82:              correlation_id,
apps/api/app/throttle.py:72:           FROM auth_login_throttle
apps/api/app/throttle.py:101:          INSERT INTO auth_login_throttle (
```

apps/api/app/throttle.py:108:   WHEN auth_login_throttle.window_started_at
apps/api/app/throttle.py:111:   ELSE auth_login_throttle.failed_attempts + 1
apps/api/app/throttle.py:114:   WHEN auth_login_throttle.window_started_at
apps/api/app/throttle.py:117:   ELSE auth_login_throttle.window_started_at
apps/api/app/throttle.py:121:   WHEN auth_login_throttle.window_started_at
apps/api/app/throttle.py:124:   ELSE auth_login_throttle.failed_attempts + 1
apps/api/app/throttle.py:127:   WHEN auth_login_throttle.blocked_until > %s
apps/api/app/throttle.py:128:   THEN auth_login_throttle.blocked_until
apps/api/app/throttle.py:154:   "DELETE FROM auth_login_throttle WHERE scope_type =
%s AND key_hash = %s",
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:6:CREATE TABLE audit_events (
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:15:   correlation_id UUID NULL,
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:32:CREATE TABLE
auth_login_throttle (
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:44:CREATE INDEX
idx_auth_login_throttle_blocked_until
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:45:   ON auth_login_throttle
(blocked_until)
=== Audit service ===
"""Reusable, redacted audit persistence for authentication security events."""

from __future__ import annotations

import json
import re
from datetime import UTC, datetime
from typing import Any
from uuid import UUID

_ALLOWED_OUTCOMES = frozenset({"success", "failure", "denied"})
_SECRET_KEY = re.compile(
    r"(password|passwd|token|cookie|authorization|api[_-]?key|secret|credential)",
    re.IGNORECASE,
)

def _redact(value: Any) -> Any:
    """Return JSON-safe audit details without values under secret-like keys."""
    if isinstance(value, dict):
        return {
            str(key): (
                "[REDACTED]" if _SECRET_KEY.search(str(key)) else _redact(item)
)
            for key, item in value.items()

```
        }
    if isinstance(value, (list, tuple)):
        return [_redact(item) for item in value]
    if isinstance(value, UUID):
        return str(value)
    if isinstance(value, datetime):
        return value.astimezone(UTC).isoformat()
    if value is None or isinstance(value, (str, int, float, bool)):
        return value
    return str(value)


def record_auth_event(
    conn: Any,
    *,
    action_key: str,
    outcome: str,
    actor_user_id: UUID | None,
    details: dict[str, Any] | None = None,
    correlation_id: UUID | None = None,
    occurred_at: datetime | None = None,
) -> None:
    """Persist one authentication audit event on the caller's transaction.

    Persistence errors intentionally propagate to the caller. The caller must
    roll back and apply the documented fail-closed policy; events are never
    reported as recorded when this insert fails.
    """
    if outcome not in _ALLOWED_OUTCOMES:
        raise ValueError("Unsupported audit outcome")
    if not action_key or len(action_key) > 120:
        raise ValueError("action_key must contain 1 to 120 characters")

    redacted_details = _redact(details or {})
    payload = json.dumps(redacted_details, separators=(",", ":"), sort_keys=True)

    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO audit_events (
                workspace_id,
                actor_user_id,
                action_key,
                resource_type,
```

```
            resource_id,
            outcome,
            correlation_id,
            details_redacted,
            occurred_at
        )
        VALUES (NULL, %s, %s, 'authentication', NULL, %s, %s, %s::jsonb, %s)
        """,
        (
            actor_user_id,
            action_key,
            outcome,
            correlation_id,
            payload,
            occurred_at or datetime.now(UTC),
        ),
    )
=== Throttle service ===
"""PostgreSQL-backed login throttling with secret-keyed identifiers."""

from __future__ import annotations

import hashlib
import hmac
import os
from dataclasses import dataclass
from datetime import UTC, datetime
from typing import Any


@dataclass(frozen=True)
class ThrottleConfig:
    secret: bytes
    max_failures: int
    window_seconds: int
    block_seconds: int


def load_throttle_config() -> ThrottleConfig:
    """Load validated throttle settings; never use a weak fallback secret."""
    secret = os.environ.get("AUTH_THROTTLE_SECRET", "")
    if len(secret.encode("utf-8")) < 32:
        raise ValueError("AUTH_THROTTLE_SECRET must contain at least 32 bytes")
```

```
    try:
        max_failures = int(os.environ.get("AUTH_LOGIN_MAX_FAILURES", "5"))
        window_seconds = int(os.environ.get("AUTH_LOGIN_WINDOW_SECONDS", "900"))
        block_seconds = int(os.environ.get("AUTH_LOGIN_BLOCK_SECONDS", "900"))
    except ValueError as exc:
        raise ValueError("Login throttle configuration is invalid") from exc

    if not 1 <= max_failures <= 100:
        raise ValueError("AUTH_LOGIN_MAX_FAILURES must be between 1 and 100")
    if not 1 <= window_seconds <= 86400:
        raise ValueError("AUTH_LOGIN_WINDOW_SECONDS must be between 1 and 86400")
    if not 1 <= block_seconds <= 86400:
        raise ValueError("AUTH_LOGIN_BLOCK_SECONDS must be between 1 and 86400")

    return ThrottleConfig(
        secret=secret.encode("utf-8"),
        max_failures=max_failures,
        window_seconds=window_seconds,
        block_seconds=block_seconds,
    )


def make_throttle_key(secret: bytes, scope_type: str, value: str) -> str:
    """Return a stable HMAC-SHA256 identifier without storing the raw value."""
    if scope_type not in {"account", "ip"}:
        raise ValueError("Unsupported throttle scope")
    if not value:
        raise ValueError("Throttle key value cannot be empty")

    message = f"finsight-login:{scope_type}:{value}".encode("utf-8")
    return hmac.new(secret, message, hashlib.sha256).hexdigest()


def is_blocked(
    conn: Any,
    *,
    scope_type: str,
    key_hash: str,
    now: datetime | None = None,
) -> bool:
    current_time = now or datetime.now(UTC)
    with conn.cursor() as cur:
        cur.execute(
            """
```

```
            SELECT blocked_until > %s
            FROM auth_login_throttle
            WHERE scope_type = %s AND key_hash = %s
            """,
            (current_time, scope_type, key_hash),
        )
        row = cur.fetchone()
    # Accept the timestamp result used by the database query, and Boolean
    # results from adapters that evaluate the comparison in SQL.
    if not row or row[0] is None:
        return False
    value = row[0]
    if isinstance(value, bool):
        return value
    return value > current_time


def record_failure(
    conn: Any,
    *,
    scope_type: str,
    key_hash: str,
    config: ThrottleConfig,
    now: datetime | None = None,
) -> datetime | None:
    """Atomically increment a bucket and return its active block expiry, if any."""
    current_time = now or datetime.now(UTC)
    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO auth_login_throttle (
                scope_type, key_hash, failed_attempts,
                window_started_at, blocked_until, updated_at
            )
            VALUES (%s, %s, 1, %s, NULL, %s)
            ON CONFLICT (scope_type, key_hash) DO UPDATE SET
                failed_attempts = CASE
                    WHEN auth_login_throttle.window_started_at
                        <= %s - make_interval(secs => %s)
                    THEN 1
                    ELSE auth_login_throttle.failed_attempts + 1
                END,
                window_started_at = CASE
                    WHEN auth_login_throttle.window_started_at
```

```
                    <= %s - make_interval(secs => %s)
                THEN %s
                ELSE auth_login_throttle.window_started_at
            END,
            blocked_until = CASE
                WHEN CASE
                    WHEN auth_login_throttle.window_started_at
                        <= %s - make_interval(secs => %s)
                    THEN 1
                    ELSE auth_login_throttle.failed_attempts + 1
                END >= %s
                THEN %s + make_interval(secs => %s)
                WHEN auth_login_throttle.blocked_until > %s
                THEN auth_login_throttle.blocked_until
                ELSE NULL
            END,
            updated_at = %s
        RETURNING blocked_until
        """,
        (
            scope_type, key_hash, current_time, current_time,
            current_time, config.window_seconds,
            current_time, config.window_seconds, current_time,
            current_time, config.window_seconds, config.max_failures,
            current_time, config.block_seconds, current_time,
            current_time,
        ),
    )
    row = cur.fetchone()

if row and row[0] and row[0] > current_time:
    return row[0]
return None


def reset_bucket(conn: Any, *, scope_type: str, key_hash: str) -> None:
    """Clear a successful login's corresponding failure counter."""
    with conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = %s AND key_hash = %s",
            (scope_type, key_hash),
        )
=== Migration 0003 ===
BEGIN;
```

```
-- Authentication/security audit trail.
-- workspace_id remains nullable because the current application schema
-- does not yet contain the workspaces table.
CREATE TABLE audit_events (
    audit_event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NULL,
    actor_user_id UUID NULL REFERENCES users(user_id) ON DELETE SET NULL,
    action_key VARCHAR(120) NOT NULL,
    resource_type VARCHAR(80) NOT NULL,
    resource_id UUID NULL,
    outcome VARCHAR(20) NOT NULL
        CHECK (outcome IN ('success', 'failure', 'denied')),
    correlation_id UUID NULL,
    details_redacted JSONB NOT NULL DEFAULT '{}'::jsonb,
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_events_occurred_at
    ON audit_events (occurred_at DESC);

CREATE INDEX idx_audit_events_actor_occurred
    ON audit_events (actor_user_id, occurred_at DESC);

CREATE INDEX idx_audit_events_action_occurred
    ON audit_events (action_key, occurred_at DESC);

-- Persistent, shared login-failure counters.
-- key_hash must be a keyed digest generated by the application;
-- raw email addresses, IP addresses, passwords and tokens do not belong here.
CREATE TABLE auth_login_throttle (
    scope_type VARCHAR(16) NOT NULL
        CHECK (scope_type IN ('account', 'ip')),
    key_hash CHAR(64) NOT NULL,
    failed_attempts INTEGER NOT NULL DEFAULT 0
        CHECK (failed_attempts >= 0),
    window_started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    blocked_until TIMESTAMPTZ NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (scope_type, key_hash)
);

CREATE INDEX idx_auth_login_throttle_blocked_until
    ON auth_login_throttle (blocked_until)
```

```
   WHERE blocked_until IS NOT NULL;

COMMIT;
=== Login and logout route locations ===
123-        if permission_key not in user.get("permissions", []):
124-            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Permission denied")
125-        return user
126-
127-    return dependency
128-
129-
130-@app.post("/auth/login", response_model=UserView)
131:def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
132-    _check_origin(request)
133-    email = payload.email.strip().lower()
134-    try:
135-        config = load_throttle_config()
136-    except ValueError as exc:
137-        raise HTTPException(
138-            status_code=503,
139-            detail="Authentication service unavailable",
140-        ) from exc
141-
142-    client_ip = request.client.host if request.client else "unknown"
143-    account_key = make_throttle_key(config.secret, "account", email)
144-    ip_key = make_throttle_key(config.secret, "ip", client_ip)
145-    correlation_id = uuid4()
146-    login_succeeded = False
147-    user = None
148-
149-    try:
150-        with connect() as conn:
151-            account_blocked = is_blocked(
152-                conn, scope_type="account", key_hash=account_key
153-            )
154-            ip_blocked = is_blocked(
155-                conn, scope_type="ip", key_hash=ip_key
156-            )
157-
158-            if account_blocked or ip_blocked:
159-                record_auth_event(
160-                    conn,
161-                    action_key="auth.login",
```

```
                outcome="denied",
                actor_user_id=None,
                details={"reason": "throttled"},
                correlation_id=correlation_id,
            )
        else:
            with conn.cursor(row_factory=dict_row) as cur:
                cur.execute(
                    """SELECT user_id, email, display_name, password_hash
                       FROM users
                       WHERE lower(email) = %s
                         AND account_status = 'active'
                         AND deleted_at IS NULL""",
                    (email,),
                )
                user = cur.fetchone()

            password_hash = (
                user["password_hash"]
                if user and user["password_hash"]
                else DUMMY_PASSWORD_HASH
            )
            password_valid = verify_password(
                password_hash, payload.password
            )
            login_succeeded = bool(
                user and user["password_hash"] and password_valid
            )

            if not login_succeeded:
                record_failure(
                    conn,
                    scope_type="account",
                    key_hash=account_key,
                    config=config,
                )
                record_failure(
                    conn,
                    scope_type="ip",
                    key_hash=ip_key,
                    config=config,
                )
                record_auth_event(
                    conn,
```

```
206-                    action_key="auth.login",
207-                    outcome="failure",
208-                    actor_user_id=user["user_id"] if user else None,
209-                    details={"reason": "invalid_credentials"},
210-                    correlation_id=correlation_id,
211-                )
212-            else:
213-                # Clear account failures only. Preserve IP-wide history.
214-                reset_bucket(
215-                    conn,
216-                    scope_type="account",
217-                    key_hash=account_key,
218-                )
219-
220-                token, digest = new_session_token()
221-                expires_at = session_expiry(_ttl_minutes())
222-
223-                with conn.cursor() as cur:
224-                    cur.execute(
225-                        """INSERT INTO auth_sessions
226-                               (user_id, refresh_token_hash, expires_at)
227-                           VALUES (%s, %s, %s)""",
228-                        (user["user_id"], digest, expires_at),
229-                    )
230-                    now = datetime.now(UTC)
231-                    cur.execute(
--
265-    return UserView(
266-        user_id=user["user_id"],
267-        email=user["email"],
268-        display_name=user["display_name"],
269-    )
270-
271-
272-@app.post("/auth/logout", status_code=204)
273:def logout(
274-    request: Request,
275-    response: Response,
276-    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
277-) -> Response:
278-    _check_origin(request)
279-    correlation_id = uuid4()
280-
281-    try:
```

```
282-        with connect() as conn:
283-            actor_user_id = None
284-
285-            if session_token:
286-                with conn.cursor() as cur:
287-                    cur.execute(
288-                        """UPDATE auth_sessions
289-                           SET revoked_at = %s, revoke_reason = 'logout'
290-                           WHERE refresh_token_hash = %s
291-                             AND revoked_at IS NULL
292-                           RETURNING user_id""",
293-                        (datetime.now(UTC), token_digest(session_token)),
294-                    )
295-                    row = cur.fetchone()
296-                    if row:
297-                        actor_user_id = row[0]
298-
299-            record_auth_event(
300-                conn,
301-                action_key="auth.logout",
302-                outcome="success" if actor_user_id else "denied",
303-                actor_user_id=actor_user_id,
304-                details={"reason": "logout"},
305-                correlation_id=correlation_id,
306-            )
307-    except psycopg.Error as exc:
308-        raise HTTPException(
309-            status_code=503,
310-            detail="Authentication service unavailable",
311-        ) from exc
312-
313-    response.delete_cookie(
314-        SESSION_COOKIE,
315-        path="/",
316-        httponly=True,
317-        secure=_secure_cookie(),
318-        samesite="lax",
319-    )
320-    response.status_code = status.HTTP_204_NO_CONTENT
321-    return response
322-
323-
324-@app.get("/auth/me", response_model=UserView)
325-def who_am_i(user: CurrentUser) -> UserView:
```

```
326-    return UserView(**user)
327-
328-
329-@app.get("/api/v1/admin/authorization/capabilities")
330-def platform_admin_capabilities(
331-    request: Request,
332-    _authorized: Annotated[dict[str, Any], Depends(require_permission("platform.admin"))],
333-) -> dict[str, Any]:
334-    """Return the authenticated caller's database-backed platform capabilities."""
335-
336-    supplied_request_id = request.headers.get("request_id", "").strip()
337-    if supplied_request_id and (
338-        len(supplied_request_id) > 128
339-        or any(ord(char) < 32 or ord(char) == 127 for char in supplied_request_id)
340-    ):
341-        raise HTTPException(status_code=400, detail="Invalid request_id")
342-
343-    # Always issue a server-generated canonical request ID.
344-    request_id = f"req_{secrets.token_urlsafe(12)}"
345-    correlation_id = supplied_request_id or request_id
346-    permissions = sorted(set(_authorized.get("permissions", [])))
347-    capabilities = (
348-        ["inspect_authorization_policy"]
349-        if "platform.admin" in permissions
350-        else []
351-    )
352-
353-    return {
354-        "data": {
355-            "subject": {
356-                "user_id": str(_authorized["user_id"]),
357-                "platform_role_code": _authorized.get("role_key"),
358-            },
359-            "effective_permissions": permissions,
360-            "capabilities": capabilities,
361-            "policy_version": "sprint-1-platform-rbac-v1",
362-        },
363-        "request_id": request_id,
364-        "correlation_id": correlation_id,
365-    }
=== Working-tree whitespace validation ===
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
340-    ):
341-        raise HTTPException(status_code=400, detail="Invalid request_id")
```

```
342-
343-    # Always issue a server-generated canonical request ID.
344-    request_id = f"req_{secrets.token_urlsafe(12)}"
345-    correlation_id = supplied_request_id or request_id
346-    permissions = sorted(set(_authorized.get("permissions", [])))
347-    capabilities = (
348-        ["inspect_authorization_policy"]
349-        if "platform.admin" in permissions
350-        else []
351-    )
352-
353-    return {
354-        "data": {
355-            "subject": {
356-                "user_id": str(_authorized["user_id"]),
357-                "platform_role_code": _authorized.get("role_key"),
358-            },
359-            "effective_permissions": permissions,
360-            "capabilities": capabilities,
361-            "policy_version": "sprint-1-platform-rbac-v1",
362-        },
363-        "request_id": request_id,
364-        "correlation_id": correlation_id,
365-    }
=== Working-tree whitespace validation ===
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

echo "=== Main imports and helpers ==="
sed -n '1,130p' apps/api/app/main.py

echo "=== Complete login route ==="
sed -n '130,270p' apps/api/app/main.py

echo "=== Audit integration tests ==="
sed -n '1,240p' apps/api/tests/test_fin14_auth_security.py

echo "=== Audit service tests ==="
sed -n '1,240p' apps/api/tests/test_audit_service.py

echo "=== Migration and release metadata references ==="
grep -n -E '000[123]_.*\.sql|database_migrations|audit_events' \
  scripts/package_release.py \
  docs/release/RELEASE_README.md \
```

```
  apps/api/migrations/*.sql
=== Main imports and helpers ===
"""FinSight AI API with PostgreSQL-backed browser sessions."""

from __future__ import annotations

import os
import secrets
from datetime import UTC, datetime
from pathlib import Path
from typing import Annotated, Any
from uuid import UUID

from uuid import UUID, uuid4

from app.audit import record_auth_event
from app.throttle import (
    is_blocked,
    load_throttle_config,
    make_throttle_key,
    record_failure,
    reset_bucket,
)

import psycopg
from dotenv import load_dotenv
from fastapi import Cookie, Depends, FastAPI, HTTPException, Request, Response, status
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, EmailStr, Field
from psycopg.rows import dict_row

from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
from app.database import connect

load_dotenv(Path(__file__).resolve().parents[3] / ".env")

app = FastAPI(title="FinSight AI API", version="0.2.0")
web_origin = os.environ.get("WEB_ORIGIN", "http://localhost:3000")
app.add_middleware(
    CORSMiddleware,
    allow_origins=[web_origin],
    allow_credentials=True,
    allow_methods=["GET", "POST"],
```

```
    allow_headers=["Content-Type"],
)

SESSION_COOKIE = "finsight_session"
GENERIC_LOGIN_ERROR = "Email or password is incorrect."
DUMMY_PASSWORD_HASH = hash_password(secrets.token_urlsafe(32))


class LoginRequest(BaseModel):
    email: EmailStr
    password: str = Field(min_length=1, max_length=1024)


class UserView(BaseModel):
    user_id: UUID
    email: str
    display_name: str


def _ttl_minutes() -> int:
    try:
        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
        session_expiry(value)
        return value
    except ValueError as exc:
        raise HTTPException(status_code=500, detail="Session configuration is invalid") from exc


def _secure_cookie() -> bool:
    configured = os.environ.get("COOKIE_SECURE")
    if configured is not None:
        return configured.lower() in {"1", "true", "yes"}
    return os.environ.get("APP_ENV", "development").lower() == "production"


def _check_origin(request: Request) -> None:
    origin = request.headers.get("origin")
    if origin and origin != web_origin:
        raise HTTPException(status_code=403, detail="Request origin is not allowed")


def get_current_user(
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> dict[str, Any]:
```

```
    if not session_token:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Authentication required")
    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT u.user_id, u.email, u.display_name, r.role_key,
                        COALESCE(array_agg(DISTINCT p.permission_key)
                            FILTER (WHERE p.permission_key IS NOT NULL), ARRAY[]::varchar[])
AS permissions
                    FROM auth_sessions s JOIN users u ON u.user_id = s.user_id
                    LEFT JOIN roles r ON r.role_id = u.platform_role_id AND r.scope = 'platform'
                    LEFT JOIN role_permissions rp ON rp.role_id = r.role_id
                    LEFT JOIN permissions p ON p.permission_id = rp.permission_id
                    WHERE s.refresh_token_hash = %s AND s.revoked_at IS NULL
                      AND s.expires_at > %s AND u.account_status = 'active'
                      AND s.last_seen_at > %s - make_interval(mins => %s)
                      AND u.deleted_at IS NULL
                    GROUP BY u.user_id, u.email, u.display_name, r.role_key""",
                (token_digest(session_token), datetime.now(UTC), datetime.now(UTC),
_ttl_minutes()),
            )
            user = cur.fetchone()
            if not user:
                raise HTTPException(status_code=401, detail="Authentication required")
            cur.execute(
                "UPDATE auth_sessions SET last_seen_at = %s WHERE refresh_token_hash = %s",
                (datetime.now(UTC), token_digest(session_token)),
            )
            return user
    except psycopg.Error as exc:
        raise HTTPException(status_code=503, detail="Authentication service unavailable") from
exc


CurrentUser = Annotated[dict[str, Any], Depends(get_current_user)]


def require_permission(permission_key: str):
    """Build a deny-by-default backend permission dependency."""
    def dependency(user: CurrentUser) -> dict[str, Any]:
        if permission_key not in user.get("permissions", []):
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Permission
denied")
```

```
        return user

    return dependency


@app.post("/auth/login", response_model=UserView)
=== Complete login route ===
@app.post("/auth/login", response_model=UserView)
def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
    _check_origin(request)
    email = payload.email.strip().lower()
    try:
        config = load_throttle_config()
    except ValueError as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    client_ip = request.client.host if request.client else "unknown"
    account_key = make_throttle_key(config.secret, "account", email)
    ip_key = make_throttle_key(config.secret, "ip", client_ip)
    correlation_id = uuid4()
    login_succeeded = False
    user = None

    try:
        with connect() as conn:
            account_blocked = is_blocked(
                conn, scope_type="account", key_hash=account_key
            )
            ip_blocked = is_blocked(
                conn, scope_type="ip", key_hash=ip_key
            )

            if account_blocked or ip_blocked:
                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="denied",
                    actor_user_id=None,
                    details={"reason": "throttled"},
                    correlation_id=correlation_id,
                )
```

```
else:
    with conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """SELECT user_id, email, display_name, password_hash
               FROM users
               WHERE lower(email) = %s
                 AND account_status = 'active'
                 AND deleted_at IS NULL""",
            (email,),
        )
        user = cur.fetchone()

    password_hash = (
        user["password_hash"]
        if user and user["password_hash"]
        else DUMMY_PASSWORD_HASH
    )
    password_valid = verify_password(
        password_hash, payload.password
    )
    login_succeeded = bool(
        user and user["password_hash"] and password_valid
    )

    if not login_succeeded:
        record_failure(
            conn,
            scope_type="account",
            key_hash=account_key,
            config=config,
        )
        record_failure(
            conn,
            scope_type="ip",
            key_hash=ip_key,
            config=config,
        )
        record_auth_event(
            conn,
            action_key="auth.login",
            outcome="failure",
            actor_user_id=user["user_id"] if user else None,
            details={"reason": "invalid_credentials"},
            correlation_id=correlation_id,
```

```
                )
            else:
                # Clear account failures only. Preserve IP-wide history.
                reset_bucket(
                    conn,
                    scope_type="account",
                    key_hash=account_key,
                )

                token, digest = new_session_token()
                expires_at = session_expiry(_ttl_minutes())

                with conn.cursor() as cur:
                    cur.execute(
                        """INSERT INTO auth_sessions
                               (user_id, refresh_token_hash, expires_at)
                           VALUES (%s, %s, %s)""",
                        (user["user_id"], digest, expires_at),
                    )
                    now = datetime.now(UTC)
                    cur.execute(
                        """UPDATE users
                           SET last_login_at = %s, updated_at = %s
                           WHERE user_id = %s""",
                        (now, now, user["user_id"]),
                    )

                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="success",
                    actor_user_id=user["user_id"],
                    details={"method": "password"},
                    correlation_id=correlation_id,
                )

    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    if not login_succeeded:
        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)
```

```
    response.set_cookie(
        SESSION_COOKIE,
        token,
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
        max_age=_ttl_minutes() * 60,
        path="/",
    )
    return UserView(
        user_id=user["user_id"],
        email=user["email"],
        display_name=user["display_name"],
    )

=== Audit integration tests ===
from __future__ import annotations

from datetime import UTC, datetime
from uuid import uuid4

from fastapi.testclient import TestClient

from app.auth import hash_password
from app.throttle import make_throttle_key
from app.database import connect
from app.main import app


def test_successful_login_is_audited_without_secrets(monkeypatch):
    monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Password-Only"
    correlation = None

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """INSERT INTO users (email, password_hash, display_name, account_status)
               VALUES (%s, %s, %s, 'active') RETURNING user_id""",
            (email, hash_password(password), "FIN14 Audit Test"),
        )
        user_id = cur.fetchone()[0]
```

```
    try:
        with TestClient(app) as client:
            response = client.post(
                "/auth/login", json={"email": email, "password": password}
            )
        assert response.status_code == 200

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT action_key, outcome, actor_user_id, details_redacted::text
                    FROM audit_events
                    WHERE action_key = 'auth.login' AND actor_user_id = %s
                    ORDER BY occurred_at DESC LIMIT 1""",
                (user_id,),
            )
            event = cur.fetchone()

        assert event is not None
        assert event[0] == "auth.login"
        assert event[1] == "success"
        assert event[2] == user_id
        assert password not in event[3]
        assert response.cookies.get("finsight_session") not in event[3]
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_failed_login_is_audited_and_repeated_failures_are_blocked(monkeypatch):
    monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
    monkeypatch.setenv("AUTH_LOGIN_MAX_FAILURES", "2")
    monkeypatch.setenv("AUTH_LOGIN_WINDOW_SECONDS", "600")
    monkeypatch.setenv("AUTH_LOGIN_BLOCK_SECONDS", "60")

    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Password-Only"
    test_started_at = datetime.now(UTC)
    ip_key = make_throttle_key(
        b"integration-test-secret-material-32-bytes",
        "ip",
        "testclient",
    )
```

```
with connect() as conn, conn.cursor() as cur:
    cur.execute(
        "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
        (ip_key,),
    )
    cur.execute(
        """INSERT INTO users (email, password_hash, display_name, account_status)
           VALUES (%s, %s, %s, 'active') RETURNING user_id""",
        (email, hash_password(password), "FIN14 Throttle Test"),
    )
    user_id = cur.fetchone()[0]

try:
    with TestClient(app) as client:
        first = client.post(
            "/auth/login", json={"email": email, "password": "wrong-password"}
        )
        second = client.post(
            "/auth/login", json={"email": email, "password": "wrong-password"}
        )
        blocked = client.post(
            "/auth/login", json={"email": email, "password": password}
        )

    assert first.status_code == 401
    assert second.status_code == 401
    assert blocked.status_code == 401
    assert first.json() == second.json() == blocked.json()

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """SELECT outcome, actor_user_id, details_redacted::text
               FROM audit_events
               WHERE action_key = 'auth.login'
                 AND details_redacted->>'reason' IN (
                     'invalid_credentials', 'throttled'
                 )
               ORDER BY occurred_at DESC LIMIT 3"""
        )
        events = cur.fetchall()

    assert len(events) == 3
    assert sum(event[0] == "failure" for event in events) == 2
```

```
        assert sum(event[0] == "denied" for event in events) == 1
        assert all(
            "wrong-password" not in event[2]
            for event in events
        )
        assert any(event[1] == user_id for event in events)
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
                (ip_key,),
            )
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_logout_is_audited_without_recording_session_token(monkeypatch):
    monkeypatch.setenv(
        "AUTH_THROTTLE_SECRET",
        "integration-test-secret-material-32-bytes",
    )
    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Logout-Password"

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """INSERT INTO users (email, password_hash, display_name, account_status)
               VALUES (%s, %s, %s, 'active') RETURNING user_id""",
            (email, hash_password(password), "FIN14 Logout Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with TestClient(app) as client:
            login = client.post(
                "/auth/login",
                json={"email": email, "password": password},
            )
            assert login.status_code == 200
            session_token = login.cookies.get("finsight_session")
            assert session_token

            logout = client.post(
                "/auth/logout",
                cookies={"finsight_session": session_token},
```

```
        )
        assert logout.status_code == 204

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """SELECT outcome, actor_user_id, details_redacted::text
               FROM audit_events
               WHERE action_key = 'auth.logout'
                 AND actor_user_id = %s
               ORDER BY occurred_at DESC LIMIT 1""",
            (user_id,),
        )
        event = cur.fetchone()

    assert event is not None
    assert event[0] == "success"
    assert event[1] == user_id
    assert session_token not in event[2]
    assert password not in event[2]
  finally:
    with connect() as conn, conn.cursor() as cur:
        cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
=== Audit service tests ===
from __future__ import annotations

from datetime import UTC, datetime
from uuid import uuid4

import pytest

from app.audit import record_auth_event


class FakeCursor:
  def __init__(self):
    self.calls = []

  def execute(self, query, params):
    self.calls.append((query, params))

  def __enter__(self):
    return self

  def __exit__(self, *args):
```

```
        return False


class FakeConnection:
    def __init__(self):
        self.cursor_instance = FakeCursor()

    def cursor(self):
        return self.cursor_instance


def test_auth_audit_uses_canonical_columns_and_parameterized_sql():
    conn = FakeConnection()
    actor_id = uuid4()
    occurred_at = datetime(2026, 10, 21, tzinfo=UTC)

    record_auth_event(
        conn,
        action_key="auth.login",
        outcome="success",
        actor_user_id=actor_id,
        occurred_at=occurred_at,
        details={"method": "password"},
    )

    assert len(conn.cursor_instance.calls) == 1
    query, params = conn.cursor_instance.calls[0]

    assert "INSERT INTO audit_events" in query
    for column in (
        "workspace_id",
        "actor_user_id",
        "action_key",
        "resource_type",
        "resource_id",
        "outcome",
        "correlation_id",
        "details_redacted",
        "occurred_at",
    ):
        assert column in query

    assert actor_id in params
    assert "auth.login" in params
```

```
    assert "success" in params
    assert occurred_at in params
    assert "password" not in query.lower()


def test_unknown_account_failure_can_be_audited_without_an_actor():
    conn = FakeConnection()

    record_auth_event(
        conn,
        action_key="auth.login",
        outcome="failure",
        actor_user_id=None,
        details={"reason": "invalid_credentials"},
    )

    _, params = conn.cursor_instance.calls[0]
    assert None in params
    assert "invalid_credentials" in str(params)


@pytest.mark.parametrize(
    "secret",
    [
        "raw-password-value",
        "raw-session-token-value",
        "raw-api-key-value",
        "cookie-secret-value",
    ],
)
def test_auth_audit_redacts_secret_values(secret):
    conn = FakeConnection()

    record_auth_event(
        conn,
        action_key="auth.login",
        outcome="failure",
        actor_user_id=None,
        details={
            "reason": "invalid_credentials",
            "password": secret,
            "session_token": secret,
            "api_key": secret,
            "cookie": secret,
```

```
        },
    )

    query, params = conn.cursor_instance.calls[0]
    persisted_values = f"{query} {params}"

    assert secret not in persisted_values
    assert "invalid_credentials" in persisted_values


def test_auth_audit_rejects_unknown_outcome_before_database_write():
    conn = FakeConnection()

    with pytest.raises(ValueError, match="outcome"):
        record_auth_event(
            conn,
            action_key="auth.login",
            outcome="maybe",
            actor_user_id=None,
        )

    assert conn.cursor_instance.calls == []
=== Migration and release metadata references ===
scripts/package_release.py:54:            "database_migrations": ["0001_auth_identity.sql",
"0002_platform_authorization.sql", "0003_auth_audit_and_login_throttling.sql"],
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:6:CREATE TABLE audit_events (
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:20:CREATE INDEX
idx_audit_events_occurred_at
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:21:    ON audit_events
(occurred_at DESC);
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:23:CREATE INDEX
idx_audit_events_actor_occurred
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:24:    ON audit_events
(actor_user_id, occurred_at DESC);
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:26:CREATE INDEX
idx_audit_events_action_occurred
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:27:    ON audit_events
(action_key, occurred_at DESC);
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
            "password": secret,
            "session_token": secret,
            "api_key": secret,
            "cookie": secret,
        },
```

```
    )

    query, params = conn.cursor_instance.calls[0]
    persisted_values = f"{query} {params}"

    assert secret not in persisted_values
    assert "invalid_credentials" in persisted_values


def test_auth_audit_rejects_unknown_outcome_before_database_write():
    conn = FakeConnection()

    with pytest.raises(ValueError, match="outcome"):
        record_auth_event(
            conn,
            action_key="auth.login",
            outcome="maybe",
            actor_user_id=None,
        )

    assert conn.cursor_instance.calls == []
=== Migration and release metadata references ===
scripts/package_release.py:54:            "database_migrations": ["0001_auth_identity.sql",
"0002_platform_authorization.sql", "0003_auth_audit_and_login_throttling.sql"],
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:6:CREATE TABLE audit_events (
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:20:CREATE INDEX
idx_audit_events_occurred_at
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:21:    ON audit_events
(occurred_at DESC);
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:23:CREATE INDEX
idx_audit_events_actor_occurred
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:24:    ON audit_events
(actor_user_id, occurred_at DESC);
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:26:CREATE INDEX
idx_audit_events_action_occurred
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:27:    ON audit_events
(action_key, occurred_at DESC);
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

echo "=== Logout route ==="
sed -n '270,330p' apps/api/app/main.py
```

```
echo "=== Existing correlation/request ID handling ==="
grep -n -E 'request_id|correlation_id|X-Correlation-ID|Correlation-ID' apps/api/app/main.py

echo "=== Test fixtures and client setup ==="
sed -n '1,180p' apps/api/tests/test_auth.py

echo "=== Current migration ==="
sed -n '1,180p' apps/api/migrations/0003_auth_audit_and_login_throttling.sql
=== Logout route ===


@app.post("/auth/logout", status_code=204)
def logout(
    request: Request,
    response: Response,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> Response:
    _check_origin(request)
    correlation_id = uuid4()

    try:
        with connect() as conn:
            actor_user_id = None

            if session_token:
                with conn.cursor() as cur:
                    cur.execute(
                        """UPDATE auth_sessions
                           SET revoked_at = %s, revoke_reason = 'logout'
                           WHERE refresh_token_hash = %s
                             AND revoked_at IS NULL
                           RETURNING user_id""",
                        (datetime.now(UTC), token_digest(session_token)),
                    )
                    row = cur.fetchone()
                    if row:
                        actor_user_id = row[0]

            record_auth_event(
                conn,
                action_key="auth.logout",
                outcome="success" if actor_user_id else "denied",
                actor_user_id=actor_user_id,
                details={"reason": "logout"},
```

```
            correlation_id=correlation_id,
        )
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    response.delete_cookie(
        SESSION_COOKIE,
        path="/",
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
    )
    response.status_code = status.HTTP_204_NO_CONTENT
    return response


@app.get("/auth/me", response_model=UserView)
def who_am_i(user: CurrentUser) -> UserView:
    return UserView(**user)


@app.get("/api/v1/admin/authorization/capabilities")
def platform_admin_capabilities(
=== Existing correlation/request ID handling ===
145:    correlation_id = uuid4()
165:                    correlation_id=correlation_id,
210:                        correlation_id=correlation_id,
244:                        correlation_id=correlation_id,
279:    correlation_id = uuid4()
305:                correlation_id=correlation_id,
336:    supplied_request_id = request.headers.get("request_id", "").strip()
337:    if supplied_request_id and (
338:        len(supplied_request_id) > 128
339:        or any(ord(char) < 32 or ord(char) == 127 for char in supplied_request_id)
341:        raise HTTPException(status_code=400, detail="Invalid request_id")
344:    request_id = f"req_{secrets.token_urlsafe(12)}"
345:    correlation_id = supplied_request_id or request_id
363:        "request_id": request_id,
364:        "correlation_id": correlation_id,
=== Test fixtures and client setup ===
from __future__ import annotations
```

```
import secrets
from uuid import uuid4

import pytest
from fastapi.testclient import TestClient
from psycopg.rows import dict_row

from app.auth import hash_password, token_digest, verify_password
from app.throttle import make_throttle_key
from app.database import connect
from app.main import SESSION_COOKIE, app

TEST_PASSWORD = secrets.token_urlsafe(32)


@pytest.fixture
def identity():
    email = f"{uuid4().hex}@example.com"
    encoded = hash_password(TEST_PASSWORD)
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO users (email, password_hash, display_name, account_status) VALUES
(%s, %s, %s, 'active') RETURNING user_id",
            (email, encoded, "Synthetic Analyst"),
        )
        user_id = cur.fetchone()[0]
    try:
        yield user_id, email, encoded
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


@pytest.fixture
def client(monkeypatch):
    monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
    monkeypatch.setenv("COOKIE_SECURE", "false")
    monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
    throttle_secret = "test-only-secret-material-at-least-32-bytes"
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
    ip_key = make_throttle_key(
        throttle_secret.encode("utf-8"), "ip", "testclient"
    )
```

```
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
        )

    try:
        with TestClient(app) as test_client:
            yield test_client
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
                (ip_key,),
            )


def test_password_hash_is_argon2_and_never_plaintext():
    encoded = hash_password(TEST_PASSWORD)
    assert encoded.startswith("argon2id")
    assert encoded != TEST_PASSWORD
    assert verify_password(encoded, TEST_PASSWORD)
    assert not verify_password(encoded, "wrong")


def test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
    user_id, email, encoded = identity
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
    assert response.json() == {"user_id": str(user_id), "email": email, "display_name": "Synthetic
Analyst"}
    assert TEST_PASSWORD not in response.text and encoded not in response.text
    cookie = response.cookies.get(SESSION_COOKIE)
    assert cookie
    set_cookie = response.headers["set-cookie"].lower()
    assert "httponly" in set_cookie and "samesite=lax" in set_cookie and "path=/" in set_cookie
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute("SELECT refresh_token_hash, expires_at, revoked_at FROM auth_sessions
WHERE user_id = %s", (user_id,))
        stored = cur.fetchone()
    assert stored["refresh_token_hash"] == token_digest(cookie)
    assert stored["refresh_token_hash"] != cookie
    assert stored["expires_at"] is not None and stored["revoked_at"] is None
```

```
    protected = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
    assert protected.status_code == 200 and protected.json()["user_id"] == str(user_id)
    captured = capsys.readouterr()
    assert TEST_PASSWORD not in caplog.text + captured.out + captured.err
    assert cookie not in caplog.text + captured.out + captured.err


def test_invalid_credentials_have_same_generic_response(client, identity):
    _, email, _ = identity
    known = client.post("/auth/login", json={"email": email, "password": "wrong-password"})
    absent = client.post("/auth/login", json={"email": "absent@example.com", "password":
"wrong-password"})
    assert (known.status_code, known.json()) == (absent.status_code, absent.json())
    assert known.status_code == 401 and known.json() == {"detail": "Email or password is
incorrect."}
    assert SESSION_COOKIE not in known.cookies and SESSION_COOKIE not in
absent.cookies


def test_logout_revokes_session_and_replay_fails(client, identity):
    _, email, _ = identity
    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    cookie = login.cookies.get(SESSION_COOKIE)
    assert cookie
    logout = client.post("/auth/logout", cookies={SESSION_COOKIE: cookie})
    assert logout.status_code == 204
    replay = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
    assert replay.status_code == 401
    with connect() as conn, conn.cursor() as cur:
        cur.execute("SELECT revoked_at FROM auth_sessions WHERE refresh_token_hash =
%s", (token_digest(cookie),))
        assert cur.fetchone()[0] is not None


def test_logout_without_session_is_idempotent(client):
    response = client.post("/auth/logout")

    assert response.status_code == 204
    assert response.headers["set-cookie"].startswith(f"{SESSION_COOKIE}=")


def test_expired_session_is_rejected(client, identity):
    user_id, email, _ = identity
    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
```

```
    cookie = login.cookies.get(SESSION_COOKIE)
    assert cookie
    with connect() as conn, conn.cursor() as cur:
        cur.execute("UPDATE auth_sessions SET last_seen_at = now() - interval '61 minutes'
WHERE user_id = %s", (user_id,))
    assert client.get("/auth/me", cookies={SESSION_COOKIE: cookie}).status_code == 401


def test_public_registration_is_not_available(client):
    response = client.post("/auth/register", json={"email": "public@example.com", "password":
TEST_PASSWORD})
    assert response.status_code == 404


def test_auth_mutation_rejects_foreign_origin(client):
    response = client.post(
        "/auth/logout", headers={"Origin": "https://attacker.invalid"},
    )
    assert response.status_code == 403


def test_secure_cookie_can_be_required_by_deployment(client, identity, monkeypatch):
    _, email, _ = identity
    monkeypatch.setenv("COOKIE_SECURE", "true")
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
    assert "secure" in response.headers["set-cookie"].lower()
=== Current migration ===
BEGIN;

-- Authentication/security audit trail.
-- workspace_id remains nullable because the current application schema
-- does not yet contain the workspaces table.
CREATE TABLE audit_events (
    audit_event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NULL,
    actor_user_id UUID NULL REFERENCES users(user_id) ON DELETE SET NULL,
    action_key VARCHAR(120) NOT NULL,
    resource_type VARCHAR(80) NOT NULL,
    resource_id UUID NULL,
    outcome VARCHAR(20) NOT NULL
        CHECK (outcome IN ('success', 'failure', 'denied')),
    correlation_id UUID NULL,
    details_redacted JSONB NOT NULL DEFAULT '{}'::jsonb,
```

```
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_events_occurred_at
    ON audit_events (occurred_at DESC);

CREATE INDEX idx_audit_events_actor_occurred
    ON audit_events (actor_user_id, occurred_at DESC);

CREATE INDEX idx_audit_events_action_occurred
    ON audit_events (action_key, occurred_at DESC);

-- Persistent, shared login-failure counters.
-- key_hash must be a keyed digest generated by the application;
-- raw email addresses, IP addresses, passwords and tokens do not belong here.
CREATE TABLE auth_login_throttle (
    scope_type VARCHAR(16) NOT NULL
        CHECK (scope_type IN ('account', 'ip')),
    key_hash CHAR(64) NOT NULL,
    failed_attempts INTEGER NOT NULL DEFAULT 0
        CHECK (failed_attempts >= 0),
    window_started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    blocked_until TIMESTAMPTZ NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (scope_type, key_hash)
);

CREATE INDEX idx_auth_login_throttle_blocked_until
    ON auth_login_throttle (blocked_until)
    WHERE blocked_until IS NOT NULL;

COMMIT;
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

-- Persistent, shared login-failure counters.
-- key_hash must be a keyed digest generated by the application;
-- raw email addresses, IP addresses, passwords and tokens do not belong here.
CREATE TABLE auth_login_throttle (
    scope_type VARCHAR(16) NOT NULL
        CHECK (scope_type IN ('account', 'ip')),
    key_hash CHAR(64) NOT NULL,
    failed_attempts INTEGER NOT NULL DEFAULT 0
        CHECK (failed_attempts >= 0),
    window_started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
    blocked_until TIMESTAMPTZ NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (scope_type, key_hash)
);

CREATE INDEX idx_auth_login_throttle_blocked_until
    ON auth_login_throttle (blocked_until)
    WHERE blocked_until IS NOT NULL;

COMMIT;
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
if source.count(old_import) != 1:
    raise SystemExit("STOP: expected duplicate UUID imports were not found exactly once.")
if source.count(old_helper) != 1:
    raise SystemExit("STOP: expected origin helper was not found exactly once.")
if source.count("correlation_id = uuid4()") != 2:
    raise SystemExit("STOP: expected exactly two auth correlation assignments.")

source = source.replace(old_import, new_import, 1)
source = source.replace(old_helper, new_helper, 1)
source = source.replace(
    "    correlation_id = uuid4()",
    "    correlation_id = _auth_correlation_id(request)",
)
source = source.replace(
    "    _check_origin(request)\n    email = payload.email.strip().lower()",
    "    _check_origin(request)\n    correlation_id = _auth_correlation_id(request)\n
response.headers['X-Correlation-ID'] = str(correlation_id)\n    email =
payload.email.strip().lower()",
    1,
)
source = source.replace(
    "    correlation_id = _auth_correlation_id(request)\n    login_succeeded = False",
    "    login_succeeded = False",
    1,
)
source = source.replace(
    "    correlation_id = _auth_correlation_id(request)\n\n    try:",
    "    correlation_id = _auth_correlation_id(request)\n    response.headers['X-Correlation-ID'] =
str(correlation_id)\n\n    try:",
    1,
)
```

```
path.write_text(source)
print("Updated authentication correlation-ID handling.")
PY

git diff --check
git diff -- apps/api/app/main.py
Updated authentication correlation-ID handling.
diff --git a/apps/api/app/main.py b/apps/api/app/main.py
index 5e0e618..c034be4 100644
--- a/apps/api/app/main.py
+++ b/apps/api/app/main.py
@@ -7,7 +7,16 @@ import secrets
 from datetime import UTC, datetime
 from pathlib import Path
 from typing import Annotated, Any
-from uuid import UUID
+from uuid import UUID, uuid4
+
+from app.audit import record_auth_event
+from app.throttle import (
+    is_blocked,
+    load_throttle_config,
+    make_throttle_key,
+    record_failure,
+    reset_bucket,
+)

 import psycopg
 from dotenv import load_dotenv
@@ -69,6 +78,20 @@ def _check_origin(request: Request) -> None:
         raise HTTPException(status_code=403, detail="Request origin is not allowed")


+def _auth_correlation_id(request: Request) -> UUID:
+    """Accept a valid UUID correlation ID or generate one for this auth request."""
+    supplied = request.headers.get("X-Correlation-ID", "").strip()
+    if not supplied:
+        return uuid4()
+    try:
+        return UUID(supplied)
+    except ValueError as exc:
+        raise HTTPException(
+            status_code=400,
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
```

```
            status_code=400,
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

echo "=== Correlation helper ==="
sed -n '65,105p' apps/api/app/main.py

echo "=== Login route ==="
sed -n '185,225p' apps/api/app/main.py

echo "=== Logout route ==="
sed -n '290,325p' apps/api/app/main.py

echo "=== All correlation assignments and response headers ==="
grep -n -E 'correlation_id =|X-Correlation-ID|record_auth_event' apps/api/app/main.py
=== Correlation helper ===
        raise HTTPException(status_code=500, detail="Session configuration is invalid") from exc


def _secure_cookie() -> bool:
    configured = os.environ.get("COOKIE_SECURE")
    if configured is not None:
        return configured.lower() in {"1", "true", "yes"}
    return os.environ.get("APP_ENV", "development").lower() == "production"


def _check_origin(request: Request) -> None:
    origin = request.headers.get("origin")
    if origin and origin != web_origin:
        raise HTTPException(status_code=403, detail="Request origin is not allowed")


def _auth_correlation_id(request: Request) -> UUID:
    """Accept a valid UUID correlation ID or generate one for this auth request."""
    supplied = request.headers.get("X-Correlation-ID", "").strip()
    if not supplied:
        return uuid4()
    try:
        return UUID(supplied)
    except ValueError as exc:
        raise HTTPException(
            status_code=400,
            detail="Invalid X-Correlation-ID",
```

```
        ) from exc


def get_current_user(
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> dict[str, Any]:
    if not session_token:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Authentication required")
    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT u.user_id, u.email, u.display_name, r.role_key,
                       COALESCE(array_agg(DISTINCT p.permission_key)
                           FILTER (WHERE p.permission_key IS NOT NULL), ARRAY[]::varchar[])
AS permissions
=== Login route ===
                     WHERE lower(email) = %s
                       AND account_status = 'active'
                       AND deleted_at IS NULL""",
                    (email,),
                )
                user = cur.fetchone()

            password_hash = (
                user["password_hash"]
                if user and user["password_hash"]
                else DUMMY_PASSWORD_HASH
            )
            password_valid = verify_password(
                password_hash, payload.password
            )
            login_succeeded = bool(
                user and user["password_hash"] and password_valid
            )

            if not login_succeeded:
                record_failure(
                    conn,
                    scope_type="account",
                    key_hash=account_key,
                    config=config,
                )
                record_failure(
```

```
                    conn,
                    scope_type="ip",
                    key_hash=ip_key,
                    config=config,
                )
                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="failure",
                    actor_user_id=user["user_id"] if user else None,
                    details={"reason": "invalid_credentials"},
                    correlation_id=correlation_id,
                )
            else:
=== Logout route ===
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers['X-Correlation-ID'] = str(correlation_id)

    try:
        with connect() as conn:
            actor_user_id = None

            if session_token:
                with conn.cursor() as cur:
                    cur.execute(
                        """UPDATE auth_sessions
                           SET revoked_at = %s, revoke_reason = 'logout'
                           WHERE refresh_token_hash = %s
                             AND revoked_at IS NULL
                           RETURNING user_id""",
                        (datetime.now(UTC), token_digest(session_token)),
                    )
                    row = cur.fetchone()
                    if row:
                        actor_user_id = row[0]

            record_auth_event(
                conn,
                action_key="auth.logout",
                outcome="success" if actor_user_id else "denied",
                actor_user_id=actor_user_id,
                details={"reason": "logout"},
```

```
            correlation_id=correlation_id,
        )
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc
=== All correlation assignments and response headers ===
12:from app.audit import record_auth_event
83:    supplied = request.headers.get("X-Correlation-ID", "").strip()
91:            detail="Invalid X-Correlation-ID",
145:    correlation_id = _auth_correlation_id(request)
146:    response.headers['X-Correlation-ID'] = str(correlation_id)
172:                record_auth_event(
217:                    record_auth_event(
251:                    record_auth_event(
292:    correlation_id = _auth_correlation_id(request)
293:    response.headers['X-Correlation-ID'] = str(correlation_id)
313:            record_auth_event(
359:    correlation_id = supplied_request_id or request_id
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
eaders['X-Correlation-ID'] = str(correlation_id)
313:            record_auth_event(
359:    correlation_id = supplied_request_id or request_id
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1
sed -n '125,190p' apps/api/app/main.py
    except psycopg.Error as exc:
        raise HTTPException(status_code=503, detail="Authentication service unavailable") from
exc


CurrentUser = Annotated[dict[str, Any], Depends(get_current_user)]


def require_permission(permission_key: str):
    """Build a deny-by-default backend permission dependency."""
    def dependency(user: CurrentUser) -> dict[str, Any]:
        if permission_key not in user.get("permissions", []):
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Permission
denied")
        return user
```

```
    return dependency


@app.post("/auth/login", response_model=UserView)
def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers['X-Correlation-ID'] = str(correlation_id)
    email = payload.email.strip().lower()
    try:
        config = load_throttle_config()
    except ValueError as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    client_ip = request.client.host if request.client else "unknown"
    account_key = make_throttle_key(config.secret, "account", email)
    ip_key = make_throttle_key(config.secret, "ip", client_ip)
    login_succeeded = False
    user = None

    try:
        with connect() as conn:
            account_blocked = is_blocked(
                conn, scope_type="account", key_hash=account_key
            )
            ip_blocked = is_blocked(
                conn, scope_type="ip", key_hash=ip_key
            )

            if account_blocked or ip_blocked:
                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="denied",
                    actor_user_id=None,
                    details={"reason": "throttled"},
                    correlation_id=correlation_id,
                )
            else:
                with conn.cursor(row_factory=dict_row) as cur:
                    cur.execute(
```

```
                """SELECT user_id, email, display_name, password_hash
                   FROM users
                   WHERE lower(email) = %s
                     AND account_status = 'active'
                     AND deleted_at IS NULL""",
                (email,),
            )
            user = cur.fetchone()
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
        AND account_status = 'active'
                     AND deleted_at IS NULL""",
                (email,),
            )
            user = cur.fetchone()
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

sed -n '1,115p' apps/api/app/main.py
sed -n '1,100p' apps/api/requirements.txt
"""FinSight AI API with PostgreSQL-backed browser sessions."""

from __future__ import annotations

import os
import secrets
from datetime import UTC, datetime
from pathlib import Path
from typing import Annotated, Any
from uuid import UUID, uuid4

from app.audit import record_auth_event
from app.throttle import (
    is_blocked,
    load_throttle_config,
    make_throttle_key,
    record_failure,
    reset_bucket,
)

import psycopg
from dotenv import load_dotenv
from fastapi import Cookie, Depends, FastAPI, HTTPException, Request, Response, status
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, EmailStr, Field
```

```
from psycopg.rows import dict_row

from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
from app.database import connect

load_dotenv(Path(__file__).resolve().parents[3] / ".env")

app = FastAPI(title="FinSight AI API", version="0.2.0")
web_origin = os.environ.get("WEB_ORIGIN", "http://localhost:3000")
app.add_middleware(
    CORSMiddleware,
    allow_origins=[web_origin],
    allow_credentials=True,
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)

SESSION_COOKIE = "finsight_session"
GENERIC_LOGIN_ERROR = "Email or password is incorrect."
DUMMY_PASSWORD_HASH = hash_password(secrets.token_urlsafe(32))


class LoginRequest(BaseModel):
    email: EmailStr
    password: str = Field(min_length=1, max_length=1024)


class UserView(BaseModel):
    user_id: UUID
    email: str
    display_name: str


def _ttl_minutes() -> int:
    try:
        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
        session_expiry(value)
        return value
    except ValueError as exc:
        raise HTTPException(status_code=500, detail="Session configuration is invalid") from exc


def _secure_cookie() -> bool:
```

```
    configured = os.environ.get("COOKIE_SECURE")
    if configured is not None:
        return configured.lower() in {"1", "true", "yes"}
    return os.environ.get("APP_ENV", "development").lower() == "production"


def _check_origin(request: Request) -> None:
    origin = request.headers.get("origin")
    if origin and origin != web_origin:
        raise HTTPException(status_code=403, detail="Request origin is not allowed")


def _auth_correlation_id(request: Request) -> UUID:
    """Accept a valid UUID correlation ID or generate one for this auth request."""
    supplied = request.headers.get("X-Correlation-ID", "").strip()
    if not supplied:
        return uuid4()
    try:
        return UUID(supplied)
    except ValueError as exc:
        raise HTTPException(
            status_code=400,
            detail="Invalid X-Correlation-ID",
        ) from exc


def get_current_user(
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> dict[str, Any]:
    if not session_token:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Authentication required")
    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT u.user_id, u.email, u.display_name, r.role_key,
                       COALESCE(array_agg(DISTINCT p.permission_key)
                           FILTER (WHERE p.permission_key IS NOT NULL), ARRAY[]::varchar[])
AS permissions
                  FROM auth_sessions s JOIN users u ON u.user_id = s.user_id
                  LEFT JOIN roles r ON r.role_id = u.platform_role_id AND r.scope = 'platform'
                  LEFT JOIN role_permissions rp ON rp.role_id = r.role_id
                  LEFT JOIN permissions p ON p.permission_id = rp.permission_id
                  WHERE s.refresh_token_hash = %s AND s.revoked_at IS NULL
```

```
                  AND s.expires_at > %s AND u.account_status = 'active'
                  AND s.last_seen_at > %s - make_interval(mins => %s)
                  AND u.deleted_at IS NULL
                GROUP BY u.user_id, u.email, u.display_name, r.role_key""",
              (token_digest(session_token), datetime.now(UTC), datetime.now(UTC),
_ttl_minutes()),
fastapi==0.133.0
starlette==1.3.1
uvicorn==0.40.0
argon2-cffi==25.1.0
psycopg[binary]==3.2.13
python-dotenv==1.2.2
email-validator==2.3.0
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
rmissions rp ON rp.role_id = r.role_id
                LEFT JOIN permissions p ON p.permission_id = rp.permission_id
                WHERE s.refresh_token_hash = %s AND s.revoked_at IS NULL
                  AND s.expires_at > %s AND u.account_status = 'active'
                  AND s.last_seen_at > %s - make_interval(mins => %s)
                  AND u.deleted_at IS NULL
                GROUP BY u.user_id, u.email, u.display_name, r.role_key""",
              (token_digest(session_token), datetime.now(UTC), datetime.now(UTC),
_ttl_minutes()),
fastapi==0.133.0
starlette==1.3.1
uvicorn==0.40.0
argon2-cffi==25.1.0
psycopg[binary]==3.2.13
python-dotenv==1.2.2
email-validator==2.3.0
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

tail -n 100 apps/api/tests/test_fin14_auth_security.py

git status --short
git diff --check
        )
        user_id = cur.fetchone()[0]

    try:
        with TestClient(app) as client:
            first = client.post(
                "/auth/login", json={"email": email, "password": "wrong-password"}
```

```
        )
        second = client.post(
            "/auth/login", json={"email": email, "password": "wrong-password"}
        )
        blocked = client.post(
            "/auth/login", json={"email": email, "password": password}
        )

    assert first.status_code == 401
    assert second.status_code == 401
    assert blocked.status_code == 401
    assert first.json() == second.json() == blocked.json()

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """SELECT outcome, actor_user_id, details_redacted::text
               FROM audit_events
               WHERE action_key = 'auth.login'
                 AND details_redacted->>'reason' IN (
                     'invalid_credentials', 'throttled'
                 )
               ORDER BY occurred_at DESC LIMIT 3"""
        )
        events = cur.fetchall()

    assert len(events) == 3
    assert sum(event[0] == "failure" for event in events) == 2
    assert sum(event[0] == "denied" for event in events) == 1
    assert all(
        "wrong-password" not in event[2]
        for event in events
    )
    assert any(event[1] == user_id for event in events)
finally:
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
        )
        cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_logout_is_audited_without_recording_session_token(monkeypatch):
    monkeypatch.setenv(
```

```
    "AUTH_THROTTLE_SECRET",
    "integration-test-secret-material-32-bytes",
)
email = f"{uuid4().hex}@example.com"
password = "Synthetic-FIN14-Logout-Password"

with connect() as conn, conn.cursor() as cur:
    cur.execute(
        """INSERT INTO users (email, password_hash, display_name, account_status)
           VALUES (%s, %s, %s, 'active') RETURNING user_id""",
        (email, hash_password(password), "FIN14 Logout Test"),
    )
    user_id = cur.fetchone()[0]

try:
    with TestClient(app) as client:
        login = client.post(
            "/auth/login",
            json={"email": email, "password": password},
        )
        assert login.status_code == 200
        session_token = login.cookies.get("finsight_session")
        assert session_token

        logout = client.post(
            "/auth/logout",
            cookies={"finsight_session": session_token},
        )
        assert logout.status_code == 204

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """SELECT outcome, actor_user_id, details_redacted::text
               FROM audit_events
               WHERE action_key = 'auth.logout'
                 AND actor_user_id = %s
               ORDER BY occurred_at DESC LIMIT 1""",
            (user_id,),
        )
        event = cur.fetchone()

    assert event is not None
    assert event[0] == "success"
    assert event[1] == user_id
```

```
        assert session_token not in event[2]
        assert password not in event[2]
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
 M .env.example
 M FUNCTION_TEST_TRACEABILITY.txt
 M IMPLEMENTATION_STATUS.txt
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/tests/test_auth.py
 M conversation.txt
 M docs/release/RELEASE_README.md
 M scripts/package_release.py
?? apps/api/app/audit.py
?? apps/api/app/throttle.py
?? apps/api/migrations/0003_auth_audit_and_login_throttling.sql
?? apps/api/tests/test_audit_service.py
?? apps/api/tests/test_fin14_auth_security.py
?? apps/api/tests/test_throttle.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
 apps/api/tests/test_fin14_auth_security.py
?? apps/api/tests/test_throttle.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest apps/api/tests/test_fin14_auth_security.py -q
...
[100%]
=============================================================================
========= warnings summary
=============================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
```

```
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
   _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
   return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
3 passed, 3 warnings in 0.73s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest apps/api/tests/test_fin14_auth_security.py -q
...
[100%]
==============================================================================
========= warnings summary
==============================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
   from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
   _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken
```

```
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
3 passed, 3 warnings in 0.73s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

grep -n '^def test_' apps/api/tests/test_fin14_auth_security.py
grep -n -E
'test_auth_correlation_id_is_returned_and_audited|test_invalid_auth_correlation_id_is_rejected'
apps/api/tests/test_fin14_auth_security.py
14:def test_successful_login_is_audited_without_secrets(monkeypatch):
56:def test_failed_login_is_audited_and_repeated_failures_are_blocked(monkeypatch):
129:def test_logout_is_audited_without_recording_session_token(monkeypatch):
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
              VALUES (%s, %s, %s, 'active') RETURNING user_id""",
            (email, hash_password(password), "FIN14 Correlation Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with TestClient(app) as client:
            response = client.post(
                "/auth/login",
                json={"email": email, "password": password},
                headers={"X-Correlation-ID": str(correlation_id)},
            )

        assert response.status_code == 200
        assert response.headers.get("X-Correlation-ID") == str(correlation_id)

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT correlation_id
                   FROM audit_events
                   WHERE action_key = 'auth.login'
                     AND actor_user_id = %s
                     AND outcome = 'success'
                   ORDER BY occurred_at DESC LIMIT 1""",
```

```
            (user_id,),
        )
        event = cur.fetchone()

    assert event is not None
    assert event[0] == correlation_id
finally:
    with connect() as conn, conn.cursor() as cur:
        cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_invalid_auth_correlation_id_is_rejected(monkeypatch):
    monkeypatch.setenv(
        "AUTH_THROTTLE_SECRET",
        "integration-test-secret-material-32-bytes",
    )

    with TestClient(app) as client:
        response = client.post(
            "/auth/login",
            json={"email": "correlation-test@example.com", "password": "not-used"},
            headers={"X-Correlation-ID": "not-a-uuid"},
        )

    assert response.status_code == 400
    assert response.json() == {"detail": "Invalid X-Correlation-ID"}
PY

echo "=== Confirm new tests exist ==="
grep -n '^def test_' apps/api/tests/test_fin14_auth_security.py

echo "=== Check patch whitespace ==="
git diff --check
```

```
(base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

grep -n '^def test_' apps/api/tests/test_fin14_auth_security.py
grep -n -E
'test_auth_correlation_id_is_returned_and_audited|test_invalid_auth_correlation_id_is_rejected'
apps/api/tests/test_fin14_auth_security.py
14:def test_successful_login_is_audited_without_secrets(monkeypatch):
56:def test_failed_login_is_audited_and_repeated_failures_are_blocked(monkeypatch):
```

```
129:def test_logout_is_audited_without_recording_session_token(monkeypatch):
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
            VALUES (%s, %s, %s, 'active') RETURNING user_id""",
        (email, hash_password(password), "FIN14 Correlation Test"),
    )
    user_id = cur.fetchone()[0]

  try:
    with TestClient(app) as client:
      response = client.post(
        "/auth/login",
        json={"email": email, "password": password},
        headers={"X-Correlation-ID": str(correlation_id)},
      )

    assert response.status_code == 200
    assert response.headers.get("X-Correlation-ID") == str(correlation_id)

    with connect() as conn, conn.cursor() as cur:
      cur.execute(
        """SELECT correlation_id
          FROM audit_events
          WHERE action_key = 'auth.login'
            AND actor_user_id = %s
            AND outcome = 'success'
          ORDER BY occurred_at DESC LIMIT 1""",
        (user_id,),
      )
      event = cur.fetchone()

    assert event is not None
    assert event[0] == correlation_id
  finally:
    with connect() as conn, conn.cursor() as cur:
      cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_invalid_auth_correlation_id_is_rejected(monkeypatch):
  monkeypatch.setenv(
    "AUTH_THROTTLE_SECRET",
    "integration-test-secret-material-32-bytes",
  )

  with TestClient(app) as client:
```

```
        response = client.post(
            "/auth/login",
            json={"email": "correlation-test@example.com", "password": "not-used"},
            headers={"X-Correlation-ID": "not-a-uuid"},
        )

    assert response.status_code == 400
    assert response.json() == {"detail": "Invalid X-Correlation-ID"}
PY

echo "=== Confirm new tests exist ==="
grep -n '^def test_' apps/api/tests/test_fin14_auth_security.py

echo "=== Check patch whitespace ==="
git diff --check
```
```
(base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest apps/api/tests/test_fin14_auth_security.py -q
(base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

echo "=== Confirm test functions ==="
grep -n '^def test_' apps/api/tests/test_fin14_auth_security.py

echo "=== Confirm new tests are present ==="
grep -n -E
'test_auth_correlation_id_is_returned_and_audited|test_invalid_auth_correlation_id_is_rejected'
apps/api/tests/test_fin14_auth_security.py

echo "=== Run the security integration tests ==="
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest -vv -s apps/api/tests/test_fin14_auth_security.py
(base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro NVDIA_Play_Book % cd FinSight_AI
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI"
pwd
ls -l apps/api/tests/test_fin14_auth_security.py
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI
-rw-r--r--@ 1 sandeeph  staff  8853 Oct 21 19:31 apps/api/tests/test_fin14_auth_security.py
```

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % .venv/bin/python -m pytest
--collect-only -q apps/api/tests/test_fin14_auth_security.py
apps/api/tests/test_fin14_auth_security.py::test_successful_login_is_audited_without_secrets
apps/api/tests/test_fin14_auth_security.py::test_failed_login_is_audited_and_repeated_failures_
are_blocked
apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken
apps/api/tests/test_fin14_auth_security.py::test_auth_correlation_id_is_returned_and_audited
apps/api/tests/test_fin14_auth_security.py::test_invalid_auth_correlation_id_is_rejected

============================================================================
========= warnings summary
============================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
5 tests collected in 0.25s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % echo "Exit status: $?"
Exit status: 0
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest -vv apps/api/tests/test_fin14_auth_security.py

echo "Pytest exit status: $?"
============================================================================
======== test session starts
```

```
============================================================================
========
platform darwin -- Python 3.14.6, pytest-9.0.3, pluggy-1.7.0 --
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI
configfile: pytest.ini
plugins: cov-6.2.1, anyio-4.15.1
collected 5 items

apps/api/tests/test_fin14_auth_security.py::test_successful_login_is_audited_without_secrets
PASSED                                                                   [ 20%]
apps/api/tests/test_fin14_auth_security.py::test_failed_login_is_audited_and_repeated_failures_
are_blocked PASSED                                                       [ 40%]
apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken PASSED                                                              [ 60%]
apps/api/tests/test_fin14_auth_security.py::test_auth_correlation_id_is_returned_and_audited
PASSED                                                                   [ 80%]
apps/api/tests/test_fin14_auth_security.py::test_invalid_auth_correlation_id_is_rejected
PASSED                                                                   [100%]

============================================================================
========= warnings summary
============================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
```

```
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=============================================================================
=== 5 passed, 3 warnings in 0.73s
=============================================================================
===
Pytest exit status: 0
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

sed -n '75,105p' apps/api/app/main.py
sed -n '135,180p' apps/api/app/main.py
sed -n '285,325p' apps/api/app/main.py
def _check_origin(request: Request) -> None:
    origin = request.headers.get("origin")
    if origin and origin != web_origin:
        raise HTTPException(status_code=403, detail="Request origin is not allowed")


def _auth_correlation_id(request: Request) -> UUID:
    """Accept a valid UUID correlation ID or generate one for this auth request."""
    supplied = request.headers.get("X-Correlation-ID", "").strip()
    if not supplied:
        return uuid4()
    try:
        return UUID(supplied)
    except ValueError as exc:
        raise HTTPException(
            status_code=400,
            detail="Invalid X-Correlation-ID",
        ) from exc


def get_current_user(
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> dict[str, Any]:
    if not session_token:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Authentication required")
```

```
    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT u.user_id, u.email, u.display_name, r.role_key,
                          COALESCE(array_agg(DISTINCT p.permission_key)
                                   FILTER (WHERE p.permission_key IS NOT NULL), ARRAY[]::varchar[])
AS permissions
        if permission_key not in user.get("permissions", []):
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Permission
denied")
        return user

    return dependency


@app.post("/auth/login", response_model=UserView)
def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers['X-Correlation-ID'] = str(correlation_id)
    email = payload.email.strip().lower()
    try:
        config = load_throttle_config()
    except ValueError as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    client_ip = request.client.host if request.client else "unknown"
    account_key = make_throttle_key(config.secret, "account", email)
    ip_key = make_throttle_key(config.secret, "ip", client_ip)
    login_succeeded = False
    user = None

    try:
        with connect() as conn:
            account_blocked = is_blocked(
                conn, scope_type="account", key_hash=account_key
            )
            ip_blocked = is_blocked(
                conn, scope_type="ip", key_hash=ip_key
            )
```

```
            if account_blocked or ip_blocked:
                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="denied",
                    actor_user_id=None,
                    details={"reason": "throttled"},
                    correlation_id=correlation_id,
                )
            else:
@app.post("/auth/logout", status_code=204)
def logout(
    request: Request,
    response: Response,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers['X-Correlation-ID'] = str(correlation_id)

    try:
        with connect() as conn:
            actor_user_id = None

            if session_token:
                with conn.cursor() as cur:
                    cur.execute(
                        """UPDATE auth_sessions
                           SET revoked_at = %s, revoke_reason = 'logout'
                           WHERE refresh_token_hash = %s
                             AND revoked_at IS NULL
                           RETURNING user_id""",
                        (datetime.now(UTC), token_digest(session_token)),
                    )
                    row = cur.fetchone()
                    if row:
                        actor_user_id = row[0]

            record_auth_event(
                conn,
                action_key="auth.logout",
                outcome="success" if actor_user_id else "denied",
                actor_user_id=actor_user_id,
                details={"reason": "logout"},
```

```
                correlation_id=correlation_id,
            )
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(base) sandeeph@Sandeeps-MacBook-Pro NVDIA_Play_Book %  source
/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/bin/activate
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro NVDIA_Play_Book % cd FinSight_AI
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
    if request.url.path not in {"/auth/login", "/auth/logout"}:
        return await call_next(request)

    supplied = request.headers.get("X-Correlation-ID", "").strip()
    try:
        correlation_id = UUID(supplied) if supplied else uuid4()
    except ValueError:
        from starlette.responses import JSONResponse

        return JSONResponse(
            status_code=400,
            content={"detail": "Invalid X-Correlation-ID"},
        )

    request.state.auth_correlation_id = correlation_id
    response = await call_next(request)
    response.headers["X-Correlation-ID"] = str(correlation_id)
    return response


web_origin = os.environ.get("WEB_ORIGIN", "http://localhost:3000")
'''

if source.count(anchor) != 1:
    raise SystemExit("STOP: app initialization anchor did not match exactly once.")

source = source.replace(anchor, replacement, 1)
path.write_text(source)
print("Added authentication correlation-ID middleware.")
PY

git diff --check
```

```
Added authentication correlation-ID middleware.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
    with TestClient(app) as client:
        response = client.post(
            "/auth/login",
            json={
                "email": f"{uuid4().hex}@example.com",
                "password": "Synthetic-Invalid-Password",
            },
            headers={"X-Correlation-ID": str(correlation_id)},
        )

    assert response.status_code == 401
    assert response.headers.get("X-Correlation-ID") == str(correlation_id)

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """SELECT correlation_id
               FROM audit_events
               WHERE action_key = 'auth.login'
                 AND outcome = 'failure'
                 AND correlation_id = %s
               ORDER BY occurred_at DESC LIMIT 1""",
            (correlation_id,),
        )
        event = cur.fetchone()

    assert event is not None
    assert event[0] == correlation_id
PY

git diff --check

grep -n '^def test_' apps/api/tests/test_fin14_auth_security.py
14:def test_successful_login_is_audited_without_secrets(monkeypatch):
56:def test_failed_login_is_audited_and_repeated_failures_are_blocked(monkeypatch):
129:def test_logout_is_audited_without_recording_session_token(monkeypatch):
182:def test_auth_correlation_id_is_returned_and_audited(monkeypatch):
229:def test_invalid_auth_correlation_id_is_rejected(monkeypatch):
246:def test_failed_login_returns_the_audited_correlation_id(monkeypatch):
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest -vv apps/api/tests/test_fin14_auth_security.py
```

```
============================================================================
======== test session starts
============================================================================
========
platform darwin -- Python 3.14.6, pytest-9.0.3, pluggy-1.7.0 --
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI
configfile: pytest.ini
plugins: cov-6.2.1, anyio-4.15.1
collected 6 items

apps/api/tests/test_fin14_auth_security.py::test_successful_login_is_audited_without_secrets
PASSED                                                                   [ 16%]
apps/api/tests/test_fin14_auth_security.py::test_failed_login_is_audited_and_repeated_failures_
are_blocked PASSED                                                       [ 33%]
apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken PASSED                                                              [ 50%]
apps/api/tests/test_fin14_auth_security.py::test_auth_correlation_id_is_returned_and_audited
PASSED                                                                   [ 66%]
apps/api/tests/test_fin14_auth_security.py::test_invalid_auth_correlation_id_is_rejected
PASSED                                                                   [ 83%]
apps/api/tests/test_fin14_auth_security.py::test_failed_login_returns_the_audited_correlation_id
PASSED                                                                   [100%]

============================================================================
========= warnings summary
============================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]
```

```
apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
  return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================================================================
=== 6 passed, 3 warnings in 0.76s
================================================================================
===
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

plugins: cov-6.2.1, anyio-4.15.1
collected 6 items

apps/api/tests/test_fin14_auth_security.py::test_successful_login_is_audited_without_secrets
PASSED                                                                   [ 16%]
apps/api/tests/test_fin14_auth_security.py::test_failed_login_is_audited_and_repeated_failures_
are_blocked PASSED                                                       [ 33%]
apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken PASSED                                                              [ 50%]
apps/api/tests/test_fin14_auth_security.py::test_auth_correlation_id_is_returned_and_audited
PASSED                                                                   [ 66%]
apps/api/tests/test_fin14_auth_security.py::test_invalid_auth_correlation_id_is_rejected
PASSED                                                                   [ 83%]
apps/api/tests/test_fin14_auth_security.py::test_failed_login_returns_the_audited_correlation_id
PASSED                                                                   [100%]

================================================================================
========= warnings summary
================================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53
```

```
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-packages/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_token

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-packages/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on the client instance instead.
  return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================================================================
=== 6 passed, 3 warnings in 0.76s
============================================================================
===
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
sed -n '1,45p' apps/api/migrations/0003_auth_audit_and_login_throttling.sql

echo "=== Existing database constraints and triggers ==="
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
.venv/bin/python - <<'PY'
from app.database import connect

with connect() as conn, conn.cursor() as cur:
  cur.execute("""
    SELECT c.conname, c.contype, pg_get_constraintdef(c.oid)
    FROM pg_constraint c
    JOIN pg_class t ON t.oid = c.conrelid
    JOIN pg_namespace n ON n.oid = t.relnamespace
    WHERE n.nspname = 'public'
      AND t.relname = 'audit_events'
    ORDER BY c.conname
  """)
  print("Constraints:")
  for row in cur.fetchall():
    print(row)

  cur.execute("""
    SELECT tgname, pg_get_triggerdef(oid)
```

```
        FROM pg_trigger
        WHERE tgrelid = 'public.audit_events'::regclass
          AND NOT tgisinternal
        ORDER BY tgname
    """)
    print("Triggers:")
    for row in cur.fetchall():
        print(row)
PY
=== Migration files ===
total 24
-rw-r--r--@ 1 sandeeph  staff  1728 Oct 18 22:05 0001_auth_identity.sql
-rw-r--r--@ 1 sandeeph  staff  2580 Oct 19 11:20 0002_platform_authorization.sql
-rw-r--r--@ 1 sandeeph  staff  1738 Oct 21 15:02 0003_auth_audit_and_login_throttling.sql
=== Audit table definition ===
BEGIN;

-- Authentication/security audit trail.
-- workspace_id remains nullable because the current application schema
-- does not yet contain the workspaces table.
CREATE TABLE audit_events (
    audit_event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NULL,
    actor_user_id UUID NULL REFERENCES users(user_id) ON DELETE SET NULL,
    action_key VARCHAR(120) NOT NULL,
    resource_type VARCHAR(80) NOT NULL,
    resource_id UUID NULL,
    outcome VARCHAR(20) NOT NULL
        CHECK (outcome IN ('success', 'failure', 'denied')),
    correlation_id UUID NULL,
    details_redacted JSONB NOT NULL DEFAULT '{}'::jsonb,
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_events_occurred_at
    ON audit_events (occurred_at DESC);

CREATE INDEX idx_audit_events_actor_occurred
    ON audit_events (actor_user_id, occurred_at DESC);

CREATE INDEX idx_audit_events_action_occurred
    ON audit_events (action_key, occurred_at DESC);

-- Persistent, shared login-failure counters.
```

```
-- key_hash must be a keyed digest generated by the application;
-- raw email addresses, IP addresses, passwords and tokens do not belong here.
CREATE TABLE auth_login_throttle (
    scope_type VARCHAR(16) NOT NULL
        CHECK (scope_type IN ('account', 'ip')),
    key_hash CHAR(64) NOT NULL,
    failed_attempts INTEGER NOT NULL DEFAULT 0
        CHECK (failed_attempts >= 0),
    window_started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    blocked_until TIMESTAMPTZ NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (scope_type, key_hash)
);

CREATE INDEX idx_auth_login_throttle_blocked_until
    ON auth_login_throttle (blocked_until)
=== Existing database constraints and triggers ===
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'app'
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
(
  cd apps/api || exit 1
  DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
  ../../.venv/bin/python - <<'PY'
from app.database import connect

with connect() as conn, conn.cursor() as cur:
    cur.execute("""
        SELECT c.conname, c.contype, pg_get_constraintdef(c.oid)
        FROM pg_constraint c
        JOIN pg_class t ON t.oid = c.conrelid
        JOIN pg_namespace n ON n.oid = t.relnamespace
        WHERE n.nspname = 'public'
          AND t.relname = 'audit_events'
        ORDER BY c.conname
    """)
    print("Constraints:")
    for row in cur.fetchall():
        print(row)

    cur.execute("""
        SELECT tgname, pg_get_triggerdef(oid)
```

```
        FROM pg_trigger
        WHERE tgrelid = 'public.audit_events'::regclass
          AND NOT tgisinternal
        ORDER BY tgname
    """)
    print("Triggers:")
    for row in cur.fetchall():
        print(row)
PY
)
=== Audit table constraints and triggers ===
<frozen site>:101: RuntimeWarning: Unexpected value in sys.prefix, expected
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv, got
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/apps/api/../../.venv
<frozen site>:101: RuntimeWarning: Unexpected value in sys.exec_prefix, expected
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv, got
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/apps/api/../../.venv
Constraints:
('audit_events_actor_user_id_fkey', 'f', 'FOREIGN KEY (actor_user_id) REFERENCES
users(user_id) ON DELETE SET NULL')
('audit_events_outcome_check', 'c', "CHECK (((outcome)::text = ANY
((ARRAY['success'::character varying, 'failure'::character varying, 'denied'::character
varying])::text[])))")
('audit_events_pkey', 'p', 'PRIMARY KEY (audit_event_id)')
Triggers:
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
Triggers:
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

echo "=== User deletion and soft-delete references ==="
grep -R -n -E 'DELETE FROM users|UPDATE users
SET.*(deleted_at|is_active|status)|deleted_at|ON DELETE (SET NULL|RESTRICT)' \
  apps/api/app apps/api/migrations apps/api/tests \
  --exclude-dir=__pycache__ 2>/dev/null | head -100

echo "=== User table definition ==="
sed -n '1,100p' apps/api/migrations/0001_auth_identity.sql
=== User deletion and soft-delete references ===
apps/api/app/provision_user.py:36:                  account_status = 'active', updated_at = now(),
deleted_at = NULL""",
apps/api/app/main.py:138:                  AND u.deleted_at IS NULL
apps/api/app/main.py:212:                        AND deleted_at IS NULL""",
```

```
apps/api/migrations/0001_auth_identity.sql:21:    platform_role_id UUID REFERENCES
roles(role_id) ON DELETE SET NULL,
apps/api/migrations/0001_auth_identity.sql:27:    deleted_at TIMESTAMPTZ
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:9:    actor_user_id UUID NULL
REFERENCES users(user_id) ON DELETE SET NULL,
apps/api/tests/test_fin14_auth_security.py:53:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_fin14_auth_security.py:126:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_fin14_auth_security.py:179:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_fin14_auth_security.py:226:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py:37:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py:42:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py:125:            cur.execute("DELETE FROM users
WHERE user_id IN (%s, %s)", (admin_id, user_id))
apps/api/tests/test_identity_constraints.py:206:                cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py:229:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py:283:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_auth.py:32:            cur.execute("DELETE FROM users WHERE user_id =
%s", (user_id,))
=== User table definition ===
-- FIN-13 identity/session subset. Derived from Database/schema (1).sql.
-- Apply only to a new development/test database; this is not the full target schema.
BEGIN;

CREATE TABLE roles (
    role_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    role_key VARCHAR(50) NOT NULL UNIQUE,
    display_name VARCHAR(100) NOT NULL,
    scope VARCHAR(20) NOT NULL CHECK (scope IN ('platform','workspace')),
    description TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(320) NOT NULL,
```

```
    password_hash TEXT,
    display_name VARCHAR(160) NOT NULL,
    account_status VARCHAR(20) NOT NULL DEFAULT 'active'
        CHECK (account_status IN ('invited','active','locked','disabled','deleted')),
    platform_role_id UUID REFERENCES roles(role_id) ON DELETE SET NULL,
    email_verified_at TIMESTAMPTZ,
    mfa_required BOOLEAN NOT NULL DEFAULT false,
    last_login_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at TIMESTAMPTZ
);
CREATE UNIQUE INDEX uq_users_email_lower ON users (lower(email));

CREATE TABLE auth_sessions (
    session_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    refresh_token_hash TEXT UNIQUE,
    ip_address INET,
    user_agent TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_seen_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at TIMESTAMPTZ NOT NULL,
    revoked_at TIMESTAMPTZ,
    revoke_reason VARCHAR(80)
);
CREATE INDEX ix_auth_sessions_active ON auth_sessions(user_id, expires_at) WHERE
revoked_at IS NULL;

COMMIT;
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
COMMIT;
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

sed -n '1,250p' apps/api/tests/test_fin14_auth_security.py
from __future__ import annotations

from datetime import UTC, datetime
from uuid import uuid4

from fastapi.testclient import TestClient

from app.auth import hash_password
```

```
from app.throttle import make_throttle_key
from app.database import connect
from app.main import app


def test_successful_login_is_audited_without_secrets(monkeypatch):
    monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Password-Only"
    correlation = None

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """INSERT INTO users (email, password_hash, display_name, account_status)
               VALUES (%s, %s, %s, 'active') RETURNING user_id""",
            (email, hash_password(password), "FIN14 Audit Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with TestClient(app) as client:
            response = client.post(
                "/auth/login", json={"email": email, "password": password}
            )
        assert response.status_code == 200

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT action_key, outcome, actor_user_id, details_redacted::text
                   FROM audit_events
                   WHERE action_key = 'auth.login' AND actor_user_id = %s
                   ORDER BY occurred_at DESC LIMIT 1""",
                (user_id,),
            )
            event = cur.fetchone()

        assert event is not None
        assert event[0] == "auth.login"
        assert event[1] == "success"
        assert event[2] == user_id
        assert password not in event[3]
        assert response.cookies.get("finsight_session") not in event[3]
    finally:
```

```
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_failed_login_is_audited_and_repeated_failures_are_blocked(monkeypatch):
    monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
    monkeypatch.setenv("AUTH_LOGIN_MAX_FAILURES", "2")
    monkeypatch.setenv("AUTH_LOGIN_WINDOW_SECONDS", "600")
    monkeypatch.setenv("AUTH_LOGIN_BLOCK_SECONDS", "60")

    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Password-Only"
    test_started_at = datetime.now(UTC)
    ip_key = make_throttle_key(
        b"integration-test-secret-material-32-bytes",
        "ip",
        "testclient",
    )

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
        )
        cur.execute(
            """INSERT INTO users (email, password_hash, display_name, account_status)
               VALUES (%s, %s, %s, 'active') RETURNING user_id""",
            (email, hash_password(password), "FIN14 Throttle Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with TestClient(app) as client:
            first = client.post(
                "/auth/login", json={"email": email, "password": "wrong-password"}
            )
            second = client.post(
                "/auth/login", json={"email": email, "password": "wrong-password"}
            )
            blocked = client.post(
                "/auth/login", json={"email": email, "password": password}
            )
```

```
        assert first.status_code == 401
        assert second.status_code == 401
        assert blocked.status_code == 401
        assert first.json() == second.json() == blocked.json()

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT outcome, actor_user_id, details_redacted::text
                   FROM audit_events
                   WHERE action_key = 'auth.login'
                     AND details_redacted->>'reason' IN (
                         'invalid_credentials', 'throttled'
                     )
                   ORDER BY occurred_at DESC LIMIT 3"""
            )
            events = cur.fetchall()

        assert len(events) == 3
        assert sum(event[0] == "failure" for event in events) == 2
        assert sum(event[0] == "denied" for event in events) == 1
        assert all(
            "wrong-password" not in event[2]
            for event in events
        )
        assert any(event[1] == user_id for event in events)
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
                (ip_key,),
            )
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_logout_is_audited_without_recording_session_token(monkeypatch):
    monkeypatch.setenv(
        "AUTH_THROTTLE_SECRET",
        "integration-test-secret-material-32-bytes",
    )
    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Logout-Password"

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
```

```
            """INSERT INTO users (email, password_hash, display_name, account_status)
               VALUES (%s, %s, %s, 'active') RETURNING user_id""",
            (email, hash_password(password), "FIN14 Logout Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with TestClient(app) as client:
            login = client.post(
                "/auth/login",
                json={"email": email, "password": password},
            )
            assert login.status_code == 200
            session_token = login.cookies.get("finsight_session")
            assert session_token

            logout = client.post(
                "/auth/logout",
                cookies={"finsight_session": session_token},
            )
            assert logout.status_code == 204

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT outcome, actor_user_id, details_redacted::text
                   FROM audit_events
                   WHERE action_key = 'auth.logout'
                     AND actor_user_id = %s
                   ORDER BY occurred_at DESC LIMIT 1""",
                (user_id,),
            )
            event = cur.fetchone()

        assert event is not None
        assert event[0] == "success"
        assert event[1] == user_id
        assert session_token not in event[2]
        assert password not in event[2]
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_auth_correlation_id_is_returned_and_audited(monkeypatch):
```

```
monkeypatch.setenv(
    "AUTH_THROTTLE_SECRET",
    "integration-test-secret-material-32-bytes",
)
email = f"{uuid4().hex}@example.com"
password = "Synthetic-FIN14-Correlation-Password"
correlation_id = uuid4()

with connect() as conn, conn.cursor() as cur:
    cur.execute(
        """INSERT INTO users (email, password_hash, display_name, account_status)
           VALUES (%s, %s, %s, 'active') RETURNING user_id""",
        (email, hash_password(password), "FIN14 Correlation Test"),
    )
    user_id = cur.fetchone()[0]

try:
    with TestClient(app) as client:
        response = client.post(
            "/auth/login",
            json={"email": email, "password": password},
            headers={"X-Correlation-ID": str(correlation_id)},
        )

    assert response.status_code == 200
    assert response.headers.get("X-Correlation-ID") == str(correlation_id)

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """SELECT correlation_id
               FROM audit_events
               WHERE action_key = 'auth.login'
                 AND actor_user_id = %s
                 AND outcome = 'success'
               ORDER BY occurred_at DESC LIMIT 1""",
            (user_id,),
        )
        event = cur.fetchone()

    assert event is not None
    assert event[0] == correlation_id
finally:
    with connect() as conn, conn.cursor() as cur:
        cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
```

```
def test_invalid_auth_correlation_id_is_rejected(monkeypatch):
    monkeypatch.setenv(
        "AUTH_THROTTLE_SECRET",
        "integration-test-secret-material-32-bytes",
    )

    with TestClient(app) as client:
        response = client.post(
            "/auth/login",
            json={"email": "correlation-test@example.com", "password": "not-used"},
            headers={"X-Correlation-ID": "not-a-uuid"},
        )

    assert response.status_code == 400
    assert response.json() == {"detail": "Invalid X-Correlation-ID"}


def test_failed_login_returns_the_audited_correlation_id(monkeypatch):
    monkeypatch.setenv(
        "AUTH_THROTTLE_SECRET",
        "integration-test-secret-material-32-bytes",
    )
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
30-    finally:
apps/api/tests/test_auth.py-31-        with connect() as conn, conn.cursor() as cur:
apps/api/tests/test_auth.py:32:            cur.execute("DELETE FROM users WHERE user_id =
%s", (user_id,))
apps/api/tests/test_auth.py-33-
apps/api/tests/test_auth.py-34-
apps/api/tests/test_auth.py-35-@pytest.fixture
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

grep -n -B 5 -A 35 -E '_auth_correlation_id|auth_correlation_middleware|allow_headers' \
  apps/api/app/main.py
32-
33-app = FastAPI(title="FinSight AI API", version="0.2.0")
34-
35-
36-@app.middleware("http")
37:async def auth_correlation_middleware(request: Request, call_next):
```

```
    """Return a correlation ID on authentication responses, including errors."""
    if request.url.path not in {"/auth/login", "/auth/logout"}:
        return await call_next(request)

    supplied = request.headers.get("X-Correlation-ID", "").strip()
    try:
        correlation_id = UUID(supplied) if supplied else uuid4()
    except ValueError:
        from starlette.responses import JSONResponse

        return JSONResponse(
            status_code=400,
            content={"detail": "Invalid X-Correlation-ID"},
        )

    request.state.auth_correlation_id = correlation_id
    response = await call_next(request)
    response.headers["X-Correlation-ID"] = str(correlation_id)
    return response


web_origin = os.environ.get("WEB_ORIGIN", "http://localhost:3000")
app.add_middleware(
    CORSMiddleware,
    allow_origins=[web_origin],
    allow_credentials=True,
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)

SESSION_COOKIE = "finsight_session"
GENERIC_LOGIN_ERROR = "Email or password is incorrect."
DUMMY_PASSWORD_HASH = hash_password(secrets.token_urlsafe(32))


class LoginRequest(BaseModel):
    email: EmailStr
    password: str = Field(min_length=1, max_length=1024)


class UserView(BaseModel):
    user_id: UUID
    email: str
    display_name: str
```

```
82-
83-
84-def _ttl_minutes() -> int:
85-    try:
86-        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
87-        session_expiry(value)
88-        return value
89-    except ValueError as exc:
90-        raise HTTPException(status_code=500, detail="Session configuration is invalid") from
exc
91-
92-
93-def _secure_cookie() -> bool:
94-    configured = os.environ.get("COOKIE_SECURE")
95-    if configured is not None:
96-        return configured.lower() in {"1", "true", "yes"}
97-    return os.environ.get("APP_ENV", "development").lower() == "production"
98-
99-
100-def _check_origin(request: Request) -> None:
101-    origin = request.headers.get("origin")
102-    if origin and origin != web_origin:
103-        raise HTTPException(status_code=403, detail="Request origin is not allowed")
104-
105-
106:def _auth_correlation_id(request: Request) -> UUID:
107-    """Accept a valid UUID correlation ID or generate one for this auth request."""
108-    supplied = request.headers.get("X-Correlation-ID", "").strip()
109-    if not supplied:
110-        return uuid4()
111-    try:
112-        return UUID(supplied)
113-    except ValueError as exc:
114-        raise HTTPException(
115-            status_code=400,
116-            detail="Invalid X-Correlation-ID",
117-        ) from exc
118-
119-
120-def get_current_user(
121-    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
122-) -> dict[str, Any]:
123-    if not session_token:
```

```
124-        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Authentication required")
125-    try:
126-        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
127-            cur.execute(
128-                """SELECT u.user_id, u.email, u.display_name, r.role_key,
129-                          COALESCE(array_agg(DISTINCT p.permission_key)
130-                                   FILTER (WHERE p.permission_key IS NOT NULL),
ARRAY[]::varchar[]) AS permissions
131-                   FROM auth_sessions s JOIN users u ON u.user_id = s.user_id
132-                   LEFT JOIN roles r ON r.role_id = u.platform_role_id AND r.scope = 'platform'
133-                   LEFT JOIN role_permissions rp ON rp.role_id = r.role_id
134-                   LEFT JOIN permissions p ON p.permission_id = rp.permission_id
135-                   WHERE s.refresh_token_hash = %s AND s.revoked_at IS NULL
136-                     AND s.expires_at > %s AND u.account_status = 'active'
137-                     AND s.last_seen_at > %s - make_interval(mins => %s)
138-                     AND u.deleted_at IS NULL
139-                   GROUP BY u.user_id, u.email, u.display_name, r.role_key""",
140-                (token_digest(session_token), datetime.now(UTC), datetime.now(UTC),
_ttl_minutes()),
141-            )
--
165-
166-
167-@app.post("/auth/login", response_model=UserView)
168-def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
169-    _check_origin(request)
170:    correlation_id = _auth_correlation_id(request)
171-    response.headers['X-Correlation-ID'] = str(correlation_id)
172-    email = payload.email.strip().lower()
173-    try:
174-        config = load_throttle_config()
175-    except ValueError as exc:
176-        raise HTTPException(
177-            status_code=503,
178-            detail="Authentication service unavailable",
179-        ) from exc
180-
181-    client_ip = request.client.host if request.client else "unknown"
182-    account_key = make_throttle_key(config.secret, "account", email)
183-    ip_key = make_throttle_key(config.secret, "ip", client_ip)
184-    login_succeeded = False
185-    user = None
186-
```

```
187-    try:
188-        with connect() as conn:
189-            account_blocked = is_blocked(
190-                conn, scope_type="account", key_hash=account_key
191-            )
192-            ip_blocked = is_blocked(
193-                conn, scope_type="ip", key_hash=ip_key
194-            )
195-
196-            if account_blocked or ip_blocked:
197-                record_auth_event(
198-                    conn,
199-                    action_key="auth.login",
200-                    outcome="denied",
201-                    actor_user_id=None,
202-                    details={"reason": "throttled"},
203-                    correlation_id=correlation_id,
204-                )
205-            else:
--
312-    request: Request,
313-    response: Response,
314-    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
315-) -> Response:
316-    _check_origin(request)
317:    correlation_id = _auth_correlation_id(request)
318-    response.headers['X-Correlation-ID'] = str(correlation_id)
319-
320-    try:
321-        with connect() as conn:
322-            actor_user_id = None
323-
324-            if session_token:
325-                with conn.cursor() as cur:
326-                    cur.execute(
327-                        """UPDATE auth_sessions
328-                           SET revoked_at = %s, revoke_reason = 'logout'
329-                           WHERE refresh_token_hash = %s
330-                             AND revoked_at IS NULL
331-                           RETURNING user_id""",
332-                        (datetime.now(UTC), token_digest(session_token)),
333-                    )
334-                    row = cur.fetchone()
335-                    if row:
```

```
336-                    actor_user_id = row[0]
337-
338-            record_auth_event(
339-                conn,
340-                action_key="auth.logout",
341-                outcome="success" if actor_user_id else "denied",
342-                actor_user_id=actor_user_id,
343-                details={"reason": "logout"},
344-                correlation_id=correlation_id,
345-            )
346-    except psycopg.Error as exc:
347-        raise HTTPException(
348-            status_code=503,
349-            detail="Authentication service unavailable",
350-        ) from exc
351-
352-    response.delete_cookie(
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
    if existing is not None:
        return existing

    supplied = request.headers.get("X-Correlation-ID", "").strip()
    if supplied:
        try:
            correlation_id = UUID(supplied)
        except ValueError as exc:
            raise HTTPException(
                status_code=400,
                detail="Invalid X-Correlation-ID",
            ) from exc
    else:
        correlation_id = uuid4()

    request.state.auth_correlation_id = correlation_id
    return correlation_id
'''

if source.count(old_cors) != 1:
    raise SystemExit("STOP: expected CORS line not found exactly once; no changes made.")
if source.count(old_helper) != 1:
    raise SystemExit("STOP: expected correlation helper not found exactly once; no changes
made.")
```

```
source = source.replace(old_cors, new_cors, 1)
source = source.replace(old_helper, new_helper, 1)
path.write_text(source)
print("Updated CORS and correlation ID helper.")
PY

git diff --check
git diff -- apps/api/app/main.py
Updated CORS and correlation ID helper.
diff --git a/apps/api/app/main.py b/apps/api/app/main.py
index 5e0e618..aa34a51 100644
--- a/apps/api/app/main.py
+++ b/apps/api/app/main.py
@@ -7,7 +7,16 @@ import secrets
 from datetime import UTC, datetime
 from pathlib import Path
 from typing import Annotated, Any
-from uuid import UUID
+from uuid import UUID, uuid4
+
+from app.audit import record_auth_event
+from app.throttle import (
+    is_blocked,
+    load_throttle_config,
+    make_throttle_key,
+    record_failure,
+    reset_bucket,
+)

 import psycopg
 from dotenv import load_dotenv
@@ -22,13 +31,38 @@ from app.database import connect
 load_dotenv(Path(__file__).resolve().parents[3] / ".env")

 app = FastAPI(title="FinSight AI API", version="0.2.0")
+
+
+@app.middleware("http")
+async def auth_correlation_middleware(request: Request, call_next):
+    """Return a correlation ID on authentication responses, including errors."""
+    if request.url.path not in {"/auth/login", "/auth/logout"}:
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd apps/api || exit 1
```

```
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
../../.venv/bin/python -m pytest -q tests/test_fin14_auth_security.py
<frozen site>:101: RuntimeWarning: Unexpected value in sys.prefix, expected
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv, got
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/apps/api/../../.venv
<frozen site>:101: RuntimeWarning: Unexpected value in sys.exec_prefix, expected
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv, got
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/apps/api/../../.venv
.....
[100%]
=============================================================================
========= warnings summary
=============================================================================
==========
../../.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

../../.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
6 passed, 3 warnings in 0.78s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro api %

    assert event is not None
    assert event[0] == parsed_correlation_id
```

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest -q apps/api/tests/test_fin14_auth_security.py
.......
[100%]
============================================================================
========= warnings summary
============================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
7 passed, 3 warnings in 0.81s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %


(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest -q apps/api/tests
```

```
.....................................................................................
[100%]
=============================================================================
========= warnings summary
=============================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py: 4 warnings
apps/api/tests/test_auth_unit.py: 3 warnings
apps/api/tests/test_fin14_auth_security.py: 1 warning
apps/api/tests/test_identity_constraints.py: 9 warnings

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
86 passed, 19 warnings in 1.98s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
 passed, 19 warnings in 1.98s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

echo "=== Whitespace validation ==="
git diff --check

echo "=== Changed files ==="
git status --short
```

```
echo "=== FIN-14 tracking status ==="
grep -n -A 24 -B 3 'FIN-14' IMPLEMENTATION_STATUS.txt
grep -n -A 12 -B 3 'TC_AUTHSEC_001' SPRINT_1_TEST_EXECUTION_STATUS.txt
grep -n -A 12 -B 3 'TC_AUTHSEC_003' SPRINT_1_TEST_EXECUTION_STATUS.txt
=== Whitespace validation ===
=== Changed files ===
 M .env.example
 M FUNCTION_TEST_TRACEABILITY.txt
 M IMPLEMENTATION_STATUS.txt
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/tests/test_auth.py
 M conversation.txt
 M docs/release/RELEASE_README.md
 M scripts/package_release.py
?? apps/api/app/audit.py
?? apps/api/app/throttle.py
?? apps/api/migrations/0003_auth_audit_and_login_throttling.sql
?? apps/api/tests/test_audit_service.py
?? apps/api/tests/test_fin14_auth_security.py
?? apps/api/tests/test_throttle.py
=== FIN-14 tracking status ===
427-  remain outside this implementation.
428-- TC_SYS_001: NOT RUN / NOT ESTABLISHED by the API suite alone.
429-- Approved decision remains platform-level authorization only. Workspace roles,
430:  membership, and workspace tables are deferred. FIN-14 is not changed.
431-- Before marking FIN-15 complete, verify the actual route path in
432-  apps/api/app/main.py and its tests, align stale route references in the
433-  tracking records, and close the remaining story-level acceptance gaps.
434-
435-FIN-15 ROUTE RECONCILIATION — 2026-10-20
436-Verified against the current FastAPI route decorator and test request paths:
437-GET /api/v1/admin/authorization/capabilities
438-Older references to GET /platform/admin/capability above are historical
439-implementation notes and do not describe the current route.
440-No application code or migration was changed by this documentation update.
441-
442-FIN-15 — FRESH QUALITY VERIFICATION ADDENDUM (2026-10-21)
443-
444-Scope: FIN-15 / FS-01-02, platform-level authorization only.
445-Database: dedicated local PostgreSQL database finsight_fin15_test.
446-Migrations: 0001_auth_identity.sql and 0002_platform_authorization.sql
447-applied successfully to the dedicated test database.
```

448-
449-Observed verification results:
450-- Focused identity/authorization integration tests:
451- 5 passed, 0 failed.
452-- Full backend test suite:
453- 58 passed, 0 failed, 18 deprecation warnings.
454-- Full suite with statement and branch coverage:
--
477-Overall FIN-15 status: PARTIAL. Do not mark accepted or VERIFIED COMPLETE
478-until the remaining applicable acceptance criteria and evidence are
479-reconciled. Workspace roles, membership, and resource-level authorization
480:remain deferred. FIN-14 is outside this verification scope.
481-
482-No commit or push was performed as part of this verification.
483-
484-
485:FIN-14 / FS-01-03 — AUTHENTICATION SECURITY IMPLEMENTATION (2026-10-21)
486-Status: PARTIAL — acceptance gaps remain.
487-
488-Implemented:
489-- Added migration 0003_auth_audit_and_login_throttling.sql for persistent
490- authentication audit events and PostgreSQL-backed login throttle state.
491-- Added apps/api/app/audit.py for authentication audit insertion and
492- redaction of secret-like fields.
493-- Added apps/api/app/throttle.py for validated configuration, HMAC-derived
494- account/IP keys, persistent failure counting, and temporary blocking.
495-- Integrated login success/failure/denied audit events and throttle checks
496- into the login flow; logout revokes the presented active session and
497- records an audit event without storing the session token.
498-- Updated .env.example, release configuration metadata, and release README
499- for the throttle settings and migration.
500-- Added tests in test_audit_service.py, test_throttle.py, and
501- test_fin14_auth_security.py; extended authentication integration coverage.
502-
503-Verification evidence:
504-- Focused auth/security suite: 66 passed, 0 failed, 10 warnings.
505-- Full API suite: 82 passed, 0 failed, 19 warnings.
506-- git diff --check passed in the recorded verification.
507-- Local release package included migration 0003 and the new audit/throttle
508- modules; archive checksum verification passed.
509-- Test database: finsight_fin15_test. Synthetic test records were cleaned up
--
517-- TC_AUTHSEC_003 remains BLOCKED: MFA policy/provider and challenge flow are
518- not implemented; inspection and termination of other active sessions are

519- also absent. Current-session logout does not satisfy cross-session controls.
520:- FIN-14 must not be marked accepted or VERIFIED COMPLETE until the relevant
521- acceptance criteria are implemented, tested, and reconciled.
522-
523-No source requirements, reference SQL/schema, diagrams, or original test plan
524-were modified. No commit or push was performed.
94-TC_AUTHZ_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
95-TC_AUTHZ_002 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
96-TC_AUTHZ_003 | Status: PASS | Execution date: 2026-10-18 | Actual result: Stale session rejected under configured TTL | Tester/owner: Codex | Evidence: apps/api/tests/test_auth.py | Linked defect: —
97:TC_AUTHSEC_001 | Status: PARTIAL | Execution date: 2026-10-21 | Actual result: Successful login, failed login, and logout auditing plus secret-redaction tests passed; end-to-endcorrelation propagation and append-only enforcement remain unverified | Tester/owner: Codex | Evidence: apps/api/tests/test_fin14_auth_security.py; apps/api/tests/test_audit_service.py | Linked defect: FIN-14 acceptance gaps
98-TC_AUTHSEC_002 | Status: PARTIAL | Execution date: 2026-10-21 | Actual result: Configurable database-backed throttling, repeated-failure blocking, configuration validation, and throttle expiry behavior are tested; the complete login-level expiry-and-successful-retry flow is not established | Tester/owner: Codex | Evidence: apps/api/tests/test_throttle.py; apps/api/tests/test_fin14_auth_security.py; PostgreSQL throttle verification recorded in IMPLEMENTATION_STATUS.txt | Linked defect: FIN-14 acceptance gap
99-TC_AUTHSEC_003 | Status: BLOCKED | Execution date: 2026-10-21 | Actual result: MFA challenge enforcement and inspection/termination of other active sessions are not implemented;current-session logout is not equivalent | Tester/owner: Codex | Evidence: apps/api/app/main.py; apps/api/tests/test_auth.py | Linked defect: MFA policy/provider decision and cross-session management
100-TC_AUTHSEC_004 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
101-TC_AUTHN_006 | Status: PASS | Execution date: 2026-10-18 | Actual result: Disposable synthetic database dump contained password/session digests; plaintext password/token absent; API test checked captured logs/stdout/stderr and response | Tester/owner: Codex | Evidence: apps/api/tests/test_auth.py; disposable scan summarized in IMPLEMENTATION_STATUS.txt (dump removed) | Linked defect: —
102-TC_AUTHN_007 | Status: BLOCKED | Execution date: 2026-10-18 | Actual result: Local cookie flags checked; deployed HTTPS verification unavailable | Tester/owner: Codex | Evidence: apps/api/tests/test_auth.py | Linked defect: —
103-TC_AUTHN_008 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
104-TC_AUTHN_009 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —

105-TC_LLM_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
106-TC_LLM_002 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
107-TC_LLM_003 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
108-TC_CFG_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
109-TC_CFG_002 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
--
220-- Warnings were reported as test-client deprecations; no test failures occurred.
221-
222-Acceptance-case reconciliation:
223:- TC_AUTHSEC_001: PARTIAL. Login success/failure and logout audit events,
224- plus secret-redaction behavior, have tests. Complete correlation propagation
225- and database-level append-only enforcement have not been established.
226-- TC_AUTHSEC_002: PARTIAL. Configurable PostgreSQL-backed throttling and
227- repeated-failure blocking are implemented. Throttle behavior/expiry has
228- been tested, but the complete login-level expiry-and-successful-retry flow
229- is not established by the recorded integration tests.
230-- TC_AUTHSEC_003: BLOCKED. MFA enforcement and listing/terminating other
231- active sessions are not implemented. Existing logout only revokes the
232- session presented to that request.
233-
234-Overall FIN-14 / FS-01-03 status: PARTIAL, not accepted or VERIFIED COMPLETE.
235-No source requirements, schema reference documents, diagrams, or original test
96-TC_AUTHZ_003 | Status: PASS | Execution date: 2026-10-18 | Actual result: Stale session rejected under configured TTL | Tester/owner: Codex | Evidence: apps/api/tests/test_auth.py | Linked defect: —
97-TC_AUTHSEC_001 | Status: PARTIAL | Execution date: 2026-10-21 | Actual result: Successful login, failed login, and logout auditing plus secret-redaction tests passed; end-to-endcorrelation propagation and append-only enforcement remain unverified | Tester/owner: Codex | Evidence: apps/api/tests/test_fin14_auth_security.py; apps/api/tests/test_audit_service.py | Linked defect: FIN-14 acceptance gaps
98-TC_AUTHSEC_002 | Status: PARTIAL | Execution date: 2026-10-21 | Actual result: Configurable database-backed throttling, repeated-failure blocking, configuration validation, and throttle expiry behavior are tested; the complete login-level expiry-and-successful-retry flow is not established | Tester/owner: Codex | Evidence: apps/api/tests/test_throttle.py; apps/api/tests/test_fin14_auth_security.py; PostgreSQL throttle verification recorded in IMPLEMENTATION_STATUS.txt | Linked defect: FIN-14 acceptance gap
99:TC_AUTHSEC_003 | Status: BLOCKED | Execution date: 2026-10-21 | Actual result: MFA challenge enforcement and inspection/termination of other active sessions are not implemented;current-session logout is not equivalent | Tester/owner: Codex | Evidence:

apps/api/app/main.py; apps/api/tests/test_auth.py | Linked defect: MFA policy/provider decision and cross-session management
100-TC_AUTHSEC_004 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
101-TC_AUTHN_006 | Status: PASS | Execution date: 2026-10-18 | Actual result: Disposable synthetic database dump contained password/session digests; plaintext password/token absent; API test checked captured logs/stdout/stderr and response | Tester/owner: Codex | Evidence: apps/api/tests/test_auth.py; disposable scan summarized in IMPLEMENTATION_STATUS.txt (dump removed) | Linked defect: —
102-TC_AUTHN_007 | Status: BLOCKED | Execution date: 2026-10-18 | Actual result: Local cookie flags checked; deployed HTTPS verification unavailable | Tester/owner: Codex | Evidence: apps/api/tests/test_auth.py | Linked defect: —
103-TC_AUTHN_008 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
104-TC_AUTHN_009 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
105-TC_LLM_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
106-TC_LLM_002 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
107-TC_LLM_003 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
108-TC_CFG_001 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
109-TC_CFG_002 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
110-TC_CFG_003 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
111-TC_CFG_004 | Status: NOT RUN | Execution date: — | Actual result: — | Tester/owner: — | Evidence: — | Linked defect: —
--
227- repeated-failure blocking are implemented. Throttle behavior/expiry has
228- been tested, but the complete login-level expiry-and-successful-retry flow
229- is not established by the recorded integration tests.
230:- TC_AUTHSEC_003: BLOCKED. MFA enforcement and listing/terminating other
231- active sessions are not implemented. Existing logout only revokes the
232- session presented to that request.
233-
234-Overall FIN-14 / FS-01-03 status: PARTIAL, not accepted or VERIFIED COMPLETE.
235-No source requirements, schema reference documents, diagrams, or original test
236-plan were changed. No commit or push was performed.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....

```
updates = {
    "IMPLEMENTATION_STATUS.txt": """\n\nFIN-14 — CORRELATION REGRESSION AND FULL API VERIFICATION (2026-10-21)\n\nLatest verification:\n- Focused FIN-14 security tests: 7 passed, 0 failed, 3 warnings.\n- Full API test suite: 86 passed, 0 failed, 19 warnings.\n- Verified supplied correlation IDs are returned and stored in audit events.\n- Added a regression test verifying generated correlation IDs match the audit record when the caller omits the header.\n- CORS configuration now permits the X-Correlation-ID request header.\n- git diff --check passed.\n\nRemaining gaps:\n- Database-level append-only enforcement is not implemented or verified.\n- The complete login-level throttle expiry-and-successful-retry flow remains unestablished.\n- MFA policy/provider and challenge enforcement remain unresolved.\n- Listing and terminating other active sessions remain unimplemented.\n\nOverall FIN-14 status remains PARTIAL. No commit or push was performed.\n""",
    "SPRINT_1_TEST_EXECUTION_STATUS.txt": """\n\nFIN-14 — LATEST VERIFICATION ADDENDUM (2026-10-21)\n- Focused security suite: 7 passed, 0 failed, 3 warnings.\n- Full API suite: 86 passed, 0 failed, 19 warnings.\n- Generated and caller-supplied correlation IDs match their audit records.\n- TC_AUTHSEC_001 remains PARTIAL because database-level append-only enforcement is unverified.\n- TC_AUTHSEC_002 remains PARTIAL because the complete login-level expiry-and-retry flow is unestablished.\n- TC_AUTHSEC_003 remains BLOCKED pending MFA and cross-session inspection/termination.\n""",
    "FUNCTION_TEST_TRACEABILITY.txt": """\n\nFIN-14 — LATEST VERIFICATION ADDENDUM (2026-10-21)\n- Focused security suite: 7 passed, 0 failed, 3 warnings.\n- Full API suite: 86 passed, 0 failed, 19 warnings.\n- Supplied and generated correlation-ID propagation is verified by integration tests.\n- TC_AUTHSEC_001 remains PARTIAL: append-only enforcement remains unverified.\n- TC_AUTHSEC_002 remains PARTIAL: end-to-end throttle expiry and successful retry remain unestablished.\n- TC_AUTHSEC_003 remains BLOCKED: MFA and cross-session inspection/termination remain unimplemented.\n""",
    "conversation.txt": """\n\nFIN-14 verification update — 2026-10-21\n- Full API suite: 86 passed, 0 failed, 19 warnings.\n- Focused FIN-14 security suite: 7 passed, 0 failed, 3 warnings.\n- Generated correlation-ID/audit consistency regression test passes.\n- FIN-14 remains partial; append-only enforcement, complete throttle expiry/retry acceptance, MFA, and cross-session controls remain open.\n- No commit or push performed.\n""",
}

for filename, addition in updates.items():
    path = root / filename
    text = path.read_text()
    if marker in text:
        print(f"Skipped duplicate addendum in {filename}")
        continue
    path.write_text(text.rstrip() + addition)
    print(f"Appended verification addendum to {filename}")
PY

git diff --check
```

```
git status --short
STOP: expected text not found in SPRINT_1_TEST_EXECUTION_STATUS.txt:
end-to-endcorrelation propagation and append-only enforcement remain unverified
 M .env.example
 M FUNCTION_TEST_TRACEABILITY.txt
 M IMPLEMENTATION_STATUS.txt
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/tests/test_auth.py
 M conversation.txt
 M docs/release/RELEASE_README.md
 M scripts/package_release.py
?? apps/api/app/audit.py
?? apps/api/app/throttle.py
?? apps/api/migrations/0003_auth_audit_and_login_throttling.sql
?? apps/api/tests/test_audit_service.py
?? apps/api/tests/test_fin14_auth_security.py
?? apps/api/tests/test_throttle.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

echo "=== Current security test status ==="
grep -n -E 'TC_AUTHSEC_001|TC_AUTHSEC_002|TC_AUTHSEC_003|correlation|82
passed|86 passed' \
  SPRINT_1_TEST_EXECUTION_STATUS.txt

echo "=== Current traceability status ==="
grep -n -E 'TC_AUTHSEC_001|TC_AUTHSEC_002|TC_AUTHSEC_003|correlation|82
passed|86 passed' \
  FUNCTION_TEST_TRACEABILITY.txt

echo "=== Existing latest verification addenda ==="
grep -n -E 'LATEST VERIFICATION ADDENDUM|CORRELATION REGRESSION AND FULL
API VERIFICATION|FIN-14 verification update' \
  IMPLEMENTATION_STATUS.txt SPRINT_1_TEST_EXECUTION_STATUS.txt \
  FUNCTION_TEST_TRACEABILITY.txt conversation.txt
=== Current security test status ===
78:TC_SYS_001 | Status: BLOCKED | Execution date: 2026-10-18 | Actual result: Browser
smoke verified current sign-in/session/logout path against live frontend, FastAPI, and
PostgreSQL. Full planned dashboard/settings, dependency health, Ollama
available/unavailable, and correlation-log workflow cannot run because those components are
not implemented | Tester/owner: Codex | Evidence: apps/web/tests/e2e/auth.spec.js;
```

scripts/run_live_e2e.py; IMPLEMENTATION_STATUS.txt | Linked defect: Current application implements auth starter only
97:TC_AUTHSEC_001 | Status: PARTIAL | Execution date: 2026-10-21 | Actual result: Successful login, failed login, and logout auditing plus secret-redaction tests passed; end-to-endcorrelation propagation and append-only enforcement remain unverified | Tester/owner: Codex | Evidence: apps/api/tests/test_fin14_auth_security.py; apps/api/tests/test_audit_service.py | Linked defect: FIN-14 acceptance gaps
98:TC_AUTHSEC_002 | Status: PARTIAL | Execution date: 2026-10-21 | Actual result: Configurable database-backed throttling, repeated-failure blocking, configuration validation, and throttle expiry behavior are tested; the complete login-level expiry-and-successful-retry flow is not established | Tester/owner: Codex | Evidence: apps/api/tests/test_throttle.py; apps/api/tests/test_fin14_auth_security.py; PostgreSQL throttle verification recorded in IMPLEMENTATION_STATUS.txt | Linked defect: FIN-14 acceptance gap
99:TC_AUTHSEC_003 | Status: BLOCKED | Execution date: 2026-10-21 | Actual result: MFA challenge enforcement and inspection/termination of other active sessions are not implemented;current-session logout is not equivalent | Tester/owner: Codex | Evidence: apps/api/app/main.py; apps/api/tests/test_auth.py | Linked defect: MFA policy/provider decision and cross-session management
217:- Full API suite: 82 passed, 0 failed, 19 warnings.
223:- TC_AUTHSEC_001: PARTIAL. Login success/failure and logout audit events,
224: plus secret-redaction behavior, have tests. Complete correlation propagation
226:- TC_AUTHSEC_002: PARTIAL. Configurable PostgreSQL-backed throttling and
230:- TC_AUTHSEC_003: BLOCKED. MFA enforcement and listing/terminating other
=== Current traceability status ===
99:- TC_AUTHSEC_001 — PARTIAL.
103: correlation propagation and append-only enforcement remain unverified.
105:- TC_AUTHSEC_002 — PARTIAL.
111:- TC_AUTHSEC_003 — BLOCKED.
117:full API suite 82 passed. Passing test counts do not establish complete
=== Existing latest verification addenda ===
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
=== Existing latest verification addenda ===
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
- TC_AUTHSEC_003 remains BLOCKED: MFA and cross-session controls remain unimplemented.
"""
    ,
    "conversation.txt": """
FIN-14 verification update — 2026-10-21
- Focused FIN-14 security suite: 7 passed, 0 failed, 3 warnings.
- Full API suite: 86 passed, 0 failed, 19 warnings.
- Generated correlation-ID/audit consistency regression test passes.
- FIN-14 remains partial; append-only enforcement, complete throttle expiry/retry acceptance, MFA, and cross-session controls remain open.

```
- No commit or push performed.
""",
}

marker = "LATEST CORRELATION AND API VERIFICATION (2026-10-21)"
for filename, addition in addenda.items():
    path = Path(filename)
    text = path.read_text()
    if marker in text:
        print(f"Skipped duplicate addendum in {filename}")
        continue
    path.write_text(text.rstrip() + "\n\n" + addition.strip() + "\n")
    print(f"Appended latest verification to {filename}")
PY

git diff --check

echo "=== Updated verification references ==="
grep -n -E 'LATEST CORRELATION AND API VERIFICATION|LATEST VERIFICATION
ADDENDUM|86 passed|correlation propagation for supplied' \
  IMPLEMENTATION_STATUS.txt SPRINT_1_TEST_EXECUTION_STATUS.txt \
  FUNCTION_TEST_TRACEABILITY.txt conversation.txt

echo "=== Working tree ==="
git status --short
STOP: expected status wording not found in SPRINT_1_TEST_EXECUTION_STATUS.txt
=== Updated verification references ===
=== Working tree ===
 M .env.example
 M FUNCTION_TEST_TRACEABILITY.txt
 M IMPLEMENTATION_STATUS.txt
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/tests/test_auth.py
 M conversation.txt
 M docs/release/RELEASE_README.md
 M scripts/package_release.py
?? apps/api/app/audit.py
?? apps/api/app/throttle.py
?? apps/api/migrations/0003_auth_audit_and_login_throttling.sql
?? apps/api/tests/test_audit_service.py
?? apps/api/tests/test_fin14_auth_security.py
?? apps/api/tests/test_throttle.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
```

```
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
- TC_AUTHSEC_001 remains PARTIAL: database-level append-only enforcement is unverified.
- TC_AUTHSEC_002 remains PARTIAL: complete login-level expiry and successful retry are
unestablished.
- TC_AUTHSEC_003 remains BLOCKED: MFA and cross-session inspection/termination are
not implemented.""",
    "FUNCTION_TEST_TRACEABILITY.txt": """FIN-14 — LATEST VERIFICATION ADDENDUM
(2026-10-21)

- Focused FIN-14 security suite: 7 passed, 0 failed, 3 warnings.
- Full API suite: 86 passed, 0 failed, 19 warnings.
- Supplied and generated correlation ID propagation is covered by integration tests.
- TC_AUTHSEC_001 remains PARTIAL: append-only enforcement is unverified.
- TC_AUTHSEC_002 remains PARTIAL: complete login-level expiry and successful retry are
unestablished.
- TC_AUTHSEC_003 remains BLOCKED: MFA and cross-session controls are not
implemented.""",
    "conversation.txt": """FIN-14 verification update — 2026-10-21

- Focused FIN-14 security suite: 7 passed, 0 failed, 3 warnings.
- Full API suite: 86 passed, 0 failed, 19 warnings.
- Generated correlation-ID/audit consistency regression test passes.
- FIN-14 remains partial; append-only enforcement, complete throttle expiry/retry acceptance,
MFA, and cross-session controls remain open.
- No commit or push performed."""
}

for filename, addition in updates.items():
    path = Path(filename)
    text = path.read_text()
    marker = addition.splitlines()[0]
    if marker in text:
        print(f"Already present; skipped: {filename}")
        continue
    path.write_text(text.rstrip() + "\n\n" + addition + "\n")
    print(f"Appended: {filename}")
PY

git diff --check
Appended: IMPLEMENTATION_STATUS.txt
Appended: SPRINT_1_TEST_EXECUTION_STATUS.txt
Appended: FUNCTION_TEST_TRACEABILITY.txt
Appended: conversation.txt
```

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % grep -n -E 'LATEST
CORRELATION AND API VERIFICATION|LATEST VERIFICATION ADDENDUM|FIN-14
verification update|86 passed' \
  IMPLEMENTATION_STATUS.txt SPRINT_1_TEST_EXECUTION_STATUS.txt \
  FUNCTION_TEST_TRACEABILITY.txt conversation.txt
IMPLEMENTATION_STATUS.txt:526:FIN-14 — LATEST CORRELATION AND API
VERIFICATION (2026-10-21)
IMPLEMENTATION_STATUS.txt:529:- Full API suite: 86 passed, 0 failed, 19 warnings.
SPRINT_1_TEST_EXECUTION_STATUS.txt:238:FIN-14 — LATEST VERIFICATION
ADDENDUM (2026-10-21)
SPRINT_1_TEST_EXECUTION_STATUS.txt:241:- Full API suite: 86 passed, 0 failed, 19
warnings.
FUNCTION_TEST_TRACEABILITY.txt:120:FIN-14 — LATEST VERIFICATION ADDENDUM
(2026-10-21)
FUNCTION_TEST_TRACEABILITY.txt:123:- Full API suite: 86 passed, 0 failed, 19 warnings.
conversation.txt:151:FIN-14 verification update — 2026-10-21
conversation.txt:154:- Full API suite: 86 passed, 0 failed, 19 warnings.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
conversation.txt:154:- Full API suite: 86 passed, 0 failed, 19 warnings.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

echo "=== Audit service ==="
sed -n '1,260p' apps/api/app/audit.py

echo "=== Existing audit and throttle migration ==="
sed -n '1,220p' apps/api/migrations/0003_auth_audit_and_login_throttling.sql

echo "=== Database connection and transaction handling ==="
sed -n '1,220p' apps/api/app/database.py

echo "=== Audit service tests ==="
sed -n '1,280p' apps/api/tests/test_audit_service.py

echo "=== Current user-deletion cleanup patterns ==="
grep -R -n -B 3 -A 4 'DELETE FROM users' \
  apps/api/tests --exclude-dir=__pycache__ 2>/dev/null | head -100

echo "=== Migration execution mechanism ==="
grep -R -n -E 'migrations/|glob.*migration|execute.*sql|migration' \
  apps/api scripts --exclude-dir=__pycache__ 2>/dev/null | head -100
=== Audit service ===
"""Reusable, redacted audit persistence for authentication security events."""
```

```
from __future__ import annotations

import json
import re
from datetime import UTC, datetime
from typing import Any
from uuid import UUID

_ALLOWED_OUTCOMES = frozenset({"success", "failure", "denied"})
_SECRET_KEY = re.compile(
    r"(password|passwd|token|cookie|authorization|api[_-]?key|secret|credential)",
    re.IGNORECASE,
)


def _redact(value: Any) -> Any:
    """Return JSON-safe audit details without values under secret-like keys."""
    if isinstance(value, dict):
        return {
            str(key): (
                "[REDACTED]" if _SECRET_KEY.search(str(key)) else _redact(item)
            )
            for key, item in value.items()
        }
    if isinstance(value, (list, tuple)):
        return [_redact(item) for item in value]
    if isinstance(value, UUID):
        return str(value)
    if isinstance(value, datetime):
        return value.astimezone(UTC).isoformat()
    if value is None or isinstance(value, (str, int, float, bool)):
        return value
    return str(value)


def record_auth_event(
    conn: Any,
    *,
    action_key: str,
    outcome: str,
    actor_user_id: UUID | None,
    details: dict[str, Any] | None = None,
    correlation_id: UUID | None = None,
    occurred_at: datetime | None = None,
```

```
) -> None:
    """Persist one authentication audit event on the caller's transaction.

    Persistence errors intentionally propagate to the caller. The caller must
    roll back and apply the documented fail-closed policy; events are never
    reported as recorded when this insert fails.
    """
    if outcome not in _ALLOWED_OUTCOMES:
        raise ValueError("Unsupported audit outcome")
    if not action_key or len(action_key) > 120:
        raise ValueError("action_key must contain 1 to 120 characters")

    redacted_details = _redact(details or {})
    payload = json.dumps(redacted_details, separators=(",", ":"), sort_keys=True)

    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO audit_events (
                workspace_id,
                actor_user_id,
                action_key,
                resource_type,
                resource_id,
                outcome,
                correlation_id,
                details_redacted,
                occurred_at
            )
            VALUES (NULL, %s, %s, 'authentication', NULL, %s, %s, %s::jsonb, %s)
            """,
            (
                actor_user_id,
                action_key,
                outcome,
                correlation_id,
                payload,
                occurred_at or datetime.now(UTC),
            ),
        )
=== Existing audit and throttle migration ===
BEGIN;

-- Authentication/security audit trail.
```

```
-- workspace_id remains nullable because the current application schema
-- does not yet contain the workspaces table.
CREATE TABLE audit_events (
    audit_event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NULL,
    actor_user_id UUID NULL REFERENCES users(user_id) ON DELETE SET NULL,
    action_key VARCHAR(120) NOT NULL,
    resource_type VARCHAR(80) NOT NULL,
    resource_id UUID NULL,
    outcome VARCHAR(20) NOT NULL
        CHECK (outcome IN ('success', 'failure', 'denied')),
    correlation_id UUID NULL,
    details_redacted JSONB NOT NULL DEFAULT '{}'::jsonb,
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_events_occurred_at
    ON audit_events (occurred_at DESC);

CREATE INDEX idx_audit_events_actor_occurred
    ON audit_events (actor_user_id, occurred_at DESC);

CREATE INDEX idx_audit_events_action_occurred
    ON audit_events (action_key, occurred_at DESC);

-- Persistent, shared login-failure counters.
-- key_hash must be a keyed digest generated by the application;
-- raw email addresses, IP addresses, passwords and tokens do not belong here.
CREATE TABLE auth_login_throttle (
    scope_type VARCHAR(16) NOT NULL
        CHECK (scope_type IN ('account', 'ip')),
    key_hash CHAR(64) NOT NULL,
    failed_attempts INTEGER NOT NULL DEFAULT 0
        CHECK (failed_attempts >= 0),
    window_started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    blocked_until TIMESTAMPTZ NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (scope_type, key_hash)
);

CREATE INDEX idx_auth_login_throttle_blocked_until
    ON auth_login_throttle (blocked_until)
    WHERE blocked_until IS NOT NULL;
```

```
COMMIT;
=== Database connection and transaction handling ===
"""Small synchronous PostgreSQL boundary for the auth endpoints."""

from __future__ import annotations

import os
from pathlib import Path

import psycopg
from dotenv import load_dotenv

load_dotenv(Path(__file__).resolve().parents[3] / ".env")


def connect() -> psycopg.Connection:
    database_url = os.environ.get("DATABASE_URL")
    if not database_url:
        raise RuntimeError("DATABASE_URL is required")
    return psycopg.connect(database_url, connect_timeout=3)
=== Audit service tests ===
from __future__ import annotations

from datetime import UTC, datetime
from uuid import uuid4

import pytest

from app.audit import record_auth_event


class FakeCursor:
    def __init__(self):
        self.calls = []

    def execute(self, query, params):
        self.calls.append((query, params))

    def __enter__(self):
        return self

    def __exit__(self, *args):
        return False
```

```
class FakeConnection:
    def __init__(self):
        self.cursor_instance = FakeCursor()

    def cursor(self):
        return self.cursor_instance


def test_auth_audit_uses_canonical_columns_and_parameterized_sql():
    conn = FakeConnection()
    actor_id = uuid4()
    occurred_at = datetime(2026, 10, 21, tzinfo=UTC)

    record_auth_event(
        conn,
        action_key="auth.login",
        outcome="success",
        actor_user_id=actor_id,
        occurred_at=occurred_at,
        details={"method": "password"},
    )

    assert len(conn.cursor_instance.calls) == 1
    query, params = conn.cursor_instance.calls[0]

    assert "INSERT INTO audit_events" in query
    for column in (
        "workspace_id",
        "actor_user_id",
        "action_key",
        "resource_type",
        "resource_id",
        "outcome",
        "correlation_id",
        "details_redacted",
        "occurred_at",
    ):
        assert column in query

    assert actor_id in params
    assert "auth.login" in params
    assert "success" in params
    assert occurred_at in params
```

```
    assert "password" not in query.lower()


def test_unknown_account_failure_can_be_audited_without_an_actor():
    conn = FakeConnection()

    record_auth_event(
        conn,
        action_key="auth.login",
        outcome="failure",
        actor_user_id=None,
        details={"reason": "invalid_credentials"},
    )

    _, params = conn.cursor_instance.calls[0]
    assert None in params
    assert "invalid_credentials" in str(params)


@pytest.mark.parametrize(
    "secret",
    [
        "raw-password-value",
        "raw-session-token-value",
        "raw-api-key-value",
        "cookie-secret-value",
    ],
)
def test_auth_audit_redacts_secret_values(secret):
    conn = FakeConnection()

    record_auth_event(
        conn,
        action_key="auth.login",
        outcome="failure",
        actor_user_id=None,
        details={
            "reason": "invalid_credentials",
            "password": secret,
            "session_token": secret,
            "api_key": secret,
            "cookie": secret,
        },
    )
```

```
    query, params = conn.cursor_instance.calls[0]
    persisted_values = f"{query} {params}"

    assert secret not in persisted_values
    assert "invalid_credentials" in persisted_values


def test_auth_audit_rejects_unknown_outcome_before_database_write():
    conn = FakeConnection()

    with pytest.raises(ValueError, match="outcome"):
        record_auth_event(
            conn,
            action_key="auth.login",
            outcome="maybe",
            actor_user_id=None,
        )

    assert conn.cursor_instance.calls == []
=== Current user-deletion cleanup patterns ===
apps/api/tests/test_fin14_auth_security.py-50-        assert
response.cookies.get("finsight_session") not in event[3]
apps/api/tests/test_fin14_auth_security.py-51-    finally:
apps/api/tests/test_fin14_auth_security.py-52-        with connect() as conn, conn.cursor() as cur:
apps/api/tests/test_fin14_auth_security.py:53:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_fin14_auth_security.py-54-
apps/api/tests/test_fin14_auth_security.py-55-
apps/api/tests/test_fin14_auth_security.py-56-def
test_failed_login_is_audited_and_repeated_failures_are_blocked(monkeypatch):
apps/api/tests/test_fin14_auth_security.py-57-
monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
--
apps/api/tests/test_fin14_auth_security.py-123-            "DELETE FROM auth_login_throttle
WHERE scope_type = 'ip' AND key_hash = %s",
apps/api/tests/test_fin14_auth_security.py-124-            (ip_key,),
apps/api/tests/test_fin14_auth_security.py-125-        )
apps/api/tests/test_fin14_auth_security.py:126:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_fin14_auth_security.py-127-
apps/api/tests/test_fin14_auth_security.py-128-
```

```
apps/api/tests/test_fin14_auth_security.py-129-def
test_logout_is_audited_without_recording_session_token(monkeypatch):
apps/api/tests/test_fin14_auth_security.py-130-    monkeypatch.setenv(
--
apps/api/tests/test_fin14_auth_security.py-176-        assert password not in event[2]
apps/api/tests/test_fin14_auth_security.py-177-    finally:
apps/api/tests/test_fin14_auth_security.py-178-        with connect() as conn, conn.cursor() as
cur:
apps/api/tests/test_fin14_auth_security.py:179:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_fin14_auth_security.py-180-
apps/api/tests/test_fin14_auth_security.py-181-
apps/api/tests/test_fin14_auth_security.py-182-def
test_auth_correlation_id_is_returned_and_audited(monkeypatch):
apps/api/tests/test_fin14_auth_security.py-183-    monkeypatch.setenv(
--
apps/api/tests/test_fin14_auth_security.py-223-        assert event[0] == correlation_id
apps/api/tests/test_fin14_auth_security.py-224-    finally:
apps/api/tests/test_fin14_auth_security.py-225-        with connect() as conn, conn.cursor() as
cur:
apps/api/tests/test_fin14_auth_security.py:226:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_fin14_auth_security.py-227-
apps/api/tests/test_fin14_auth_security.py-228-
apps/api/tests/test_fin14_auth_security.py-229-def
test_invalid_auth_correlation_id_is_rejected(monkeypatch):
apps/api/tests/test_fin14_auth_security.py-230-    monkeypatch.setenv(
--
apps/api/tests/test_identity_constraints.py-34-                "INSERT INTO auth_sessions (user_id,
refresh_token_hash, expires_at) VALUES (%s, %s, %s)",
apps/api/tests/test_identity_constraints.py-35-                (user_id, digest, session_expiry(5)),
apps/api/tests/test_identity_constraints.py-36-            )
apps/api/tests/test_identity_constraints.py:37:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py-38-            cur.execute("SELECT count(*) FROM
auth_sessions WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py-39-            assert cur.fetchone()[0] == 0
apps/api/tests/test_identity_constraints.py-40-    finally:
apps/api/tests/test_identity_constraints.py-41-        with connect() as conn, conn.cursor() as cur:
apps/api/tests/test_identity_constraints.py:42:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py-43-
apps/api/tests/test_identity_constraints.py-44-
```

```
apps/api/tests/test_identity_constraints.py-45-def
test_platform_permission_is_resolved_from_database_role_and_session():
apps/api/tests/test_identity_constraints.py-46-    admin_email =
f"platform-admin-{uuid4().hex}@example.test"
--
apps/api/tests/test_identity_constraints.py-122-        assert denied.json() == {"detail":
"Permission denied"}
apps/api/tests/test_identity_constraints.py-123-    finally:
apps/api/tests/test_identity_constraints.py-124-        with connect() as conn, conn.cursor() as
cur:
apps/api/tests/test_identity_constraints.py:125:            cur.execute("DELETE FROM users
WHERE user_id IN (%s, %s)", (admin_id, user_id))
apps/api/tests/test_identity_constraints.py-126-
apps/api/tests/test_identity_constraints.py-127-
apps/api/tests/test_identity_constraints.py-128-
apps/api/tests/test_identity_constraints.py-129-def
test_permission_grant_revocation_takes_effect_on_next_request():
--
apps/api/tests/test_identity_constraints.py-203-    finally:
apps/api/tests/test_identity_constraints.py-204-        with connect() as conn, conn.cursor() as
cur:
apps/api/tests/test_identity_constraints.py-205-            if user_id is not None:
apps/api/tests/test_identity_constraints.py:206:                cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py-207-            if role_id is not None:
apps/api/tests/test_identity_constraints.py-208-                cur.execute("DELETE FROM roles
WHERE role_id = %s", (role_id,))
apps/api/tests/test_identity_constraints.py-209-
apps/api/tests/test_identity_constraints.py-210-def
test_platform_role_assignment_rejects_workspace_scoped_roles():
--
apps/api/tests/test_identity_constraints.py-226-                cur.execute("UPDATE users SET
platform_role_id = %s WHERE user_id = %s", (workspace_role_id, user_id))
apps/api/tests/test_identity_constraints.py-227-    finally:
apps/api/tests/test_identity_constraints.py-228-        with connect() as conn, conn.cursor() as
cur:
apps/api/tests/test_identity_constraints.py:229:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py-230-            cur.execute("DELETE FROM roles
WHERE role_id = %s", (workspace_role_id,))
apps/api/tests/test_identity_constraints.py-231-
apps/api/tests/test_identity_constraints.py-232-
apps/api/tests/test_identity_constraints.py-233-
--
```

```
apps/api/tests/test_identity_constraints.py-280-            assert next_response.json() == {"detail":
"Permission denied"}
apps/api/tests/test_identity_constraints.py-281-    finally:
apps/api/tests/test_identity_constraints.py-282-        with connect() as conn, conn.cursor() as
cur:
apps/api/tests/test_identity_constraints.py:283:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py-284-
apps/api/tests/test_identity_constraints.py-285-
apps/api/tests/test_identity_constraints.py-286-# def
test_capabilities_request_id_is_canonical_and_correlation_is_preserved():
apps/api/tests/test_identity_constraints.py-287-#     with connect() as conn, conn.cursor() as cur:
--
apps/api/tests/test_auth.py-29-        yield user_id, email, encoded
apps/api/tests/test_auth.py-30-    finally:
apps/api/tests/test_auth.py-31-        with connect() as conn, conn.cursor() as cur:
apps/api/tests/test_auth.py:32:            cur.execute("DELETE FROM users WHERE user_id =
%s", (user_id,))
apps/api/tests/test_auth.py-33-
apps/api/tests/test_auth.py-34-
apps/api/tests/test_auth.py-35-@pytest.fixture
apps/api/tests/test_auth.py-36-def client(monkeypatch):
=== Migration execution mechanism ===
scripts/package_release.py:45:        shutil.copytree(ROOT / "apps/api/migrations", backend /
"migrations")
scripts/package_release.py:54:            "database_migrations": ["0001_auth_identity.sql",
"0002_platform_authorization.sql", "0003_auth_audit_and_login_throttling.sql"],
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
st_identity_constraints.py-228-        with connect() as conn, conn.cursor() as cur:
apps/api/tests/test_identity_constraints.py:229:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
apps/api/tests/test_identity_constraints.py-230-            cur.execute("DELETE FROM roles
WHERE role_id = %s", (workspace_role_id,))
apps/api/tests/test_identity_constraints.py-231-
apps/api/tests/test_identity_constraints.py-232-
apps/api/tests/test_identity_constraints.py-233-
--
apps/api/tests/test_identity_constraints.py-280-            assert next_response.json() == {"detail":
"Permission denied"}
apps/api/tests/test_identity_constraints.py-281-    finally:
apps/api/tests/test_identity_constraints.py-282-        with connect() as conn, conn.cursor() as
cur:
apps/api/tests/test_identity_constraints.py:283:            cur.execute("DELETE FROM users
WHERE user_id = %s", (user_id,))
```

```
apps/api/tests/test_identity_constraints.py-284-
apps/api/tests/test_identity_constraints.py-285-
apps/api/tests/test_identity_constraints.py-286-# def
test_capabilities_request_id_is_canonical_and_correlation_is_preserved():
apps/api/tests/test_identity_constraints.py-287-#     with connect() as conn, conn.cursor() as cur:
--
apps/api/tests/test_auth.py-29-        yield user_id, email, encoded
apps/api/tests/test_auth.py-30-    finally:
apps/api/tests/test_auth.py-31-        with connect() as conn, conn.cursor() as cur:
apps/api/tests/test_auth.py:32:            cur.execute("DELETE FROM users WHERE user_id =
%s", (user_id,))
apps/api/tests/test_auth.py-33-
apps/api/tests/test_auth.py-34-
apps/api/tests/test_auth.py-35-@pytest.fixture
apps/api/tests/test_auth.py-36-def client(monkeypatch):
=== Migration execution mechanism ===
scripts/package_release.py:45:        shutil.copytree(ROOT / "apps/api/migrations", backend /
"migrations")
scripts/package_release.py:54:            "database_migrations": ["0001_auth_identity.sql",
"0002_platform_authorization.sql", "0003_auth_audit_and_login_throttling.sql"],
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

echo "=== User lifecycle and deletion logic ==="
grep -R -n -E 'deleted_at|account_status|DELETE FROM users|UPDATE users SET' \
  apps/api/app apps/api/migrations --exclude-dir=__pycache__ 2>/dev/null | head -180

echo "=== Session schema and authentication dependency ==="
grep -R -n -E 'CREATE TABLE auth_sessions|CREATE TABLE.*sessions|def get_current|def
require.*session|session_id|revoked_at|expires_at' \
  apps/api/app apps/api/migrations --exclude-dir=__pycache__ 2>/dev/null | head -220

echo "=== Login throttle configuration and behavior ==="
sed -n '1,300p' apps/api/app/throttle.py
sed -n '1,280p' apps/api/tests/test_fin14_auth_security.py

echo "=== Release manifest builder ==="
sed -n '1,180p' scripts/package_release.py

echo "=== MFA and session routes already present ==="
grep -R -n -E 'mfa|multi.factor|totp|active.session|auth/sessions|auth/logout' \
  apps/api/app apps/api/tests --exclude-dir=__pycache__ 2>/dev/null | head -180
=== User lifecycle and deletion logic ===
```

apps/api/app/provision_user.py:29:   """INSERT INTO users (email, password_hash,
display_name, account_status, platform_role_id)
apps/api/app/provision_user.py:36:   account_status = 'active', updated_at = now(),
deleted_at = NULL""",
apps/api/app/main.py:144:   AND s.expires_at > %s AND u.account_status = 'active'
apps/api/app/main.py:146:   AND u.deleted_at IS NULL
apps/api/app/main.py:219:   AND account_status = 'active'
apps/api/app/main.py:220:   AND deleted_at IS NULL""",
apps/api/migrations/0001_auth_identity.sql:19:   account_status VARCHAR(20) NOT NULL
DEFAULT 'active'
apps/api/migrations/0001_auth_identity.sql:20:   CHECK (account_status IN
('invited','active','locked','disabled','deleted')),
apps/api/migrations/0001_auth_identity.sql:27:   deleted_at TIMESTAMPTZ
=== Session schema and authentication dependency ===
apps/api/app/main.py:128:def get_current_user(
apps/api/app/main.py:143:   WHERE s.refresh_token_hash = %s AND s.revoked_at
IS NULL
apps/api/app/main.py:144:   AND s.expires_at > %s AND u.account_status = 'active'
apps/api/app/main.py:267:   expires_at = session_expiry(_ttl_minutes())
apps/api/app/main.py:272:   (user_id, refresh_token_hash, expires_at)
apps/api/app/main.py:274:   (user["user_id"], digest, expires_at),
apps/api/app/main.py:336:   SET revoked_at = %s, revoke_reason = 'logout'
apps/api/app/main.py:338:   AND revoked_at IS NULL
apps/api/migrations/0001_auth_identity.sql:31:CREATE TABLE auth_sessions (
apps/api/migrations/0001_auth_identity.sql:32:   session_id UUID PRIMARY KEY DEFAULT
gen_random_uuid(),
apps/api/migrations/0001_auth_identity.sql:39:   expires_at TIMESTAMPTZ NOT NULL,
apps/api/migrations/0001_auth_identity.sql:40:   revoked_at TIMESTAMPTZ,
apps/api/migrations/0001_auth_identity.sql:43:CREATE INDEX ix_auth_sessions_active ON
auth_sessions(user_id, expires_at) WHERE revoked_at IS NULL;
=== Login throttle configuration and behavior ===
"""PostgreSQL-backed login throttling with secret-keyed identifiers."""

from __future__ import annotations

import hashlib
import hmac
import os
from dataclasses import dataclass
from datetime import UTC, datetime
from typing import Any

@dataclass(frozen=True)

```
class ThrottleConfig:
    secret: bytes
    max_failures: int
    window_seconds: int
    block_seconds: int


def load_throttle_config() -> ThrottleConfig:
    """Load validated throttle settings; never use a weak fallback secret."""
    secret = os.environ.get("AUTH_THROTTLE_SECRET", "")
    if len(secret.encode("utf-8")) < 32:
        raise ValueError("AUTH_THROTTLE_SECRET must contain at least 32 bytes")

    try:
        max_failures = int(os.environ.get("AUTH_LOGIN_MAX_FAILURES", "5"))
        window_seconds = int(os.environ.get("AUTH_LOGIN_WINDOW_SECONDS", "900"))
        block_seconds = int(os.environ.get("AUTH_LOGIN_BLOCK_SECONDS", "900"))
    except ValueError as exc:
        raise ValueError("Login throttle configuration is invalid") from exc

    if not 1 <= max_failures <= 100:
        raise ValueError("AUTH_LOGIN_MAX_FAILURES must be between 1 and 100")
    if not 1 <= window_seconds <= 86400:
        raise ValueError("AUTH_LOGIN_WINDOW_SECONDS must be between 1 and 86400")
    if not 1 <= block_seconds <= 86400:
        raise ValueError("AUTH_LOGIN_BLOCK_SECONDS must be between 1 and 86400")

    return ThrottleConfig(
        secret=secret.encode("utf-8"),
        max_failures=max_failures,
        window_seconds=window_seconds,
        block_seconds=block_seconds,
    )


def make_throttle_key(secret: bytes, scope_type: str, value: str) -> str:
    """Return a stable HMAC-SHA256 identifier without storing the raw value."""
    if scope_type not in {"account", "ip"}:
        raise ValueError("Unsupported throttle scope")
    if not value:
        raise ValueError("Throttle key value cannot be empty")

    message = f"finsight-login:{scope_type}:{value}".encode("utf-8")
    return hmac.new(secret, message, hashlib.sha256).hexdigest()
```

```
def is_blocked(
    conn: Any,
    *,
    scope_type: str,
    key_hash: str,
    now: datetime | None = None,
) -> bool:
    current_time = now or datetime.now(UTC)
    with conn.cursor() as cur:
        cur.execute(
            """
            SELECT blocked_until > %s
            FROM auth_login_throttle
            WHERE scope_type = %s AND key_hash = %s
            """,
            (current_time, scope_type, key_hash),
        )
        row = cur.fetchone()
    # Accept the timestamp result used by the database query, and Boolean
    # results from adapters that evaluate the comparison in SQL.
    if not row or row[0] is None:
        return False
    value = row[0]
    if isinstance(value, bool):
        return value
    return value > current_time


def record_failure(
    conn: Any,
    *,
    scope_type: str,
    key_hash: str,
    config: ThrottleConfig,
    now: datetime | None = None,
) -> datetime | None:
    """Atomically increment a bucket and return its active block expiry, if any."""
    current_time = now or datetime.now(UTC)
    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO auth_login_throttle (
```

```
            scope_type, key_hash, failed_attempts,
            window_started_at, blocked_until, updated_at
        )
        VALUES (%s, %s, 1, %s, NULL, %s)
        ON CONFLICT (scope_type, key_hash) DO UPDATE SET
            failed_attempts = CASE
                WHEN auth_login_throttle.window_started_at
                    <= %s - make_interval(secs => %s)
                THEN 1
                ELSE auth_login_throttle.failed_attempts + 1
            END,
            window_started_at = CASE
                WHEN auth_login_throttle.window_started_at
                    <= %s - make_interval(secs => %s)
                THEN %s
                ELSE auth_login_throttle.window_started_at
            END,
            blocked_until = CASE
                WHEN CASE
                    WHEN auth_login_throttle.window_started_at
                        <= %s - make_interval(secs => %s)
                    THEN 1
                    ELSE auth_login_throttle.failed_attempts + 1
                END >= %s
                THEN %s + make_interval(secs => %s)
                WHEN auth_login_throttle.blocked_until > %s
                THEN auth_login_throttle.blocked_until
                ELSE NULL
            END,
            updated_at = %s
        RETURNING blocked_until
        """,
        (
            scope_type, key_hash, current_time, current_time,
            current_time, config.window_seconds,
            current_time, config.window_seconds, current_time,
            current_time, config.window_seconds, config.max_failures,
            current_time, config.block_seconds, current_time,
            current_time,
        ),
    )
    row = cur.fetchone()

if row and row[0] and row[0] > current_time:
```

```
        return row[0]
    return None


def reset_bucket(conn: Any, *, scope_type: str, key_hash: str) -> None:
    """Clear a successful login's corresponding failure counter."""
    with conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = %s AND key_hash = %s",
            (scope_type, key_hash),
        )
from __future__ import annotations

from datetime import UTC, datetime
from uuid import UUID, uuid4

from fastapi.testclient import TestClient

from app.auth import hash_password
from app.throttle import make_throttle_key
from app.database import connect
from app.main import app


def test_successful_login_is_audited_without_secrets(monkeypatch):
    monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Password-Only"
    correlation = None

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """INSERT INTO users (email, password_hash, display_name, account_status)
               VALUES (%s, %s, %s, 'active') RETURNING user_id""",
            (email, hash_password(password), "FIN14 Audit Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with TestClient(app) as client:
            response = client.post(
                "/auth/login", json={"email": email, "password": password}
            )
```

```
        assert response.status_code == 200

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT action_key, outcome, actor_user_id, details_redacted::text
                    FROM audit_events
                    WHERE action_key = 'auth.login' AND actor_user_id = %s
                    ORDER BY occurred_at DESC LIMIT 1""",
                (user_id,),
            )
            event = cur.fetchone()

        assert event is not None
        assert event[0] == "auth.login"
        assert event[1] == "success"
        assert event[2] == user_id
        assert password not in event[3]
        assert response.cookies.get("finsight_session") not in event[3]
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_failed_login_is_audited_and_repeated_failures_are_blocked(monkeypatch):
    monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
    monkeypatch.setenv("AUTH_LOGIN_MAX_FAILURES", "2")
    monkeypatch.setenv("AUTH_LOGIN_WINDOW_SECONDS", "600")
    monkeypatch.setenv("AUTH_LOGIN_BLOCK_SECONDS", "60")

    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Password-Only"
    test_started_at = datetime.now(UTC)
    ip_key = make_throttle_key(
        b"integration-test-secret-material-32-bytes",
        "ip",
        "testclient",
    )

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
        )
```

```
    cur.execute(
        """INSERT INTO users (email, password_hash, display_name, account_status)
           VALUES (%s, %s, %s, 'active') RETURNING user_id""",
        (email, hash_password(password), "FIN14 Throttle Test"),
    )
    user_id = cur.fetchone()[0]

try:
    with TestClient(app) as client:
        first = client.post(
            "/auth/login", json={"email": email, "password": "wrong-password"}
        )
        second = client.post(
            "/auth/login", json={"email": email, "password": "wrong-password"}
        )
        blocked = client.post(
            "/auth/login", json={"email": email, "password": password}
        )

    assert first.status_code == 401
    assert second.status_code == 401
    assert blocked.status_code == 401
    assert first.json() == second.json() == blocked.json()

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """SELECT outcome, actor_user_id, details_redacted::text
               FROM audit_events
               WHERE action_key = 'auth.login'
                 AND details_redacted->>'reason' IN (
                     'invalid_credentials', 'throttled'
                 )
               ORDER BY occurred_at DESC LIMIT 3"""
        )
        events = cur.fetchall()

    assert len(events) == 3
    assert sum(event[0] == "failure" for event in events) == 2
    assert sum(event[0] == "denied" for event in events) == 1
    assert all(
        "wrong-password" not in event[2]
        for event in events
    )
    assert any(event[1] == user_id for event in events)
```

```
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
                (ip_key,),
            )
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_logout_is_audited_without_recording_session_token(monkeypatch):
    monkeypatch.setenv(
        "AUTH_THROTTLE_SECRET",
        "integration-test-secret-material-32-bytes",
    )
    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Logout-Password"

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """INSERT INTO users (email, password_hash, display_name, account_status)
               VALUES (%s, %s, %s, 'active') RETURNING user_id""",
            (email, hash_password(password), "FIN14 Logout Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with TestClient(app) as client:
            login = client.post(
                "/auth/login",
                json={"email": email, "password": password},
            )
            assert login.status_code == 200
            session_token = login.cookies.get("finsight_session")
            assert session_token

            logout = client.post(
                "/auth/logout",
                cookies={"finsight_session": session_token},
            )
            assert logout.status_code == 204

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT outcome, actor_user_id, details_redacted::text
```

```
                FROM audit_events
                WHERE action_key = 'auth.logout'
                  AND actor_user_id = %s
                ORDER BY occurred_at DESC LIMIT 1""",
              (user_id,),
            )
            event = cur.fetchone()

        assert event is not None
        assert event[0] == "success"
        assert event[1] == user_id
        assert session_token not in event[2]
        assert password not in event[2]
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_auth_correlation_id_is_returned_and_audited(monkeypatch):
    monkeypatch.setenv(
        "AUTH_THROTTLE_SECRET",
        "integration-test-secret-material-32-bytes",
    )
    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Correlation-Password"
    correlation_id = uuid4()

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """INSERT INTO users (email, password_hash, display_name, account_status)
               VALUES (%s, %s, %s, 'active') RETURNING user_id""",
            (email, hash_password(password), "FIN14 Correlation Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with TestClient(app) as client:
            response = client.post(
                "/auth/login",
                json={"email": email, "password": password},
                headers={"X-Correlation-ID": str(correlation_id)},
            )

        assert response.status_code == 200
```

```
        assert response.headers.get("X-Correlation-ID") == str(correlation_id)

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT correlation_id
                   FROM audit_events
                   WHERE action_key = 'auth.login'
                     AND actor_user_id = %s
                     AND outcome = 'success'
                   ORDER BY occurred_at DESC LIMIT 1""",
                (user_id,),
            )
            event = cur.fetchone()

        assert event is not None
        assert event[0] == correlation_id
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_invalid_auth_correlation_id_is_rejected(monkeypatch):
    monkeypatch.setenv(
        "AUTH_THROTTLE_SECRET",
        "integration-test-secret-material-32-bytes",
    )

    with TestClient(app) as client:
        response = client.post(
            "/auth/login",
            json={"email": "correlation-test@example.com", "password": "not-used"},
            headers={"X-Correlation-ID": "not-a-uuid"},
        )

    assert response.status_code == 400
    assert response.json() == {"detail": "Invalid X-Correlation-ID"}


def test_failed_login_returns_the_audited_correlation_id(monkeypatch):
    monkeypatch.setenv(
        "AUTH_THROTTLE_SECRET",
        "integration-test-secret-material-32-bytes",
    )
    correlation_id = uuid4()
```

```
    with TestClient(app) as client:
        response = client.post(
            "/auth/login",
            json={
                "email": f"{uuid4().hex}@example.com",
                "password": "Synthetic-Invalid-Password",
            },
            headers={"X-Correlation-ID": str(correlation_id)},
        )

    assert response.status_code == 401
    assert response.headers.get("X-Correlation-ID") == str(correlation_id)

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """SELECT correlation_id
               FROM audit_events
               WHERE action_key = 'auth.login'
                 AND outcome = 'failure'
                 AND correlation_id = %s
               ORDER BY occurred_at DESC LIMIT 1""",
            (correlation_id,),
        )
        event = cur.fetchone()

    assert event is not None
    assert event[0] == correlation_id

=== Release manifest builder ===
#!/usr/bin/env python3
"""Build a versioned FinSight application bundle from locked/build outputs."""

from __future__ import annotations

import argparse
import hashlib
import json
import os
import re
import shutil
import tarfile
import tempfile
from datetime import UTC, datetime
```

```
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
VERSION_PATTERN = re.compile(r"\A[0-9]+\.[0-9]+\.[0-9]+(?:[-+][0-9A-Za-z.-]+)?\Z")
SHA_PATTERN = re.compile(r"\A[0-9a-f]{40}\Z")


def package(frontend_dir: Path, version: str, commit_sha: str | None, output_dir: Path) ->
tuple[Path, Path, Path]:
    if not VERSION_PATTERN.fullmatch(version):
        raise ValueError("version must be a numeric semantic version, optionally with a
prerelease/build suffix")
    if commit_sha and not SHA_PATTERN.fullmatch(commit_sha):
        raise ValueError("commit SHA must be a full 40-character lowercase hexadecimal SHA")
    if not frontend_dir.is_dir() or not (frontend_dir / "server.js").is_file():
        raise FileNotFoundError("frontend standalone build is missing server.js")

    web_static = ROOT / "apps/web/.next/static"
    runtime_config = ROOT / "apps/web/app/api/runtime-config/route.js"
    if not web_static.is_dir() or not runtime_config.is_file():
        raise FileNotFoundError("build the frontend before packaging")

    output_dir.mkdir(parents=True, exist_ok=True)
    artifact_base = f"finsight-{version}" + (f"-{commit_sha[:12]}" if commit_sha else "")
    with tempfile.TemporaryDirectory(prefix="finsight-release-") as temp_dir:
        payload = Path(temp_dir) / "finsight"
        frontend = payload / "frontend"
        shutil.copytree(frontend_dir, frontend, symlinks=True)
        shutil.copytree(web_static, frontend / ".next/static", symlinks=True)

        backend = payload / "backend"
        shutil.copytree(ROOT / "apps/api/app", backend / "app",
ignore=shutil.ignore_patterns("__pycache__", "*.pyc"))
        shutil.copytree(ROOT / "apps/api/migrations", backend / "migrations")
        shutil.copy2(ROOT / "apps/api/requirements.lock", backend / "requirements.lock")

        metadata = {
            "artifact": artifact_base,
            "version": version,
            "commit_sha": commit_sha,
            "frontend": {"runtime": "Node.js 22", "entrypoint": "node frontend/server.js"},
            "backend": {"runtime": "Python 3.12", "entrypoint": "python -m uvicorn app.main:app"},
            "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql",
"0003_auth_audit_and_login_throttling.sql"],
```

```
        "configuration": "Runtime values are supplied through environment variables; see
RELEASE_README.md.",
    }
    (payload / "RELEASE-METADATA.json").write_text(json.dumps(metadata, indent=2) + "\n",
encoding="utf-8")
    release_readme = ROOT / "docs/release/RELEASE_README.md"
    shutil.copy2(release_readme, payload / "RELEASE_README.md")

    artifact_path = output_dir / f"{artifact_base}.tar.gz"
    def normalized_member(member: tarfile.TarInfo) -> tarfile.TarInfo:
        member.mtime = 0
        member.uid = 0
        member.gid = 0
        member.uname = ""
        member.gname = ""
        return member

    with artifact_path.open("wb") as raw:
        import gzip

        with gzip.GzipFile(fileobj=raw, mode="wb", mtime=0, filename="") as compressed:
            with tarfile.open(fileobj=compressed, mode="w", format=tarfile.PAX_FORMAT) as
archive:
                for item in sorted(payload.rglob("*")):
                    archive.add(
                        item,
                        arcname=(Path("finsight") / item.relative_to(payload)).as_posix(),
                        recursive=False,
                        filter=normalized_member,
                    )

  digest_builder = hashlib.sha256()
  with artifact_path.open("rb") as stream:
      for chunk in iter(lambda: stream.read(1024 * 1024), b""):
          digest_builder.update(chunk)
  digest = digest_builder.hexdigest()
  checksum_path = output_dir / f"{artifact_path.name}.sha256"
  checksum_path.write_text(f"{digest}  {artifact_path.name}\n", encoding="ascii")
  build_info_path = output_dir / f"{artifact_path.name}.build-info.json"
  build_info = {
      **metadata,
      "built_at_utc": datetime.now(UTC).replace(microsecond=0).isoformat(),
      "sha256": digest,
  }
```

```
    build_info_path.write_text(json.dumps(build_info, indent=2) + "\n", encoding="utf-8")
    return artifact_path, checksum_path, build_info_path


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--frontend-dir", type=Path, required=True, help="Next standalone
output directory")
    parser.add_argument("--version", required=True)
    parser.add_argument("--commit-sha", default=os.environ.get("GITHUB_SHA"))
    parser.add_argument("--output-dir", type=Path, default=ROOT / "dist")
    args = parser.parse_args()
    artifact, checksum, build_info = package(args.frontend_dir, args.version, args.commit_sha,
args.output_dir)
    print(f"Created {artifact}")
    print(f"Created {checksum}")
    print(f"Created {build_info}")


if __name__ == "__main__":
    main()
=== MFA and session routes already present ===
apps/api/app/main.py:39:    if request.url.path not in {"/auth/login", "/auth/logout"}:
apps/api/app/main.py:318:@app.post("/auth/logout", status_code=204)
apps/api/tests/test_fin14_auth_security.py:156:                "/auth/logout",
apps/api/tests/test_auth.py:108:    logout = client.post("/auth/logout",
cookies={SESSION_COOKIE: cookie})
apps/api/tests/test_auth.py:118:    response = client.post("/auth/logout")
apps/api/tests/test_auth.py:141:        "/auth/logout", headers={"Origin": "https://attacker.invalid"},
apps/api/tests/test_auth_unit.py:139:        ("post", "/auth/logout", {"cookies": {"finsight_session":
"synthetic-token"}}),
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
AS $$
BEGIN
    RAISE EXCEPTION 'audit_events is append-only; % is not permitted', TG_OP
        USING ERRCODE = '55000';
END;
$$;

CREATE TRIGGER audit_events_append_only
BEFORE UPDATE OR DELETE ON audit_events
FOR EACH ROW
EXECUTE FUNCTION reject_audit_events_mutation();
```

```
COMMIT;
""", encoding="utf-8")

print(f"Created {migration}")

packager = Path("scripts/package_release.py")
source = packager.read_text(encoding="utf-8")
old = '"database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql",
"0003_auth_audit_and_login_throttling.sql"],'
new = '"database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql",
"0003_auth_audit_and_login_throttling.sql", "0004_immutable_auth_audit.sql"],'
if source.count(old) != 1:
    raise SystemExit("Migration created, but release metadata did not match exactly. Inspect
scripts/package_release.py manually.")
packager.write_text(source.replace(old, new), encoding="utf-8")
print("Updated release migration metadata")
PY

echo "=== Review migration ==="
cat apps/api/migrations/0004_immutable_auth_audit.sql

echo "=== Check diff formatting ==="
git diff --check
Created apps/api/migrations/0004_immutable_auth_audit.sql
Updated release migration metadata
=== Review migration ===
BEGIN;

-- Preserve the actor UUID and prevent hard deletion from mutating history.
ALTER TABLE audit_events
    DROP CONSTRAINT audit_events_actor_user_id_fkey;

ALTER TABLE audit_events
    ADD CONSTRAINT audit_events_actor_user_id_fkey
    FOREIGN KEY (actor_user_id)
    REFERENCES users(user_id)
    ON DELETE RESTRICT;

-- Audit events are append-only. Corrections must be represented by new events.
CREATE FUNCTION reject_audit_events_mutation()
RETURNS trigger
LANGUAGE plpgsql
AS $$
```

```
BEGIN
   RAISE EXCEPTION 'audit_events is append-only; % is not permitted', TG_OP
      USING ERRCODE = '55000';
END;
$$;

CREATE TRIGGER audit_events_append_only
BEFORE UPDATE OR DELETE ON audit_events
FOR EACH ROW
EXECUTE FUNCTION reject_audit_events_mutation();

COMMIT;
=== Check diff formatting ===
=== Review migration ===
BEGIN;

-- Preserve the actor UUID and prevent hard deletion from mutating history.
ALTER TABLE audit_events
   DROP CONSTRAINT audit_events_actor_user_id_fkey;

ALTER TABLE audit_events
   ADD CONSTRAINT audit_events_actor_user_id_fkey
   FOREIGN KEY (actor_user_id)
   REFERENCES users(user_id)
   ON DELETE RESTRICT;

-- Audit events are append-only. Corrections must be represented by new events.
CREATE FUNCTION reject_audit_events_mutation()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
   RAISE EXCEPTION 'audit_events is append-only; % is not permitted', TG_OP
      USING ERRCODE = '55000';
END;
$$;

CREATE TRIGGER audit_events_append_only
BEFORE UPDATE OR DELETE ON audit_events
FOR EACH ROW
EXECUTE FUNCTION reject_audit_events_mutation();

COMMIT;
=== Check diff formatting ===
```

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....

echo "=== Auth integration fixture cleanup ==="
sed -n '1,180p' apps/api/tests/test_auth.py

echo "=== Identity constraint tests ==="
sed -n '1,330p' apps/api/tests/test_identity_constraints.py

echo "=== Current database constraints and triggers ==="
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
.venv/bin/python - <<'PY'
from app.database import connect

with connect() as conn, conn.cursor() as cur:
    cur.execute("""
        SELECT conname, pg_get_constraintdef(oid)
        FROM pg_constraint
        WHERE conrelid = 'audit_events'::regclass
        ORDER BY conname
    """)
    print("Constraints:")
    for row in cur.fetchall():
        print(row)

    cur.execute("""
        SELECT tgname
        FROM pg_trigger
        WHERE tgrelid = 'audit_events'::regclass
          AND NOT tgisinternal
        ORDER BY tgname
    """)
    print("Triggers:", cur.fetchall())
PY
=== Audit and login test cleanup ===
from __future__ import annotations

from datetime import UTC, datetime
from uuid import UUID, uuid4

from fastapi.testclient import TestClient

from app.auth import hash_password
from app.throttle import make_throttle_key
```

```
from app.database import connect
from app.main import app


def test_successful_login_is_audited_without_secrets(monkeypatch):
    monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Password-Only"
    correlation = None

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """INSERT INTO users (email, password_hash, display_name, account_status)
               VALUES (%s, %s, %s, 'active') RETURNING user_id""",
            (email, hash_password(password), "FIN14 Audit Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with TestClient(app) as client:
            response = client.post(
                "/auth/login", json={"email": email, "password": password}
            )
        assert response.status_code == 200

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT action_key, outcome, actor_user_id, details_redacted::text
                   FROM audit_events
                   WHERE action_key = 'auth.login' AND actor_user_id = %s
                   ORDER BY occurred_at DESC LIMIT 1""",
                (user_id,),
            )
            event = cur.fetchone()

        assert event is not None
        assert event[0] == "auth.login"
        assert event[1] == "success"
        assert event[2] == user_id
        assert password not in event[3]
        assert response.cookies.get("finsight_session") not in event[3]
    finally:
        with connect() as conn, conn.cursor() as cur:
```

```
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_failed_login_is_audited_and_repeated_failures_are_blocked(monkeypatch):
    monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
    monkeypatch.setenv("AUTH_LOGIN_MAX_FAILURES", "2")
    monkeypatch.setenv("AUTH_LOGIN_WINDOW_SECONDS", "600")
    monkeypatch.setenv("AUTH_LOGIN_BLOCK_SECONDS", "60")

    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Password-Only"
    test_started_at = datetime.now(UTC)
    ip_key = make_throttle_key(
        b"integration-test-secret-material-32-bytes",
        "ip",
        "testclient",
    )

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
        )
        cur.execute(
            """INSERT INTO users (email, password_hash, display_name, account_status)
               VALUES (%s, %s, %s, 'active') RETURNING user_id""",
            (email, hash_password(password), "FIN14 Throttle Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with TestClient(app) as client:
            first = client.post(
                "/auth/login", json={"email": email, "password": "wrong-password"}
            )
            second = client.post(
                "/auth/login", json={"email": email, "password": "wrong-password"}
            )
            blocked = client.post(
                "/auth/login", json={"email": email, "password": password}
            )

        assert first.status_code == 401
```

```
        assert second.status_code == 401
        assert blocked.status_code == 401
        assert first.json() == second.json() == blocked.json()

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT outcome, actor_user_id, details_redacted::text
                   FROM audit_events
                   WHERE action_key = 'auth.login'
                     AND details_redacted->>'reason' IN (
                         'invalid_credentials', 'throttled'
                     )
                   ORDER BY occurred_at DESC LIMIT 3"""
            )
            events = cur.fetchall()

        assert len(events) == 3
        assert sum(event[0] == "failure" for event in events) == 2
        assert sum(event[0] == "denied" for event in events) == 1
        assert all(
            "wrong-password" not in event[2]
            for event in events
        )
        assert any(event[1] == user_id for event in events)
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
                (ip_key,),
            )
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_logout_is_audited_without_recording_session_token(monkeypatch):
    monkeypatch.setenv(
        "AUTH_THROTTLE_SECRET",
        "integration-test-secret-material-32-bytes",
    )
    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Logout-Password"

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """INSERT INTO users (email, password_hash, display_name, account_status)
```

```
            VALUES (%s, %s, %s, 'active') RETURNING user_id""",
            (email, hash_password(password), "FIN14 Logout Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with TestClient(app) as client:
            login = client.post(
                "/auth/login",
                json={"email": email, "password": password},
            )
            assert login.status_code == 200
            session_token = login.cookies.get("finsight_session")
            assert session_token

            logout = client.post(
                "/auth/logout",
                cookies={"finsight_session": session_token},
            )
            assert logout.status_code == 204

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """SELECT outcome, actor_user_id, details_redacted::text
                   FROM audit_events
                   WHERE action_key = 'auth.logout'
                     AND actor_user_id = %s
                   ORDER BY occurred_at DESC LIMIT 1""",
                (user_id,),
            )
            event = cur.fetchone()

        assert event is not None
        assert event[0] == "success"
        assert event[1] == user_id
        assert session_token not in event[2]
        assert password not in event[2]
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_auth_correlation_id_is_returned_and_audited(monkeypatch):
    monkeypatch.setenv(
```

```
    "AUTH_THROTTLE_SECRET",
    "integration-test-secret-material-32-bytes",
)
email = f"{uuid4().hex}@example.com"
password = "Synthetic-FIN14-Correlation-Password"
correlation_id = uuid4()

with connect() as conn, conn.cursor() as cur:
    cur.execute(
        """INSERT INTO users (email, password_hash, display_name, account_status)
           VALUES (%s, %s, %s, 'active') RETURNING user_id""",
        (email, hash_password(password), "FIN14 Correlation Test"),
    )
    user_id = cur.fetchone()[0]

try:
    with TestClient(app) as client:
        response = client.post(
            "/auth/login",
            json={"email": email, "password": password},
            headers={"X-Correlation-ID": str(correlation_id)},
        )

    assert response.status_code == 200
    assert response.headers.get("X-Correlation-ID") == str(correlation_id)

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """SELECT correlation_id
               FROM audit_events
               WHERE action_key = 'auth.login'
                 AND actor_user_id = %s
                 AND outcome = 'success'
               ORDER BY occurred_at DESC LIMIT 1""",
            (user_id,),
        )
        event = cur.fetchone()

    assert event is not None
    assert event[0] == correlation_id
finally:
    with connect() as conn, conn.cursor() as cur:
        cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
```

```
def test_invalid_auth_correlation_id_is_rejected(monkeypatch):
    monkeypatch.setenv(
        "AUTH_THROTTLE_SECRET",
        "integration-test-secret-material-32-bytes",
    )

    with TestClient(app) as client:
        response = client.post(
            "/auth/login",
            json={"email": "correlation-test@example.com", "password": "not-used"},
            headers={"X-Correlation-ID": "not-a-uuid"},
        )

    assert response.status_code == 400
    assert response.json() == {"detail": "Invalid X-Correlation-ID"}


def test_failed_login_returns_the_audited_correlation_id(monkeypatch):
    monkeypatch.setenv(
        "AUTH_THROTTLE_SECRET",
        "integration-test-secret-material-32-bytes",
    )
    correlation_id = uuid4()

    with TestClient(app) as client:
        response = client.post(
            "/auth/login",
            json={
                "email": f"{uuid4().hex}@example.com",
                "password": "Synthetic-Invalid-Password",
            },
            headers={"X-Correlation-ID": str(correlation_id)},
        )

    assert response.status_code == 401
    assert response.headers.get("X-Correlation-ID") == str(correlation_id)

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """SELECT correlation_id
               FROM audit_events
               WHERE action_key = 'auth.login'
                 AND outcome = 'failure'
```

```
              AND correlation_id = %s
            ORDER BY occurred_at DESC LIMIT 1""",
          (correlation_id,),
        )
        event = cur.fetchone()

    assert event is not None
    assert event[0] == correlation_id


def test_generated_auth_correlation_id_matches_audit_record(monkeypatch):
    monkeypatch.setenv(
        "AUTH_THROTTLE_SECRET",
        "integration-test-secret-material-32-bytes",
    )
    email = f"{uuid4().hex}@example.com"

    with TestClient(app) as client:
        response = client.post(
            "/auth/login",
            json={"email": email, "password": "Synthetic-Invalid-Password"},
        )

    assert response.status_code == 401
    correlation_id = response.headers.get("X-Correlation-ID")
    assert correlation_id is not None
    parsed_correlation_id = UUID(correlation_id)

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """SELECT correlation_id
              FROM audit_events
              WHERE action_key = 'auth.login'
                AND outcome = 'failure'
                AND correlation_id = %s""",
            (parsed_correlation_id,),
        )
        event = cur.fetchone()

    assert event is not None
    assert event[0] == parsed_correlation_id
=== Auth integration fixture cleanup ===
from __future__ import annotations
```

```
import secrets
from uuid import uuid4

import pytest
from fastapi.testclient import TestClient
from psycopg.rows import dict_row

from app.auth import hash_password, token_digest, verify_password
from app.throttle import make_throttle_key
from app.database import connect
from app.main import SESSION_COOKIE, app

TEST_PASSWORD = secrets.token_urlsafe(32)


@pytest.fixture
def identity():
    email = f"{uuid4().hex}@example.com"
    encoded = hash_password(TEST_PASSWORD)
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO users (email, password_hash, display_name, account_status) VALUES
(%s, %s, %s, 'active') RETURNING user_id",
            (email, encoded, "Synthetic Analyst"),
        )
        user_id = cur.fetchone()[0]
    try:
        yield user_id, email, encoded
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


@pytest.fixture
def client(monkeypatch):
    monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
    monkeypatch.setenv("COOKIE_SECURE", "false")
    monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
    throttle_secret = "test-only-secret-material-at-least-32-bytes"
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
    ip_key = make_throttle_key(
        throttle_secret.encode("utf-8"), "ip", "testclient"
    )
```

```
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
        )

    try:
        with TestClient(app) as test_client:
            yield test_client
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
                (ip_key,),
            )


def test_password_hash_is_argon2_and_never_plaintext():
    encoded = hash_password(TEST_PASSWORD)
    assert encoded.startswith("argon2id")
    assert encoded != TEST_PASSWORD
    assert verify_password(encoded, TEST_PASSWORD)
    assert not verify_password(encoded, "wrong")


def test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
    user_id, email, encoded = identity
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
    assert response.json() == {"user_id": str(user_id), "email": email, "display_name": "Synthetic
Analyst"}
    assert TEST_PASSWORD not in response.text and encoded not in response.text
    cookie = response.cookies.get(SESSION_COOKIE)
    assert cookie
    set_cookie = response.headers["set-cookie"].lower()
    assert "httponly" in set_cookie and "samesite=lax" in set_cookie and "path=/" in set_cookie
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute("SELECT refresh_token_hash, expires_at, revoked_at FROM auth_sessions
WHERE user_id = %s", (user_id,))
        stored = cur.fetchone()
    assert stored["refresh_token_hash"] == token_digest(cookie)
    assert stored["refresh_token_hash"] != cookie
    assert stored["expires_at"] is not None and stored["revoked_at"] is None
    protected = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
```

```
    assert protected.status_code == 200 and protected.json()["user_id"] == str(user_id)
    captured = capsys.readouterr()
    assert TEST_PASSWORD not in caplog.text + captured.out + captured.err
    assert cookie not in caplog.text + captured.out + captured.err


def test_invalid_credentials_have_same_generic_response(client, identity):
    _, email, _ = identity
    known = client.post("/auth/login", json={"email": email, "password": "wrong-password"})
    absent = client.post("/auth/login", json={"email": "absent@example.com", "password":
"wrong-password"})
    assert (known.status_code, known.json()) == (absent.status_code, absent.json())
    assert known.status_code == 401 and known.json() == {"detail": "Email or password is
incorrect."}
    assert SESSION_COOKIE not in known.cookies and SESSION_COOKIE not in
absent.cookies


def test_logout_revokes_session_and_replay_fails(client, identity):
    _, email, _ = identity
    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    cookie = login.cookies.get(SESSION_COOKIE)
    assert cookie
    logout = client.post("/auth/logout", cookies={SESSION_COOKIE: cookie})
    assert logout.status_code == 204
    replay = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
    assert replay.status_code == 401
    with connect() as conn, conn.cursor() as cur:
        cur.execute("SELECT revoked_at FROM auth_sessions WHERE refresh_token_hash =
%s", (token_digest(cookie),))
        assert cur.fetchone()[0] is not None


def test_logout_without_session_is_idempotent(client):
    response = client.post("/auth/logout")

    assert response.status_code == 204
    assert response.headers["set-cookie"].startswith(f"{SESSION_COOKIE}=")


def test_expired_session_is_rejected(client, identity):
    user_id, email, _ = identity
    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    cookie = login.cookies.get(SESSION_COOKIE)
```

```
    assert cookie
    with connect() as conn, conn.cursor() as cur:
        cur.execute("UPDATE auth_sessions SET last_seen_at = now() - interval '61 minutes'
WHERE user_id = %s", (user_id,))
    assert client.get("/auth/me", cookies={SESSION_COOKIE: cookie}).status_code == 401


def test_public_registration_is_not_available(client):
    response = client.post("/auth/register", json={"email": "public@example.com", "password":
TEST_PASSWORD})
    assert response.status_code == 404


def test_auth_mutation_rejects_foreign_origin(client):
    response = client.post(
        "/auth/logout", headers={"Origin": "https://attacker.invalid"},
    )
    assert response.status_code == 403


def test_secure_cookie_can_be_required_by_deployment(client, identity, monkeypatch):
    _, email, _ = identity
    monkeypatch.setenv("COOKIE_SECURE", "true")
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
    assert "secure" in response.headers["set-cookie"].lower()
=== Identity constraint tests ===
from __future__ import annotations

from uuid import uuid4

import psycopg
import pytest
from fastapi.testclient import TestClient

from app.auth import new_session_token, session_expiry
from app.database import connect
from app.main import SESSION_COOKIE, app, require_permission


def test_identity_schema_rejects_duplicate_email_and_cascades_sessions():
    email = f"constraint-{uuid4().hex}@example.com"
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
```

```
            "INSERT INTO users (email, password_hash, display_name) VALUES (%s, %s, %s)
RETURNING user_id",
            (email, "synthetic-hash", "Synthetic Constraint Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with pytest.raises(psycopg.errors.UniqueViolation):
            with connect() as conn, conn.cursor() as cur:
                cur.execute(
                    "INSERT INTO users (email, password_hash, display_name) VALUES (%s, %s,
%s)",
                    (email.upper(), "synthetic-hash", "Duplicate Synthetic Test"),
                )

        _, digest = new_session_token()
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at) VALUES
(%s, %s, %s)",
                (user_id, digest, session_expiry(5)),
            )
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
            cur.execute("SELECT count(*) FROM auth_sessions WHERE user_id = %s", (user_id,))
            assert cur.fetchone()[0] == 0
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_platform_permission_is_resolved_from_database_role_and_session():
    admin_email = f"platform-admin-{uuid4().hex}@example.test"
    user_email = f"unprivileged-{uuid4().hex}@example.test"
    tokens = []
    with connect() as conn, conn.cursor() as cur:
        cur.execute("SELECT role_id FROM roles WHERE role_key = 'platform_admin' AND
scope = 'platform'")
        admin_role_id = cur.fetchone()[0]
        cur.execute(
            "INSERT INTO users (email, display_name, platform_role_id) VALUES (%s, %s, %s)
RETURNING user_id",
            (admin_email, "Synthetic Platform Admin", admin_role_id),
        )
        admin_id = cur.fetchone()[0]
```

```
        cur.execute(
            "INSERT INTO users (email, display_name) VALUES (%s, %s) RETURNING user_id",
            (user_email, "Synthetic Standard User"),
        )
        user_id = cur.fetchone()[0]
        for owner_id in (admin_id, user_id):
            token, digest = new_session_token()
            tokens.append(token)
            cur.execute(
                "INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at) VALUES
(%s, %s, %s)",
                (owner_id, digest, session_expiry(5)),
            )

    try:
        with TestClient(app) as client:
            anonymous = client.get("/api/v1/admin/authorization/capabilities")
            assert anonymous.status_code == 401
            assert anonymous.json() == {"detail": "Authentication required"}

            invalid_session = client.get(
                "/api/v1/admin/authorization/capabilities",
                cookies={SESSION_COOKIE: "invalid-session-token"},
            )
            assert invalid_session.status_code == 401

            allowed = client.get(
                "/api/v1/admin/authorization/capabilities",
                cookies={SESSION_COOKIE: tokens[0]},
            )
            #₹ The request_id header is optional, but if supplied it must be validated and preserved
as the correlation_id.
            correlated = client.get(
                "/api/v1/admin/authorization/capabilities",
                cookies={SESSION_COOKIE: tokens[0]},
                headers={"request_id": "client-correlation-123"},
            )
            assert correlated.status_code == 200
            correlated_body = correlated.json()
            assert correlated_body["request_id"].startswith("req_")
            assert correlated_body["request_id"] != "client-correlation-123"
            assert correlated_body["correlation_id"] == "client-correlation-123"

            malformed = client.get(
```

```
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: tokens[0]},
            headers={"request_id": "x" * 129},
        )
        assert malformed.status_code == 400

        # The platform-admin user should be allowed to inspect the authorization policy.
        assert allowed.status_code == 200
        body = allowed.json()
        assert body["data"]["subject"]["user_id"] == str(admin_id)
        assert body["data"]["subject"]["platform_role_code"] == "platform_admin"
        assert body["data"]["effective_permissions"] == ["platform.admin"]
        assert body["data"]["capabilities"] == ["inspect_authorization_policy"]
        assert body["data"]["policy_version"] == "sprint-1-platform-rbac-v1"
        assert body["request_id"].startswith("req_")
        assert body["correlation_id"] == body["request_id"]

        denied = client.get(
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: tokens[1]},
            headers={"X-Role": "platform_admin"},
        )
        assert denied.status_code == 403
        assert denied.json() == {"detail": "Permission denied"}
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id IN (%s, %s)", (admin_id, user_id))



def test_permission_grant_revocation_takes_effect_on_next_request():
    suffix = uuid4().hex
    role_key = f"test-admin-{suffix}"
    email = f"permission-revocation-{suffix}@example.test"
    user_id = None
    role_id = None
    permission_id = None

    try:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
                INSERT INTO roles (role_key, display_name, scope, description)
                VALUES (%s, %s, 'platform', %s)
```

```
        RETURNING role_id
        """,
        (role_key, "Synthetic Permission Revocation Role", "Test-only role"),
    )
    role_id = cur.fetchone()[0]

    cur.execute(
        "SELECT permission_id FROM permissions WHERE permission_key = %s",
        ("platform.admin",),
    )
    row = cur.fetchone()
    assert row is not None, "platform.admin permission must exist"
    permission_id = row[0]

    cur.execute(
        "INSERT INTO role_permissions (role_id, permission_id) VALUES (%s, %s)",
        (role_id, permission_id),
    )
    cur.execute(
        """
        INSERT INTO users (email, display_name, platform_role_id)
        VALUES (%s, %s, %s)
        RETURNING user_id
        """,
        (email, "Synthetic Permission Revocation User", role_id),
    )
    user_id = cur.fetchone()[0]

    token, digest = new_session_token()
    cur.execute(
        """
        INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at)
        VALUES (%s, %s, %s)
        """,
        (user_id, digest, session_expiry(5)),
    )

with TestClient(app) as client:
    first_response = client.get(
        "/api/v1/admin/authorization/capabilities",
        cookies={SESSION_COOKIE: token},
    )
    assert first_response.status_code == 200
    assert first_response.json()["data"]["effective_permissions"] == [
```

```
            "platform.admin"
        ]

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM role_permissions WHERE role_id = %s AND permission_id =
%s",
                (role_id, permission_id),
            )

        next_response = client.get(
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: token},
        )
        assert next_response.status_code == 403
        assert next_response.json() == {"detail": "Permission denied"}

    finally:
        with connect() as conn, conn.cursor() as cur:
            if user_id is not None:
                cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
            if role_id is not None:
                cur.execute("DELETE FROM roles WHERE role_id = %s", (role_id,))

def test_platform_role_assignment_rejects_workspace_scoped_roles():
    email = f"wrong-scope-{uuid4().hex}@example.test"
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO roles (role_key, display_name, scope) VALUES (%s, %s, 'workspace')
RETURNING role_id",
            (f"ws-{uuid4().hex[:12]}", "Synthetic Workspace Role"),
        )
        workspace_role_id = cur.fetchone()[0]
        cur.execute(
            "INSERT INTO users (email, display_name) VALUES (%s, 'Synthetic Scope User')
RETURNING user_id",
            (email,),
        )
        user_id = cur.fetchone()[0]
    try:
        with pytest.raises(psycopg.errors.CheckViolation):
            with connect() as conn, conn.cursor() as cur:
                cur.execute("UPDATE users SET platform_role_id = %s WHERE user_id = %s",
(workspace_role_id, user_id))
```

```
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
            cur.execute("DELETE FROM roles WHERE role_id = %s", (workspace_role_id,))




def test_platform_role_revocation_takes_effect_on_next_request():
    email = f"revoked-platform-admin-{uuid4().hex}@example.test"
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "SELECT role_id FROM roles WHERE role_key = 'platform_admin' AND scope =
'platform'"
        )
        row = cur.fetchone()
        assert row is not None, "Platform-admin role must exist"
        admin_role_id = row[0]
        cur.execute(
            "INSERT INTO users (email, display_name, platform_role_id) "
            "VALUES (%s, %s, %s) RETURNING user_id",
            (email, "Synthetic Revocation Test", admin_role_id),
        )
        user_id = cur.fetchone()[0]
        token, digest = new_session_token()
        cur.execute(
            "INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at) "
            "VALUES (%s, %s, %s)",
            (user_id, digest, session_expiry(5)),
        )

    try:
        with TestClient(app) as client:
            first_response = client.get(
                "/api/v1/admin/authorization/capabilities",
                cookies={SESSION_COOKIE: token},
            )
            assert first_response.status_code == 200
            assert first_response.json()["data"]["effective_permissions"] == [
                "platform.admin"
            ]

            # Revoke this test user's role without changing shared role permissions.
            with connect() as conn, conn.cursor() as cur:
```

```
            cur.execute(
                "UPDATE users SET platform_role_id = NULL WHERE user_id = %s",
                (user_id,),
            )

        next_response = client.get(
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: token},
        )
        assert next_response.status_code == 403
        assert next_response.json() == {"detail": "Permission denied"}
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


# def test_capabilities_request_id_is_canonical_and_correlation_is_preserved():
#     with connect() as conn, conn.cursor() as cur:
#         cur.execute(
#         """
#         SELECT u.user_id, s.refresh_token_hash
#         FROM users u
#         JOIN roles r ON r.role_id = u.platform_role_id
#         JOIN role_permissions rp ON rp.role_id = r.role_id
#         JOIN permissions p ON p.permission_id = rp.permission_id
#         JOIN auth_sessions s ON s.user_id = u.user_id
#         WHERE r.role_key = 'platform_admin'
#         AND p.permission_key = 'platform.admin'
#         AND s.revoked_at IS NULL
#         AND s.expires_at > now()
#         LIMIT 1
#         """
#         )
#     row = cur.fetchone()

#     if row is None:
#         pytest.skip("No active platform-admin test session is available")

#     # The database stores only the token digest, so use a dedicated test
#     # session instead of attempting to recover an existing session token.
#     pytest.skip("Use the dedicated-session integration fixture for this assertion")

# def test_capabilities_rejects_malformed_request_id():
#     # Authentication runs before the endpoint validates its header.
```

```
#     # An unauthenticated request must not be mistaken for a header-validation test.
#     response = TestClient(app).get(
#     "/api/v1/admin/authorization/capabilities",
#     headers={"request_id": "x" * 129},
#     )
#     assert response.status_code == 401=== Current database constraints and triggers ===
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'app'
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
.venv/bin/python - <<'PY'
from app.database import connect

with connect() as conn, conn.cursor() as cur:
    cur.execute("""
        SELECT conname, pg_get_constraintdef(oid)
        FROM pg_constraint
        WHERE conrelid = 'audit_events'::regclass
        ORDER BY conname
    """)
    print("=== Audit constraints ===")
    for row in cur.fetchall():
        print(row)

    cur.execute("""
        SELECT tgname
        FROM pg_trigger
        WHERE tgrelid = 'audit_events'::regclass
          AND NOT tgisinternal
        ORDER BY tgname
    """)
    print("=== Audit triggers ===")
    for row in cur.fetchall():
        print(row)
PY
=== Audit constraints ===
('audit_events_actor_user_id_fkey', 'FOREIGN KEY (actor_user_id) REFERENCES
users(user_id) ON DELETE SET NULL')
```

```
('audit_events_outcome_check', "CHECK (((outcome)::text = ANY ((ARRAY['success'::character
varying, 'failure'::character varying, 'denied'::character varying])::text[])))")
('audit_events_pkey', 'PRIMARY KEY (audit_event_id)')
=== Audit triggers ===
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
    ),
    Path("apps/api/tests/test_auth.py"): (
        'cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))',
        """cur.execute(
                "UPDATE auth_sessions SET revoked_at = COALESCE(revoked_at, now()), "
                "revoke_reason = COALESCE(revoke_reason, 'test_cleanup') "
                "WHERE user_id = %s",
                (user_id,),
            )
            cur.execute(
                "UPDATE users SET account_status = 'deleted', deleted_at =
COALESCE(deleted_at, now()) "
                "WHERE user_id = %s",
                (user_id,),
            )""",
        1,
    ),
}

for path, (old, new, expected_count) in changes.items():
    source = path.read_text(encoding="utf-8")
    actual_count = source.count(old)
    if actual_count != expected_count:
        raise SystemExit(
            f"STOP: {path}: expected {expected_count} matching cleanup statements, found
{actual_count}; no changes made to this file."
        )
    path.write_text(source.replace(old, new), encoding="utf-8")
    print(f"Updated {expected_count} cleanup statement(s) in {path}")

PY

git diff --check
git diff -- apps/api/tests/test_fin14_auth_security.py apps/api/tests/test_auth.py
Updated 4 cleanup statement(s) in apps/api/tests/test_fin14_auth_security.py
Updated 1 cleanup statement(s) in apps/api/tests/test_auth.py
diff --git a/apps/api/tests/test_auth.py b/apps/api/tests/test_auth.py
index 2d8dd8f..b77d907 100644
```

```
--- a/apps/api/tests/test_auth.py
+++ b/apps/api/tests/test_auth.py
@@ -8,6 +8,7 @@ from fastapi.testclient import TestClient
 from psycopg.rows import dict_row

 from app.auth import hash_password, token_digest, verify_password
+from app.throttle import make_throttle_key
 from app.database import connect
 from app.main import SESSION_COOKIE, app

@@ -28,7 +29,17 @@ def identity():
         yield user_id, email, encoded
     finally:
         with connect() as conn, conn.cursor() as cur:
-            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
+            cur.execute(
+                "UPDATE auth_sessions SET revoked_at = COALESCE(revoked_at, now()), "
+                "revoke_reason = COALESCE(revoke_reason, 'test_cleanup') "
+                "WHERE user_id = %s",
+                (user_id,),
+            )
+            cur.execute(
+                "UPDATE users SET account_status = 'deleted', deleted_at =
COALESCE(deleted_at, now()) "
+                "WHERE user_id = %s",
+                (user_id,),
+            )


 @pytest.fixture
@@ -36,8 +47,27 @@ def client(monkeypatch):
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
index 2d8dd8f..b77d907 100644
--- a/apps/api/tests/test_auth.py
+++ b/apps/api/tests/test_auth.py
@@ -8,6 +8,7 @@ from fastapi.testclient import TestClient
 from psycopg.rows import dict_row

 from app.auth import hash_password, token_digest, verify_password
+from app.throttle import make_throttle_key
 from app.database import connect
 from app.main import SESSION_COOKIE, app

@@ -28,7 +29,17 @@ def identity():
```

```
        yield user_id, email, encoded
    finally:
        with connect() as conn, conn.cursor() as cur:
-           cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
+           cur.execute(
+               "UPDATE auth_sessions SET revoked_at = COALESCE(revoked_at, now()), "
+               "revoke_reason = COALESCE(revoke_reason, 'test_cleanup') "
+               "WHERE user_id = %s",
+               (user_id,),
+           )
+           cur.execute(
+               "UPDATE users SET account_status = 'deleted', deleted_at =
COALESCE(deleted_at, now()) "
+               "WHERE user_id = %s",
+               (user_id,),
+           )


 @pytest.fixture
@@ -36,8 +47,27 @@ def client(monkeypatch):
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
               "DELETE FROM audit_events WHERE audit_event_id = %s",
               (audit_event_id,),
           )
    assert delete_error.value.sqlstate == "55000"

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            UPDATE users
            SET account_status = 'deleted',
                deleted_at = COALESCE(deleted_at, now())
            WHERE user_id = %s
            """,
            (user_id,),
        )
        cur.execute(
            """
            SELECT actor_user_id, action_key, outcome
            FROM audit_events
            WHERE audit_event_id = %s
            """,
            (audit_event_id,),
```

```
        )
        event = cur.fetchone()

    assert event == (user_id, action_key, "success")
", encoding="utf-8")

print(f"Created {path}")
PY

git diff --check
Created apps/api/tests/test_audit_immutability.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....

migration = Path("apps/api/migrations/0004_immutable_auth_audit.sql")
if not migration.exists():
    raise SystemExit(f"STOP: migration not found: {migration}")

with connect() as conn, conn.cursor() as cur:
    cur.execute("""
        SELECT
            EXISTS (
                SELECT 1 FROM pg_trigger
                WHERE tgrelid = 'audit_events'::regclass
                  AND tgname = 'audit_events_append_only'
                  AND NOT tgisinternal
            ),
            (
                SELECT confdeltype
                FROM pg_constraint
                WHERE conrelid = 'audit_events'::regclass
                  AND conname = 'audit_events_actor_user_id_fkey'
            )
    """)
    trigger_exists, delete_rule = cur.fetchone()

    if trigger_exists or delete_rule == "r":
        raise SystemExit(
            f"STOP: migration appears already applied (trigger={trigger_exists},
delete_rule={delete_rule})."
        )

    cur.execute(migration.read_text(encoding="utf-8"))

print("Applied migration 0004 successfully.")
```

```
PY
Applied migration 0004 successfully.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest \
  apps/api/tests/test_audit_immutability.py \
  apps/api/tests/test_audit_service.py \
  apps/api/tests/test_throttle.py \
  apps/api/tests/test_fin14_auth_security.py \
  apps/api/tests/test_auth.py \
  -q
.....................................
[100%]
========================================================================
========= warnings summary
========================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken
apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
```

```
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
38 passed, 7 warnings in 1.68s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
38 passed, 7 warnings in 1.68s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE key_hash IN (%s, %s)",
                (account_key, ip_key),
            )
            cur.execute(
                """UPDATE auth_sessions
                   SET revoked_at = COALESCE(revoked_at, now()),
                       revoke_reason = COALESCE(revoke_reason, 'test_cleanup')
                   WHERE user_id = %s""",
                (user_id,),
            )
            cur.execute(
                """UPDATE users
                   SET account_status = 'deleted',
                       deleted_at = COALESCE(deleted_at, now())
                   WHERE user_id = %s""",
                (user_id,),
            )
'''

path.write_text(source, encoding="utf-8")
print(f"Added throttle expiry/retry test to {path}")
PY

git diff --check

PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest \

apps/api/tests/test_fin14_auth_security.py::test_throttle_expires_and_correct_login_succeeds_a
fter_retry \
```

```
 -q
Added throttle expiry/retry test to apps/api/tests/test_fin14_auth_security.py
F
[100%]
================================================================================
============= FAILURES
================================================================================
==============

_____________________________________________________________________
test_throttle_expires_and_correct_login_succeeds_after_retry
_____________________________________________________________________

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x12417ce90>

    def test_throttle_expires_and_correct_login_succeeds_after_retry(monkeypatch):
        secret = "integration-test-secret-material-32-bytes"
        monkeypatch.setenv("AUTH_THROTTLE_SECRET", secret)
        monkeypatch.setenv("AUTH_LOGIN_MAX_FAILURES", "1")
        monkeypatch.setenv("AUTH_LOGIN_WINDOW_SECONDS", "60")
        monkeypatch.setenv("AUTH_LOGIN_BLOCK_SECONDS", "1")

        email = f"{uuid4().hex}@example.com"
        password = "Synthetic-FIN14-Expiry-Retry-Password"
        failed_correlation = uuid4()
        blocked_correlation = uuid4()
        retry_correlation = uuid4()
        account_key = make_throttle_key(secret.encode("utf-8"), "account", email.lower())
        ip_key = make_throttle_key(secret.encode("utf-8"), "ip", "testclient")

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """DELETE FROM auth_login_throttle
                   WHERE key_hash IN (%s, %s)""",
                (account_key, ip_key),
            )
            cur.execute(
                """INSERT INTO users (email, password_hash, display_name, account_status)
                   VALUES (%s, %s, %s, 'active') RETURNING user_id""",
                (email, hash_password(password), "FIN14 Throttle Expiry Test"),
            )
            user_id = cur.fetchone()[0]

        try:
            with TestClient(app) as client:
```

```
            failed = client.post(
                "/auth/login",
                json={"email": email, "password": "wrong-password"},
                headers={"X-Correlation-ID": str(failed_correlation)},
            )
            assert failed.status_code == 401

            blocked = client.post(
                "/auth/login",
                json={"email": email, "password": password},
                headers={"X-Correlation-ID": str(blocked_correlation)},
            )
>           assert blocked.status_code == 401
E           assert 200 == 401
E            +  where 200 = <Response [200 OK]>.status_code

apps/api/tests/test_fin14_auth_security.py:398: AssertionError
=============================================================================
========= warnings summary
=============================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=============================================================================
====== short test summary info
=============================================================================
======
FAILED
apps/api/tests/test_fin14_auth_security.py::test_throttle_expires_and_correct_login_succeeds_a
fter_retry - assert 200 == 401
1 failed, 2 warnings in 0.44s
```

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

echo "=== Throttle implementation ==="
sed -n '1,280p' apps/api/app/throttle.py

echo "=== Login route and throttle configuration ==="
grep -n -E
'AUTH_LOGIN_MAX_FAILURES|AUTH_LOGIN_WINDOW_SECONDS|AUTH_LOGIN_BLOCK
_SECONDS|make_throttle_key|record_failure|is_blocked|reset_bucket|auth.login'
apps/api/app/main.py

echo "=== Relevant login route ==="
sed -n '220,430p' apps/api/app/main.py

echo "=== Throttle unit tests ==="
sed -n '1,300p' apps/api/tests/test_throttle.py
=== Throttle implementation ===
"""PostgreSQL-backed login throttling with secret-keyed identifiers."""

from __future__ import annotations

import hashlib
import hmac
import os
from dataclasses import dataclass
from datetime import UTC, datetime
from typing import Any


@dataclass(frozen=True)
class ThrottleConfig:
    secret: bytes
    max_failures: int
    window_seconds: int
    block_seconds: int


def load_throttle_config() -> ThrottleConfig:
    """Load validated throttle settings; never use a weak fallback secret."""
    secret = os.environ.get("AUTH_THROTTLE_SECRET", "")
    if len(secret.encode("utf-8")) < 32:
```

```
        raise ValueError("AUTH_THROTTLE_SECRET must contain at least 32 bytes")

    try:
        max_failures = int(os.environ.get("AUTH_LOGIN_MAX_FAILURES", "5"))
        window_seconds = int(os.environ.get("AUTH_LOGIN_WINDOW_SECONDS", "900"))
        block_seconds = int(os.environ.get("AUTH_LOGIN_BLOCK_SECONDS", "900"))
    except ValueError as exc:
        raise ValueError("Login throttle configuration is invalid") from exc

    if not 1 <= max_failures <= 100:
        raise ValueError("AUTH_LOGIN_MAX_FAILURES must be between 1 and 100")
    if not 1 <= window_seconds <= 86400:
        raise ValueError("AUTH_LOGIN_WINDOW_SECONDS must be between 1 and 86400")
    if not 1 <= block_seconds <= 86400:
        raise ValueError("AUTH_LOGIN_BLOCK_SECONDS must be between 1 and 86400")

    return ThrottleConfig(
        secret=secret.encode("utf-8"),
        max_failures=max_failures,
        window_seconds=window_seconds,
        block_seconds=block_seconds,
    )


def make_throttle_key(secret: bytes, scope_type: str, value: str) -> str:
    """Return a stable HMAC-SHA256 identifier without storing the raw value."""
    if scope_type not in {"account", "ip"}:
        raise ValueError("Unsupported throttle scope")
    if not value:
        raise ValueError("Throttle key value cannot be empty")

    message = f"finsight-login:{scope_type}:{value}".encode("utf-8")
    return hmac.new(secret, message, hashlib.sha256).hexdigest()


def is_blocked(
    conn: Any,
    *,
    scope_type: str,
    key_hash: str,
    now: datetime | None = None,
) -> bool:
    current_time = now or datetime.now(UTC)
    with conn.cursor() as cur:
```

```
        cur.execute(
            """
            SELECT blocked_until > %s
            FROM auth_login_throttle
            WHERE scope_type = %s AND key_hash = %s
            """,
            (current_time, scope_type, key_hash),
        )
        row = cur.fetchone()
    # Accept the timestamp result used by the database query, and Boolean
    # results from adapters that evaluate the comparison in SQL.
    if not row or row[0] is None:
        return False
    value = row[0]
    if isinstance(value, bool):
        return value
    return value > current_time


def record_failure(
    conn: Any,
    *,
    scope_type: str,
    key_hash: str,
    config: ThrottleConfig,
    now: datetime | None = None,
) -> datetime | None:
    """Atomically increment a bucket and return its active block expiry, if any."""
    current_time = now or datetime.now(UTC)
    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO auth_login_throttle (
                scope_type, key_hash, failed_attempts,
                window_started_at, blocked_until, updated_at
            )
            VALUES (%s, %s, 1, %s, NULL, %s)
            ON CONFLICT (scope_type, key_hash) DO UPDATE SET
                failed_attempts = CASE
                    WHEN auth_login_throttle.window_started_at
                        <= %s - make_interval(secs => %s)
                    THEN 1
                    ELSE auth_login_throttle.failed_attempts + 1
                END,
```

```
            window_started_at = CASE
                WHEN auth_login_throttle.window_started_at
                    <= %s - make_interval(secs => %s)
                THEN %s
                ELSE auth_login_throttle.window_started_at
            END,
            blocked_until = CASE
                WHEN CASE
                    WHEN auth_login_throttle.window_started_at
                        <= %s - make_interval(secs => %s)
                    THEN 1
                    ELSE auth_login_throttle.failed_attempts + 1
                END >= %s
                THEN %s + make_interval(secs => %s)
                WHEN auth_login_throttle.blocked_until > %s
                THEN auth_login_throttle.blocked_until
                ELSE NULL
            END,
            updated_at = %s
        RETURNING blocked_until
        """,
        (
            scope_type, key_hash, current_time, current_time,
            current_time, config.window_seconds,
            current_time, config.window_seconds, current_time,
            current_time, config.window_seconds, config.max_failures,
            current_time, config.block_seconds, current_time,
            current_time,
        ),
    )
    row = cur.fetchone()

if row and row[0] and row[0] > current_time:
    return row[0]
return None


def reset_bucket(conn: Any, *, scope_type: str, key_hash: str) -> None:
    """Clear a successful login's corresponding failure counter."""
    with conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = %s AND key_hash = %s",
            (scope_type, key_hash),
        )
```

```
=== Login route and throttle configuration ===
14:    is_blocked,
16:    make_throttle_key,
17:    record_failure,
18:    reset_bucket,
39:    if request.url.path not in {"/auth/login", "/auth/logout"}:
175:@app.post("/auth/login", response_model=UserView)
190:    account_key = make_throttle_key(config.secret, "account", email)
191:    ip_key = make_throttle_key(config.secret, "ip", client_ip)
197:            account_blocked = is_blocked(
200:            ip_blocked = is_blocked(
207:                    action_key="auth.login",
238:                    record_failure(
244:                    record_failure(
252:                        action_key="auth.login",
260:                    reset_bucket(
286:                        action_key="auth.login",
=== Relevant login route ===
                        AND deleted_at IS NULL""",
                    (email,),
                )
                user = cur.fetchone()

            password_hash = (
                user["password_hash"]
                if user and user["password_hash"]
                else DUMMY_PASSWORD_HASH
            )
            password_valid = verify_password(
                password_hash, payload.password
            )
            login_succeeded = bool(
                user and user["password_hash"] and password_valid
            )

            if not login_succeeded:
                record_failure(
                    conn,
                    scope_type="account",
                    key_hash=account_key,
                    config=config,
                )
                record_failure(
                    conn,
```

```
            scope_type="ip",
            key_hash=ip_key,
            config=config,
        )
        record_auth_event(
            conn,
            action_key="auth.login",
            outcome="failure",
            actor_user_id=user["user_id"] if user else None,
            details={"reason": "invalid_credentials"},
            correlation_id=correlation_id,
        )
    else:
        # Clear account failures only. Preserve IP-wide history.
        reset_bucket(
            conn,
            scope_type="account",
            key_hash=account_key,
        )

        token, digest = new_session_token()
        expires_at = session_expiry(_ttl_minutes())

        with conn.cursor() as cur:
            cur.execute(
                """INSERT INTO auth_sessions
                       (user_id, refresh_token_hash, expires_at)
                   VALUES (%s, %s, %s)""",
                (user["user_id"], digest, expires_at),
            )
            now = datetime.now(UTC)
            cur.execute(
                """UPDATE users
                   SET last_login_at = %s, updated_at = %s
                   WHERE user_id = %s""",
                (now, now, user["user_id"]),
            )

        record_auth_event(
            conn,
            action_key="auth.login",
            outcome="success",
            actor_user_id=user["user_id"],
            details={"method": "password"},
```

```
                    correlation_id=correlation_id,
                )

    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    if not login_succeeded:
        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

    response.set_cookie(
        SESSION_COOKIE,
        token,
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
        max_age=_ttl_minutes() * 60,
        path="/",
    )
    return UserView(
        user_id=user["user_id"],
        email=user["email"],
        display_name=user["display_name"],
    )


@app.post("/auth/logout", status_code=204)
def logout(
    request: Request,
    response: Response,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers['X-Correlation-ID'] = str(correlation_id)

    try:
        with connect() as conn:
            actor_user_id = None

            if session_token:
                with conn.cursor() as cur:
```

```
                cur.execute(
                    """UPDATE auth_sessions
                       SET revoked_at = %s, revoke_reason = 'logout'
                       WHERE refresh_token_hash = %s
                         AND revoked_at IS NULL
                       RETURNING user_id""",
                    (datetime.now(UTC), token_digest(session_token)),
                )
                row = cur.fetchone()
                if row:
                    actor_user_id = row[0]

            record_auth_event(
                conn,
                action_key="auth.logout",
                outcome="success" if actor_user_id else "denied",
                actor_user_id=actor_user_id,
                details={"reason": "logout"},
                correlation_id=correlation_id,
            )
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    response.delete_cookie(
        SESSION_COOKIE,
        path="/",
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
    )
    response.status_code = status.HTTP_204_NO_CONTENT
    return response


@app.get("/auth/me", response_model=UserView)
def who_am_i(user: CurrentUser) -> UserView:
    return UserView(**user)


@app.get("/api/v1/admin/authorization/capabilities")
def platform_admin_capabilities(
```

```
    request: Request,
    _authorized: Annotated[dict[str, Any], Depends(require_permission("platform.admin"))],
) -> dict[str, Any]:
    """Return the authenticated caller's database-backed platform capabilities."""

    supplied_request_id = request.headers.get("request_id", "").strip()
    if supplied_request_id and (
        len(supplied_request_id) > 128
        or any(ord(char) < 32 or ord(char) == 127 for char in supplied_request_id)
    ):
        raise HTTPException(status_code=400, detail="Invalid request_id")

    # Always issue a server-generated canonical request ID.
    request_id = f"req_{secrets.token_urlsafe(12)}"
    correlation_id = supplied_request_id or request_id
    permissions = sorted(set(_authorized.get("permissions", [])))
    capabilities = (
        ["inspect_authorization_policy"]
        if "platform.admin" in permissions
        else []
    )

    return {
        "data": {
            "subject": {
                "user_id": str(_authorized["user_id"]),
                "platform_role_code": _authorized.get("role_key"),
            },
            "effective_permissions": permissions,
            "capabilities": capabilities,
            "policy_version": "sprint-1-platform-rbac-v1",
        },
        "request_id": request_id,
        "correlation_id": correlation_id,
    }
=== Throttle unit tests ===
from __future__ import annotations

from datetime import UTC, datetime, timedelta

import pytest

from app.throttle import (
    ThrottleConfig,
```

```
    is_blocked,
    load_throttle_config,
    make_throttle_key,
)


def test_throttle_config_requires_a_strong_secret(monkeypatch):
    monkeypatch.delenv("AUTH_THROTTLE_SECRET", raising=False)

    with pytest.raises(ValueError, match="AUTH_THROTTLE_SECRET"):
        load_throttle_config()

    monkeypatch.setenv("AUTH_THROTTLE_SECRET", "short-secret")
    with pytest.raises(ValueError, match="AUTH_THROTTLE_SECRET"):
        load_throttle_config()


def test_throttle_config_accepts_valid_settings(monkeypatch):
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", "x" * 32)
    monkeypatch.setenv("AUTH_LOGIN_MAX_FAILURES", "4")
    monkeypatch.setenv("AUTH_LOGIN_WINDOW_SECONDS", "600")
    monkeypatch.setenv("AUTH_LOGIN_BLOCK_SECONDS", "300")

    config = load_throttle_config()

    assert config.max_failures == 4
    assert config.window_seconds == 600
    assert config.block_seconds == 300


@pytest.mark.parametrize(
    ("name", "value"),
    [
        ("AUTH_LOGIN_MAX_FAILURES", "0"),
        ("AUTH_LOGIN_MAX_FAILURES", "101"),
        ("AUTH_LOGIN_WINDOW_SECONDS", "0"),
        ("AUTH_LOGIN_WINDOW_SECONDS", "86401"),
        ("AUTH_LOGIN_BLOCK_SECONDS", "0"),
        ("AUTH_LOGIN_BLOCK_SECONDS", "86401"),
        ("AUTH_LOGIN_MAX_FAILURES", "not-a-number"),
    ],
)
def test_throttle_config_rejects_invalid_limits(monkeypatch, name, value):
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", "x" * 32)
```

```
    monkeypatch.setenv(name, value)

    with pytest.raises(ValueError):
        load_throttle_config()


def test_throttle_keys_are_stable_and_scope_separated():
    secret = b"test-only-secret-material-at-least-32-bytes"
    first = make_throttle_key(secret, "account", "analyst@example.test")

    assert first == make_throttle_key(secret, "account", "analyst@example.test")
    assert first != make_throttle_key(secret, "ip", "analyst@example.test")
    assert first != make_throttle_key(b"different-secret-material-32-bytes", "account",
"analyst@example.test")
    assert "analyst@example.test" not in first


def test_throttle_key_rejects_unknown_scope():
    with pytest.raises(ValueError, match="scope"):
        make_throttle_key(b"x" * 32, "email", "analyst@example.test")


class FakeCursor:
    def __init__(self, row):
        self.row = row
        self.calls = []

    def execute(self, query, params):
        self.calls.append((query, params))

    def fetchone(self):
        return self.row

    def __enter__(self):
        return self

    def __exit__(self, *args):
        return False


class FakeConnection:
    def __init__(self, row):
        self.cursor_instance = FakeCursor(row)
```

```
    def cursor(self):
        return self.cursor_instance


def test_is_blocked_returns_true_only_for_future_expiry():
    now = datetime(2026, 10, 21, tzinfo=UTC)
    conn = FakeConnection((now + timedelta(minutes=5),))

    assert is_blocked(
        conn, scope_type="account", key_hash="a" * 64, now=now
    )
    query, params = conn.cursor_instance.calls[0]
    assert "auth_login_throttle" in query
    assert params == (now, "account", "a" * 64)


def test_is_blocked_returns_false_for_expired_or_missing_bucket():
    now = datetime(2026, 10, 21, tzinfo=UTC)

    assert not is_blocked(
        FakeConnection((now - timedelta(seconds=1),)),
        scope_type="ip",
        key_hash="b" * 64,
        now=now,
    )
    assert not is_blocked(
        FakeConnection(None),
        scope_type="ip",
        key_hash="b" * 64,
        now=now,
    )


def test_throttle_config_defaults_are_documented_values(monkeypatch):
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", "x" * 32)
    for name in (
        "AUTH_LOGIN_MAX_FAILURES",
        "AUTH_LOGIN_WINDOW_SECONDS",
        "AUTH_LOGIN_BLOCK_SECONDS",
    ):
        monkeypatch.delenv(name, raising=False)

    config = load_throttle_config()
    assert config == ThrottleConfig(
```

```
        secret=b"x" * 32,
        max_failures=5,
        window_seconds=900,
        block_seconds=900,
    )
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

sed -n '160,275p' apps/api/app/main.py

echo "=== Current failing test ==="
sed -n '350,450p' apps/api/tests/test_fin14_auth_security.py

echo "=== Database throttle rows for recent attempts ==="
PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
.venv/bin/python - <<'PY'
from app.database import connect

with connect() as conn, conn.cursor() as cur:
    cur.execute("""
        SELECT scope_type, failed_attempts, window_started_at,
               blocked_until, updated_at
        FROM auth_login_throttle
        ORDER BY updated_at DESC
        LIMIT 10
    """)
    for row in cur.fetchall():
        print(row)
PY


CurrentUser = Annotated[dict[str, Any], Depends(get_current_user)]


def require_permission(permission_key: str):
    """Build a deny-by-default backend permission dependency."""
    def dependency(user: CurrentUser) -> dict[str, Any]:
        if permission_key not in user.get("permissions", []):
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Permission
denied")
```

```
        return user

    return dependency


@app.post("/auth/login", response_model=UserView)
def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers['X-Correlation-ID'] = str(correlation_id)
    email = payload.email.strip().lower()
    try:
        config = load_throttle_config()
    except ValueError as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    client_ip = request.client.host if request.client else "unknown"
    account_key = make_throttle_key(config.secret, "account", email)
    ip_key = make_throttle_key(config.secret, "ip", client_ip)
    login_succeeded = False
    user = None

    try:
        with connect() as conn:
            account_blocked = is_blocked(
                conn, scope_type="account", key_hash=account_key
            )
            ip_blocked = is_blocked(
                conn, scope_type="ip", key_hash=ip_key
            )

            if account_blocked or ip_blocked:
                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="denied",
                    actor_user_id=None,
                    details={"reason": "throttled"},
                    correlation_id=correlation_id,
                )
            else:
```

```
with conn.cursor(row_factory=dict_row) as cur:
    cur.execute(
        """SELECT user_id, email, display_name, password_hash
           FROM users
           WHERE lower(email) = %s
             AND account_status = 'active'
             AND deleted_at IS NULL""",
        (email,),
    )
    user = cur.fetchone()

password_hash = (
    user["password_hash"]
    if user and user["password_hash"]
    else DUMMY_PASSWORD_HASH
)
password_valid = verify_password(
    password_hash, payload.password
)
login_succeeded = bool(
    user and user["password_hash"] and password_valid
)

if not login_succeeded:
    record_failure(
        conn,
        scope_type="account",
        key_hash=account_key,
        config=config,
    )
    record_failure(
        conn,
        scope_type="ip",
        key_hash=ip_key,
        config=config,
    )
    record_auth_event(
        conn,
        action_key="auth.login",
        outcome="failure",
        actor_user_id=user["user_id"] if user else None,
        details={"reason": "invalid_credentials"},
        correlation_id=correlation_id,
    )
```

```
            else:
                # Clear account failures only. Preserve IP-wide history.
                reset_bucket(
                    conn,
                    scope_type="account",
                    key_hash=account_key,
                )

                token, digest = new_session_token()
                expires_at = session_expiry(_ttl_minutes())

                with conn.cursor() as cur:
                    cur.execute(
                        """INSERT INTO auth_sessions
                               (user_id, refresh_token_hash, expires_at)
                           VALUES (%s, %s, %s)""",
                        (user["user_id"], digest, expires_at),
                    )
=== Current failing test ===
        event = cur.fetchone()

    assert event is not None
    assert event[0] == parsed_correlation_id


def test_throttle_expires_and_correct_login_succeeds_after_retry(monkeypatch):
    secret = "integration-test-secret-material-32-bytes"
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", secret)
    monkeypatch.setenv("AUTH_LOGIN_MAX_FAILURES", "1")
    monkeypatch.setenv("AUTH_LOGIN_WINDOW_SECONDS", "60")
    monkeypatch.setenv("AUTH_LOGIN_BLOCK_SECONDS", "1")

    email = f"{uuid4().hex}@example.com"
    password = "Synthetic-FIN14-Expiry-Retry-Password"
    failed_correlation = uuid4()
    blocked_correlation = uuid4()
    retry_correlation = uuid4()
    account_key = make_throttle_key(secret.encode("utf-8"), "account", email.lower())
    ip_key = make_throttle_key(secret.encode("utf-8"), "ip", "testclient")

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """DELETE FROM auth_login_throttle
               WHERE key_hash IN (%s, %s)""",
```

```
        (account_key, ip_key),
    )
    cur.execute(
        """INSERT INTO users (email, password_hash, display_name, account_status)
           VALUES (%s, %s, %s, 'active') RETURNING user_id""",
        (email, hash_password(password), "FIN14 Throttle Expiry Test"),
    )
    user_id = cur.fetchone()[0]

try:
    with TestClient(app) as client:
        failed = client.post(
            "/auth/login",
            json={"email": email, "password": "wrong-password"},
            headers={"X-Correlation-ID": str(failed_correlation)},
        )
        assert failed.status_code == 401

        blocked = client.post(
            "/auth/login",
            json={"email": email, "password": password},
            headers={"X-Correlation-ID": str(blocked_correlation)},
        )
        assert blocked.status_code == 401
        assert blocked.json() == failed.json()

        time.sleep(1.15)

        retry = client.post(
            "/auth/login",
            json={"email": email, "password": password},
            headers={"X-Correlation-ID": str(retry_correlation)},
        )
        assert retry.status_code == 200
        assert retry.json()["user_id"] == str(user_id)
        assert retry.headers["X-Correlation-ID"] == str(retry_correlation)

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """SELECT correlation_id, outcome
               FROM audit_events
               WHERE correlation_id = ANY(%s)
               ORDER BY occurred_at""",
            ([failed_correlation, blocked_correlation, retry_correlation],),
```

```
            )
            outcomes = {row[0]: row[1] for row in cur.fetchall()}

        assert outcomes.get(failed_correlation) == "failure"
        assert outcomes.get(blocked_correlation) == "denied"
        assert outcomes.get(retry_correlation) == "success"
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE key_hash IN (%s, %s)",
                (account_key, ip_key),
            )
            cur.execute(
                """UPDATE auth_sessions
                   SET revoked_at = COALESCE(revoked_at, now()),
                       revoke_reason = COALESCE(revoke_reason, 'test_cleanup')
                   WHERE user_id = %s""",
                (user_id,),
            )
            cur.execute(
                """UPDATE users
                   SET account_status = 'deleted',
                       deleted_at = COALESCE(deleted_at, now())
                   WHERE user_id = %s""",
                (user_id,),
            )
=== Database throttle rows for recent attempts ===
('account', 1, datetime.datetime(2026, 10, 21, 20, 13, 26, 310844,
tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')), None, datetime.datetime(2026, 10, 21, 20, 13,
26, 310844, tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')))
('account', 1, datetime.datetime(2026, 10, 21, 20, 13, 26, 269474,
tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')), None, datetime.datetime(2026, 10, 21, 20, 13,
26, 269474, tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')))
('account', 1, datetime.datetime(2026, 10, 21, 20, 13, 25, 978732,
tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')), None, datetime.datetime(2026, 10, 21, 20, 13,
25, 978732, tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')))
('account', 1, datetime.datetime(2026, 10, 21, 20, 13, 25, 929039,
tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')), None, datetime.datetime(2026, 10, 21, 20, 13,
25, 929039, tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')))
('account', 2, datetime.datetime(2026, 10, 21, 20, 13, 25, 630371,
tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')), datetime.datetime(2026, 10, 21, 20, 14, 25,
668186, tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')), datetime.datetime(2026, 10, 21, 20, 13,
25, 668186, tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')))
```

```
('account', 1, datetime.datetime(2026, 10, 21, 19, 58, 7, 427523,
tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')), None, datetime.datetime(2026, 10, 21, 19, 58, 7,
427523, tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')))
('account', 1, datetime.datetime(2026, 10, 21, 19, 58, 7, 379049,
tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')), None, datetime.datetime(2026, 10, 21, 19, 58, 7,
379049, tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')))
('account', 2, datetime.datetime(2026, 10, 21, 19, 58, 7, 73900,
tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')), datetime.datetime(2026, 10, 21, 19, 59, 7,
114745, tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')), datetime.datetime(2026, 10, 21, 19, 58,
7, 114745, tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')))
('account', 1, datetime.datetime(2026, 10, 21, 19, 58, 6, 369589,
tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')), None, datetime.datetime(2026, 10, 21, 19, 58, 6,
369589, tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')))
('account', 1, datetime.datetime(2026, 10, 21, 19, 55, 5, 660972,
tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')), None, datetime.datetime(2026, 10, 21, 19, 55, 5,
660972, tzinfo=zoneinfo.ZoneInfo(key='Asia/Kolkata')))
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
info.ZoneInfo(key='Asia/Kolkata')))
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
    max_failures=1,
    window_seconds=60,
    block_seconds=30,
)

try:
    with connect() as conn:
        blocked_until = record_failure(
            conn,
            scope_type="account",
            key_hash=key,
            config=config,
            now=now,
        )
        with conn.cursor() as cur:
            cur.execute(
                """SELECT failed_attempts, blocked_until
                   FROM auth_login_throttle
                   WHERE scope_type = 'account' AND key_hash = %s""",
                (key,),
            )
            row = cur.fetchone()

    print("record_failure returned:", blocked_until)
    print("database row:", row)
```

```
finally:
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'account' AND key_hash =
%s",
            (key,),
        )
PY
record_failure returned: None
database row: (1, None)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI" || exit 1

echo "=== Throttle table schema ==="
grep -n -A35 -B5 'CREATE TABLE.*auth_login_throttle' \
  apps/api/migrations/0003_auth_audit_and_login_throttling.sql

echo "=== Exact record_failure SQL ==="
sed -n '100,175p' apps/api/app/throttle.py

echo "=== Existing tests for failure threshold ==="
grep -n -E 'record_failure|max_failures|blocked_until|failed_attempts' \
  apps/api/tests/test_throttle.py
=== Throttle table schema ===
27-    ON audit_events (action_key, occurred_at DESC);
28-
29--- Persistent, shared login-failure counters.
30--- key_hash must be a keyed digest generated by the application;
31--- raw email addresses, IP addresses, passwords and tokens do not belong here.
32:CREATE TABLE auth_login_throttle (
33-    scope_type VARCHAR(16) NOT NULL
34-        CHECK (scope_type IN ('account', 'ip')),
35-    key_hash CHAR(64) NOT NULL,
36-    failed_attempts INTEGER NOT NULL DEFAULT 0
37-        CHECK (failed_attempts >= 0),
38-    window_started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
39-    blocked_until TIMESTAMPTZ NULL,
40-    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
41-    PRIMARY KEY (scope_type, key_hash)
42-);
43-
44-CREATE INDEX idx_auth_login_throttle_blocked_until
```

```
45-    ON auth_login_throttle (blocked_until)
46-    WHERE blocked_until IS NOT NULL;
47-
48-COMMIT;
=== Exact record_failure SQL ===
            """
            INSERT INTO auth_login_throttle (
               scope_type, key_hash, failed_attempts,
               window_started_at, blocked_until, updated_at
            )
            VALUES (%s, %s, 1, %s, NULL, %s)
            ON CONFLICT (scope_type, key_hash) DO UPDATE SET
               failed_attempts = CASE
                  WHEN auth_login_throttle.window_started_at
                      <= %s - make_interval(secs => %s)
                  THEN 1
                  ELSE auth_login_throttle.failed_attempts + 1
               END,
               window_started_at = CASE
                  WHEN auth_login_throttle.window_started_at
                      <= %s - make_interval(secs => %s)
                  THEN %s
                  ELSE auth_login_throttle.window_started_at
               END,
               blocked_until = CASE
                  WHEN CASE
                     WHEN auth_login_throttle.window_started_at
                         <= %s - make_interval(secs => %s)
                     THEN 1
                     ELSE auth_login_throttle.failed_attempts + 1
                  END >= %s
                  THEN %s + make_interval(secs => %s)
                  WHEN auth_login_throttle.blocked_until > %s
                  THEN auth_login_throttle.blocked_until
                  ELSE NULL
               END,
               updated_at = %s
            RETURNING blocked_until
            """,
            (
               scope_type, key_hash, current_time, current_time,
               current_time, config.window_seconds,
               current_time, config.window_seconds, current_time,
               current_time, config.window_seconds, config.max_failures,
```

```
                current_time, config.block_seconds, current_time,
                current_time,
            ),
        )
        row = cur.fetchone()

    if row and row[0] and row[0] > current_time:
        return row[0]
    return None


def reset_bucket(conn: Any, *, scope_type: str, key_hash: str) -> None:
    """Clear a successful login's corresponding failure counter."""
    with conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = %s AND key_hash = %s",
            (scope_type, key_hash),
        )
=== Existing tests for failure threshold ===
34:    assert config.max_failures == 4
141:        max_failures=5,
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
```